

# DIGITAL ELECTRONICS CIRCUITS

Course Code: BEL5T16C • Semester V

Complete First-Principles Comprehensive Lecture Notes,  
Hardware Reality Checks, Solved University Problems & Practical Lab Manual

Core Authorized Textbooks Synthesized:

- A. Anand Kumar • *Fundamentals of Digital Circuits* (4th Edition, PHI)
- M. Morris Mano & Michael D. Ciletti • *Digital Design* (6th Global Ed., Pearson)
- Thomas L. Floyd • *Digital Fundamentals: A Systems Approach* (1st Ed., Pearson)
- R. P. Jain • *Modern Digital Electronics* (4th Edition, McGraw-Hill)

40,900+

Exhaustive Words

80 Figures

Authentic 300 DPI Crops

10 Labs

Standard Worksheets

# Digital Electronics and Circuits (DEC)

---

## Complete First-Principles Comprehensive Lecture Notes

---

### Course & Curriculum Reference

- **Course Title:** Digital Electronics Circuits (DEC)
  - **Course Code:** BEL5T16C
  - **Target Audience:** Engineering Students (Sem V, B.Tech Electrical/Electronics Engineering) & Beginners with **Zero Prior Background** in Electronics.
  - **Syllabus Baseline:** Rashtrasant Tukadoji Maharaj Nagpur University (RTMNU) NEP Curriculum, aligned with national model curricula (AICTE/GATE).
  - **Core Authorized Source Books:**
  - **A. Anand Kumar**, *Fundamentals of Digital Circuits*, 4th Edition, PHI Learning Private Limited, 2016. [Cited throughout as *Kumar*].
  - **M. Morris Mano & Michael D. Ciletti**, *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*, 6th Global Edition, Pearson Education, 2017. [Cited throughout as *Mano*].
  - **Thomas L. Floyd**, *Digital Fundamentals: A Systems Approach*, 1st Edition, Pearson, 2013. [Cited throughout as *Floyd*].
  - **R. P. Jain**, *Modern Digital Electronics*, McGraw-Hill Education, 2009. [Cited throughout as *Jain*].
- 

### Pedagogical Architecture & Reader Contract

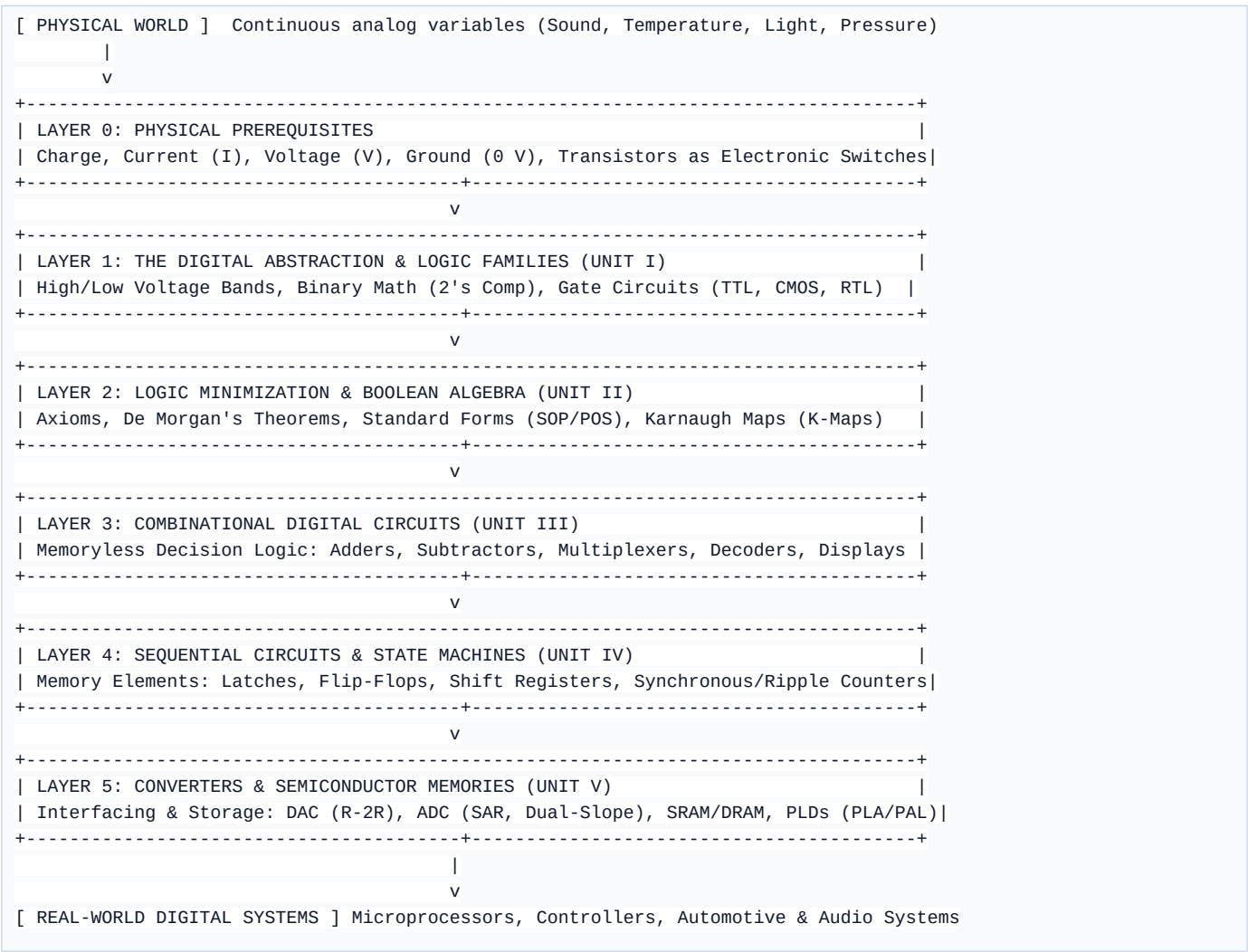
This document is engineered under a strict pedagogical mandate: **assume the reader has never seen an electric circuit, never written a binary number, and never encountered Boolean logic in their life**. It is neither an exam cram-sheet nor a compressed summary. Every concept is constructed systematically from physical reality, following thirteen core structural rules:

1. **Entry Point Assuming Nothing:** Every topic begins with everyday physical intuition and plain words before any formal mathematical notation appears. Every variable and symbol (**V**, **I**, **A**, **B**, **Q**,  $\oplus$ , etc.) is defined inline at its immediate point of use, even if previously defined.
2. **One New Idea Per Step:** Concepts are unpacked sequentially. Derivations explain what each step is *doing conceptually*, not just the algebraic transformation.
3. **Concrete Before Abstract:** Simple, round numerical examples and tangible physical scenarios precede generalized formulas.
4. **Three Parallel Representations:** Every core concept is presented in three synchronized formats: **plain words**, an **authentic textbook diagram** sourced directly from the reference literature, and a **governing equation or truth table**.
5. **The "Why" Behind the "How":** Engineering tricks, circuit simplifications, and mathematical conventions are accompanied by the reason they were invented and what disaster happens without them.
6. **Understanding Checkpoints & Dual Misconception Audits:** Each major topic concludes with an immediate self-check question (with answer supplied immediately below) along with explicit corrections of both **conceptual misconceptions** (fundamental mental errors) and **arithmetic/bookkeeping slip-ups**.
7. **Exhaustive Ground-Up Prerequisites:** A comprehensive foundational section precedes Unit I, developing voltage, current, ground, transistors as switches, and timing waveforms from first principles.
8. **Explicit Pacing Signals:** Every section explicitly alerts the reader whether to read slowly and deliberately for deep conceptual grounding, or to move briskly through routine algorithmic procedures.
9. **Universal Concept Spine (Need o Chain o How o Check):**
10. **Need (Problem-First):** What physical failure or real-world bottleneck forces this idea to exist?
11. **Chain (Cause and Effect):** How does one physical action trigger the next?
12. **How (Mechanism & Math):** The complete physical working, circuit operation, and mathematical derivation.
13. **Check:** An immediate understanding verification question.
14. **Deliberate Vocabulary Building:** Technical terms are defined upon first introduction and used consistently throughout.
15. **Continuous Subject Mapping:** The master architectural map below is echoed at the head of every individual unit.
16. **Style Anchoring:** Plainspoken engineering prose grounded in real physical devices.
17. **Explicit Added Characteristic — Physical Wire & Internal Node Signal Walkthrough (Hardware Reality Check):** *Added pedagogical rationale:* To prevent a zero-background reader from confusing digital electronics with purely abstract paper mathematics, every circuit topic includes a step-by-step physical walkthrough of moving charge carriers, node voltages (e.g.,

`0 ext{ V}` vs. `5 ext{ V}`}, and transistor conduction states across internal circuit wires.

## Master Subject Map: How Electricity Becomes a Digital Computer

To understand Digital Electronics, you must see the complete five-layer pyramid before exploring its individual stones:



- **Layer 0 (Prerequisites):** Establishes that electricity is moving charge, voltage is electrical push, and a transistor is a valve that can turn electricity ON or OFF without human fingers.
- **Layer 1 (Unit I):** Defines that `0 ext{ V}` represents the symbol 0 and `+5 ext{ V}` represents 1. We show how basic transistors combine into primitive decision blocks called **Logic Gates** (AND, OR, NOT, NAND, NOR, XOR), build binary number systems, and examine the silicon hardware families (**RTL**, **TTL**, **CMOS**) that realize them.
- **Layer 2 (Unit II):** Demonstrates how to take complex word problems or large truth tables and mathematically boil them down to the smallest possible number of gates using **Boolean Algebra** and visual **Karnaugh Maps (K-maps)**.
- **Layer 3 (Unit III):** Combines minimized gates into instantaneous calculating machines (**Combinational Circuits**) that have no memory—calculating sums (Adders), steering signals (Multiplexers), and decoding symbols for human eyes (Seven-Segment Displays).
- **Layer 4 (Unit IV):** Adds **feedback** to gates, creating circuits that remember their previous state (**Sequential Circuits**). This gives rise to 1-bit memory cells (Flip-Flops), serial data pipelines (Shift Registers), and rhythmic step-counters (Counters for Traffic Light Controllers).
- **Layer 5 (Unit V):** Completes the digital universe by bridging back to the physical world through **Digital-to-Analog Converters (DAC)** and **Analog-to-Digital Converters (ADC)**, and organizing massive arrays of bit storage (**Semiconductor Memories: SRAM, DRAM, ROM**) and field-customizable chips (**PLDs: PLA, PAL**).

## Comprehensive Ground-Up Prerequisites

# Building the Physical and Mathematical Foundations from Zero

## Reader Guidance & Scope Notice:

Before reading Unit I, you do not need to have taken courses in semiconductor physics, electromagnetic theory, or advanced calculus. This dedicated section builds every single required concept from scratch—beginning with the physical nature of an electron and ending with how an integrated circuit chip is numbered.

*Sourcing transparency:* Foundational electrical concepts not explicitly covered in the digital textbook chapters are clearly designated as **"Added background (not from the provided books)"**. All digital signal waveforms, pulse parameters, and integrated circuit package descriptions are sourced directly from *Floyd* (pp. 6–35) and *Kumar* (pp. 32–45).

## Prerequisites Inventory

Before proceeding into digital logic, ensure you understand this sequenced dependency chain: 1. **Physical Electricity:** Charge (**Q**), Electric Current (**I**), Voltage (**V**), Resistance (**R**), and Ohm's Law (**V = IR**). 2. **Circuit References:** Ground (**0 V**), Supply Rails (**V<sub>CC</sub>** / **V<sub>DD</sub>**), and the necessity of a closed conductive loop. 3. **The Electronic Switch:** How a 3-terminal semiconductor device (BJT or MOSFET) acts like a light switch flipped by a control voltage rather than a human finger. 4. **The Digital Abstraction:** Why real-world circuits convert continuous, messy analog voltages into two rigid digital states: 0 (LOW) and 1 (HIGH). 5. **Time and Waveforms:** Pulse definitions, rising edges, falling edges, rise time (**t<sub>r</sub>**), fall time (**t<sub>f</sub>**), pulse width (**t<sub>w</sub>**), frequency (**f**), and clock periods (**T**). 6. **Mathematical Notation:** Powers of 2, radix subscripts (e.g., **101<sub>2</sub>** vs. **101<sub>10</sub>**), Boolean variables, and truth table reading. 7. **Physical Hardware Reality:** Silicon dies, wire bonds, Dual In-line Packages (DIP), and pin numbering rules.

## Prerequisite 1: Electric Charge, Current, Voltage, and Resistance

*Added background (not from the provided books)*

**[Pacing: Slow & Intuitive — Read Every Word]**

### Need (Problem-First)

All computing hardware is made of physical matter. If you do not understand what electricity is physically doing inside a copper trace or a silicon channel, digital logic chips will seem like magic black boxes. Without understanding the relationship between voltage and current, you cannot understand why a gate draws power, why chips get hot, or why an output pin can only drive a limited number of other chips before failing.

### Chain of Cause and Effect

Subatomic particles have charge → mobile electrons move through a conductor under an electric field → this movement of charge constitutes an electric current → the force pushing these charges is electrical potential difference (voltage) → materials resist this motion (resistance) → energy is converted to heat.

### How It Works (The Physical Mechanism)

1. **Electric Charge (Q):** Matter is composed of atoms containing positively charged protons and negatively charged electrons. In conductive metals (like copper wires) and treated semiconductors (silicon), outer electrons can detach from their host atoms and drift freely. Charge is measured in **Coulombs (C)**. One electron carries an elementary negative charge of approximately  **$1.602 \times 10^{-19}$  C**.
2. **Electric Current (I):** Current is the rate at which electric charge flows past a specific cross-section of a wire:

$$I = (dQ / dt)$$

Current is measured in **Amperes (A)**, where **1 A = 1 Coulomb per second**. In digital circuits, currents are typically tiny, measured in **milliamperes (1 mA =  $10^{-3}$  A)** or **microamperes (1  $\mu$  A =  $10^{-6}$  A)**. 3. **Voltage (V or E):** Voltage is the electrical pressure or potential energy difference between two distinct points. It measures how much work is required to move a unit charge between those points:

$$V = (dW / dQ)$$

Voltage is measured in **Volts (V)**, where **1 V = 1 Joule per Coulomb**. A battery or power supply creates an excess of electrons at its negative terminal and a deficit at its positive terminal. This difference in potential creates an electric field that pushes electrons through an external path. 4. **Resistance (R):** As electrons travel through a conductor, they collide with lattice atoms, hindering their flow. This opposition is electrical resistance, measured in **Ohms ( $\Omega$ )**. 5. **Ohm's Law:** In an ideal resistive

conductor, the current flowing through is directly proportional to the applied voltage difference across its terminals:

$$V = I \cdot R \text{ iff } I = (V / R) \text{ iff } R = (V / I)$$

*Concrete Round-Number Example:* Suppose a **+5 V** voltage is applied across a resistor of **R = 1000 Ω (1 k Ω)**. The resulting current is:

$$I = \frac{5 \text{ V}}{1000 \text{ Ω}} = 0.005 \text{ A} = 5 \text{ mA}$$

### Understanding Checkpoint

- **Question:** If you have a **+5 V** power supply connected to a resistor and you double the resistance from **1 k Ω** to **2 k Ω**, what happens to the electric current flowing through it?
- **Answer:** The current is cut in half, dropping from **5 mA** to **2.5 mA**, because current is inversely proportional to resistance (**I = V/R**).

---

## Prerequisite 2: Ground Reference (0 ext{ V}) and Supply Rails (V\_{CC} / V\_{DD})

*Added background (not from the provided books)*

[Pacing: Fundamental Concept — Establish Firmly]

### Need (Problem-First)

Beginners often ask: "If a wire is labeled **+5 V**, where does the electricity go? Why doesn't a circuit work with just one wire?" In physics, an absolute voltage at a single isolated point does not exist. Voltage is strictly a *difference* between two points. If you do not establish a shared zero-volt baseline across all components on a circuit board, none of the chips can agree on what a 0 or 1 is.

### Chain of Cause and Effect

Electrons need a complete loop to circulate → circuits designate a shared common conductor called Ground (**0 V**) → a power source maintains a fixed positive potential difference above this ground (e.g., **+5 V**) → current leaves the positive supply, travels through the gates, and returns to ground.

### How It Works (The Physical Mechanism)

- **Ground (GND / 0 V):** In digital electronics, "Ground" does not necessarily mean driving a copper rod into the literal earth outside. It is simply the common return wire or circuit board copper plane that we arbitrarily define as the **0.00 V** baseline reference point. All other voltages in the circuit are measured *with respect to this ground node*.
- **Positive Supply Rail (V\_{CC} or V\_{DD}):** In bipolar transistor logic (TTL), the positive power supply pin is labeled **V\_{CC}** (standing for Collector Supply Voltage, historically **+5.0 V**). In MOS/CMOS transistor logic, the positive power supply pin is labeled **V\_{DD}** (standing for Drain Supply Voltage, typically **+5.0 V**, **+3.3 V**, or **+1.8 V** in modern silicon).
- **The Closed Loop Requirement:** Electric charge cannot accumulate indefinitely on an open wire. For steady current to flow, every milliampere that leaves the **+5 V** supply terminal must physically travel through components and enter the ground return terminal back to the power supply.

### Understanding Checkpoint

- **Question:** If a voltmeter lead touches a wire carrying **+5 V** while the meter's black reference lead is left hanging in empty air, what voltage does the meter read?
- **Answer:** It reads an unpredictable, floating noise value (essentially meaningless), because voltage can only be measured as the potential difference *between two connected points*.

---

## Prerequisite 3: The Electronic Switch — How Transistors Turn Signals ON and OFF

*Added background (not from the provided books), supported by Kumar (Ch. 16, pp. 892, 917)*

[Pacing: Conceptually Critical — Read Slowly]

### Need (Problem-First)

To build a machine that calculates, we cannot have human hands mechanically toggling wall switches millions of times per second. We need an electrical switch that can be toggled by *another electrical voltage*. That device is the semiconductor transistor.

## Chain of Cause and Effect

A physical switch connects or disconnects two terminals → an electronic transistor has a conduction channel whose resistance can be made near zero or near infinite → applying a control voltage to the gate/base turns this channel ON or OFF → this enables one circuit to control another without moving mechanical parts.

## How It Works (The Physical Mechanism)

In digital circuits, transistors are **never used as linear amplifiers** (as they are in radio transmitters or audio equipment). They are operated exclusively at their extreme outer limits: 1. **Cutoff Region (The OPEN Switch)**: When the control voltage is removed (or set to **0 V**), the internal conduction path between the output terminals has massive resistance (hundreds of megohms). No current flows. The switch is **OPEN** (OFF). 2. **Saturation Region (The CLOSED Switch)**: When a sufficient control voltage is applied, mobile charge carriers flood the internal channel. The electrical resistance between the output terminals collapses to near zero ohms (a fraction of an ohm to a few ohms). Current flows freely. The switch is **CLOSED** (ON).

Two main families of transistors perform this electronic switching in digital history: - **Bipolar Junction Transistor (BJT)**: Used in Transistor-Transistor Logic (**TTL**). A small input current injected into the middle terminal (Base, **B**) controls a large current between the Collector (**C**) and Emitter (**E**). - **Metal-Oxide-Semiconductor Field-Effect Transistor (MOSFET)**: Used in Complementary MOS (**CMOS**). A voltage applied to the insulated control terminal (Gate, **G**) sets up an electric field across an oxide insulator, opening or closing a conductive path between the Drain (**D**) and Source (**S**). Because the Gate is insulated by silicon dioxide, the steady-state control current is essentially **zero** ( $I_G \approx 0$ ), making CMOS consume vastly less electrical power than BJT logic.

*Transistor Inverter Action (The Pull-Down Mechanism)*: Consider a simple switch circuit where a resistor connects an output wire to **+5 V**, and an electronic transistor switch connects that same output wire to Ground (**0 V**): - When the transistor switch is **OPEN (control input = 0 V)**: No current can flow through the transistor to ground. The output wire is pulled up to **+5 V** through the resistor. Output = **+5 V** (HIGH). - When the transistor switch is **CLOSED (control input = +5 V)**: The transistor creates a direct short-circuit path from the output wire to Ground (**0 V**). Current rushes through the resistor and drains straight into ground. The output wire is clamped to **0 V**. Output = **0 V** (LOW). Notice what just happened: an input of **0 V** produced an output of **+5 V**, and an input of **+5 V** produced an output of **0 V**. **This is the fundamental electronic INVERTER (NOT gate).**

## Understanding Checkpoint

- **Question:** When an electronic transistor switch is fully ON (saturated), what is the voltage drop across its main switching terminals?
- **Answer:** Near zero volts (ideally **0 V**, practically **0.1 V** to **0.2 V** in silicon BJTs).

---

## Prerequisite 4: The Digital Abstraction — Logic Levels, Voltage Bands, and Noise Margins

Source: Floyd (Ch. 1, pp. 6–15); Kumar (Ch. 1, pp. 32–36)

[Pacing: Core Foundation — Essential Reading]

### Need (Problem-First)

Real electronic components are imperfect. Power supplies ripple, radio waves induce stray voltages into wires, and temperature fluctuations shift component values. If a circuit relied on precise analog voltages—such that **2.500 V** meant "number 5" and **2.505 V** meant "number 6"—stray electrical noise would immediately corrupt every calculation. Digital systems achieve complete immunity to this noise through the **Digital Abstraction**.

## Chain of Cause and Effect

Continuous analog voltages are inherently vulnerable to physical noise → engineers define two broad, non-overlapping voltage ranges separated by an illegal buffer zone → any voltage falling in the upper band is treated identically as logic 1 → any voltage in the lower band is treated identically as logic 0 → noise that stays within the allowed bands is completely rejected.

## How It Works (The Physical Mechanism)

In binary digital electronics, circuits recognize only two operational states: - **Logic 1 (HIGH)**: Represents the assertion of a condition, a binary TRUE, or a binary digit 1. - **Logic 0 (LOW)**: Represents the non-assertion of a condition, a binary FALSE, or a binary digit 0.

Instead of requiring an exact voltage like **+5.000 V**, practical IC logic families define **voltage bands** with four critical thresholds (Kumar, p. 896): 1.  **$V_{OH(min)}$**  (**Minimum Output High Voltage**): The lowest voltage that a transmitting logic gate will output when it asserts a HIGH state (e.g., **+2.7 V** in standard TTL, or **+4.9 V** in CMOS). 2.  **$V_{IH(min)}$**  (**Minimum Input High**

**Voltage):** The lowest voltage that a receiving logic gate will reliably accept as a legitimate HIGH input (e.g., **+2.0 V** in standard TTL, or **+3.5 V** in CMOS). 3.  **$V_{IL(max)}$**  (**Maximum Input Low Voltage**): The highest voltage that a receiving logic gate will reliably accept as a legitimate LOW input (e.g., **+0.8 V** in standard TTL, or **+1.5 V** in CMOS). 4.  **$V_{OL(max)}$**  (**Maximum Output Low Voltage**): The highest voltage that a transmitting logic gate will output when it asserts a LOW state (e.g., **+0.4 V** in standard TTL, or **+0.1 V** in CMOS).

Between  **$V_{IL(max)}$**  and  **$V_{IH(min)}$**  lies the **Forbidden / Undefined Region** (e.g., between **0.8 V** and **2.0 V** in standard TTL). A gate's input must never be allowed to float or dwell in this middle zone during steady-state operation; otherwise, internal transistors can enter unpredictable conduction states, causing false outputs or burning excessive power.

#### Noise Margin (NM):

The noise margin is the maximum amplitude of unwanted noise voltage that can be superimposed on a digital signal without causing the receiving gate to misinterpret the logic level (Kumar, p. 896): - **High-state Noise Margin ( $NM_H$ )**:

$$NM_H = V_{OH(min)} - V_{IH(min)}$$

*Example (Standard TTL):*  **$NM_H = 2.7\text{ V} - 2.0\text{ V} = 0.7\text{ V}$** . A noise spike would have to pull the output down by more than **0.7 V** before the receiver misreads it. - **Low-state Noise Margin ( $NM_L$ )**:

$$NM_L = V_{IL(max)} - V_{OL(max)}$$

*Example (Standard TTL):*  **$NM_L = 0.8\text{ V} - 0.4\text{ V} = 0.4\text{ V}$** . A positive noise spike would have to raise the ground potential by more than **0.4 V** before the receiver misreads a LOW as a HIGH.

#### Understanding Checkpoint

- **Question:** A digital sensor produces an output of **+1.2 V** connected to a standard TTL logic gate whose input thresholds are  **$V_{IL(max)} = 0.8\text{ V}$**  and  **$V_{IH(min)} = 2.0\text{ V}$** . Does the gate read this as a 0 or a 1?
- **Answer:** Neither reliably. **+1.2 V** falls directly in the forbidden/indeterminate region between **0.8 V** and **2.0 V**. The circuit behavior is unpredictable and represents a design fault.

---

## Prerequisite 5: Pulse Waveforms, Clock Signals, and Timing Parameters

Source: Kumar (Ch. 1, pp. 34–36, Figures 1.1 & 1.2); Floyd (Ch. 1, pp. 11–14)

[Pacing: Conceptually Important — Master the Waveform Vocabulary]

### Need (Problem-First)

Digital circuits do not live in frozen time. Voltages must switch back and forth to carry messages, count events, and clock calculations. In textbook theory, people draw square waves that change instantly. In real silicon, electrons take time to travel and parasitic capacitors take time to charge. If an engineer assumes pulses switch instantaneously, high-speed sequential circuits will glitch and fail.

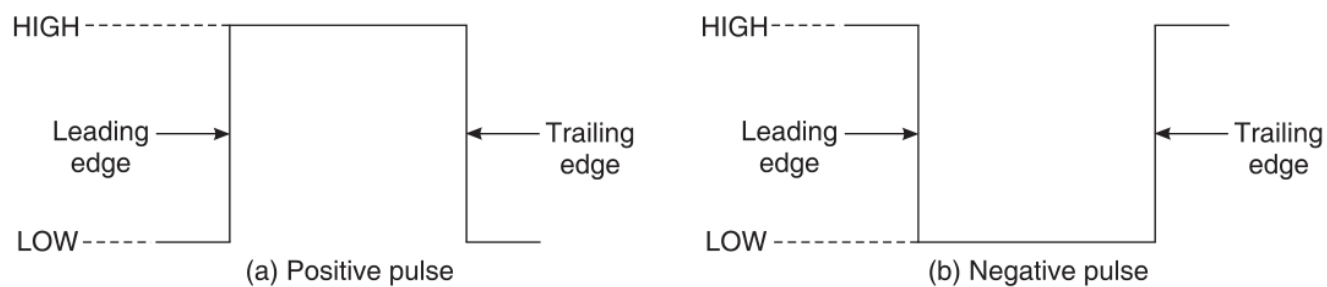
### Chain of Cause and Effect

Transistors turn ON and OFF → voltages rise from LOW to HIGH and fall from HIGH to LOW → capacitive loads in physical wires prevent instant voltage changes → pulses have finite rise times and fall times → engineers define exact 10% and 90% measurement points to characterize signal speed.

### How It Works (The Physical Mechanism)

#### 1. Ideal Pulses vs. Real-World Pulses

An **ideal pulse** changes between the LOW and HIGH level in zero time, forming perfectly vertical edges: - **Positive Pulse:** Starts at the baseline LOW level, transitions to the HIGH level at its **leading edge** (or rising edge), remains HIGH for duration  **$t_w$** , and returns to the LOW level at its **trailing edge** (or falling edge) (Kumar, p. 35, Figure 1.1a). - **Negative Pulse:** Starts at the baseline HIGH level, transitions to the LOW level at its leading edge, and returns to HIGH at its trailing edge (Kumar, p. 35, Figure 1.1b).

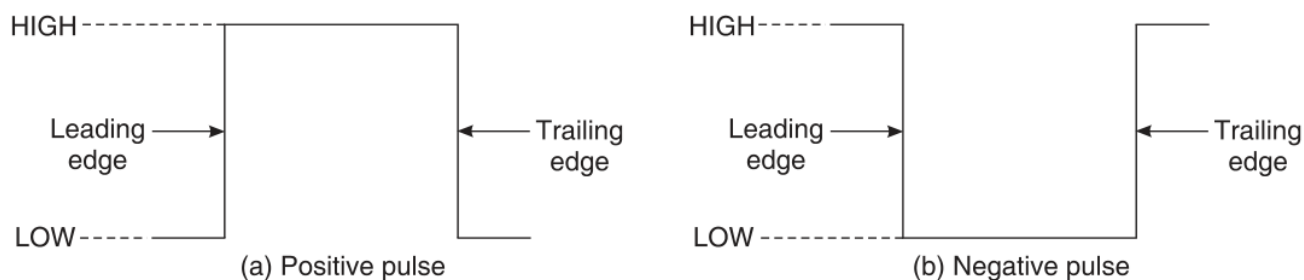


**Figure 1.1** Ideal positive and negative pulses.

Figure

**1.1:** Ideal positive and negative pulses showing leading and trailing edges. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 35  
(printed p. 3).

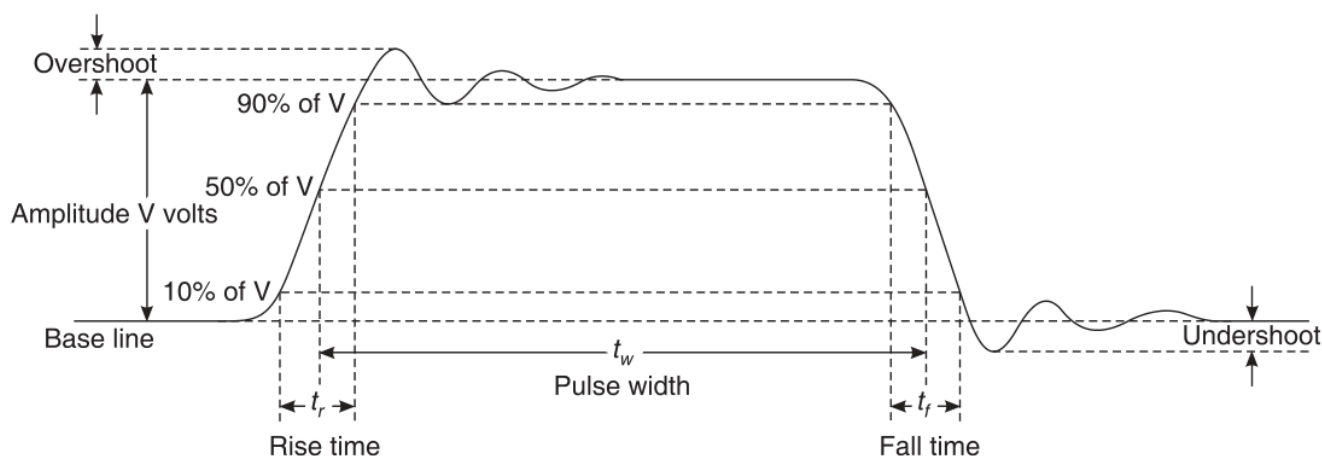
In physical circuits, no voltage can change in zero picoseconds. A real-world non-ideal pulse is shaped by finite transition intervals (Kumar, p. 35, Figure 1.2):



**Figure 1.1** Ideal positive and negative pulses.

As indicated in Figure 1.1, a pulse has two edges: a leading edge and a trailing edge. For a positive pulse, the leading edge is a positive going transition (PGT or rising edge) and the trailing edge is a negative going transition (NGT or falling edge), whereas for a negative pulse, the leading edge is a negative going transition (NGT) and the trailing edge is a positive going transition (PGT). The pulses shown in Figure 1.1 are ideal, because the rising and falling edges change instantaneously, i.e. in zero time. Practical pulses do not change instantaneously from LOW to HIGH or from HIGH to LOW.

A non-ideal pulse is shown in Figure 1.2. It has finite rise and fall times. The time taken by the pulse to rise from LOW to HIGH is called the *rise time* and the time taken by the pulse to go from HIGH to LOW is called the *fall time*. Because of the nonlinearities that commonly occur at the bottom and top of the pulse, the rise time is defined as the time taken by the pulse to rise from 10% to 90% of the pulse amplitude, and the fall time is defined as the time taken by the pulse to fall from 90% to 10% of the pulse amplitude. The duration of the pulse is usually indicated by pulse width  $t_w$  which is defined as the time between the 50% points on the rising and falling edges.



**Figure 1.2** Non-ideal pulse characteristics.

Figure

1.2: Non-ideal pulse characteristics defining rise time ( $t_r$ ), fall time ( $t_f$ ), and pulse width ( $t_w$ ) measured between 10%, 50%, and 90% amplitude thresholds.

Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 35 (printed p. 3).

## 2. Key Non-Ideal Pulse Parameters Defined Inline:

1. **Rise Time ( $t_r$ ):** The time required for the pulse voltage to transition from **10%** of its final amplitude up to **90%** of its final amplitude (Kumar, p. 35). The lower **10%** and upper **10%** regions are excluded from the measurement because non-linear rounding ("knees") occurs near the supply rails.
2. **Fall Time ( $t_f$ ):** The time required for the pulse voltage to transition downward from **90%** of its peak amplitude down to **10%** of its amplitude (Kumar, p. 35).
3. **Pulse Width ( $t_w$ ):** The operational duration of the active pulse, standardized by international convention as the elapsed time between the **50% amplitude point on the rising edge** and the **50% amplitude point on the falling edge** (Kumar, p. 35).

4. **Period (T):** In a repeating periodic waveform (such as a system clock), the period **T** is the total elapsed time for one complete cycle to occur, measured from a point on one pulse to the identical corresponding point on the next pulse.
5. **Frequency (f):** The rate at which pulses repeat per unit time, measured in **Hertz (Hz)**, where **1 Hz = 1 cycle per second**:

$$f = (1 / T) \text{ iff } T = (1 / f)$$

*Concrete Round-Number Example:* If a microprocessor clock has a period of **T = 10 nanoseconds** ( $10 \times 10^{-9}$  s), its operating frequency is:

$$f = \frac{1}{10 \times 10^{-9} \text{ s}} = 100,000,000 \text{ Hz} = 100 \text{ MHz}$$

1. **Duty Cycle:** The ratio of the active pulse width (**t<sub>w</sub>**) to the total repeating period (**T**), typically expressed as a percentage:

$$\text{Duty Cycle} = \left( t_w / T \right) \times 100\%$$

A symmetrical square wave has a duty cycle of exactly **50%** (**t<sub>w</sub> = T/2**).

### Understanding Checkpoint

- **Question:** A periodic clock signal has a frequency of **2 MHz** ( $2 \times 10^6$  Hz). What is its time period **T**, and if its pulse width is **0.1 μs**, what is its duty cycle?
- **Answer:**

$$T = \frac{1}{2 \times 10^6 \text{ s}^{-1}} = 0.5 \times 10^{-6} \text{ s} = 0.5 \text{ μs} = 500 \text{ ns}$$

$$\text{Duty Cycle} = \frac{0.1 \text{ μs}}{0.5 \text{ μs}} \times 100\% = 20\%$$

## Prerequisite 6: Mathematical Foundations — Positional Notation, Powers of Two, and Truth Tables

Source: *Floyd (Ch. 2, pp. 52–58); Kumar (Ch. 2, pp. 59–63)*

[Pacing: Systematic Bookkeeping — Follow the Method]

### Need (Problem-First)

Human beings count in decimal (base 10) because we have ten fingers. But electronic switches can only exist stably in two physical states: fully OPEN or fully CLOSED. To map numbers and decisions onto two-state hardware, we must understand positional numbering in arbitrary bases—especially base 2.

### Chain of Cause and Effect

Positional notation weights each column by powers of the base → decimal weights are powers of **10** (**10<sup>0</sup>, 10<sup>1</sup>, 10<sup>2</sup> dots**) → binary weights are powers of **2** (**2<sup>0</sup>, 2<sup>1</sup>, 2<sup>2</sup> dots**) → any positive integer can be uniquely expressed as a sum of active powers of two.

### How It Works (The Mechanism & Math)

#### 1. Positional Number Systems

In any positional number system with base (or radix) **r**, a number sequence **d<sub>n</sub> d<sub>{n-1}</sub> dots d<sub>1</sub> d<sub>0</sub>** represents the polynomial value:

$$\text{Value} = \sum_{i=0}^n d_i \cdot r^i = d_n r^n + d_{n-1} r^{n-1} + \text{dots} + d_1 r^1 + d_0 r^0$$

- In base 10 (**r=10**), the symbols are **{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}**.  
Example: **742<sub>{10}</sub> = 7 × 10<sup>2</sup> + 4 × 10<sup>1</sup> + 2 × 10<sup>0</sup> = 700 + 40 + 2 = 742.**
- In base 2 (**r=2**), the only allowed symbols are **{0, 1}**.  
Each binary digit is called a **bit** (portmanteau of **binary digit**).

#### 2. The Powers of Two (Memorize These Columns):

$$2^0 = 1, 2^1 = 2, 2^2 = 4, 2^3 = 8, 2^4 = 16, 2^5 = 32, 2^6 = 64, 2^7 = 128, 2^8 = 256, 2^9 = 512, 2^{10} = 1024$$

Concrete Round-Number Example: Evaluate the 4-bit binary number **1101<sub>2</sub>**:

$$1101_2 = (1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) = 8 + 4 + 0 + 1 = 13_{10}$$

- The rightmost bit ( $2^0 = 1$ ) is called the **Least Significant Bit (LSB)** because it has the smallest numerical weight.
- The leftmost bit ( $2^3 = 8$  in a 4-bit number) is called the **Most Significant Bit (MSB)** because it carries the largest numerical weight.

### 3. Truth Table Reading

A **Truth Table** is an exhaustive tabular ledger listing every possible combination of input logic states alongside the resulting output logic state for a digital circuit. - If a circuit has **n** independent binary inputs, there are exactly  $2^n$  unique input combinations. - 1 input (**n=1**):  $2^1 = 2$  rows (**0, 1**). - 2 inputs (**n=2**):  $2^2 = 4$  rows (**00, 01, 10, 11**). - 3 inputs (**n=3**):  $2^3 = 8$  rows (**000 to 111**). - 4 inputs (**n=4**):  $2^4 = 16$  rows (**0000 to 1111**). The standard convention is to write the input rows in ascending binary order (**0, 1, 2, 3 dots**) so no combination is accidentally omitted or duplicated.

#### Understanding Checkpoint

- **Question:** How many rows must a truth table contain for a logic circuit with 5 digital inputs?
- **Answer:** Exactly  $2^5 = 32$  rows.

## Prerequisite 7: Physical Integrated Circuit Hardware — Dies, Wire Bonds, and Package Pinouts

Source: *Floyd (Ch. 1, pp. 30–32, Figures 41 & 44); Mano (Ch. 2, pp. 90–93)*

[Pacing: Physical Reality — High Practical Value]

### Need (Problem-First)

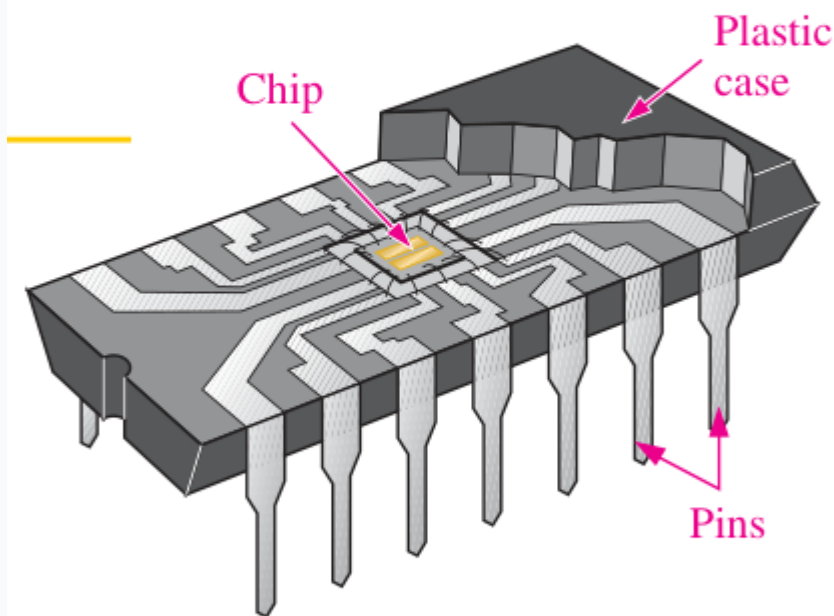
Students who only study logic diagrams on paper often fail completely when given physical components in an engineering laboratory. A logic gate drawn on paper as a triangular symbol is actually an microscopic silicon die sealed inside a black plastic package with metal legs called pins. If you wire power to the wrong pin or insert the package upside down, you will destroy the chip within milliseconds.

### Chain of Cause and Effect

Silicon wafers are fabricated with microscopic transistors → individual dies are sliced and mounted into durable protective packages → tiny microscopic bond wires connect the silicon pads to sturdy exterior metal pins → an international indexing convention (the notch/dot) allows engineers to identify Pin 1 without ambiguity.

### How It Works (The Physical Mechanism)

Figure 41 from *Floyd (p. 30)* illustrates the internal cutaway view of a fixed-function IC package:



**FIGURE 41** Cutaway view of one type of fixed-function surface-mount IC package, showing the chip mounted inside and connections to input and output pins.

Figure 41: Cutaway view of a fixed-function IC

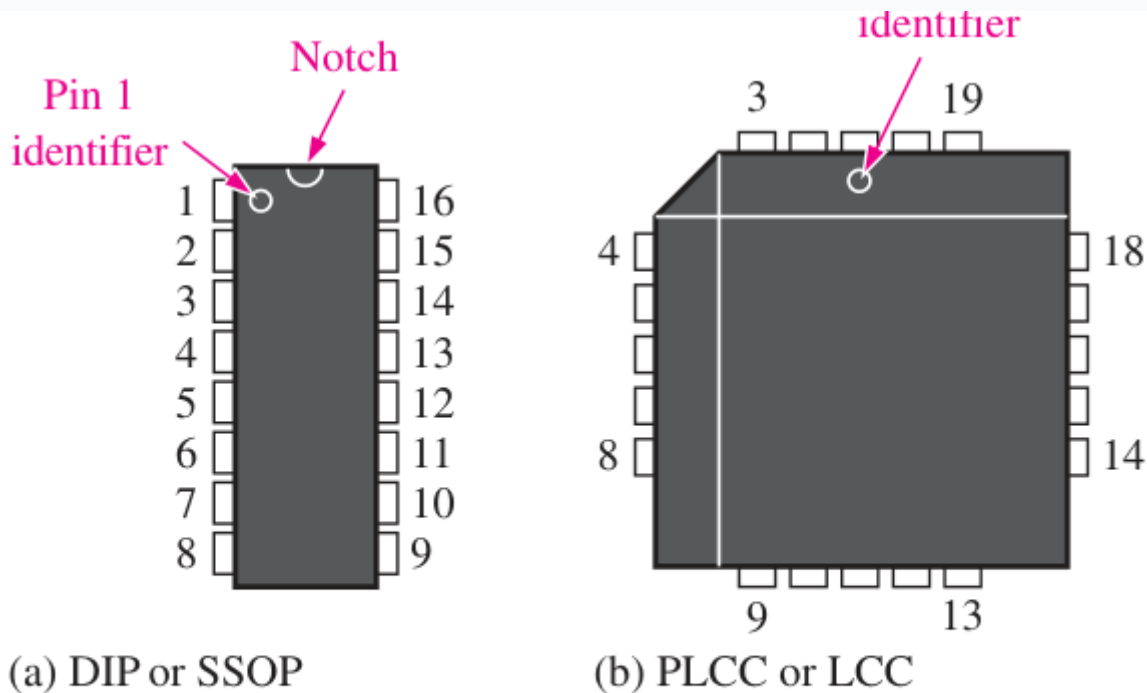
package showing the internal silicon chip die, lead frame, and microscopic wire bonds connecting the silicon pads to external metal pins. Sourced from

Thomas L. Floyd, *Digital Fundamentals: A Systems Approach* (1st Ed.), p. 30 (printed p. 24).

- **The Silicon Die:** A microscopic sliver of high-purity silicon (typically a few millimeters square) containing dozens to billions of interconnected transistors, diodes, and resistors.
- **Wire Bonds:** Microscopic gold or aluminum wires (thinner than a human hair) that bridge the pads on the silicon die to the exterior metal lead frame.
- **Packaging Types:**
- **DIP (Dual In-line Package):** Standard through-hole package with two parallel rows of sturdy pins spaced **0.1 inch (2.54 mm)** apart, ideal for laboratory breadboarding and educational circuit assembly.
- **SMT / SOIC (Surface Mount Technology / Small Outline IC):** Modern compact packages whose pins sit flat against copper surface pads on a printed circuit board (PCB), saving space and reducing lead inductance.

### The Universal Pin Numbering Rule

How do you know which pin is Pin 1? Figure 44 from *Floyd* (p. 32) demonstrates the universal standard:



**FIGURE 44** Pin numbering for standard types of IC packages. Top views are shown.

Figure 44: Pin

numbering conventions for standard IC packages (top view). Pin 1 is always located to the immediate left of the identification notch or marked by an indented dot, with pin numbers proceeding counterclockwise around the package perimeter. Sourced from Thomas L. Floyd, *Digital Fundamentals: A Systems Approach* (1st Ed.), p. 32 (printed p. 26).

**The Counterclockwise Rule:** 1. Place the IC flat on the table, viewing it from the **top**. 2. Locate the **polarizing marker**: either a small semi-circular notch on one end of the package, or a small indented circular dot near one corner. 3. Orient the chip so the notch faces **upward** (or the dot is at the top left). 4. **Pin 1** is always the top-left pin. 5. Count **downward** along the left edge (**1, 2, 3 dots 7** on a 14-pin IC). 6. Jump across to the bottom right and continue counting **upward** along the right edge (**8, 9, 10 dots 14**). 7. In standard 14-pin 7400-series TTL logic ICs: - **Pin 7 is always connected to Ground (0 V)**. - **Pin 14 is always connected to Supply Voltage ( $V_{CC} = +5.0$  V)**.

## Prerequisites Audit: Misconceptions and Pitfalls to Avoid

### 1. Conceptual Misconceptions

- Misconception:** "A digital 0 means zero volts, and a digital 1 means exactly five volts."
 

**Correction:** Digital electronics relies on *bands* of voltages, not exact values. In TTL logic, any input voltage between **0.0 V** and **0.8 V** is accepted as a valid 0, and any input between **2.0 V** and **5.0 V** is accepted as a valid 1. Real circuits routinely output **0.2 V** for LOW and **3.4 V** for HIGH.
- Misconception:** "Transistors inside digital chips store electricity like tiny batteries."
 

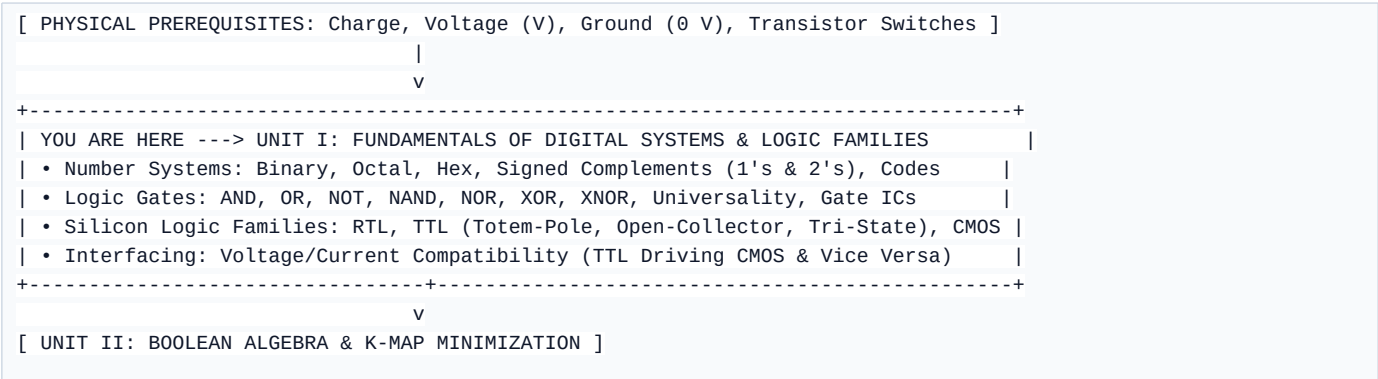
**Correction:** Transistors do not store power; they act as *valves* directing electric current from an external power supply to ground or output pins. Memory in digital circuits is created by active feedback loops (as taught in Unit IV), not by electrostatic charge storage in gate switches.

### 2. Practical Slip-Ups

- Pin Numbering Reversal:** Beginners often count down the left side (1 to 7) and then continue counting down the right side (8 to 14). **This is backwards!** The numbering always wraps around **counterclockwise**, meaning Pin 8 is directly opposite Pin 7 at the bottom!
- Unconnected Power Pins:** Beginners frequently wire logic gates on a breadboard but forget to connect Pin 14 to **+5 V** and Pin 7 to GND. Without power connections to the silicon substrate, the internal transistors cannot operate.

# Unit I: Fundamentals of Digital Systems and Logic Families

## Master Subject Map: Unit I Position



*In 3 lines:* Unit I establishes the language of digital electronics. We translate human numbers into patterns of binary voltages, use semiconductor transistors to construct primitive decision-making blocks called logic gates, and analyze the physical silicon families (RTL, TTL, CMOS) that power real integrated circuits.

## Prerequisites for this Unit

Before studying Unit I, you need the foundational concepts established in the Prerequisites section: 1. **Voltage and Ground (Prereq 1 & 2):** Current flows from a positive potential ( $V_{CC}$  /  $V_{DD}$ ) to Ground ( $0\text{ V}$ ). 2. **Transistors as Electronic Switches (Prereq 3):** An active control input creates a conducting short to ground (pull-down), while absence of control leaves the path open (pull-up). 3. **Logic Levels & Voltage Bands (Prereq 4):** A HIGH signal is a voltage in the upper band (near  $+5\text{ V}$ ), while a LOW signal is in the lower band (near  $0\text{ V}$ ). 4. **Positional Numbering & Powers of Two (Prereq 6):** Powers of two (**1, 2, 4, 8, 16 dots**) form the basis of binary representations.

## PART 1: Number Systems, Signed Binary Arithmetic, and Codes

### 1.1 Number Base Systems: Binary, Octal, and Hexadecimal

Source: Kumar (Ch. 2, pp. 59–68); Floyd (Ch. 2, pp. 52–65); Mano (Ch. 1, pp. 18–28)

[Pacing: Systematic Bookkeeping — Follow the Step-by-Step Algorithm]

#### Need (Problem-First)

Human beings communicate in decimal (base 10), but computer processors physically store and manipulate bits in base 2. However, long strings of binary digits (such as **1101111010101101\_2**) are notoriously difficult for human engineers to read, write, and debug without making transcription errors. We need compact, human-friendly representations that map directly into binary bits without requiring painful mathematical long-division. This creates the need for **Octal (base 8)** and **Hexadecimal (base 16)**.

#### Chain of Cause and Effect

Binary is machine-native but unwieldy for humans → bases that are exact powers of two (**8 =  $2^3$**  and **16 =  $2^4$** ) allow direct bit-grouping → each octal digit corresponds to exactly 3 binary bits, and each hexadecimal digit corresponds to exactly 4 binary bits → complex memory addresses and machine instructions can be inspected with ease.

#### How It Works (Mechanism & Algorithms)

##### 1. Positional Bases Defined

- Decimal (Base 10):** Radix  $r=10$ . Digits in **{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}**.
- Binary (Base 2):** Radix  $r=2$ . Digits in **{0, 1}**.
- Octal (Base 8):** Radix  $r=8$ . Digits in **{0, 1, 2, 3, 4, 5, 6, 7}**.
- Hexadecimal (Base 16):** Radix  $r=16$ . Digits in **{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F}**.

Where letters represent numbers 10 through 15 inline:

$$A_{16} = 10_{10}, B_{16} = 11_{10}, C_{16} = 12_{10}, D_{16} = 13_{10}, E_{16} = 14_{10}, F_{16} = 15_{10}$$

## 2. Conversions to Decimal (Sum-of-Weights Method)

Multiply each digit by its positional weight  $r^i$  and compute the arithmetic sum (Kumar, p. 63). - **Binary to Decimal:**

Example: Convert  $11010.11_2$  to decimal:

$$11010.11_2 = (1 \times 2^4) + (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (0 \times 2^0) + (1 \times 2^{-1}) + (1 \times 2^{-2})$$

$$= 16 + 8 + 0 + 2 + 0 + 0.5 + 0.25 = 26.75_{10}$$

- **Hexadecimal to Decimal:**

Example: Convert  $2A6_{16}$  to decimal:

$$2A6_{16} = (2 \times 16^2) + (10 \times 16^1) + (6 \times 16^0) = (2 \times 256) + (10 \times 16) + (6 \times 1) = 512 + 160 + 6 = 678_{10}$$

## 3. Decimal to Binary Conversion (Double-Dabble Method)

- **Integer Part (Repeated Division by 2):** Divide the decimal integer repeatedly by 2, recording the remainder (0 or 1) at each step, until the quotient reaches 0. The **first remainder is the LSB**, and the **last remainder is the MSB** (Kumar, p. 64).

Example: Convert  $25_{10}$  to binary:

- $25 \div 2 = 12$ , remainder 1 (LSB)
- $12 \div 2 = 6$ , remainder 0
- $6 \div 2 = 3$ , remainder 0
- $3 \div 2 = 1$ , remainder 1
- $1 \div 2 = 0$ , remainder 1 (MSB)

Reading remainders from bottom to top:  $25_{10} = 11001_2$ .

- **Fractional Part (Repeated Multiplication by 2):** Multiply the fraction repeatedly by 2, recording the integer carry (0 or 1) generated at each step, until the fraction becomes zero or the desired precision is reached. The **first integer carry is the first fractional digit** (Kumar, p. 66). Example: Convert  $0.625_{10}$  to binary:

- $0.625 \times 2 = 1.250$  (carry 1)
- $0.250 \times 2 = 0.500$  (carry 0)
- $0.500 \times 2 = 1.000$  (carry 1, fraction zeroed out)

Reading carries top to bottom:  $0.625_{10} = 0.101_2$ .

## 4. Octal and Hexadecimal Bit-Grouping (The Fast Direct Shortcut)

Because  $8 = 2^3$ , every octal digit converts into exactly 3 binary bits.

Because  $16 = 2^4$ , every hexadecimal digit converts into exactly 4 binary bits. - **Binary to Hexadecimal:** Group bits into sets of 4 starting from the binary point moving left for integers and right for fractions (padding with leading/trailing zeros if needed), then substitute the hex digit: Example: Convert  $101110010_2$  to hex: - Group:  $\underbrace{0010}_2 \underbrace{1111}_F \underbrace{0010}_2$  (padded with two leading zeros on MSB group) - Result:  $101110010_2 = 2F2_{16}$ .

- **Hexadecimal to Binary:** Directly expand each hex digit into its 4-bit binary equivalent: Example: Convert  $3B_{16}$  to binary: -  $3_{16} = 0011_2$  -  $B_{16} = 11_{10} = 1011_2$  - Result:  $3B_{16} = 00111011_2$ .

### Understanding Checkpoint

- **Question:** Convert  $11110110_2$  directly to hexadecimal.
- **Answer:** Split into two 4-bit groups:  $1111_2$  and  $0110_2$ .  $1111_2 = 15_{10} = F_{16}$ , and  $0110_2 = 6_{16}$ . Therefore,  $11110110_2 = F6_{16}$ .

## 1.2 Binary Arithmetic: Addition, Subtraction, Multiplication, and Division

Source: Kumar (Ch. 2, pp. 68–71); Floyd (Ch. 2, pp. 58–63)

[Pacing: Routine Bookkeeping — Master the Elementary Rules]

## Need (Problem-First)

To make hardware compute, we must translate mathematical operations into primitive bit manipulations that can be executed by electronic logic circuits.

### The Four Elementary Rules of Binary Arithmetic

#### 1. Binary Addition (Kumar, p. 68):

2.  $0 + 0 = 0$  (Sum 0, Carry 0)
3.  $0 + 1 = 1$  (Sum 1, Carry 0)
4.  $1 + 0 = 1$  (Sum 1, Carry 0)
5.  $1 + 1 = 10_2$  (Sum 0, Carry 1 to the next higher positional column)
6.  $1 + 1 + 1 = 11_2$  (Sum 1, Carry 1) *Worked Example: Add  $1101_2$  ( $13_{10}$ ) and  $0111_2$  ( $7_{10}$ ):* `` 1 1 1 <-- Carry  
row 1 1 0 1 (13)
  - 0 1 1 1 (7)

---

1 0 1 0 0 (20 in decimal:  $16 + 4 = 20$ ) ``

#### 7. Binary Subtraction (Kumar, p. 68):

8.  $0 - 0 = 0$
9.  $1 - 0 = 1$
10.  $1 - 1 = 0$
11.  $0 - 1 = 1$  with a **Borrow of 1** from the next higher column (which has a weight of 2).
12. **Binary Multiplication (Kumar, p. 69):** Identical to decimal long multiplication, but vastly simpler because multiplier bits are only 0 or 1. A 1 copies the multiplicand; a 0 yields all zeros.
13. **Binary Division (Kumar, p. 70):** Performed by successive shifting and subtraction, matching long division in decimal.

### Understanding Checkpoint

- **Question:** Compute  $101_2 + 011_2$ .
- **Answer:**  $101_2$  (5) +  $011_2$  (3) =  $1000_2$  (8).

---

## 1.3 Signed Binary Numbers: Sign-Magnitude, 1's Complement, and 2's Complement

Source: Kumar (Ch. 2, pp. 71–85); Mano (Ch. 1, pp. 34–39); Floyd (Ch. 2, pp. 65–75)

[Pacing: Conceptually Crucial — Read Slowly and Methodically]

### Need (Problem-First)

In everyday math, we indicate negative numbers by simply writing a minus sign "-" in front (e.g., -15). But an electronic circuit wire can only carry two physical voltage states: HIGH or LOW. It cannot carry a "-" symbol. How can hardware represent negative numbers, and more importantly, how can an Arithmetic Logic Unit (ALU) perform subtraction *without having to build completely separate, expensive subtraction circuitry*?

### Chain of Cause and Effect

Circuits allocate the leftmost bit (MSB) as a sign indicator ( $0 = +$ ,  $1 = -$ ) → direct sign-magnitude creates two zeros (+0 and -0) and requires complex decision logic → complement systems convert subtraction into simple addition ( $A - B = A + (-B)$ ) → 2's complement eliminates duplicate zeros and discards the final carry → the exact same hardware adder circuit performs both addition and subtraction.

### How It Works (The Three Signed Formats Defined)

In an  $n$ -bit binary word, the **Most Significant Bit (MSB)** is strictly reserved as the **Sign Bit**: - **MSB = 0** ■ **Positive number (+)** - **MSB = 1** ■ **Negative number (-)**

#### 1. Sign-Magnitude Representation (Kumar, p. 71)

The MSB indicates the sign, and the remaining ( $n-1$ ) bits represent the absolute magnitude of the number: - For an 8-bit byte: -  $+25_{10} = \mathbf{00011001_2}$  -  $-25_{10} = \mathbf{10011001_2}$  *Fatal Engineering Flaw:* Sign-magnitude has **two distinct representations for zero**:  $+0 = 00000000_2$  and  $-0 = 10000000_2$ . This wastes bit combinations and forces processors to execute extra comparison cycles.

## 2. 1's Complement Representation (Kumar, p. 72)

- Positive numbers are written in standard binary with **MSB = 0**.
- A negative number is formed by taking the positive binary number and **inverting every single bit** (**0 → 1** and **1 → 0**).
- *Example (8 bits): +25\_{10} = 00011001\_2.*  
To form **-25\_{10}**, invert all bits: **-25\_{10} = 11100110\_2**. *Limitation:* Like sign-magnitude, 1's complement still suffers from two zeros (**+0 = 00000000\_2** and **-0 = 11111111\_2**). Furthermore, subtraction produces an "end-around carry" that must be re-added to the LSB, slowing down the processor.

## 3. 2's Complement Representation (Kumar, p. 74)

The universal standard for all modern computing architectures. - Positive numbers are written in standard binary with **MSB = 0**. - A negative number is formed by taking its 1's complement and **adding 1 to the LSB**:

$$\text{2's Complement} = (\text{1's Complement}) + 1$$

- *Worked Example (8-bit representation of -25\_{10}):*
- Start with positive **+25\_{10}**: 00011001
- Invert all bits (1's complement): 11100110
- Add 1:  $\begin{array}{r} 11100110 \\ + 1 \\ \hline \end{array}$

11100111 <-- -25 in 2's complement

- *The Direct Shortcut for 2's Complement:* Working from right to left (LSB to MSB), copy all bits unchanged up to and including the first 1, then invert every bit remaining to the left!  
For 00011001: first bit is 1 (copy it), invert rest → 11100111. Exactly identical result in one second!

## 4. Subtraction Using 2's Complement Arithmetic (Kumar, p. 81)

To compute **A - B**, the ALU converts the problem into:

$$A - B = A + (\text{2's complement of } B)$$

**The Two Golden Rules of 2's Complement Subtraction:** 1. If a final carry of 1 is generated past the MSB, DISCARD IT. The remaining bits are the correct positive answer in standard binary (Kumar, p. 81). 2. If NO final carry is generated, the result is **NEGATIVE** and is in **2's complement form**. To read its human-readable decimal magnitude, take the 2's complement of the result and attach a minus sign (Kumar, p. 81).

*Concrete Worked Scenario 1 (Larger minus Smaller: 28 - 15 in 8 bits):* - **+28\_{10} = 00011100\_2** - **+15\_{10} = 00001111\_2** ■ **-15\_{10}** in 2's complement = **11110000 + 1 = 11110001\_2**. - Add them:  $\begin{array}{r} 00011100 \text{ (+28)} \\ + 11110001 \text{ (-15)} \\ \hline \end{array}$

1 00001101 ^ Discard this end carry! Result = **00001101\_2 = 8 + 4 + 1 = +13\_{10}**. Perfect!

*Concrete Worked Scenario 2 (Smaller minus Larger: 15 - 28 in 8 bits):* - **+15\_{10} = 00001111\_2** - **-28\_{10} = 11100100\_2** (2's complement of 28) - Add them:  $\begin{array}{r} 00001111 \text{ (+15)} \\ + 11100100 \text{ (-28)} \\ \hline \end{array}$

11110011 (No end carry generated!)

Since no carry appeared, the result is negative and stored in 2's complement. To find its magnitude: take 2's complement of 11110011 invert bits (00001100) + 1 = 00001101 (**13\_{10}**).

Therefore, the answer is **-13\_{10}**. Perfect!

## 5. Arithmetic Overflow

Source: Kumar (p. 84); Mano (p. 37)

In an **n-bit** signed system, the range of representable numbers is:

$$-[2^{n-1}] \text{ to } +[2^{n-1} - 1]$$

For an 8-bit system: **-128 to +127**.

**Overflow occurs if the sum of two numbers with the same sign exceeds this range.**

- If you add two positive numbers and the MSB becomes 1 (negative), **overflow has occurred**. - If you add two negative numbers and the MSB becomes 0 (positive), **overflow has occurred**. - Hardware detection rule: Overflow occurs if and only if the **carry into the sign bit (C<sub>in</sub>)** is different from the **carry out of the sign bit (C<sub>out</sub>)**:

$$\text{Overflow} = C_{in} \oplus C_{out}$$

### Understanding Checkpoint

- **Question:** What is the 8-bit 2's complement representation of **-1**?
- **Answer:** **+1 = 00000001<sub>2</sub>**. Invert all bits → **11111110<sub>2</sub>**. Add **1** → **11111111<sub>2</sub>**. Thus, **-1** is represented as all ones (11111111).

## 1.4 Binary Codes: BCD, Excess-3, Gray Code, and ASCII

Source: Kumar (Ch. 3, pp. 117–134); Floyd (Ch. 2, pp. 75–85); Mano (Ch. 1, pp. 39–48)

[Pacing: Systematic Vocabulary — High Relevance to Practical Displays]

### Need (Problem-First)

Raw binary arithmetic is great for calculations, but digital systems must interface with decimal keypads, seven-segment displays, mechanical rotary encoders, and human text. If you directly send pure binary to a 7-segment display driver, the circuitry required to decode 16-bit numbers into decimal digits becomes massive. Specialized **codes** bridge these real-world interfaces.

### The Four Core Codes Defined

#### 1. Binary Coded Decimal (BCD / 8421 Code)

Source: Kumar (Ch. 3, pp. 117–120)

- **Mechanism:** Each decimal digit (**0** through **9**) is independently encoded using its 4-bit binary equivalent with weights **8, 4, 2, 1**. - Because 4 bits can represent 16 states (**0** to **15**), the combinations from **10<sub>10</sub>** to **15<sub>10</sub>** (**1010** to **1111**) are **strictly invalid in BCD**. - *Example:* Convert **952<sub>10</sub>** to BCD:

$$9_{10} = 1001_2, 5_{10} = 0101_2, 2_{10} = 0010_2 \blacksquare 952_{\text{BCD}} = 1001 \ 0101 \ 0010$$

Notice that **952<sub>BCD</sub>** requires 12 bits, whereas pure binary **952<sub>10</sub>** requires only 10 bits (**1110111000<sub>2</sub>**). BCD trades storage density for effortless decimal interfacing.

#### 2. Excess-3 Code (XS-3)

Source: Kumar (Ch. 3, pp. 120–122)

- **Mechanism:** An unweighted code derived by adding decimal **3 (0011<sub>2</sub>)** to each BCD code group:

$$\text{XS-3} = \text{BCD} + 0011_2$$

*Example:* Decimal **4** → BCD **0100** → XS-3 = **0100 + 0011 = 0111**. - **Why It Matters (Self-Complementing Property):** The 1's complement of an Excess-3 number produces the Excess-3 code of its 9's complement! This made XS-3 immensely valuable in early computing hardware for automating decimal subtraction.

#### 3. Gray Code (Reflected Binary / Unit Distance Code)

Source: Kumar (Ch. 3, pp. 122–126); Floyd (Ch. 2, pp. 78–81)

- **Need:** When an optical rotary shaft encoder measures the position of a rotating motor, standard binary transitions can be catastrophic. When transitioning from state **3 (011)** to state **4 (100)**, **all three bits must change simultaneously**. Because physical sensor contacts never switch in perfect mathematical synchrony, the encoder may momentarily register spurious intermediate states like **000** or **111**, causing false position errors. - **The Gray Code Solution:** In Gray code, **only one bit changes state at a time** between any two consecutive numbers (Unit Distance property). - **Binary to Gray Code Conversion (Kumar, p. 123):** 1. The MSB of the Gray code is identical to the MSB of the binary number: **G<sub>n</sub> = B<sub>n</sub>**. 2. Each subsequent

Gray bit  $G_i$  is formed by XORing the current binary bit  $B_{i+1}$  with the next lower binary bit  $B_i$ :

$$G_i = B_{i+1} \oplus B_i$$

*Worked Example:* Convert binary  $1101_2$  to Gray code: -  $G_3 = B_3 = 1$  -  $G_2 = B_3 \oplus B_2 = 1 \oplus 1 = 0$  -  $G_1 = B_2 \oplus B_1 = 1 \oplus 0 = 1$  -  $G_0 = B_1 \oplus B_0 = 0 \oplus 1 = 1$

Result:  $1101_2 = 1011_{\text{Gray}}$ . - **Gray Code to Binary Conversion (Kumar, p. 124):** 1. The MSB of the binary number is identical to the Gray MSB:  $B_n = G_n$ . 2. Each subsequent binary bit is obtained by XORing the previously calculated binary bit with the current Gray bit:

$$B_i = B_{i+1} \oplus G_i$$

#### 4. Alphanumeric Code: ASCII

*Source: Floyd (Ch. 2, pp. 83–85); Mano (Ch. 1, pp. 44–46)*

The **American Standard Code for Information Interchange (ASCII)** uses 7 bits to encode 128 characters, including upper and lowercase letters, numbers, punctuation, and control characters (e.g., 'A' =  $1000001_2 = 41_{16} = 65_{10}$ ; '0' =  $0110000_2 = 30_{16} = 48_{10}$ ).

#### Understanding Checkpoint

- **Question:** What is the Gray code for binary  $1000_2$ ?
- **Answer:**  $G_3 = 1$   
 $G_2 = 1 \oplus 0 = 1$   
 $G_1 = 0 \oplus 0 = 0$   
 $G_0 = 0 \oplus 0 = 0$   
Gray code =  $1100$ .

---

## PART 2: Basic Digital Circuits, Universal Gates, and Gate ICs

### 1.5 The Fundamental Logic Gates: AND, OR, and NOT

*Source: Kumar (Ch. 4, pp. 163–168); Mano (Ch. 2, pp. 84–90); Floyd (Ch. 3, pp. 118–138)*

[Pacing: Core Visual Foundation — Study the Truth Tables & Symbols]

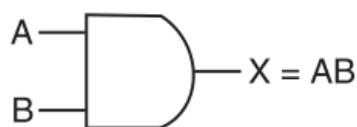
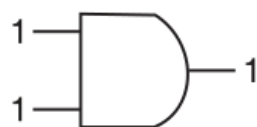
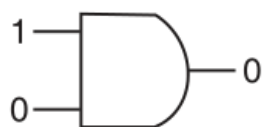
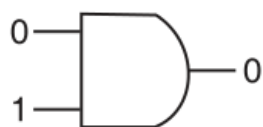
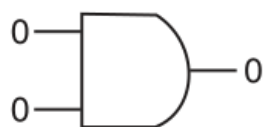
#### Need (Problem-First)

Mathematical statements require logical operators: "Activate the emergency cooling pump if the temperature is HIGH **AND** pressure is HIGH." "Sound the alarm if door 1 is open **OR** door 2 is open." We need hardware circuits that execute these logical decisions.

#### The Three Fundamental Gates

##### 1. The AND Gate (Kumar, p. 164)

- **Plain Words:** The output is HIGH (1) if and only if all inputs are HIGH (1). If any input is LOW (0), the output is clamped to LOW (0).
- **Boolean Equation:**  $X = A \cdot B$  (or simply  $X = AB$ ).
- **Authentic Diagram:**



(a) Logic symbol

| Inputs |   | Output |
|--------|---|--------|
| A      | B | X      |
| 0      | 0 | 0      |
| 0      | 1 | 0      |
| 1      | 0 | 0      |
| 1      | 1 | 1      |

(b) Truth table

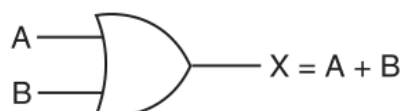
**Figure 4.1** A two-input AND gate.

Figure

4.1: A two-input AND gate showing operational logic states, circuit symbol, and complete truth table. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 164 (printed p. 132).

## 2. The OR Gate (Kumar, p. 166)

- **Plain Words:** The output is HIGH (1) if any input is HIGH (1). The output is LOW (0) only when all inputs are simultaneously LOW (0).
- **Boolean Equation:**  $X = A + B$ .
- **Authentic Diagram:**



(a) Logic symbol

| Inputs |   | Output |
|--------|---|--------|
| A      | B | X      |
| 0      | 0 | 0      |
| 0      | 1 | 1      |
| 1      | 0 | 1      |
| 1      | 1 | 1      |

(b) Truth table

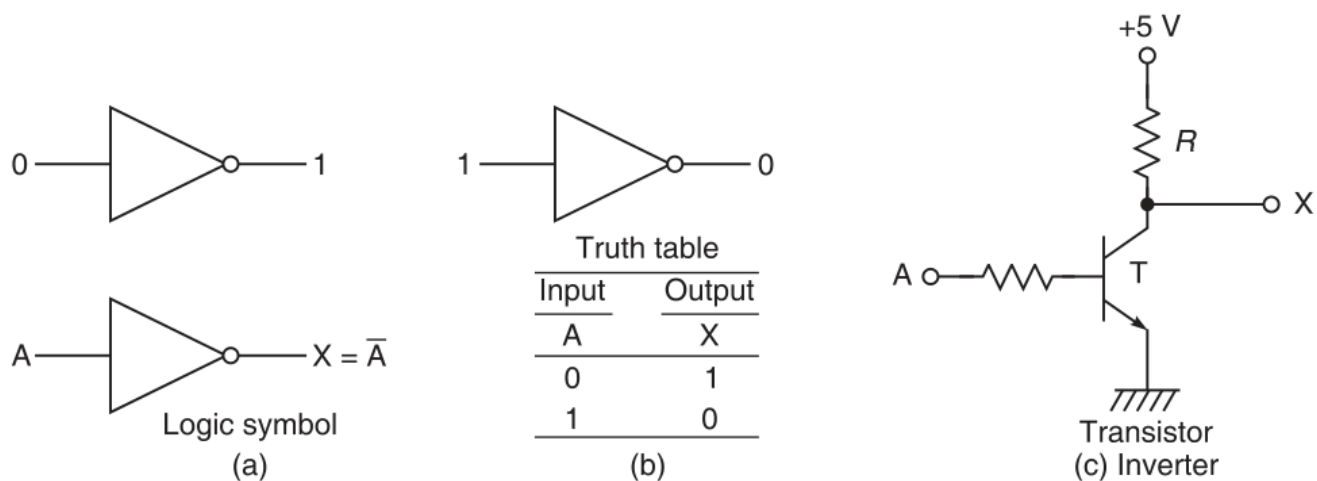
**Figure 4.4** A two-input OR gate.

Figure

4.4: A two-input OR gate showing operational logic states, circuit symbol, and complete truth table. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 166 (printed p. 134).

## 3. The NOT Gate (Inverter) (Kumar, p. 168)

- **Plain Words:** The output is the exact logical complement (inverse) of the input. If the input is 0, the output is 1; if the input is 1, the output is 0.
- **Boolean Equation:**  $X = A$  (or  $X = A'$ ).
- **The Inversion Bubble:** In schematic convention, the triangle represents a buffer (current amplifier), and the small circular bubble represents logical inversion.
- **Authentic Diagram:**



**Figure 4.7** The inverter.

Figure

4.7: The logic symbol and truth table of an inverter (NOT gate). Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 168 (printed p. 136).

## 1.6 Universal Gates: NAND and NOR Operations & Proofs of Universality

Source: Kumar (Ch. 4, pp. 169–173); Mano (Ch. 3, pp. 119–126)

[Pacing: Conceptually Crucial — Read Carefully]

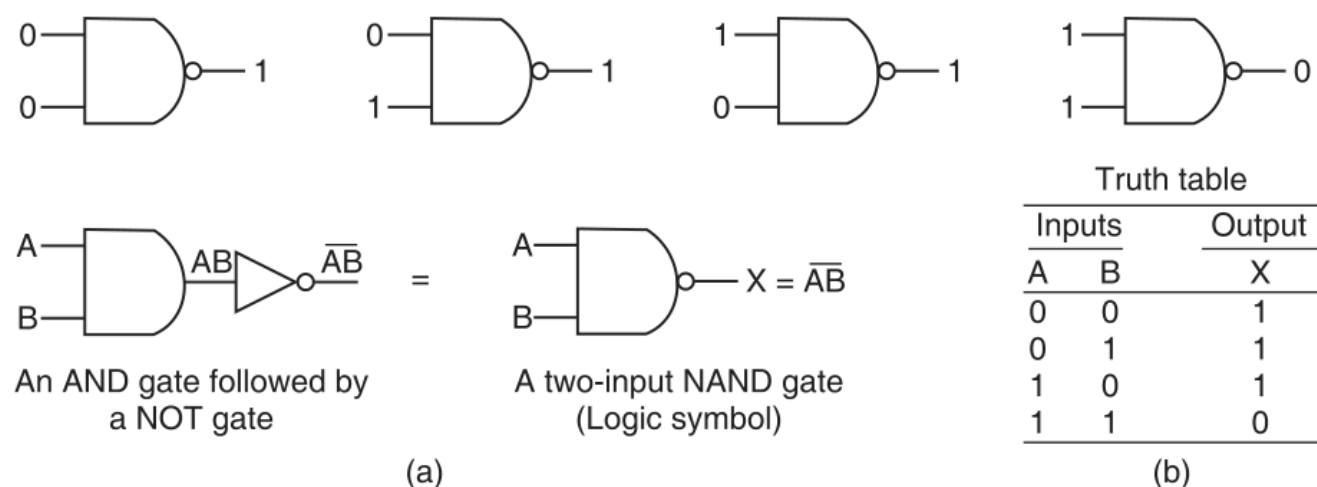
### Need (Problem-First)

In manufacturing semiconductor integrated circuits, fabricating multiple distinct types of gates (some AND, some OR, some NOT) requires complex, multi-step silicon doping masks, driving up production costs and inventory complexity. Semiconductor manufacturers sought a **single universal building block** that could be mass-produced identically and wired to perform any Boolean function whatsoever. Those blocks are **NAND** and **NOR**.

### The Universal Gates Defined

#### 1. The NAND Gate (Kumar, p. 169)

- **Plain Words:** A combination of an AND gate followed immediately by an inverter. The output is LOW (0) **only when all inputs are HIGH (1)**. For all other combinations, the output is HIGH (1).
- **Boolean Equation:**  $X = A \cdot B$ .
- **Authentic Diagram:**



**Figure 4.8** A two-input NAND gate.

Figure

4.8: A two-input NAND gate showing operational states, logic symbol with inversion bubble, and truth table. Sourced from A. Anand Kumar, *Fundamentals*

2. The NOR Gate (Kumar, p. 171)

- **Plain Words:** A combination of an OR gate followed by an inverter. The output is HIGH (1) only when all inputs are LOW (0). If any input is HIGH (1), the output is LOW (0).
- **Boolean Equation:**  $X = A + B$ .
- **Authentic Diagram:**

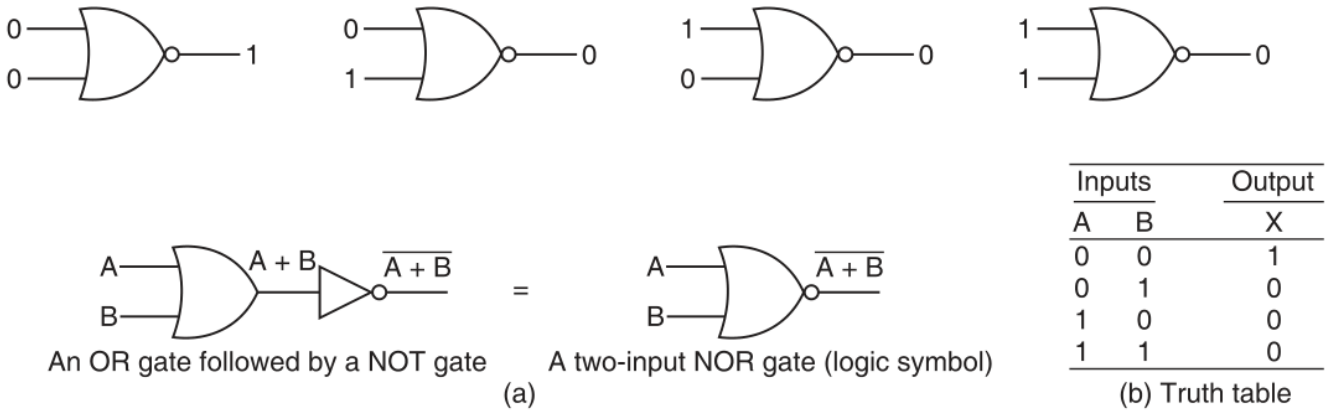


Figure 4.14 A two-input NOR gate.

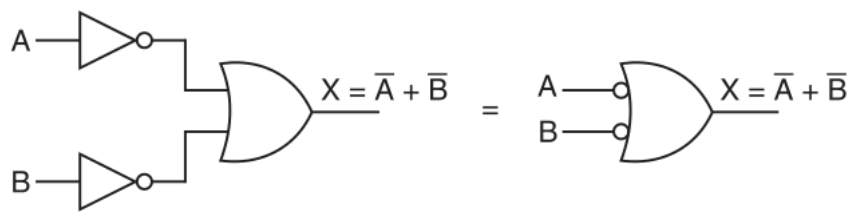
Figure

4.14: A two-input NOR gate showing operational states, logic symbol, and truth table. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 171 (printed p. 139).

Proofs of Universality: Realizing NOT, AND, and OR

1. Universality of the NAND Gate (Kumar, pp. 170–171)

- **Realization of NOT using NAND (Kumar, Figure 4.11):**  
Tie both inputs of a 2-input NAND gate together:  $X = A \cdot A = A$ .



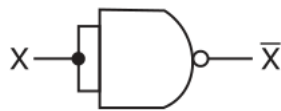
(a)

| Inputs |   | Inverted inputs |           | Output |
|--------|---|-----------------|-----------|--------|
| A      | B | $\bar{A}$       | $\bar{B}$ | X      |
| 0      | 0 | 1               | 1         | 1      |
| 0      | 1 | 1               | 0         | 1      |
| 1      | 0 | 0               | 1         | 1      |
| 1      | 1 | 0               | 0         | 0      |

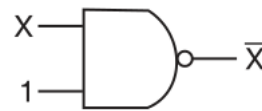
(b) Truth table

**Figure 4.10** Bubbled OR gate.

**NAND gate as an inverter:** A NAND gate can also be used as an inverter by tying all its input terminals together and applying the signal to be inverted to the common terminal (Figure 4.11a). Alternatively, by connecting all its input terminals except one, to logic 1 and applying the signal to be inverted to the remaining terminal as shown in Figure 4.11b. In the latter form, it is said to act as a controlled inverter.



(a) NAND gate as an inverter



(b) NAND gate as a controlled inverter

**Figure 4.11** NAND gate as an inverter.

Figure

4.11: Realizing a NOT gate (inverter) using a NAND gate by tying inputs together or using a controlled HIGH input. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 170 (printed p. 138).

- Realization of AND using NAND:**

Pass inputs through a NAND gate, then invert the result with a second NAND-inverter:

$$X = A \cdot B = A \cdot B$$

- Realization of OR using NAND (De Morgan's Equivalence):**

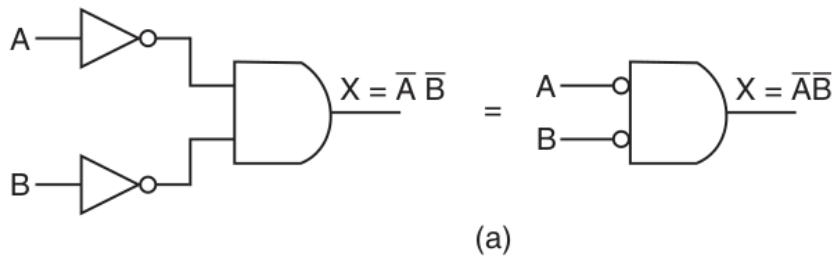
Invert **A** with one NAND to get **A**; invert **B** with another NAND to get **B**; feed both into a third NAND gate:

$$X = A \cdot B = A + B = A + B$$

## 2. Universality of the NOR Gate (Kumar, pp. 171–172)

- Realization of NOT using NOR (Kumar, Figure 4.17):**

Tie both inputs together:  $X = A + A = A$ .



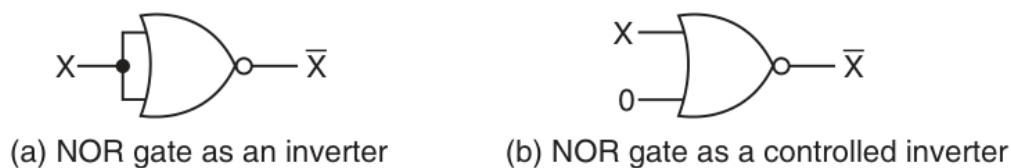
| Inputs |   | Inverted inputs |           | Output |
|--------|---|-----------------|-----------|--------|
| A      | B | $\bar{A}$       | $\bar{B}$ | X      |
| 0      | 0 | 1               | 1         | 1      |
| 0      | 1 | 1               | 0         | 0      |
| 1      | 0 | 0               | 1         | 0      |
| 1      | 1 | 0               | 0         | 0      |

(b) Truth table

**Figure 4.16** Bubbled AND gate.

The AND gate with inverted inputs (Figure 4.16a) is called a *bubbled* AND gate. So, it is equivalent to a bubbled AND gate whose truth table is shown in Figure 4.16b. A bubbled AND gate is also called a *negative* AND gate. Since its output assumes the HIGH state only when all inputs are in LOW state, a NOR gate is also called an active-LOW AND gate.

**NOR gate as an inverter:** A NOR gate can also be used as an inverter, by tying all its input terminals together and applying the signal to be inverted to the common terminal (Figure 4.17a). Alternatively, connecting all its input terminals except one to logic 0, and applying the signal to be inverted to the remaining terminal as shown in Figure 4.17b. In the latter form it is said to act as a controlled inverter.



**Figure 4.17** NOR gate as an inverter.

Figure

4.17: Realizing a NOT gate using a NOR gate. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 172 (printed p. 140).

- Realization of OR using NOR:**

Feed inputs to a NOR gate, then invert with a second NOR-inverter:

$$X = A + B = \overline{\overline{A + B}} = \overline{A \cdot B}$$

- Realization of AND using NOR (De Morgan's Equivalence):**

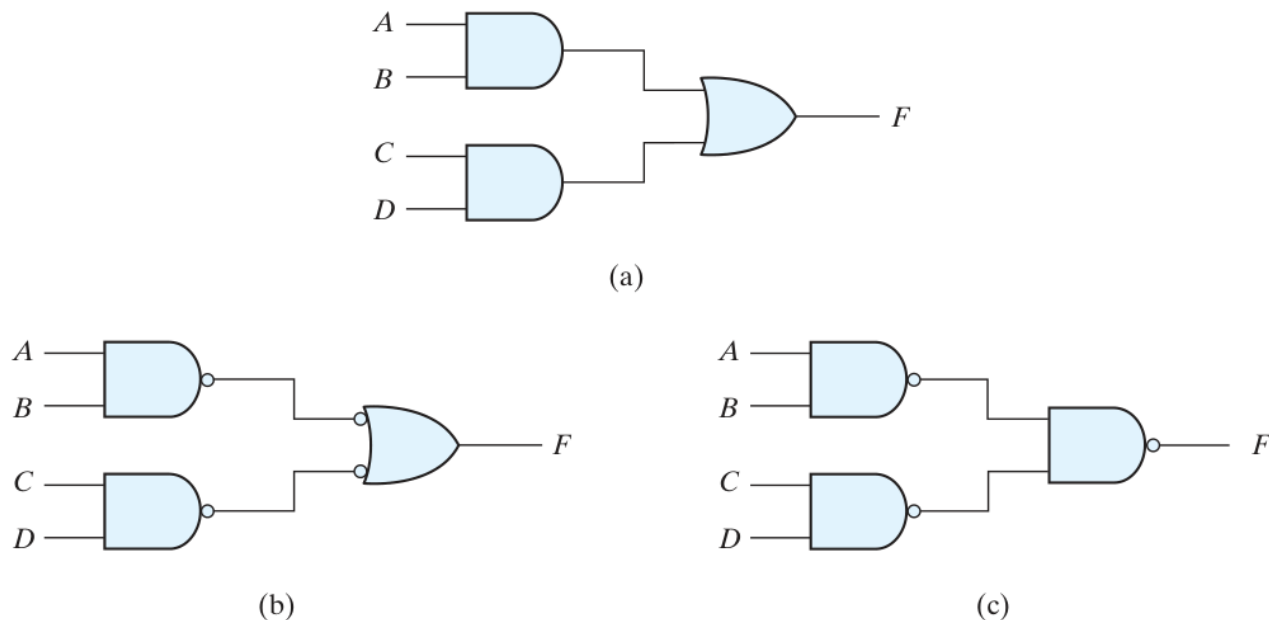
Invert inputs to get  $\bar{A}$  and  $\bar{B}$ , then feed into a third NOR gate:

$$X = \overline{\overline{A + B}} = \overline{\overline{A} \cdot \overline{B}} = A \cdot B$$

### 3. Two-Level NAND-NAND Implementation (*Mano*, p. 120)

Because an AND-OR network directly converts into a two-level NAND-NAND network under De Morgan's theorem, any Sum-of-Products (SOP) expression can be wired using **only NAND gates**:

AND gates are replaced by NAND gates and the OR gate is replaced by a NAND gate with an invert-OR graphic symbol. Remember that a bubble denotes complementation and two bubbles along the same line represent double complementation, so both can be



**FIGURE 3.18**  
Three ways to implement  $F = AB + CD$

Figure

3.18: Demonstrating equivalence of AND-OR logic and two-level NAND-NAND logic for the function  $F = AB + CD$ . Sourced from M. Morris Mano & Michael D. Ciletti, *Digital Design* (6th Global Ed.), p. 120 (printed p. 119).

## 1.7 Exclusive-OR (XOR) and Exclusive-NOR (XNOR) Gates

Source: Kumar (Ch. 4, pp. 173–176); Mano (Ch. 3, pp. 132–137)

[Pacing: Conceptually Important — The Core of Arithmetic & Parity]

### Need (Problem-First)

Standard OR includes the case where both inputs are **1** ("inclusive OR"). But in basic arithmetic addition (**1 + 1 = 0 with carry 1**), the sum bit must be **0** when both inputs are **1**. Furthermore, in error-detection (parity checking), we need a circuit that detects whether an odd or even number of bits are asserted.

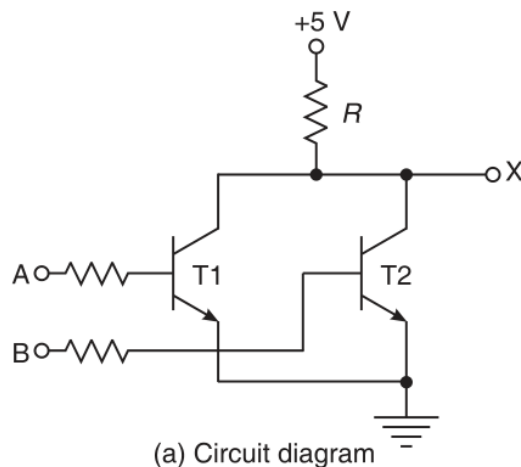
### How It Works (The Mechanisms)

#### 1. The XOR Gate (Kumar, p. 173)

- **Plain Words:** The output is HIGH (**1**) if **either input is HIGH, but not both**. The output is **1** if the inputs are different (**01** or **10**), and **0** if they are identical (**00** or **11**).
- **Boolean Equation:**

$$X = A \oplus B = AB + \overline{A}\overline{B}$$

- **Authentic Diagram:**



| Input |     | Output |
|-------|-----|--------|
| A     | B   | X      |
| 0 V   | 0 V | 5 V    |
| 0 V   | 5 V | 0 V    |
| 5 V   | 0 V | 0 V    |
| 5 V   | 5 V | 0 V    |

(a) Circuit diagram

(b) Truth table

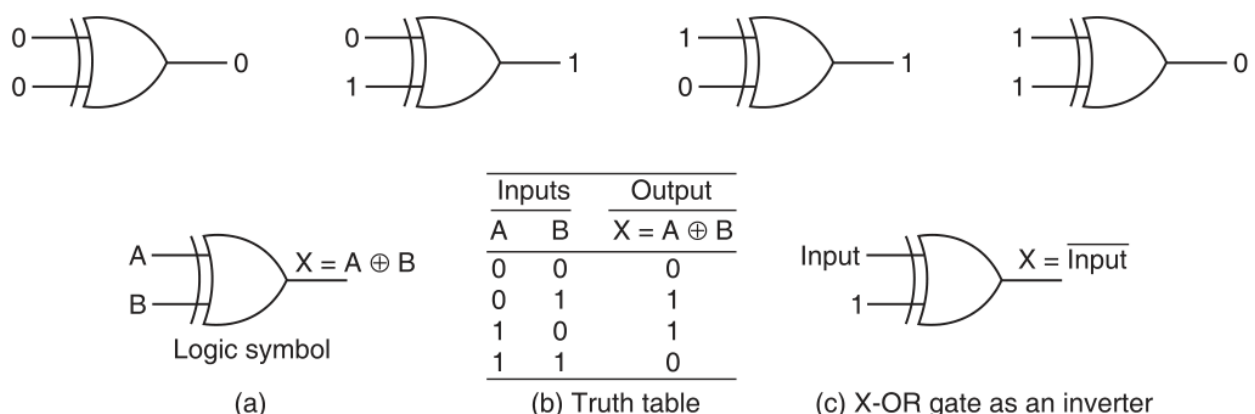
**Figure 4.19** Discrete two-input NOR gate.

## 4.6 THE EXCLUSIVE-OR (X-OR) GATE

An X-OR gate is a two input, one output logic circuit, whose output assumes a logic 1 state when one and only one of its two inputs assumes a logic 1 state. Under the conditions when both inputs assume the logic 0 state, or when both the inputs assume the logic 1 state, the output assumes a logic 0 state.

Since an X-OR gate produces an output 1 only when the inputs are not equal, it is called *anti-coincidence* gate or *inequality detector*. The output of an X-OR gate is the modulo sum of two inputs. The name Exclusive-OR is derived from the fact that its output is a 1, only when exclusively one of its inputs is a 1 (it excludes the condition when both the inputs are 1).

The logic symbol and truth table of a two-input X-OR gate are shown in Figures 4.20a and 4.20b, respectively. If the input variables are represented by A and B and the output variable by the expression for the output of this gate is written as  $X = A \oplus B = A\bar{B} + \bar{A}B$  and read as 'X equal to A ex-or B'.



**Figure 4.20** Exclusive-OR gate.

Figure

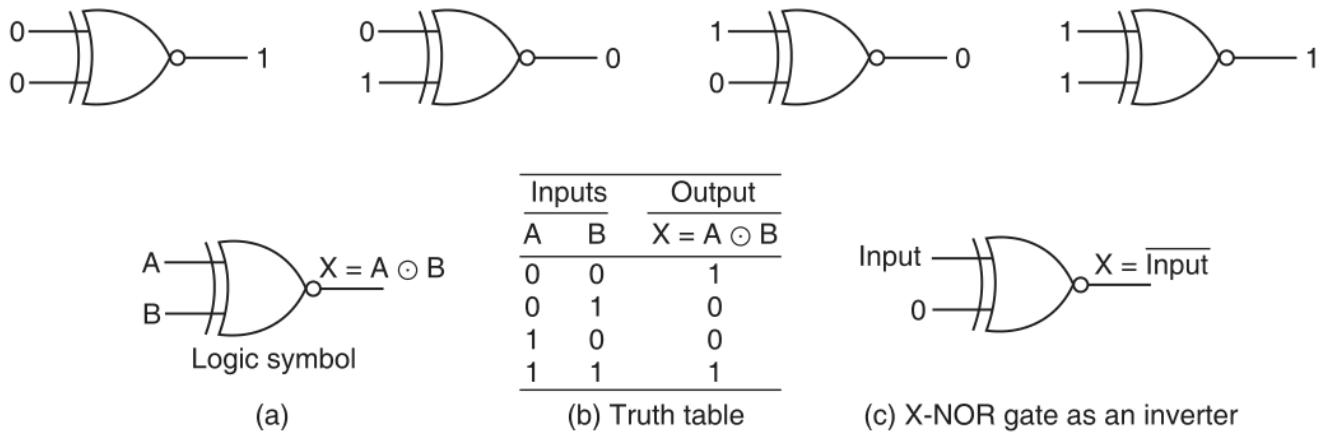
**4.20:** Exclusive-OR (XOR) gate showing logic symbol, truth table, and gate realization. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 173 (printed p. 141).

### 2. The XNOR Gate (Equivalence Gate) (Kumar, p. 175)

- **Plain Words:** The exact inverse of XOR. The output is HIGH (1) if both inputs are identical (00 or 11).
- **Boolean Equation:**

$$X = A \odot B = A \oplus B = AB + A \bar{B}$$

• **Authentic Diagram:**



**Figure 4.21** Exclusive-NOR gate.

Figure

**4.21:** Exclusive-NOR (XNOR) gate showing logic symbol and truth table. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 175 (printed p. 143).

**Understanding Checkpoint**

- **Question:** If  $A = 1$  and  $B = 1$ , what are the outputs of an XOR gate and an XNOR gate?
- **Answer:** XOR output = 0; XNOR output = 1.

## PART 3: Digital Logic Families (RTL, TTL, CMOS) and Interfacing

### 1.8 Logic Family Performance Metrics (Figures of Merit)

Source: Kumar (Ch. 16, pp. 892–897); Mano (Ch. 2, pp. 91–93)

[Pacing: Conceptually Essential — Know What Makes a Family Good or Bad]

#### Need (Problem-First)

Logic gates are not ideal abstract equations; they are made of physical transistors. When selecting chips for an engineering project, how do you decide whether to use TTL, CMOS, or ECL? We need rigorous quantitative figures of merit to measure their speed, power, driving capability, and electrical robustness.

#### The Core Figures of Merit Defined Inline:

1. **Propagation Delay ( $t_{pd}$ ):** The time delay between the application of an input signal transition and the resulting output transition (Kumar, p. 892):
2.  **$t_{pHL}$ :** Propagation delay time from HIGH to LOW output.
3.  **$t_{pLH}$ :** Propagation delay time from LOW to HIGH output.
4. Average propagation delay:  $t_{pd} = \frac{t_{pHL} + t_{pLH}}{2}$ . Measured in **nanoseconds (ns)**. Shorter delay means a faster processor.
5. **Power Dissipation ( $P_D$ ):** The electrical power consumed by the gate, measured in **milliwatts (mW)**:

$$P_D = V_{CC} \times I_{CC}$$

6. **Speed-Power Product (SPP):** The ultimate figure of merit balancing speed against energy consumption (Kumar, p. 894):

$$SPP = t_{pd} \times P_D$$

Measured in **picojoules (pJ)**. A lower SPP indicates a superior, more energy-efficient silicon architecture. 4. **Fan-Out:** The maximum number of standard logic inputs of the same family that the output of a gate can reliably drive without causing output voltages to degrade outside legitimate logic bands (Kumar, p. 893). 5. **Fan-In:** The number of independent input terminals

available on a single gate. 6. **DC Noise Margins (NM<sub>H</sub>, NM<sub>L</sub>)**: The maximum voltage amplitude of electrical noise that can be tolerated without causing false switching (Kumar, p. 896, Figure 16.4):

where environment noise may be high.

Figure 16.4a shows the range of output voltages that can occur in a logic circuit. Voltages greater than  $V_{OH}(min)$  are considered as a logic 1 and voltages lower than  $V_{OL}(max)$  are considered as a logic 0. Voltages in the disallowed range should not appear at a logic circuit output under normal conditions. Figure 16.4b shows the input voltage requirements of a logic circuit. The logic circuit will respond to any input greater than  $V_{IH}(min)$  as a logic 1 and to any input lower than  $V_{IL}(max)$  as a logic 0. Voltages in the indeterminate range will produce an unpredictable response and should not be used.

Figure

16.4: Graphical definition of DC noise margins showing transmitting output voltage levels ( $V_{OH}$ ,  $V_{OL}$ ) and receiving input threshold levels ( $V_{IH}$ ,  $V_{IL}$ ). Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 896 (printed p. 864).

### 1.9 Resistor-Transistor Logic (RTL)

Source: Kumar (Ch. 4, p. 173, Figure 4.19; Ch. 16, p. 892)

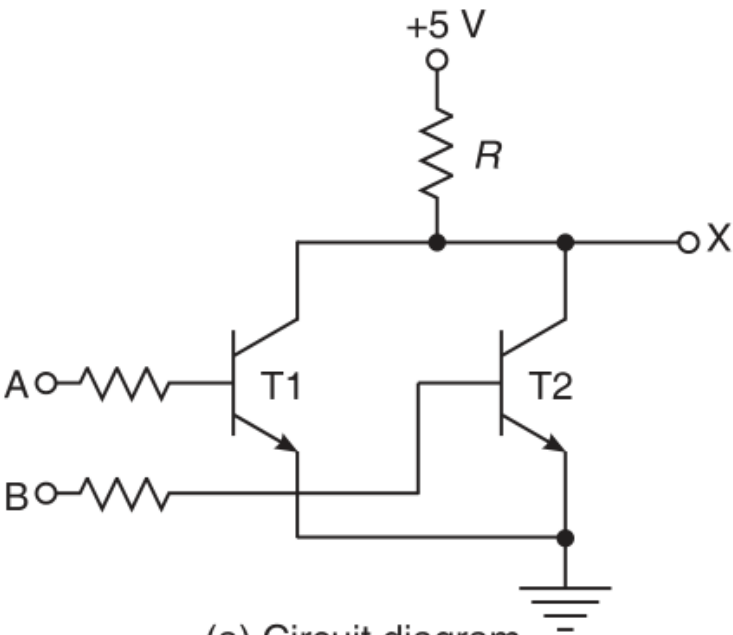
[Pacing: Historical Foundation — How Transistor Logic Began]

#### Need (Problem-First)

The very first integrated circuits in the 1960s (including the Apollo Guidance Computer) needed the simplest possible way to implement logic using basic bipolar transistors and resistors.

#### How It Works (The Physical Mechanism)

Figure 4.19 from Kumar (p. 173) shows a discrete two-input RTL NOR gate:



(a) Circuit diagram

| Input |     | Output |
|-------|-----|--------|
| A     | B   | X      |
| 0 V   | 0 V | 5 V    |
| 0 V   | 5 V | 0 V    |
| 5 V   | 0 V | 0 V    |
| 5 V   | 5 V | 0 V    |

(b) Truth table

**Figure 4.19** Discrete two-input NOR gate.

Figure

4.19: Discrete two-input Resistor-Transistor Logic (RTL) NOR gate. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 173 (printed p. 141).

#### Characteristic 13 Walkthrough: Hardware Reality Check

- **Case 1 (A = 0 V, B = 0 V)**: Neither transistor **T\_1** nor **T\_2** receives base current. Both transistors are in **cutoff (OPEN)**. No current flows through collector resistor **R**. With zero voltage drop across **R**, the output node **X** is pulled directly up to **+5 V**

(Logic 1).

- **Case 2 ( $A = 5\text{ V}$  or  $B = 5\text{ V}$ ):** Base current floods the corresponding transistor, driving it into **saturation (CLOSED)**. The saturated transistor creates a low-resistance path from output node **X** directly to Ground ( $0\text{ V}$ ). The output voltage collapses to  $V_{CE(sat)} \approx 0.2\text{ V}$  (Logic 0).
- This produces the exact truth table of a **NOR gate**.
- **Fatal Flaw of RTL:** The collector resistor **R** severely limits speed. When switching HIGH, parasitic capacitances must charge through **R**, giving poor rise times, tiny noise margins ( $0.2\text{ V}$ ), and poor fan-out (only 4 to 5 loads).

---

## 1.10 Transistor-Transistor Logic (TTL): Totem-Pole, Open-Collector, and Tri-State

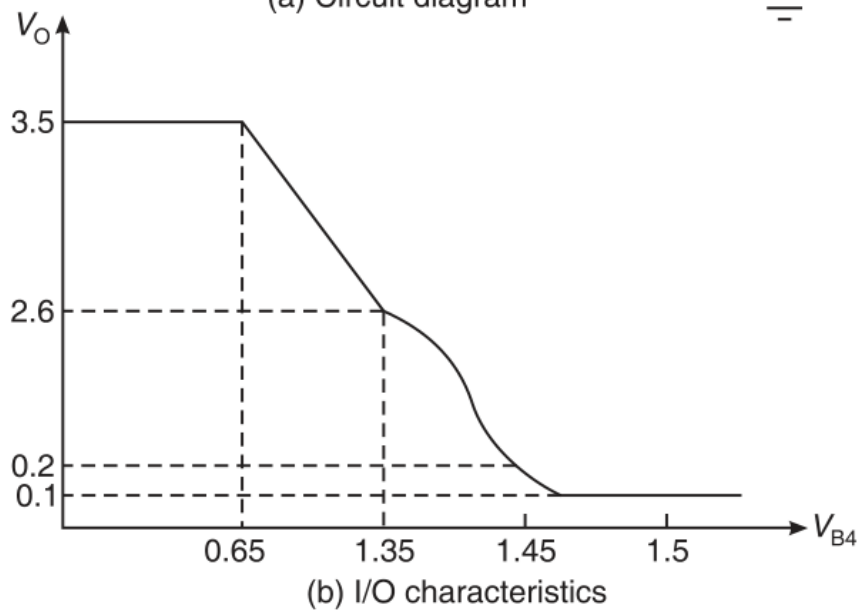
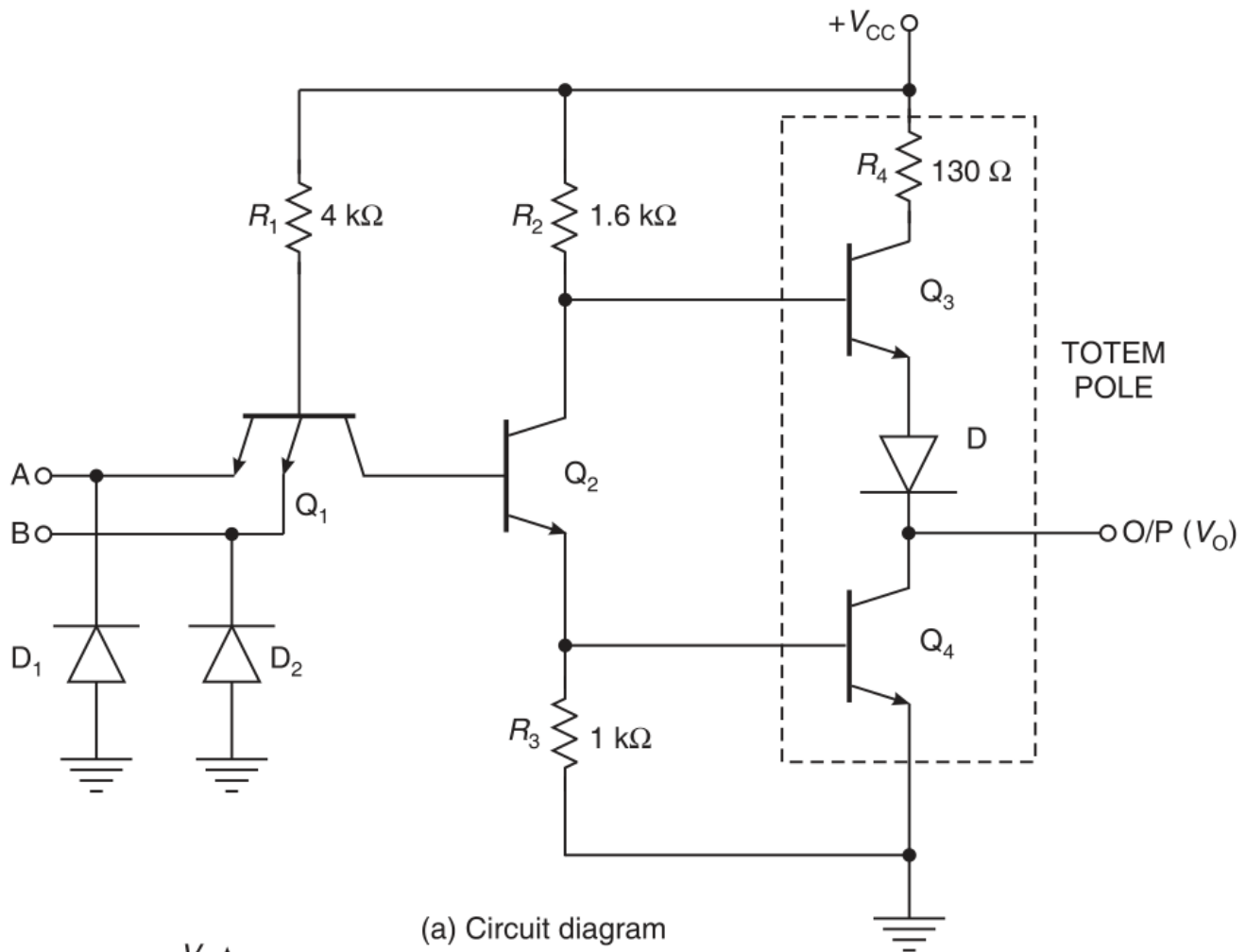
Source: Kumar (Ch. 16, pp. 898–907, Figures 16.5, 16.7, 16.8, 16.9)

[Pacing: Conceptually Heavy — Study the Transistor Conduction States Carefully]

### Need (Problem-First)

RTL was too slow and noise-sensitive. Engineers realized that if they replaced the passive pull-up resistor with an **active transistor switch** (push-pull / totem-pole), the circuit could actively drive both HIGH and LOW transitions with blazing speed, giving rise to TTL (the 7400 series).

The Standard Totem-Pole TTL NAND Gate (*Kumar*, p. 898, Figure 16.5)



**Figure 16.5** TTL NAND gate.

Figure

**16.5:** Schematic diagram and voltage transfer characteristics of a standard Transistor-Transistor Logic (TTL) NAND gate with active totem-pole output stage. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 898 (printed p. 866).

#### Four Key Functional Stages in TTL:

1. **Multi-Emitter Input Transistor ( $Q_1$ ):** Has multiple emitters (one for each input **A**, **B**). It performs the AND operation directly on electrons.

2. **Phase Splitter Transistor (Q<sub>2</sub>):** Produces two out-of-phase output voltages simultaneously: its collector voltage and emitter voltage are complementary.
3. **Totem-Pole Output Stage (Q<sub>3</sub>, D, Q<sub>4</sub>):**
4. **Q<sub>4</sub> (Pull-down transistor):** When turned ON, it pulls the output node down to Ground (0 V).
5. **Q<sub>3</sub> (Pull-up transistor):** When turned ON, it pulls the output node up to +V<sub>CC</sub> (+5 V).
6. **Diode D:** Crucial engineering component! It ensures Q<sub>3</sub> and Q<sub>4</sub> can **never be ON at the same time**. Without diode D, the base-emitter drops would allow both transistors to conduct simultaneously during transitions, creating a massive current spike from V<sub>CC</sub> straight to ground.

### Characteristic 13 Walkthrough: Hardware Reality Check

- **Condition 1 (Any Input A or B is LOW, 0.2 V):**

Current flows from V<sub>CC</sub> through R<sub>1</sub> (4 kΩ) and out through the LOW input emitter to ground. The base of Q<sub>1</sub> is clamped to 0.2 V + 0.7 V = 0.9 V. Because Q<sub>2</sub> and Q<sub>4</sub> need 0.7 V + 0.7 V = 1.4 V across their base-emitter junctions to turn ON, Q<sub>2</sub> and Q<sub>4</sub> are **completely OFF (cutoff)**. With Q<sub>2</sub> OFF, no current flows through R<sub>2</sub>, pulling the base of Q<sub>3</sub> HIGH. Q<sub>3</sub> turns ON, actively driving the output node V<sub>O</sub> to:

$$V_{OH} = V_{CC} - V_{BE3} - V_D = 5.0 \text{ V} - 0.7 \text{ V} - 0.7 \text{ V} \approx 3.6 \text{ V (Logic 1)}$$

- **Condition 2 (Both Inputs A and B are HIGH, 3.5 V):**

The emitter junctions of Q<sub>1</sub> are reverse-biased. Current from V<sub>CC</sub> through R<sub>1</sub> now flows forward through the base-collector junction of Q<sub>1</sub> directly into the base of Q<sub>2</sub>. Q<sub>2</sub> **turns ON hard**, which in turn floods the base of Q<sub>4</sub> with current, turning Q<sub>4</sub> **ON into saturation**. Saturated Q<sub>4</sub> pulls the output node down to ground:

$$V_{OL} = V_{CE4(sat)} \approx 0.2 \text{ V (Logic 0)}$$

Meanwhile, the collector of Q<sub>2</sub> drops to V<sub>CE2(sat)</sub> + V<sub>BE4</sub> ≈ 0.2 + 0.7 = 0.9 V. To turn Q<sub>3</sub> ON would require V<sub>O</sub> + V<sub>D</sub> + V<sub>BE3</sub> ≈ 0.2 + 0.7 + 0.7 = 1.6 V. Since 0.9 V < 1.6 V, Q<sub>3</sub> is **held completely OFF**. - Output = 0 only when all inputs are 1. **This is a NAND gate!**

---

### Open-Collector TTL Gates & Wired-AND

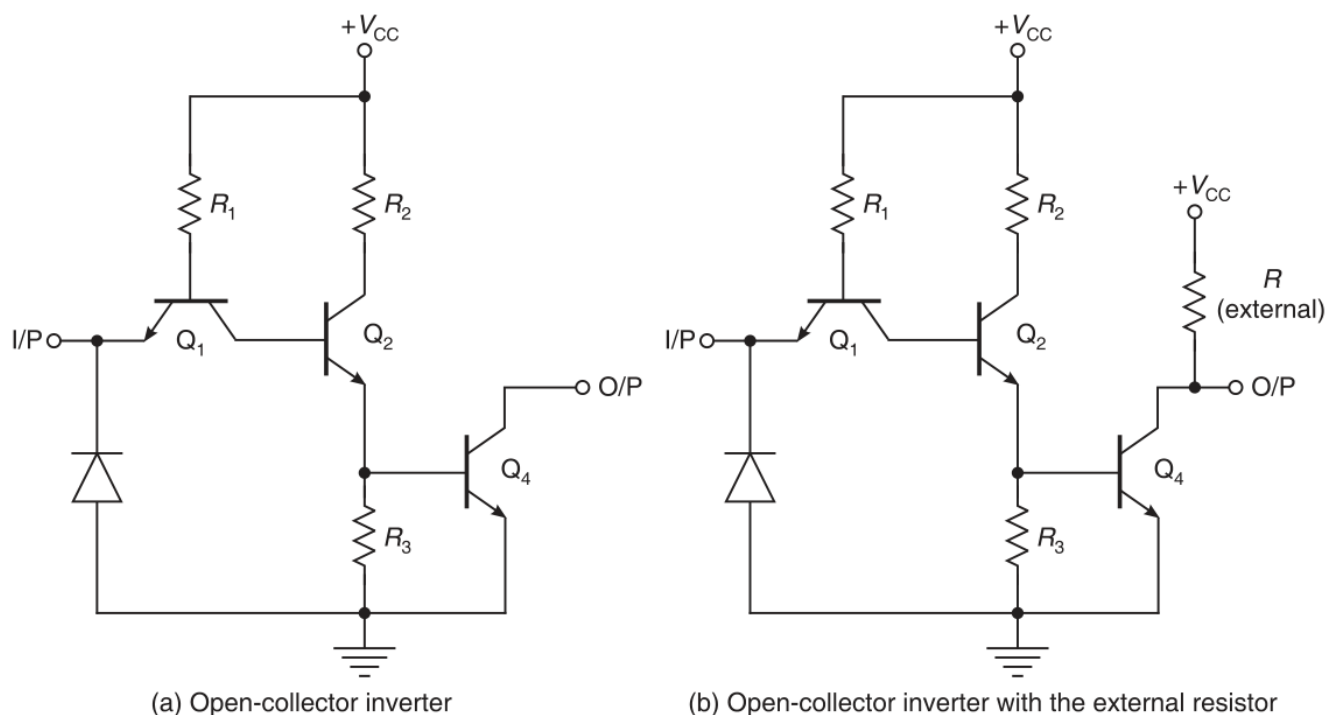
Source: Kumar (Ch. 16, pp. 903–905, Figures 16.7 & 16.8)

#### Need (Problem-First)

Totem-pole outputs can **never be directly connected together**! If Gate 1 outputs HIGH (+3.6 V) while Gate 2 outputs LOW (0.2 V), a direct low-resistance short circuit is created between V<sub>CC</sub> and Ground through Q<sub>3</sub> of Gate 1 and Q<sub>4</sub> of Gate 2. Enormous current (>100 mA) will surge through the transistors, permanently destroying both chips. How can multiple gates safely share a single common communication wire (a bus)?

#### The Solution: Open-Collector Output (Kumar, p. 904, Figure 16.7)

In an **open-collector** gate, the upper totem-pole components (Q<sub>3</sub>, diode D, and resistor R<sub>4</sub>) are completely omitted. The collector of pull-down transistor Q<sub>4</sub> is left floating in open air.



(a) Open-collector inverter

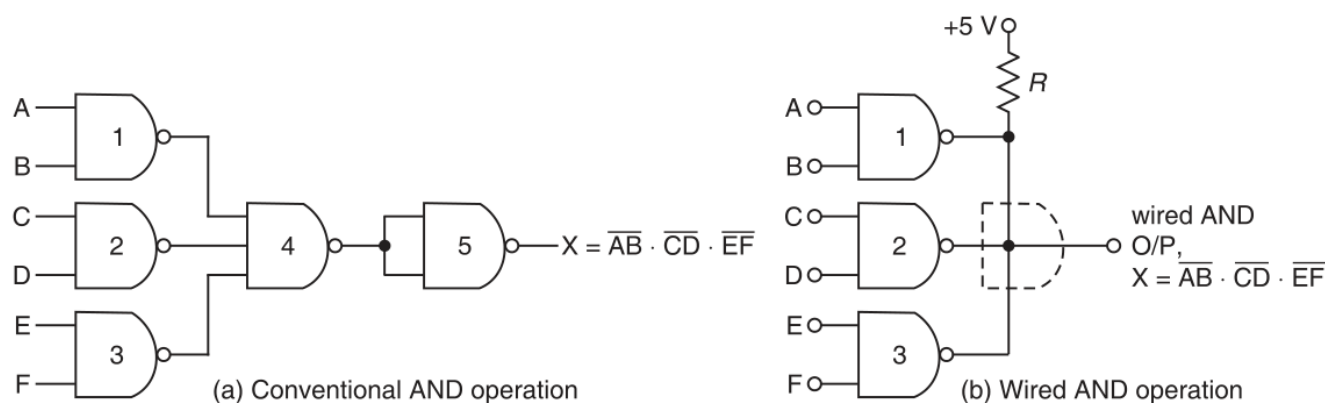
(b) Open-collector inverter with the external resistor

**Figure 16.7** Circuit diagram and logic symbol of open-collector inverter.

Figure

**16.7:** Circuit diagram and logic symbol (showing internal open-collector marking) of an open-collector TTL inverter. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 904 (printed p. 872).

- **Requirement:** An external **pull-up resistor ( $R_L$ )** must be connected between the output wire and  **$+V_{CC}$** .
- **Wired-AND Operation (Kumar, p. 905, Figure 16.8):** Multiple open-collector outputs can be tied directly to the same wire! If any gate turns its **Q<sub>4</sub>** ON, it pulls the shared wire to **0 V**. The wire only reaches **+5 V** if **all** gates release their transistors into cutoff. This performs an automatic **Wired-AND** function without requiring an extra physical AND gate:



(a) Conventional AND operation

(b) Wired AND operation

**Figure 16.8** ANDing of TTL gates.

Figure

**16.8:** Direct wired-AND connection of two open-collector TTL gates using an external pull-up resistor. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 905 (printed p. 873).

## Tri-State (Three-State) TTL Logic

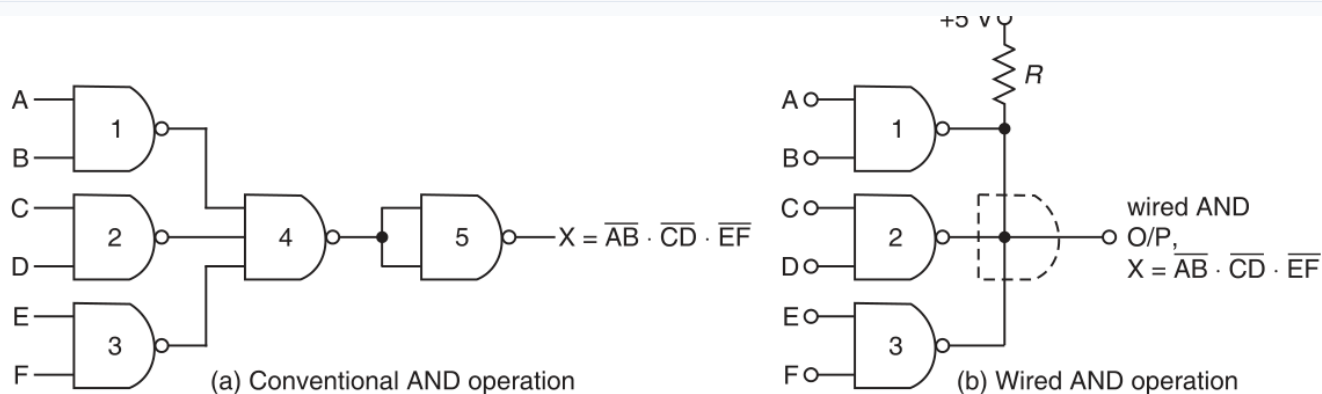
Source: Kumar (Ch. 16, pp. 905–907, Figure 16.9)

### Need (Problem-First)

Open-collector gates solve the bus contention problem, but they are relatively slow because the output line rises slowly through external resistor  **$R_L$** . We want the blazing speed of an active totem-pole output, but the bus-sharing ability of an open collector.

### The Solution: Tri-State Logic

A **tri-state gate** has three distinct output states: 1. **Logic 0**: Low impedance path to Ground (**Q<sub>4</sub>** ON, **Q<sub>3</sub>** OFF). 2. **Logic 1**: Low impedance path to **+V<sub>CC</sub>** (**Q<sub>3</sub>** ON, **Q<sub>4</sub>** OFF). 3. **High-Impedance State (Z / Hi-Z)**: Both **Q<sub>3</sub>** and **Q<sub>4</sub>** are turned **OFF simultaneously!** The gate physically disconnects itself from the output wire, behaving like an open switch with infinite resistance (**>10 M  $\Omega$** ).



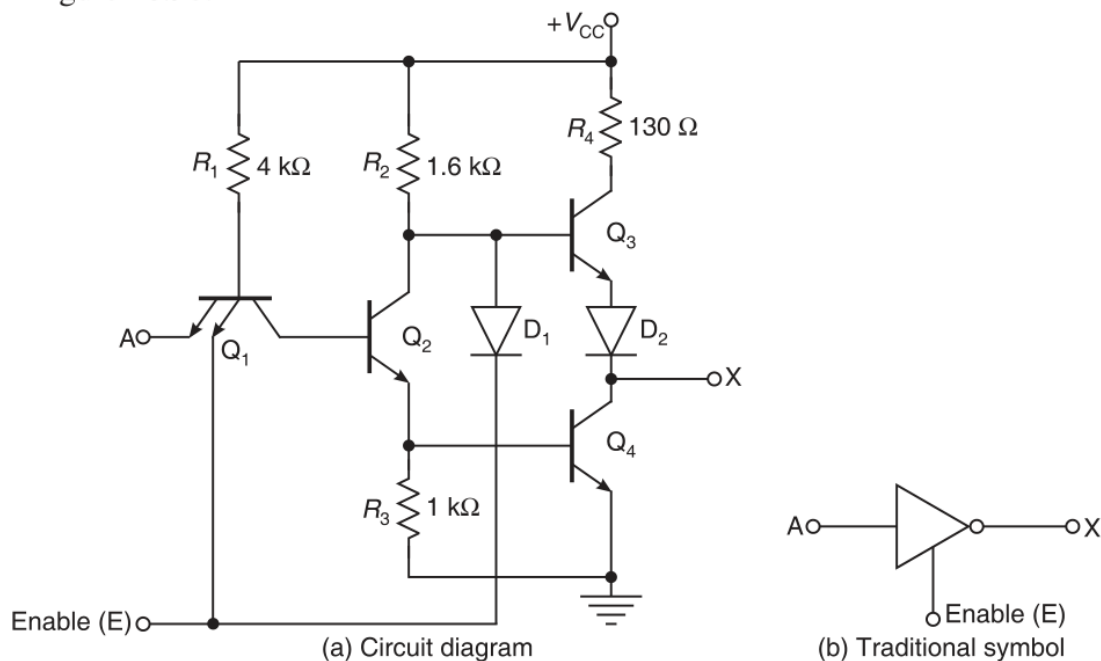
**Figure 16.8** ANDing of TTL gates.

Because of the absence of pull-up transistors, the wired-AND connections significantly reduce switching speeds. However, they are useful in reducing the chip count of a system when speed is not a consideration.

### 16.5.2 Tri-state (3-state) TTL

The third TTL configuration is the tri-state configuration. It utilizes the advantage of high speed of operation of the totem-pole configuration and wire ANDing of the open-collector configuration. It is called the *tri-state* TTL, because it allows three possible output states: HIGH, LOW, and HIGH impedance (Hi-Z). In the Hi-Z state, both the transistors in the totem-pole arrangement are turned off, so that the output terminal is a HIGH impedance to ground or  $V_{CC}$ . In fact, the output is an open or floating terminal, that is, neither a LOW nor a HIGH. In practice, the output terminal is not an exact open circuit, but has a resistance of several  $M\Omega$  or more relative to ground and  $V_{CC}$ .

The circuit of a tri-state inverter is shown in Figure 16.9a. The tri-state operation is obtained by modifying the basic totem-pole circuit of Figure 16.5. The circuit has two inputs—A is the normal logic input and E is an enable input that can produce the Hi-Z. The traditional symbol is shown in Figure 16.9b.



**Figure 16.9** Tri-state TTL inverter.

Figure

- **Operation:** When the **Enable** line is asserted, the gate behaves as a normal high-speed inverter. When Enable is de-asserted, internal diode steering holds the bases of both **Q\_3** and **Q\_4** at ground, placing the output into the floating **Z** state. This enables microprocessors to connect hundreds of memory and peripheral chips to a single shared data bus.

### 1.11 CMOS Logic: Inverter, NAND, and NOR Gates

Source: Kumar (Ch. 16, pp. 917–924, Figures 16.23, 16.24, 16.25)

[Pacing: The King of Modern VLSI — Master This Section Thoroughly]

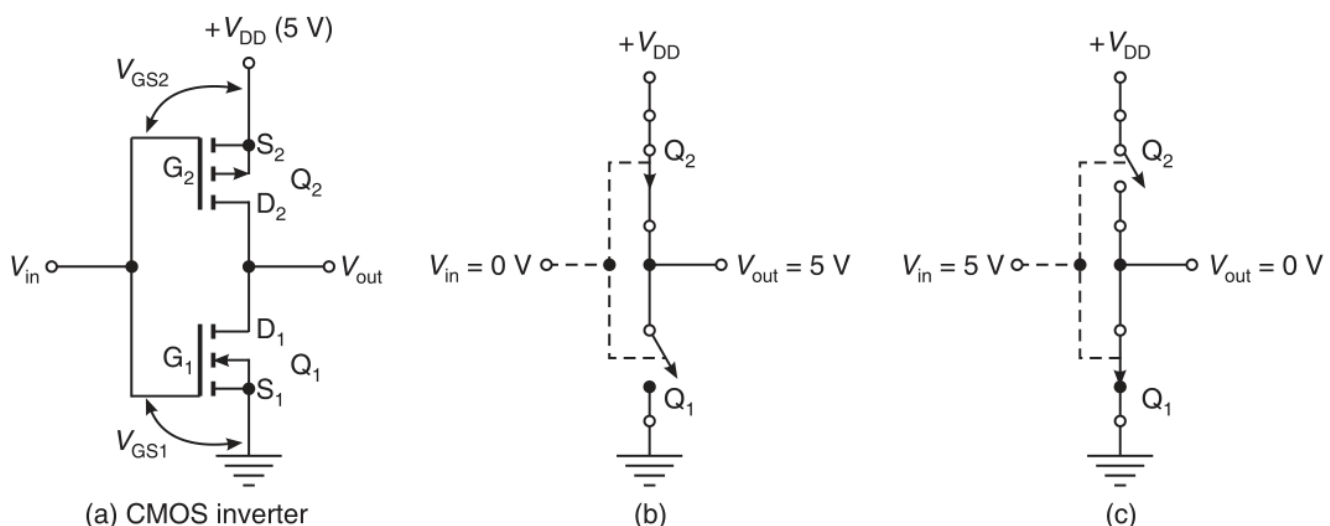
#### Need (Problem-First)

TTL gates consume substantial continuous electric power because current flows through internal resistors even when the gate is not switching. In a modern computer processor with billions of transistors, TTL logic would consume kilowatts of power and instantly melt the silicon chip. The world needed a logic family that draws **virtually zero electric power** when idling. That technology is **CMOS** (Complementary Metal-Oxide-Semiconductor).

#### The Secret of CMOS: Complementary Pairs

CMOS pairs an **n-channel MOSFET (NMOS)** with a **p-channel MOSFET (PMOS)**: - **NMOS (Q\_1)**: Turns **ON** when Gate voltage is HIGH (**+5 V**), and turns **OFF** when Gate is LOW (**0 V**). Connected between output and Ground (the **Pull-Down Network**). - **PMOS (Q\_2)**: Turns **ON** when Gate voltage is LOW (**0 V**), and turns **OFF** when Gate is HIGH (**+5 V**). Connected between **+V\_{DD}** and output (the **Pull-Up Network**).

#### 1. The CMOS Inverter (Kumar, p. 921, Figure 16.23)



**Figure 16.23** Circuit diagram and equivalent circuits for various inputs of the CMOS inverter.

Figure

16.23: CMOS Inverter schematic and equivalent switch circuits for LOW and HIGH input states. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 921 (printed p. 889).

#### Characteristic 13 Walkthrough: Hardware Reality Check

- When  $V_{in} = 0 \text{ V}$  (LOW):  
 $V_{GS1} = 0 \text{ V}$  ■ NMOS **Q\_1** is **OFF** (OPEN switch).  
 $V_{GS2} = -5 \text{ V}$  ■ PMOS **Q\_2** is **ON** (CLOSED switch).  
 Output node  $V_{out}$  is connected directly to **+V\_{DD}** through the conducting PMOS channel, while completely isolated from Ground.

$$V_{out} = +V_{DD} = +5 \text{ V (Logic 1)}$$

- When  $V_{in} = +5 \text{ V}$  (HIGH):  
 $V_{GS1} = +5 \text{ V}$  ■ NMOS **Q\_1** is **ON** (CLOSED switch).

$V_{GS2} = 0\text{ V}$  ■ PMOS  $Q_2$  is **OFF (OPEN switch)**.

Output node  $V_{out}$  is connected directly to Ground through the conducting NMOS channel, while completely isolated from  $+V_{DD}$ .

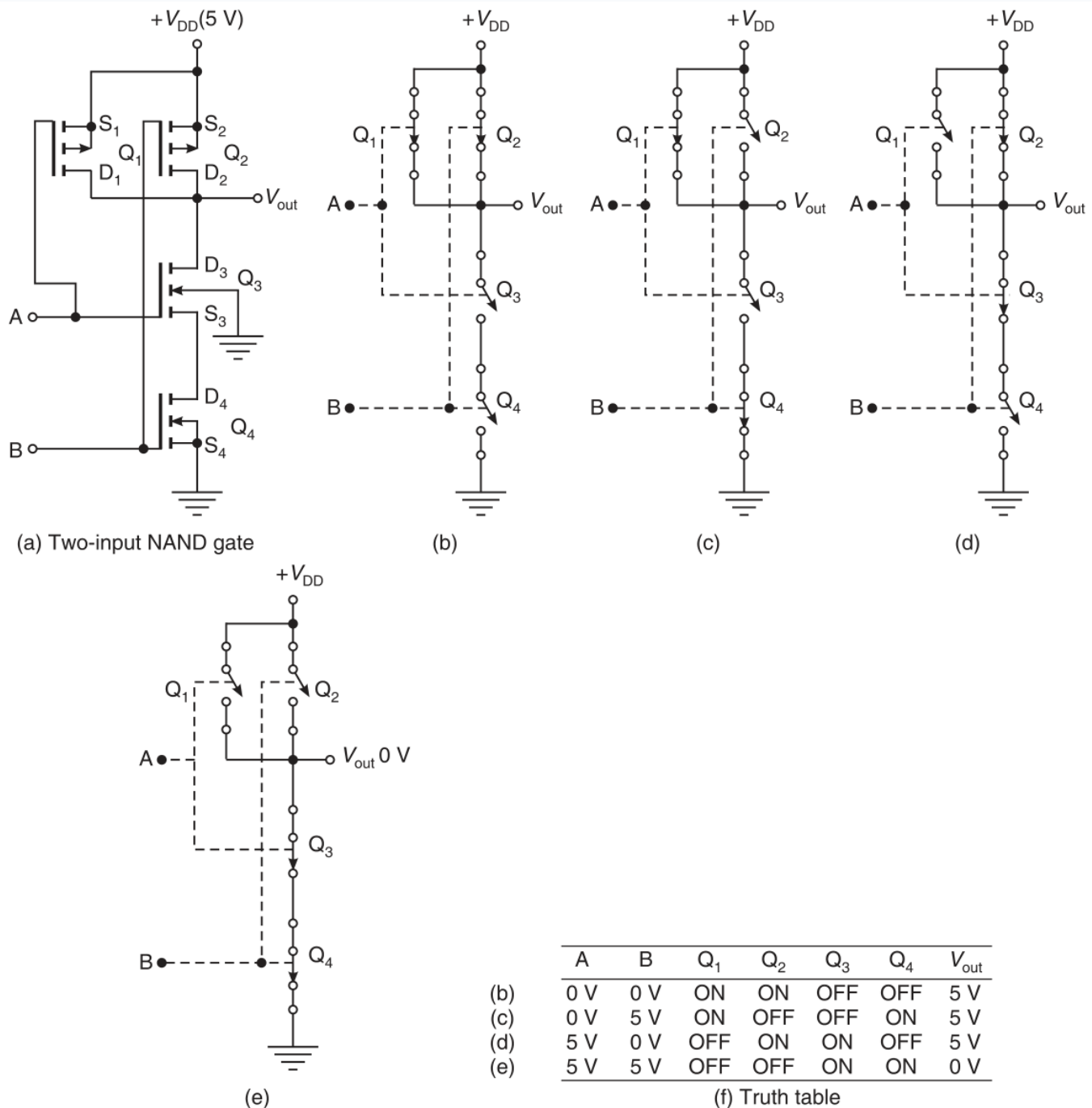
$$V_{out} = 0\text{ V (Logic 0)}$$

- **Why Static Power is ZERO:** In both steady-state conditions, **one of the two series transistors is always completely OFF**. There is never a direct DC path from  $+V_{DD}$  to Ground! Current only flows for a few picoseconds during the actual transition while charging the load capacitance.

---

## 2. The CMOS NAND Gate (*Kumar*, p. 922, Figure 16.24)

- **Architecture:** Two PMOS transistors ( $Q_1$ ,  $Q_2$ ) connected in **parallel** between  $+V_{DD}$  and output; two NMOS transistors ( $Q_3$ ,  $Q_4$ ) connected in **series** between output and Ground.



**Figure 16.24** Circuit diagram and equivalent circuits for various inputs of the CMOS NAND gate.

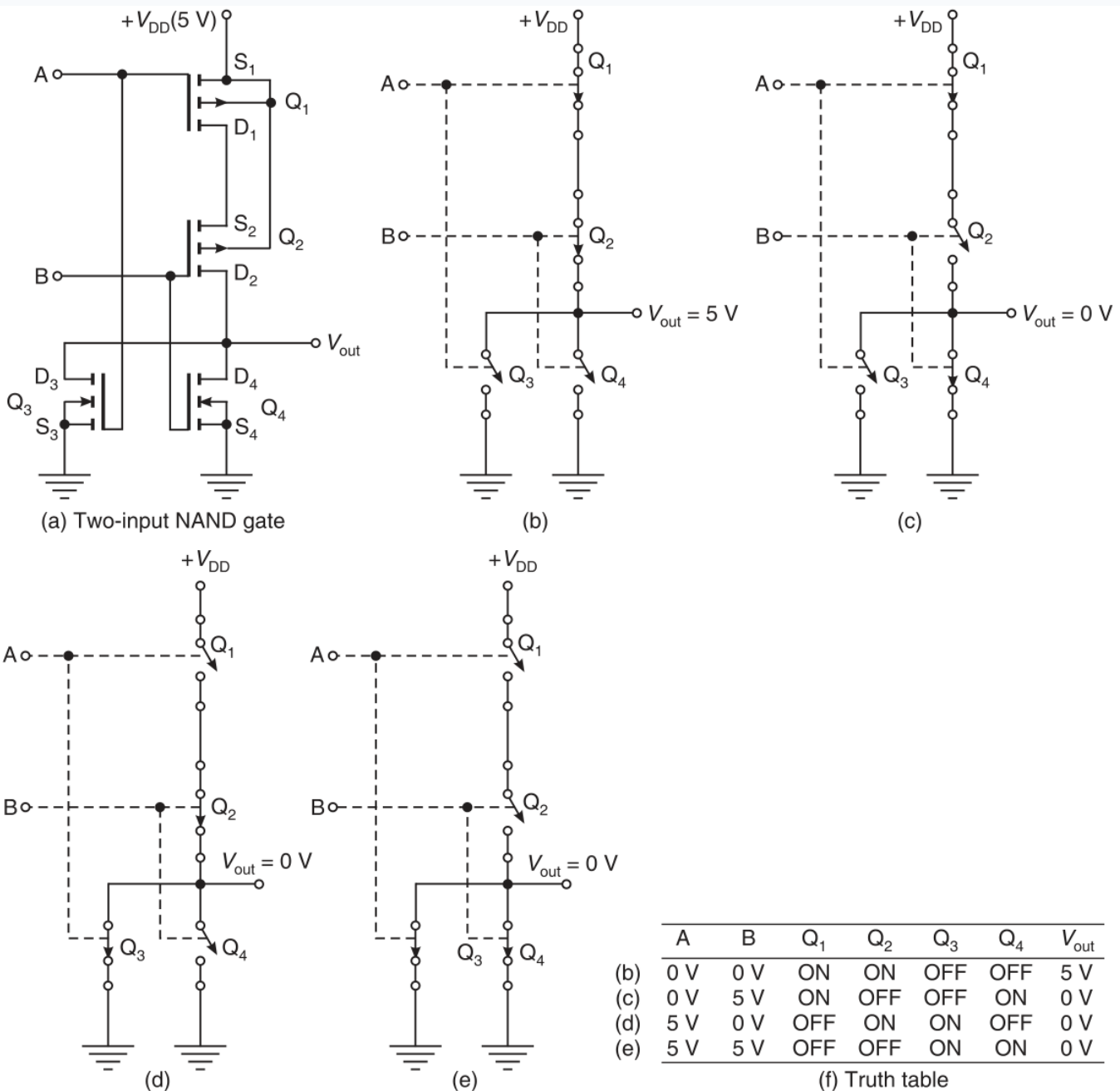
Figure

16.24: CMOS two-input NAND gate schematic, equivalent switch circuits, and operational truth table. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 922 (printed p. 890).

- **Operation:**
- If *either* A or B is 0 V, the corresponding PMOS is ON, pulling V<sub>out</sub> to +5 V. Because NMOS are in series, the path to ground is broken.
- Only when *both* A and B are +5 V are both series NMOS (Q<sub>3</sub>, Q<sub>4</sub>) ON simultaneously, while both parallel PMOS are OFF. Output is pulled to 0 V.

### 3. The CMOS NOR Gate (Kumar, p. 923, Figure 16.25)

- **Architecture:** Two PMOS transistors (Q<sub>1</sub>, Q<sub>2</sub>) connected in **series** between +V<sub>DD</sub> and output; two NMOS transistors (Q<sub>3</sub>, Q<sub>4</sub>) connected in **parallel** between output and Ground.



**Figure 16.25** Circuit diagram and equivalent circuits for various inputs of the CMOS NOR gate.

Figure

16.25: CMOS two-input NOR gate schematic, equivalent switch circuits, and operational truth table. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 923 (printed p. 891). (Note: Sub-label (a) contains an authentic textbook typographical misprint labeling the schematic as NAND; the circuit is verified as NOR by its series PMOS and parallel NMOS configuration and verified caption).

- **Operation:**
- If either **A** or **B** is **+5 V**, the corresponding parallel NMOS conducts, clamping **V<sub>out</sub>** to **0 V**.
- Only when both **A** and **B** are **0 V** are both series PMOS transistors ON simultaneously, pulling **V<sub>out</sub>** to **+5 V**.

## 1.12 Interfacing CMOS and TTL Logic Families

Source: Kumar (Ch. 16, pp. 930–933, Figures 16.32 & 16.33)

[Pacing: Critical Practical Engineering — Read Deliberately]

### Need (Problem-First)

In practical engineering systems, you often need to connect a TTL microprocessor to a modern CMOS memory chip, or vice versa. If you blindly connect a copper wire between their pins without analyzing voltage and current compatibility, the circuit will fail to recognize logic states or destroy the chips.

The Fundamental Voltage & Current Mismatch

| Parameter                       | Standard TTL (5 V) | Standard CMOS (5 V)       | Compatibility Conflict                |
|---------------------------------|--------------------|---------------------------|---------------------------------------|
| $V_{OH(min)}$ (Output High Min) | +2.4 V to +2.7 V   | +4.9 V                    | TTL output high is too low for CMOS!  |
| $V_{IH(min)}$ (Input High Min)  | +2.0 V             | +3.5 V (70% of $V_{DD}$ ) | CMOS requires $\geq 3.5$ V to see a 1 |
| $V_{OL(max)}$ (Output Low Max)  | +0.4 V             | +0.1 V                    | Compatible ( $0.4$ V < $1.5$ V)       |
| $V_{IL(max)}$ (Input Low Max)   | +0.8 V             | +1.5 V (30% of $V_{DD}$ ) | Compatible                            |

Case 1: TTL Driving CMOS (Kumar, p. 931, Figure 16.32)

- The Physical Problem:** Standard TTL guarantees a minimum output HIGH voltage of only  $V_{OH} = 2.4$  V to  $2.7$  V. But a  $5$  V CMOS gate requires at least  $V_{IH} = 3.5$  V to reliably recognize a HIGH! A standard TTL output falls directly into the CMOS forbidden/undefined region.
- The Solution:** Connect an external pull-up resistor ( $R_p$ , typically  $1$  k  $\Omega$  to  $3.3$  k  $\Omega$ ) from the interconnect wire to  $+V_{CC}$  (Kumar, Figure 16.32a), or use a dedicated TTL-compatible CMOS buffer (such as the 74HCT series). When the TTL output goes HIGH,  $R_p$  pulls the line all the way up to  $+5.0$  V, well above the CMOS  $3.5$  V threshold.

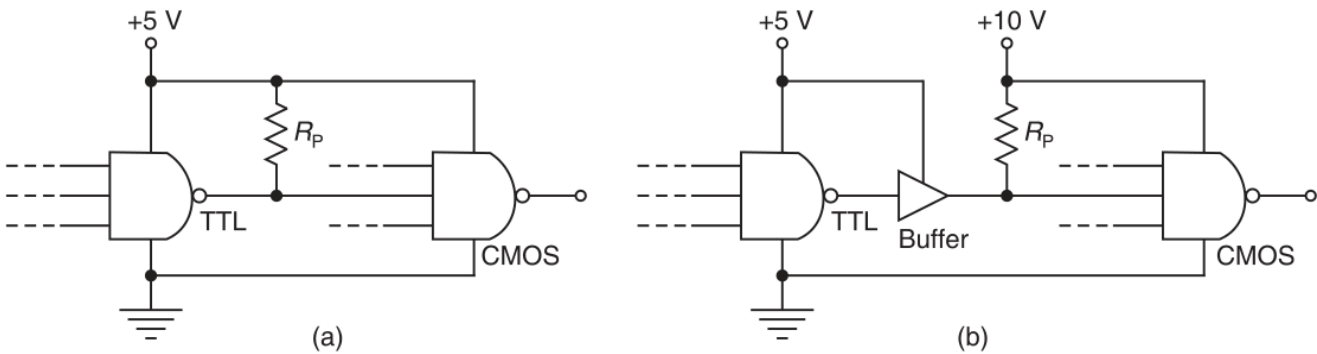


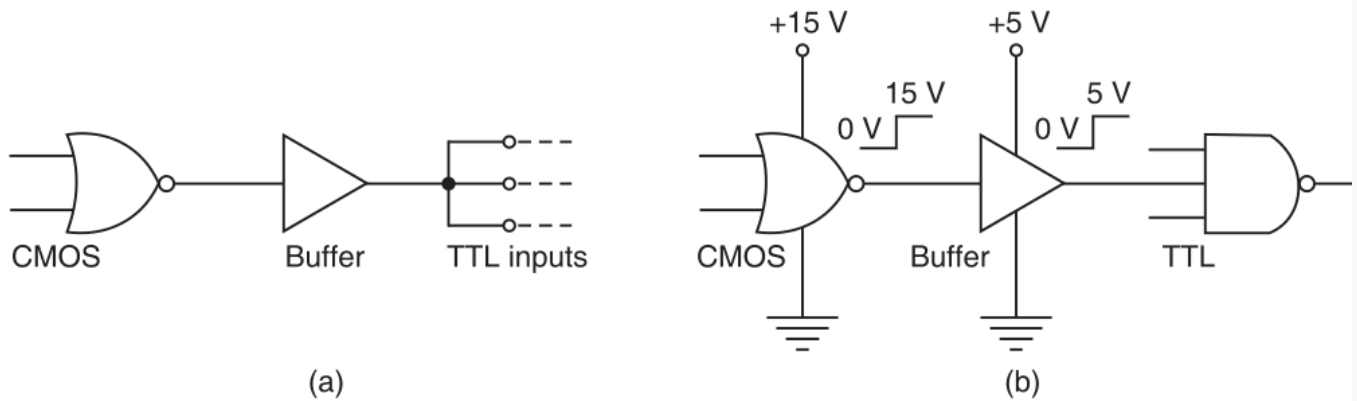
Figure 16.32 TTL to CMOS interfacing.

Figure

16.32: Interfacing TTL to CMOS logic using an external pull-up resistor or a supply-level shifting transistor. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 931 (printed p. 899).

Case 2: CMOS Driving TTL (Kumar, p. 932, Figure 16.33)

- Voltage Compatibility:** CMOS  $V_{OH(min)} = 4.9$  V, which easily exceeds TTL  $V_{IH(min)} = 2.0$  V. CMOS  $V_{OL(max)} = 0.1$  V, which is well below TTL  $V_{IL(max)} = 0.8$  V. Voltages are 100% compatible!
- The Current Sinking Problem:** When a TTL input is driven LOW, current physically flows out of the TTL emitter into the driving CMOS output ( $I_{IL} \approx 1.6$  mA per standard TTL gate). A standard 4000-series CMOS gate can only sink about  $0.4$  mA to  $1.0$  mA before its internal NMOS channel resistance causes  $V_{OL}$  to rise above  $0.8$  V!
- The Solution:** Use a high-current CMOS buffer (such as 74HCT / 74ACT) or limit the fan-out to a single low-power Schottky TTL (74LS) input:



**Figure 16.33** CMOS to TTL interfacing.

Figure

16.33: Interfacing CMOS to TTL showing buffer requirements for driving standard TTL loads. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 932 (printed p. 900).

## Unit I Understanding Checkpoints & Misconception Audit

### 1. Does This Make Sense? Self-Check

- **Question 1:** Why is two-level NAND-NAND logic completely equivalent to AND-OR logic?
- **Answer:** By De Morgan's theorem, an inverted-input OR gate (bubble-OR) is logically identical to a NAND gate ( $A + B = \overline{\overline{A} \cdot \overline{B}}$ ). The output bubbles of the first-stage NAND gates cancel the input bubbles of the second-stage NAND gate, preserving the original AND-OR function.
- **Question 2:** Why must you never directly tie two standard totem-pole TTL outputs together?
- **Answer:** If one outputs HIGH and the other outputs LOW, a direct, destructive short circuit is created from  $V_{CC}$  to Ground through their low-impedance transistors, burning out the devices.
- **Question 3:** What component is mandatory when interfacing a standard TTL output to a standard CMOS input?
- **Answer:** A pull-up resistor to  $+V_{CC}$ , because standard TTL output HIGH ( $\approx 2.7 \text{ V}$ ) is below the CMOS input HIGH threshold ( $3.5 \text{ V}$ ).

### 2. Conceptual Misconceptions Corrected

- **Misconception:** "A CMOS gate consumes zero power."  
**Correction:** It consumes virtually zero *static* (idle) power. But every time the output switches between 0 and 1, internal capacitances must be charged and discharged ( $P_{\text{dynamic}} = C \cdot V^2 \cdot f$ ). At high clock frequencies (gigahertz), CMOS chips consume immense power and generate significant heat!
- **Misconception:** "In 2's complement subtraction, if an end-carry is produced, you must add it back to the LSB."  
**Correction:** No! That is the obsolete rule for 1's complement. In 2's complement, an end carry past the MSB is **strictly discarded**.

## Unit II: Logic Function and Minimization

### Master Subject Map: Unit II Position

[ UNIT I: FUNDAMENTALS OF DIGITAL SYSTEMS & LOGIC FAMILIES ]

|  
v

```

+-----+
| YOU ARE HERE ---> UNIT II: LOGIC FUNCTION & MINIMIZATION |
| • Boolean Algebra: Axioms, Theorems (De Morgan's, Involution, Absorption), Duality |
| • Standard Representations: Minterms, Maxterms, SOP ( $\Sigma m$ ), POS ( $\Pi M$ ) Forms |
| • Karnaugh Maps (2, 3, and 4 Variables): Gray Adjacency, Grouping Rules (2, 4, 8) |
| • Prime Implicants, Essential Prime Implicants, Don't Care Conditions (d / X) |
  
```

*In 3 lines:* In Unit I, we learned how logic gates operate physically. But when designing a real digital system, naive designs require dozens of redundant gates. Unit II provides the mathematical machinery (Boolean Algebra) and graphical spatial tools (Karnaugh Maps) to strip away every redundant transistor, yielding the cheapest, fastest possible circuit.

## Prerequisites for this Unit

Before studying Unit II, ensure you recall these concepts from earlier sections: 1. **Basic Gates (Unit I, Section 1.5):** AND ( $X = AB$ ), OR ( $X = A + B$ ), and NOT ( $X = \overline{A}$ ). 2. **Truth Tables (Prereq 6):** An exhaustive list of outputs for all  $2^n$  combinations of  $n$  binary inputs. 3. **Powers of Two (Prereq 6):** **1, 2, 4, 8, 16** (governs the size of allowed K-map grouping loops). 4. **Gray Code Adjacency (Unit I, Section 1.4):** Consecutive codes differ by only a single bit (**00, 01, 11, 10**).

## PART 1: Boolean Algebra Axioms, Theorems, and Standard Forms

### 2.1 The Postulates and Axioms of Boolean Algebra

Source: Kumar (Ch. 5, pp. 207–211); Mano (Ch. 2, pp. 58–64)

[Pacing: Pure Foundations — Read Deliberately to Internalize Rules]

#### Need (Problem-First)

In high-school ordinary algebra, variables represent real numbers and arithmetic follows standard operations ( $1 + 1 = 2$ ). But electronic switches only have two states: **0** and **1**. Applying ordinary algebra to switches produces nonsense (for example, in ordinary math,  $1 + 1 = 2$ , but in an OR gate, turning on switch A and switch B simply keeps the bulb ON:  $1 + 1 = 1$ ). We need a rigorous mathematical system specifically tailored for two-valued logic. This is **Boolean Algebra**, formulated by George Boole in 1854 and adapted to switching circuits by Claude Shannon in 1938.

#### Chain of Cause and Effect

Switches are two-valued  $\rightarrow$  mathematicians define formal closure under OR (+) and AND ( $\cdot$ ) operations  $\rightarrow$  axioms establish identity, commutativity, distributivity, and complements  $\rightarrow$  these axioms permit rigorous mathematical simplification of electronic hardware.

#### The Huntington Postulates (Axioms) Defined

A set of elements  $B = \{0, 1\}$  together with two binary operators + (logical OR) and  $\cdot$  (logical AND) satisfies the following axioms (Kumar, p. 208; Mano, p. 60): 1. **Closure:** For every  $A, B$  in  $B$ : -  $A + B$  in  $B$  (the result of ORing two bits is always a bit). -  $A \cdot B$  in  $B$  (the result of ANDing two bits is always a bit). 2. **Identity Elements:** - Identity for OR is **0**:  $A + 0 = A$ . (ORing with **0** does not change the signal). - Identity for AND is **1**:  $A \cdot 1 = A$ . (ANDing with **1** preserves the signal). 3. **Commutativity:** -  $A + B = B + A$  -  $A \cdot B = B \cdot A$  (The order of physical input wires to a gate does not alter the output). 4. **Distributivity:** - **AND distributes over OR:**  $A \cdot (B + C) = (A \cdot B) + (A \cdot C)$ . - **OR distributes over AND (Crucial Boolean Special Rule!):**

$$A + (B \cdot C) = (A + B) \cdot (A + C)$$

*Note for beginners:* This second distributive law is **false in ordinary high-school algebra**, but **100% TRUE in Boolean algebra!**

5. **Complement (Inverse):** For every element  $A$  in  $B$ , there exists a unique element  $A$  in  $B$  such that: -  $A + \overline{A} = 1$  (A wire ORed with its inverse is always connected to **+5 V**). -  $A \cdot \overline{A} = 0$  (A wire ANDed with its inverse can never be closed; it is always **0 V**).

### 2.2 Core Theorems of Boolean Algebra and De Morgan's Laws

Source: Kumar (Ch. 5, pp. 211–219); Mano (Ch. 2, pp. 64–67)

[Pacing: Conceptually Critical — Memorize and Understand These Transformation Tools]

#### Need (Problem-First)

When an engineer writes down the raw logic equation for a real-world controller, the equation often contains ten or twenty terms. If built directly, the circuit would require dozens of integrated circuit packages. We need rigorous mathematical theorems to cancel out redundant terms on paper before soldering chips.

## The Core Theorems

### 1. Idempotent Laws (*Kumar*, p. 211):

$$A + A = A \quad A \cdot A = A$$

*Physical Meaning:* Tying two inputs of an OR or AND gate to the exact same wire produces the input itself. 2. **Boundedness (Null / Annihilation) Laws** (*Kumar*, p. 212):

$$A + 1 = 1 \quad A \cdot 0 = 0$$

*Physical Meaning:* An OR gate with one input tied permanently to +5 V is permanently locked HIGH (1). An AND gate with one input tied to Ground (0 V) is permanently locked LOW (0). 3. **Involution (Double Complement)** (*Kumar*, p. 212):

$$A = A$$

*Physical Meaning:* Two inverters in series cancel each other out. 4. **Absorption Laws** (*Kumar*, p. 213):

$$A + (A \cdot B) = A \quad A \cdot (A + B) = A$$

*Conceptual Proof of  $A + AB = A$ :*

$$\text{Step 1: Factor using identity axiom } (A = A \cdot 1): A + AB = A \cdot 1 + A \cdot B$$

$$\text{Step 2: Apply distributive law: } = A \cdot (1 + B)$$

$$\text{Step 3: Apply boundedness law } (1 + B = 1): = A \cdot (1)$$

$$\text{Step 4: Apply identity law: } = A$$

*Physical Meaning:* If the condition **A** is already asserted, the term **AB** is completely redundant hardware that can be cut from the circuit board! 5. **Redundant Literal Rule (Consensus Variant)** (*Kumar*, p. 214):

$$A + AB = A + B$$

*Conceptual Proof:*

$$\text{Step 1: Apply Boolean distributive law } (A + BC = (A+B)(A+C)): A + AB = (A + A) \cdot (A + B)$$

$$\text{Step 2: Apply complement law } (A + A = 1): = 1 \cdot (A + B)$$

$$\text{Step 3: Apply identity law: } = A + B$$

*Physical Hardware Reality:* The **A** term was completely useless!

---

## 6. De Morgan's Theorems

*Source: Kumar (Ch. 5, pp. 215–218); Mano (Ch. 2, pp. 65–67)*

[Pacing: Slow & Deliberate — The Most Important Theorem in Digital Electronics]

### Theorem 1: Complementation of a Product (The NAND Theorem)

"The complement of a product of variables is equal to the sum of their individual complements" (*Kumar*, p. 215):

$$A \cdot B = A + B$$

*Physical Gate Meaning:* A **NAND gate** is logically identical to an **OR gate with inverted inputs** (a Bubbled OR gate).

Theorem 2: Complementation of a Sum (The NOR Theorem)

"The complement of a sum of variables is equal to the product of their individual complements" (Kumar, p. 216):

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

Physical Gate Meaning: A **NOR gate** is logically identical to an **AND gate with inverted inputs** (a Bubbled AND gate).

Mathematical Verification by Truth Table:

Let us prove Theorem 1 ( $\overline{AB} = \overline{A} + \overline{B}$ ) exhaustively across all four possible input combinations:

| A | B | AB | AB (LHS) | $\overline{A}$ | $\overline{B}$ | $\overline{A} + \overline{B}$ (RHS) | Match? |
|---|---|----|----------|----------------|----------------|-------------------------------------|--------|
| 0 | 0 | 0  | 1        | 1              | 1              | 1                                   | Yes    |
| 0 | 1 | 0  | 1        | 1              | 0              | 1                                   | Yes    |
| 1 | 0 | 0  | 1        | 0              | 1              | 1                                   | Yes    |
| 1 | 1 | 1  | 0        | 0              | 0              | 0                                   | Yes    |

Every single row of Column 4 (**AB**) matches Column 7 ( **$\overline{A} + \overline{B}$** ) exactly. The theorem is proven beyond doubt.

2.3 The Principle of Duality

Source: Kumar (Ch. 5, p. 210); Mano (Ch. 2, p. 64)

[Pacing: Fast & Methodical — A Powerful Symmetry Tool]

Need & How It Works

The **Duality Principle** states that any true Boolean algebraic identity remains true if you perform three simultaneous swaps (Kumar, p. 210): 1. Change every OR operator (+) to an AND operator (·). 2. Change every AND operator (·) to an OR operator (+). 3. Change every identity 0 to 1, and every 1 to 0. (Leave variable literals **A**, **B**, **C** uncomplemented!) - Example: The dual of the identity  **$A + A = 1$**  is:

$$A \cdot A = 0$$

This halves the effort required in formal mathematical proofs.

2.4 Standard Representations: Minterms, Maxterms, SOP, and POS

Source: Kumar (Ch. 5, pp. 220–228); Mano (Ch. 2, pp. 73–82)

[Pacing: Systematic Bookkeeping — Foundation for K-Maps]

Need (Problem-First)

A Boolean function can be written in dozens of arbitrary algebraic forms (e.g.,  **$F = AB + C(D + E)$**  vs  **$F = (A + C)(B + C)D + CE$** ). To communicate unambiguously, build systematic automated tools, and enter logic into Karnaugh Maps, engineers require standardized **canonical forms**.

Definitions: Minterms vs. Maxterms

1. Minterms (Standard Product Terms) (Kumar, p. 220)

A **minterm** is an AND product of all **n** literals in the function, where each variable appears exactly once (either in unprimed form **A** or primed form **A**). - **The Rule for Minterms:** A minterm is defined such that it equals **1 for exactly one specific combination of input variables**. - If variable **A = 1**, write it uncomplemented: **A**. - If variable **A = 0**, write it complemented: **A**. - *Designation:* Lowercase **m<sub>i</sub>**, where index **i** is the decimal equivalent of the binary row. - For 3 variables (**A**, **B**, **C**): - Row **0 (000<sub>2</sub>)**: **m<sub>0</sub> = A B C** - Row **1 (001<sub>2</sub>)**: **m<sub>1</sub> = A B C** - Row **5 (101<sub>2</sub>)**: **m<sub>5</sub> = A B C** - Row **7 (111<sub>2</sub>)**: **m<sub>7</sub> = A B C**

## 2. Maxterms (Standard Sum Terms) (Kumar, p. 222)

A **maxterm** is an OR sum of all  $n$  literals, where each variable appears exactly once. - **The Rule for Maxterms:** A maxterm is defined such that it equals **0** for **exactly one specific combination of input variables**. - If variable  $A = 0$ , write it uncomplemented:  $A$ . - If variable  $A = 1$ , write it complemented:  $\bar{A}$ . (Notice this is the exact opposite of minterm convention!) - *Designation:* Uppercase  $M_i$ , where index  $i$  is the decimal row number. - For 3 variables ( $A, B, C$ ): - Row 0 (000<sub>2</sub>):  $M_0 = A + B + C$  - Row 5 (101<sub>2</sub>):  $M_5 = A + B + C$  - Row 7 (111<sub>2</sub>):  $M_7 = A + B + C$

## 3. Canonical Sum-of-Products (SOP) Form ( $\sum m$ )

The canonical SOP form expresses a logic function as the logical OR (sum) of all the minterms for which the output function equals **1**:

$$F(A, B, C) = \sum m(1, 4, 5, 7) = m_1 + m_4 + m_5 + m_7 = A\bar{B}C + AB\bar{C} + ABC + \bar{A}BC$$

## 4. Canonical Product-of-Sums (POS) Form (prod M)

The canonical POS form expresses a logic function as the logical AND (product) of all the maxterms for which the output function equals **0**:

$$F(A, B, C) = \text{prod } M(0, 2, 3, 6) = M_0 \cdot M_2 \cdot M_3 \cdot M_6 = (A + B + C)(A + \bar{B} + C)(A + B + \bar{C})(\bar{A} + B + C)$$

**The Fundamental Conversion Rule (Kumar, p. 226):**

The minterms where  $F = 1$  and the maxterms where  $F = 0$  are exact complements of the full universe of states:

$$\sum m(1, 4, 5, 7) \text{ equiv } \text{prod } M(0, 2, 3, 6)$$

Any function given in SOP can be instantly converted to POS by simply listing the **missing index numbers**!

# PART 2: Karnaugh Maps (K-Maps) and Logic Simplification

## 2.5 The Karnaugh Map (K-Map) Philosophy and Gray-Code Adjacency

Source: Kumar (Ch. 6, pp. 263–268); Mano (Ch. 3, pp. 99–107); Floyd (Ch. 4, pp. 195–215)

[Pacing: Conceptually Critical — The Core Tool for Minimization]

### Need (Problem-First)

Minimizing logic functions using Boolean algebra theorems by hand is like doing high-wire acrobatics without a safety net: 1. It is **non-systematic**: there is no fixed algorithm telling you which theorem to apply next. 2. It relies heavily on human pattern-recognition and intuition. 3. It is notoriously **error-prone**: missing a single prime or bar destroys the entire circuit.

We need a **foolproof graphical method** where simplification happens automatically through simple visual pattern grouping. That method is the **Karnaugh Map (K-map)**, invented by Maurice Karnaugh in 1953.

### The Core Mechanism: Gray Code Spatial Adjacency

A K-map is a diagram made up of squares, where each square represents one specific minterm (Kumar, p. 263).

The profound genius of the K-map lies in its coordinate labeling: **the rows and columns are arranged in Gray code sequence (00, 01, 11, 10)**, NEVER standard binary (00, 01, 10, 11)! - Why? In Gray code, **adjacent squares differ by only ONE bit variable**. - When two adjacent squares both contain a 1, they represent two minterms that are identical except for one variable (e.g.,  $\bar{A}B$  and  $AB$ ). - By the Boolean theorem  $\bar{A}B + AB = A(B + \bar{B}) = A(1) = A$ , combining two adjacent squares **physically eliminates the variable that changes**!

## 2.6 Two-Variable and Three-Variable K-Maps

### 1. Two-Variable K-Map (Kumar, pp. 265–267)

A 2-variable function  $F(A, B)$  has  $2^2 = 4$  minterms, arranged in a  $2 \times 2$  grid:

|   |   |                               |                         |
|---|---|-------------------------------|-------------------------|
|   |   | B                             |                         |
|   |   | 0                             | 1                       |
| A | 0 | $\bar{A}\bar{B}$ <sup>0</sup> | $\bar{A}B$ <sup>1</sup> |
|   | 1 | $A\bar{B}$ <sup>2</sup>       | $AB$ <sup>3</sup>       |

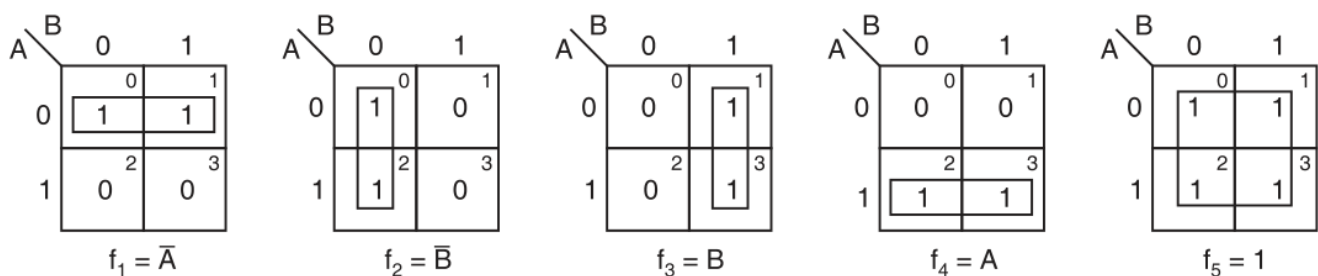
**Figure 6.1** The minterms of a two-variable K-map.

Figure

6.1: Format and minterm cell assignments for a two-variable Karnaugh Map. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p.

265 (printed p. 233).

- **Allowed Groupings in 2-Variable K-Map (Kumar, p. 266, Figure 6.4):**
- An isolated single cell ( $2^0 = 1$  cell): yields a 2-variable term.
- A pair of adjacent cells ( $2^1 = 2$  cells): eliminates 1 variable, yielding a 1-variable term.
- A block of four cells ( $2^2 = 4$  cells): eliminates 2 variables, yielding the constant logic 1.



**Figure 6.4** The possible minterm groupings in a two-variable K-map.

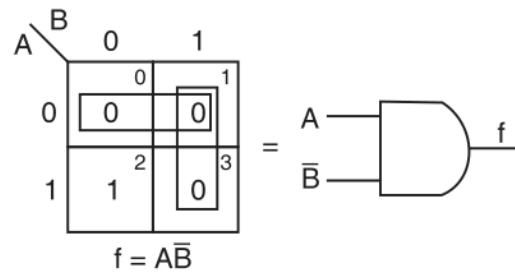
Figure

6.4: Possible minterm groupings (isolated cell, 2-cell pair, and 4-cell full map) in a two-variable K-map. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 266 (printed p. 234).

## 2. Three-Variable K-Map (Kumar, pp. 269–275)

A 3-variable function  $F(A, B, C)$  has  $2^3 = 8$  minterms, arranged in a  $2 \times 4$  grid (Kumar, Figure 6.10):

shown in the figure.



**Figure 6.9** Example 6.4: K-map in POS form and logic diagram.

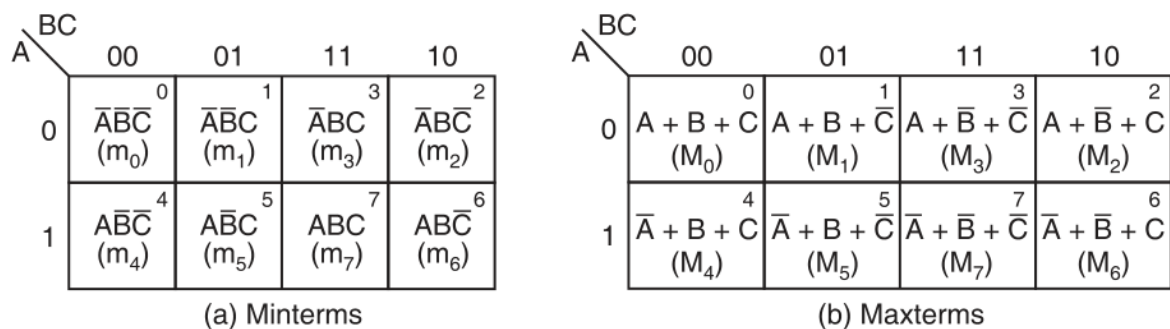
In the given expression, the maxterm  $M_2$  is absent. This is indicated by a 1 on the K-map. The corresponding SOP expression is  $\Sigma m_2$  or  $A\bar{B}$ . This realization is the same as that for POS form.

### 6.3 THREE-VARIABLE K-MAP

A function in three variables (A, B, C) expressed in the standard SOP form can have eight possible combinations:  $\bar{A}\bar{B}\bar{C}$ ,  $\bar{A}\bar{B}C$ ,  $\bar{A}B\bar{C}$ ,  $\bar{A}BC$ ,  $A\bar{B}\bar{C}$ ,  $A\bar{B}C$ ,  $AB\bar{C}$ , and  $ABC$ . Each one of these combinations designated by  $m_0, m_1, m_2, m_3, m_4, m_5, m_6$ , and  $m_7$ , respectively, is called a minterm. A is the MSB of the minterm designator and C is the LSB.

In the standard POS form, the eight possible combinations are:  $A + B + C$ ,  $A + B + \bar{C}$ ,  $A + \bar{B} + C$ ,  $A + \bar{B} + \bar{C}$ ,  $\bar{A} + B + C$ ,  $\bar{A} + B + \bar{C}$ ,  $\bar{A} + \bar{B} + C$ , and  $\bar{A} + \bar{B} + \bar{C}$ . Each one of these combinations designated by  $M_0, M_1, M_2, M_3, M_4, M_5, M_6$ , and  $M_7$ , respectively, is called a maxterm. A is the MSB of the maxterm designator and C is the LSB.

A three-variable K-map has, therefore,  $8 (= 2^3)$  squares or cells, and each square on the map represents a minterm or maxterm as shown in Figure 6.10a and Figure 6.10b. The small number on the top right corner of each cell indicates the minterm or maxterm designation.

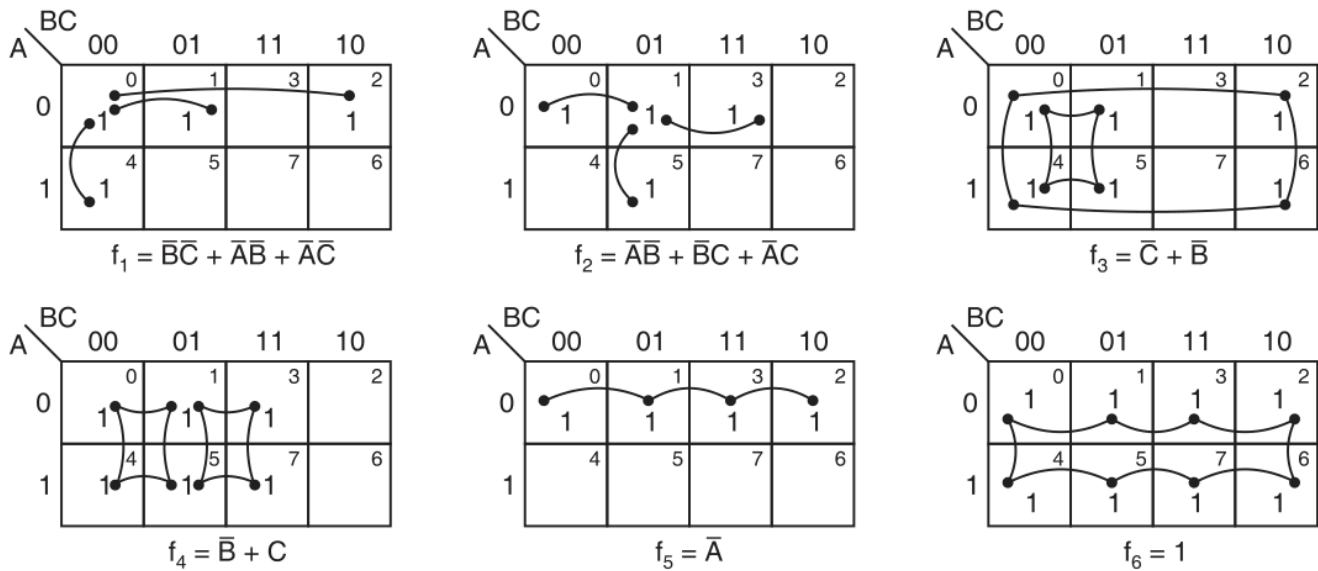


**Figure 6.10** The three-variable K-map.

Figure

**6.10:** Three-variable Karnaugh map structure and minterm cell numbering showing Gray code column sequence (00, 01, 11, 10). Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 269 (printed p. 237).

- **Notice the Column Reversal:** The column sequence is **00, 01, 11, 10**. Therefore, the cell numbers in the top row are **0, 1, 3, 2** (cells 3 and 2 are swapped!), and in the bottom row are **4, 5, 7, 6**! This is the most common place beginners make arithmetic slip-ups.
- **Topological Torus (Wrap-Around Adjacency):** The extreme left column (**00**) is physically adjacent to the extreme right column (**10**)! A grouping can roll over the outer edges:



**Figure 6.13** Some possible combinations of minterms in a three-variable K-map (SOP form).

Figure

6.13: Valid grouping patterns in a three-variable K-map (singletons, pairs, quads, and end-wrap combinations). Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 272 (printed p. 240).

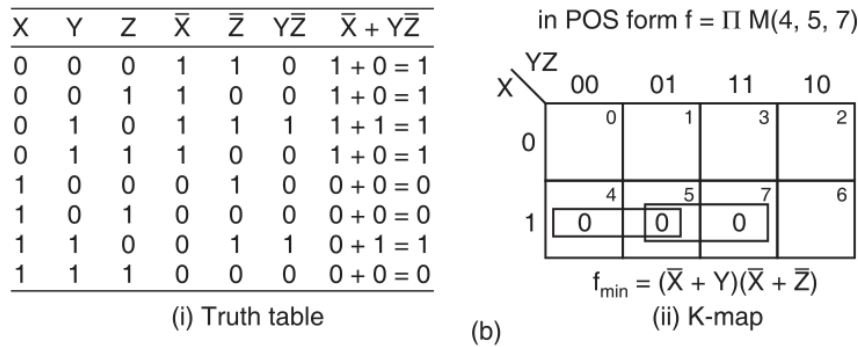
## 2.7 The Four-Variable K-Map (Up to 4 Variables)

Source: Kumar (Ch. 6, pp. 276–284); Mano (Ch. 3, pp. 107–112)

[Pacing: The Core Workhorse of Logic Minimization — Study Carefully]

### Structure and Geometry

A 4-variable function  $F(A, B, C, D)$  has  $2^4 = 16$  minterms, arranged in a symmetrical  $4 \times 4$  grid (Kumar, p. 276, Figure 6.19):

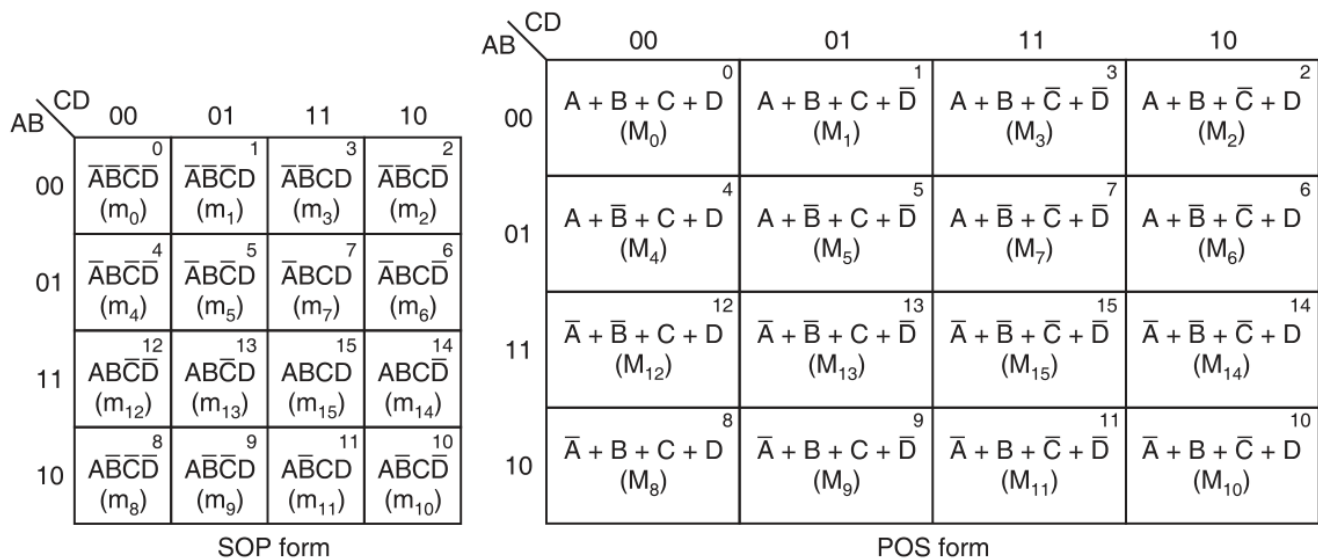


**Figure 6.18** Example 6.10.

## 6.4 FOUR-VARIABLE K-MAP

A four-variable (A, B, C, D) expression can have  $2^4 = 16$  possible combinations of input variables such as  $\bar{A}\bar{B}\bar{C}\bar{D}$ ,  $\bar{A}\bar{B}\bar{C}D$ , ...,  $ABCD$  with minterm designations  $m_0, m_1, \dots, m_{15}$ , respectively, in the SOP form and as  $A + B + C + D$ ,  $A + B + C + \bar{D}$ , ...,  $\bar{A} + \bar{B} + \bar{C} + \bar{D}$  with maxterm designations  $M_0, M_1, \dots, M_{15}$ , respectively, in the POS form. A four-variable K-map has  $2^4 = 16$  squares or cells and each square on the map represents either a minterm or a maxterm as shown in Figure 6.19.

The binary number designations of the rows and columns are in the Gray code. Here 11 follows 01 and 10 follows 11. This is called *adjacency ordering*. The binary numbers along the top of the map indicate the conditions of C and D along any column and binary numbers along the left side indicate the conditions of A and B along any row. The numbers in the top right corners of the squares indicate the minterm or maxterm designations as usual.



**Figure 6.19** The minterms and maxterms of a four-variable K-map.

Figure

**6.19:** Four-variable Karnaugh Map layout showing row and column Gray code sequences and corresponding minterm / maxterm decimal indices. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 276 (printed p. 244).

### The Universal 16-Cell Index Map:

|    |         |    |    |    |
|----|---------|----|----|----|
|    | CD      |    |    |    |
| AB | 00      | 01 | 11 | 10 |
| 00 | +-----+ |    |    |    |
|    | 0       | 1  | 3  | 2  |
| 01 | +-----+ |    |    |    |

|    |                           |    |  |    |  |    |  |    |  |                                |
|----|---------------------------|----|--|----|--|----|--|----|--|--------------------------------|
|    |                           | 4  |  | 5  |  | 7  |  | 6  |  |                                |
| 11 | +-----+-----+-----+-----+ |    |  |    |  |    |  |    |  |                                |
|    |                           | 12 |  | 13 |  | 15 |  | 14 |  | <-- Row 11 is row index 12-15! |
| 10 | +-----+-----+-----+-----+ |    |  |    |  |    |  |    |  |                                |
|    |                           | 8  |  | 9  |  | 11 |  | 10 |  | <-- Row 10 is row index 8-11!  |
|    | +-----+-----+-----+-----+ |    |  |    |  |    |  |    |  |                                |

**Crucial Double Swap Warning:** Notice that both the 3rd column (**CD=11**) and the 3rd row (**AB=11**) are swapped with their 4th counterparts to preserve Gray-code single-bit transitions!

### The Rigorous Step-by-Step K-Map Minimization Algorithm:

1. **Plot the Function:** Place a 1 in every cell corresponding to a minterm of the function. Leave all other cells as 0 (or blank).
2. **Identify Isolated Cells:** Any 1 that has no adjacent 1s cannot be combined. It must be circled as a 1-cell group ( $2^0$ ).
3. **Form the Largest Possible Groups (Powers of Two Only):**
4. **Group of 16 ( $2^4$ ):** Covers entire map ■ **F = 1** (eliminates 4 variables).
5. **Group of 8 (Octet,  $2^3$ ):** Eliminates 3 variables, leaving a 1-variable term.
6. **Group of 4 (Quad,  $2^2$ ):** Eliminates 2 variables, leaving a 2-variable term.
7. **Group of 2 (Pair,  $2^1$ ):** Eliminates 1 variable, leaving a 3-variable term.
8. **Wrap-Around Adjacencies (The 3D Cylindrical Surface):**
9. The top row (**AB=00**) is adjacent to the bottom row (**AB=10**).
10. The left column (**CD=00**) is adjacent to the right column (**CD=10**).
11. **The Four Corners:** Cells (**0, 2, 8, 10**) form a single valid **Quad** ( $2^2 = 4$  cells)!
12. **Eliminate Redundant Groups:** Every 1 must be covered at least once. If a group contains *only* 1s that are already covered by other necessary groups, that group is completely redundant and must be erased!

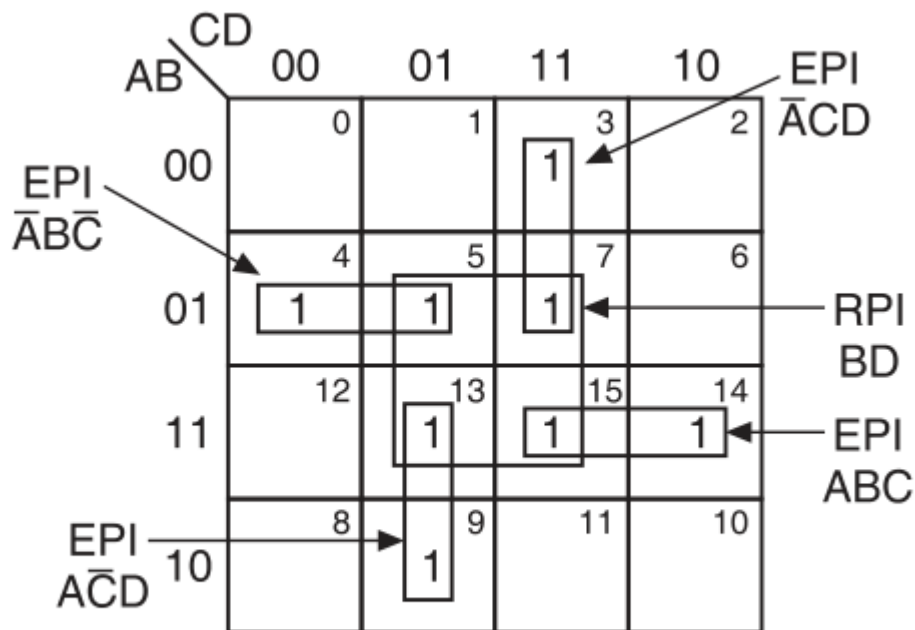
## 2.8 Prime Implicants and Essential Prime Implicants

Source: Kumar (Ch. 6, pp. 280–283); Mano (Ch. 3, pp. 109–112)

[Pacing: Conceptually Rigorous — Key for Formal Engineering Exams]

### Definitions Defined Inline:

- **Implicant:** Any single minterm or valid grouping of minterms (of size **1, 2, 4, 8 dots**) for which the output is **1**.
- **Prime Implicant (PI):** A rectangle of **1s** (size  $2^k$ ) that **cannot be merged into any larger rectangle** (Kumar, p. 281). It is maximal.
- **Essential Prime Implicant (EPI):** A Prime Implicant that contains **at least one 1 that is not covered by any other prime implicant** (Kumar, p. 281). **Every EPI must appear in the final minimal expression!**
- **Redundant Prime Implicant:** A prime implicant whose **1s** are all completely covered by essential prime implicants. It must be discarded.



**Figure 6.24** Essential and redundant prime implicants.

Figure

6.24: Distinguishing Essential Prime Implicants from Redundant Prime Implicants on a Karnaugh map. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 281 (printed p. 249).

#### Concrete Fully-Worked 4-Variable Example

Minimize:  $F(A, B, C, D) = \sum m(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$  - **Step 1 (Plotting)**: Mark cells 0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14 with 1. - **Step 2 (Identify Groups)**: - Look at columns 1 & 2 (where  $CD = 00$  and  $01$ ): cells (0, 1, 4, 5, 12, 13, 8, 9) form a massive **Octet (8 cells)** spanning all four rows! - Across these 8 cells:  $A$  varies (0, 1),  $B$  varies (0, 1),  $D$  varies (0, 1). The only variable that remains constant is  $C = 0$ . - This octet reduces to the single literal:  $C$ . - Next, look at the four corners: cells (0, 2, 8, dots) but wait! Look at the top two rows ( $AB=00$  and  $01$ ): cells (0, 1, 2, 4, 5, 6) and cells (4, 5, 6, 12, 13, 14). - Cells (0, 2, 4, 6) form a Quad: row  $A=0$ , and columns  $CD=00, 10$  ( $D=0$ ). Term:  $A D$ . - Cells (4, 6, 12, 14) form a Quad: row  $B=1$ , and columns  $CD=00, 10$  ( $D=0$ ). Term:  $BD$ . - **Final Minimal SOP Expression**:

$$F = C + A D + B D$$

Notice that an original expression containing 11 four-variable terms (requiring dozens of gates) collapsed into just three tiny 2-literal terms!

## 2.9 Don't Care Conditions (d or x)

Source: Kumar (Ch. 6, pp. 278–280); Mano (Ch. 3, pp. 116–119)

[Pacing: Conceptually Heavy — A Hardware Designer's Best Friend]

### Need (Problem-First)

In many real systems, certain input combinations **can never physically occur**. - *Concrete Example*: A circuit processes BCD digits (0 through 9, binary 0000 to 1001). What happens if the inputs are 1010 (10\_{10}) through 1111 (15\_{10})? In a legitimate BCD system, the upstream hardware will *never* produce those codes. - Because those input combinations will never happen in real life, **the designer does not care whether the circuit outputs a 0 or a 1 for those rows**.

### The Strategic Grouping Rule for Don't Cares (d or x)

A **Don't Care** condition (written as  $d$  or  $x$  in the K-map) is a wildcard: - **Rule 1**: You may treat an  $x$  as a 1 **if and only if it helps you form a larger grouping** (e.g., expanding a pair into a quad, or a quad into an octet). A larger group eliminates more variables, making the circuit cheaper. - **Rule 2**: You treat an  $x$  as a 0 (leave it un-circled) **if it does not help enlarge any group of 1s**. You are **never required to cover an x**! Only true 1s are mandatory to cover.

Concrete Worked Example: Minimize:  $F(A, B, C, D) = \sum m(1, 3, 7, 11, 15) + \sum d(0, 2, 5)$  - Analysis: - Mandatory 1s are at cells 1, 3, 7, 11, 15. - Don't cares (X) are at cells 0, 2, 5. - Without don't cares, cells (3, 7, 11, 15) form a vertical Quad (term: CD). Cell 1 would have to pair with 3 (term: A BD). - With Don't Cares: By including X\_5, cells (1, 3, 5, 7) form a Quad in the upper half! Even better: cells (0, 1, 2, 3) in the top row are all 1s or Xs! By treating X\_0, X\_2, X\_5 strategically, we group cells (1, 3, 5, 7) with (9, 11, 13, 15) if present, or group (3, 7, 11, 15) as a Quad (CD) and group (1, 3, 5, 7) as a Quad (AD). - Minimized Result:  $F = CD + AD = D(C + A)$ .

## Unit II Understanding Checkpoints & Misconception Audit

### 1. Does This Make Sense? Self-Check

- **Question 1:** Why is a group of three adjacent 1s (3 cells) illegal in a Karnaugh map?
- **Answer:** Because grouping relies on the Boolean theorem  $X + X = 1$ , which only eliminates a variable when powers of two are factored ( $2^1 = 2$ ,  $2^2 = 4$ ,  $2^3 = 8$ ). A group of 3 cannot factor out a single variable cleanly.
- **Question 2:** If all 16 cells of a 4-variable K-map contain 1s, what is the minimized Boolean function?
- **Answer:**  $F = 1$  (the output is permanently HIGH, requiring zero logic gates).
- **Question 3:** Are you required to draw circles around every Don't Care (X) on a K-map?
- **Answer:** Absolutely not! You only include an X if it helps expand a group containing genuine 1s. An X left alone simply defaults to a 0.

### 2. Conceptual Misconceptions & Arithmetic Slip-Ups Corrected

- **Conceptual Misconception:** "A Karnaugh map can minimize any function with any number of variables."  
**Correction:** K-maps are human graphical tools effective only up to **4 variables** (or awkwardly 5 to 6 variables using stacked 3D layers). For 7 or more variables, visualization collapses and industry uses tabular computer algorithms like the **Quine-McCluskey method** or **Espresso**.
- **Arithmetic / Notation Slip-Up:** When writing the K-map grid, students constantly label rows or columns as 00, 01, 10, 11. **This is completely wrong!** The order must be Gray-coded: 00, 01, 11, 10. Swapping columns 11 and 10 destroys the geometric adjacency and corrupts all resulting terms.

## Unit III: Combinational Digital Circuits

### Master Subject Map: Unit III Position

|                                                                                   |   |
|-----------------------------------------------------------------------------------|---|
| [ UNIT II: BOOLEAN ALGEBRA & K-MAP MINIMIZATION ]                                 |   |
|                                                                                   | ↓ |
|                                                                                   | v |
| +-----+                                                                           |   |
| YOU ARE HERE ---> UNIT III: COMBINATIONAL DIGITAL CIRCUITS                        |   |
| • Arithmetic Blocks: Half/Full Adders, Half/Full Subtractors, 4-bit Parallel, CLA |   |
| • Data Converters: Binary-to-Gray, Gray-to-Binary, BCD-to-Excess-3                |   |
| • Data Routing & Selection: Multiplexers (2:1, 4:1), Demultiplexers (1:4)         |   |
| • Code & Display Decoders: 3:8 Decoders (74138), Priority Encoders (74148)        |   |
| • Visual Human Interfaces: BCD-to-7-Segment Decoders (7447/7448), Common A/C LED  |   |
| +-----+                                                                           |   |
|                                                                                   | v |
| [ UNIT IV: SEQUENTIAL CIRCUITS & STATE MACHINES ]                                 |   |

In 3 lines: In Unit II, we mastered logic minimization on paper. Unit III combines those minimized gates into practical computing blocks called **Combinational Circuits**. These memoryless circuits perform instantaneous calculations (Adders/Subtractors), route data streams (Multiplexers), and translate binary code into human-readable symbols (Seven-Segment Displays).

### Prerequisites for this Unit

Before studying Unit III, ensure you recall these concepts: 1. **Universal & Exclusive Gates (Unit I, Sections 1.5–1.7):** XOR ( $A \oplus B = AB + \overline{A}B$ ), AND ( $AB$ ), OR ( $A+B$ ), and Inverters. 2. **K-Map Simplification (Unit II, Section 2.7):** Grouping cells to produce

minimal Sum-of-Products equations. 3. **Binary Arithmetic & 2's Complement Subtraction (Unit I, Sections 1.2–1.3):** Converting subtraction into controlled addition. 4. **Gray Code & BCD Rules (Unit I, Section 1.4):** Single-bit transitions and 4-bit decimal representations.

### 3.1 Combinational Logic: Definition and Formal Design Procedure

Source: Kumar (Ch. 7, pp. 357–358); Mano (Ch. 4, pp. 164–172)  
[Pacing: Methodical Framework — Learn the Standard Design Steps]

#### Need (Problem-First)

How do professional engineers transform a fuzzy English problem statement (e.g., "design a voting machine for a 3-member committee where the majority wins") into a functional electronic schematic? We need a universal, repeatable step-by-step design procedure.

#### Physical Definition of Combinational Logic

A **Combinational Circuit** is a connected network of logic gates whose output voltages at any given instant of time depend **strictly and solely on the input voltages applied at that exact same instant** (Kumar, p. 357).  
- It has **no feedback loops** (outputs are never fed back to inputs). - It has **no memory elements** (it cannot remember what happened one microsecond ago). - It is entirely governed by combinational Boolean functions:  $Y_j = f_j(I_1, I_2, \text{dots}, I_n)$ .

#### The Standard 5-Step Design Procedure (Kumar, p. 358; Mano, p. 170):

- Problem Formulation:** State the circuit's behavior clearly in words, identifying all required input and output variables.
- Variable Assignment:** Assign algebraic letter symbols (e.g., **A, B, C** for inputs; **X, Y, S** for outputs).
- Truth Table Construction:** Exhaustively list all  $2^n$  input combinations and define the required output for each row.
- Logic Minimization:** Plot each output variable onto a Karnaugh Map and derive the minimal Boolean expression (in SOP or POS).
- Logic Schematic Realization:** Draw the circuit diagram using standard gate symbols, or convert to universal NAND/NOR logic for fabrication.

## PART 1: Arithmetic Building Blocks (Adders, Subtractors, and Lookahead Carry)

### 3.2 The Half Adder and Full Adder

Source: Kumar (Ch. 7, pp. 358–363); Mano (Ch. 4, pp. 173–178); Floyd (Ch. 5, pp. 248–256)  
[Pacing: Fundamental Arithmetic — Study Every Derivation Step]

#### Need (Problem-First)

All computing—from calculating 3D graphics in video games to processing banking transactions—boils down to basic binary addition. Without a circuit that can add two electrical bits and handle carries, modern computers cannot exist.

#### 1. The Half Adder (Kumar, pp. 358–360)

- Definition:** A combinational circuit that adds **two single binary bits (A and B)**, producing two outputs: a **Sum bit (S)** and a **Carry bit (C)**. It is called a "half" adder because it has *no provision to accept a carry arriving from a previous lower stage*.
- Truth Table (Kumar, p. 358):**

| Input A | Input B | Sum (S) | Carry (C) | Arithmetic Meaning |
|---------|---------|---------|-----------|--------------------|
| 0       | 0       | 0       | 0         | $0 + 0 = 0$        |
| 0       | 1       | 1       | 0         | $0 + 1 = 1$        |
| 1       | 0       | 1       | 0         | $1 + 0 = 1$        |

| Input A | Input B | Sum (S) | Carry (C) | Arithmetic Meaning              |
|---------|---------|---------|-----------|---------------------------------|
| 1       | 1       | 0       | 1         | $1 + 1 = 10_2$ (Sum 0, Carry 1) |

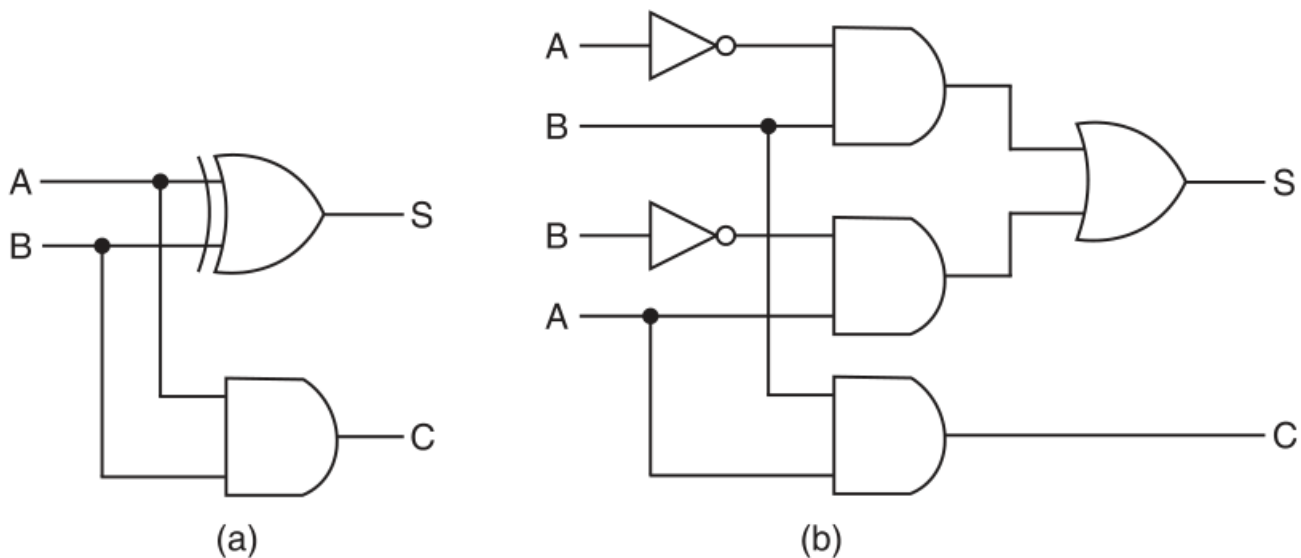
- **Boolean Equations:**
- Notice the Sum column: **S** is **1** only when inputs are different ■ **XOR Gate:**

$$S = AB + \overline{A}B + A\overline{B} = A \oplus B$$

- Notice the Carry column: **C** is **1** only when both inputs are **1** ■ **AND Gate:**

$$C = AB$$

- **Authentic Diagram (Kumar, p. 359, Figure 7.3):**



**Figure 7.3** Logic diagrams of half-adder.

Figure

7.3: Half-adder circuit implementations: (a) using basic AND, OR, NOT gates; (b) using an XOR gate and an AND gate. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 359 (printed p. 327).

## 2. The Full Adder (Kumar, pp. 360–363)

- **Need:** When adding multi-bit binary numbers (e.g.,  $1101 + 1011$ ), each column after the LSB must add **three bits simultaneously**: the augend bit (**A**), the addend bit (**B**), and the **incoming carry** (**C<sub>in</sub>**) generated by the previous column! A half adder cannot do this. We need a **Full Adder**.
- **Truth Table (Kumar, p. 360):**

| A | B | C <sub>in</sub> | Sum (S) | Carry Out (C <sub>out</sub> ) | Arithmetic Sum (A + B + C <sub>in</sub> ) |
|---|---|-----------------|---------|-------------------------------|-------------------------------------------|
| 0 | 0 | 0               | 0       | 0                             | 0                                         |
| 0 | 0 | 1               | 1       | 0                             | 1                                         |
| 0 | 1 | 0               | 1       | 0                             | 1                                         |
| 0 | 1 | 1               | 0       | 1                             | $2 = 10_2$                                |

| A | B | C <sub>{in}</sub> | Sum (S) | Carry Out (C <sub>{out}</sub> ) | Arithmetic Sum (A + B + C <sub>{in}</sub> ) |
|---|---|-------------------|---------|---------------------------------|---------------------------------------------|
| 1 | 0 | 0                 | 1       | 0                               | 1                                           |
| 1 | 0 | 1                 | 0       | 1                               | 2 = 10 <sub>2</sub>                         |
| 1 | 1 | 0                 | 0       | 1                               | 2 = 10 <sub>2</sub>                         |
| 1 | 1 | 1                 | 1       | 1                               | 3 = 11 <sub>2</sub>                         |

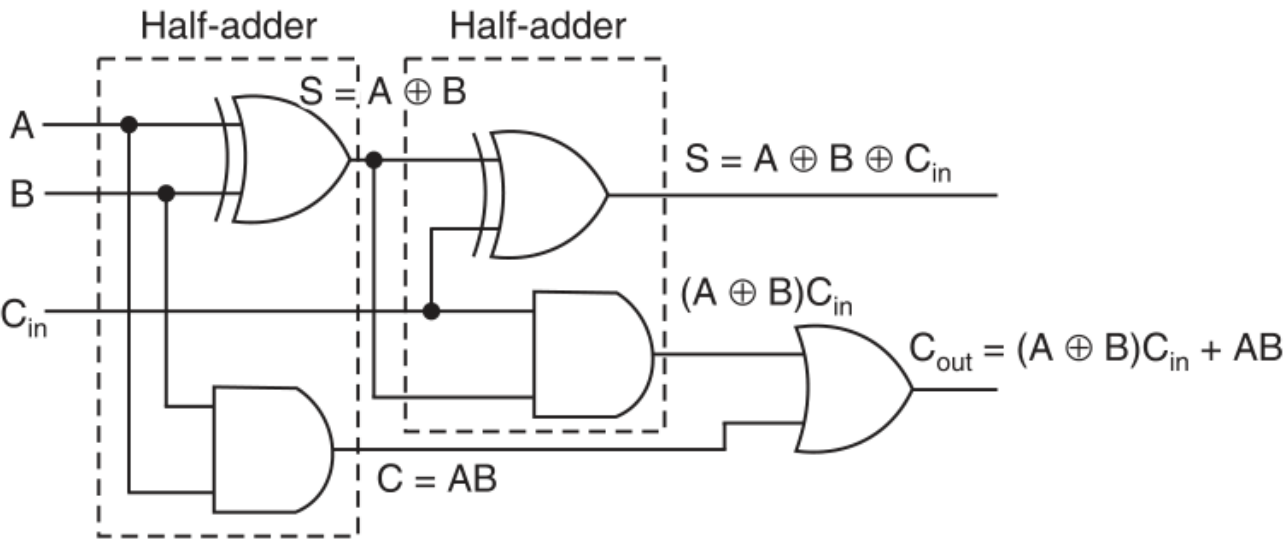
- **Derivation of Boolean Expressions:**
- **Sum (S):** S is 1 whenever an **odd number of inputs are 1** (parity function):

$$S = A \overline{BC_{in}} + A\overline{B}C_{in} + AB\overline{C_{in}} + ABC_{in} = A \oplus B \oplus C_{in}$$

- **Carry Out (C<sub>{out}</sub>):** C<sub>{out}</sub> is 1 whenever **two or more inputs are 1**:

$$C_{out} = AB + BC_{in} + AC_{in} = AB + C_{in}(A \oplus B)$$

- **Realization Using Two Half Adders (Kumar, p. 361, Figure 7.7):**  
A Full Adder can be constructed by cascading two Half Adders and one OR gate:
- Half Adder 1 adds A and B, generating partial sum S<sub>1</sub> = A ⊕ B and carry C<sub>1</sub> = AB.
- Half Adder 2 adds S<sub>1</sub> and C<sub>{in}</sub>, generating final sum S = (A ⊕ B) ⊕ C<sub>{in}</sub> and carry C<sub>2</sub> = (A ⊕ B)C<sub>{in}</sub>.
- The final carry out is formed by ORing the two partial carries: C<sub>{out}</sub> = C<sub>1</sub> + C<sub>2</sub> = AB + C<sub>{in}</sub>(A ⊕ B).



**Figure 7.7** Logic diagram of a full-adder using two half-adders.

Figure

7.7: Construction of a Full Adder using two cascaded Half Adders and an OR gate. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 361 (printed p. 329).

### 3.3 The Half Subtractor and Full Subtractor

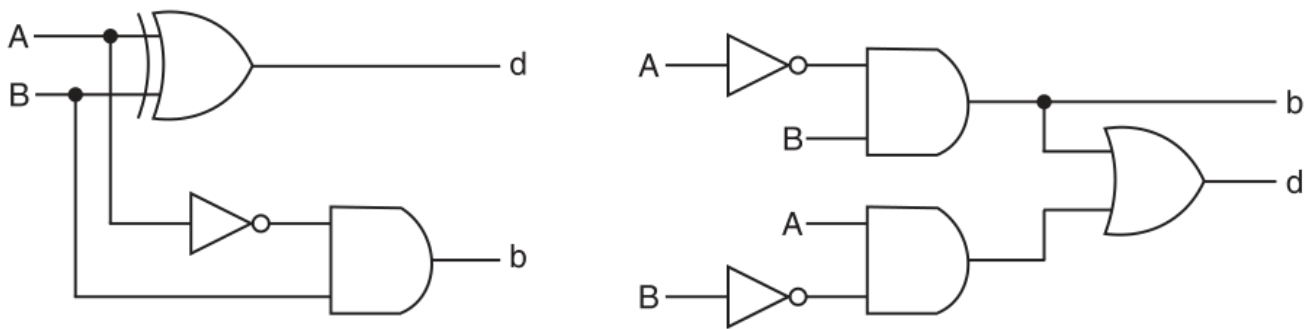
Source: Kumar (Ch. 7, pp. 363–367); Mano (Ch. 4, pp. 178–181)

[Pacing: Parallel Structure — Compare Direct Subtractors to Adders]

#### 1. The Half Subtractor (Kumar, pp. 363–365)

- **Definition:** A circuit that subtracts bit B from bit A, producing a **Difference (D)** and a **Borrow out (B<sub>{out}</sub>)**.
- **Truth Table & Equations:**

- $D = AB + \overline{A}B = A \oplus B$  (Notice: Difference equation is identical to Half Adder Sum!)
- $B_{out} = AB$  (Borrow is needed only when subtracting 1 from 0:  $0 - 1 = 1$  with borrow 1).



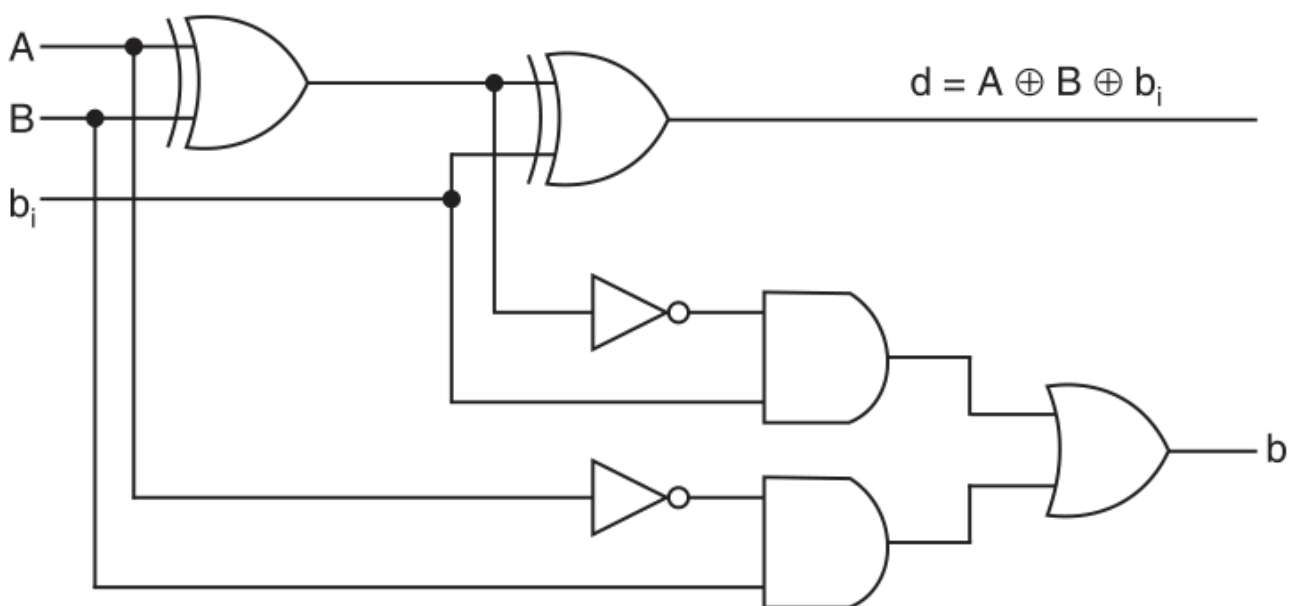
**Figure 7.13** Logic diagrams of a half-subtractor.

Figure

7.13: Logic diagrams of a Half Subtractor using basic gates and XOR-AND configuration. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 364 (printed p. 332).

## 2. The Full Subtractor (Kumar, pp. 365–367)

- **Definition:** Subtracts bits **B** and incoming borrow **B<sub>in</sub>** from bit **A** ( $A - B - B_{in}$ ).
- **Boolean Equations:**
- $D = A \oplus B \oplus B_{in}$
- $B_{out} = AB + AB_{in} + BB_{in} = AB + B_{in}(A \oplus B)$



**Figure 7.17** Logic diagram of a full-subtractor.

Figure

7.17: Logic diagram of a Full Subtractor constructed using two Half Subtractors and an OR gate. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 366 (printed p. 334).

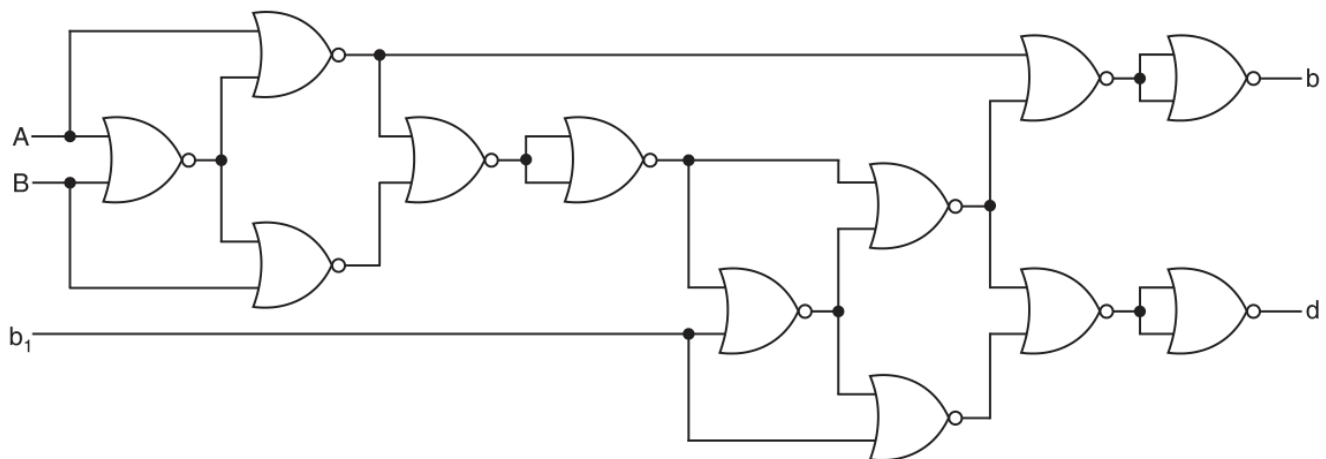
## 3.4 Multi-Bit Adders and the 4-Bit Parallel Adder/Subtractor

Source: Kumar (Ch. 7, pp. 367–373); Mano (Ch. 4, pp. 173–178); Floyd (Ch. 5, pp. 250–256)

[Pacing: Hardware Realization — High Practical Value]

### 1. The 4-Bit Binary Parallel Adder (Ripple Carry Adder) (Kumar, p. 367)

To add two 4-bit binary numbers  $A = A_3 A_2 A_1 A_0$  and  $B = B_3 B_2 B_1 B_0$ , four Full Adders (FA) are chained together in parallel (Kumar, Figure 7.20):



**Figure 7.19** Logic diagram of a full subtractor using only 2-input NOR gates.

## 7.5 BINARY PARALLEL ADDER

A binary parallel adder is a digital circuit that adds two binary numbers in parallel form and produces the arithmetic sum of those numbers in parallel form. It consists of full adders connected in a chain, with the output carry from each full-adder connected to the input carry of the next full-adder in the chain.

Figure 7.20 shows the interconnection of four full-adder (FA) circuits to provide a 4-bit parallel adder. The augend bits of A and addend bits of B are designated by subscript numbers from right to left, with subscript 1 denoting the lower-order bit. The carries are connected in a chain through the full-adders. The input carry to the adder is  $C_{in}$  and the output carry is  $C_4$ . The S outputs generate the required sum bits. When the 4-bit full-adder circuit is enclosed within an IC package, it has four terminals for the augend bits, four terminals for the addend bits, four terminals

Figure

7.20: Four-bit parallel binary adder (Ripple Carry Adder) chaining four Full Adders together. Sourced from A. Anand Kumar, Fundamentals of Digital

Circuits (4th Ed.), p. 367 (printed p. 335).

- Stage 0 adds **A<sub>0</sub>**, **B<sub>0</sub>** and input carry **C<sub>0</sub>**, producing Sum **S<sub>0</sub>** and carry **C<sub>1</sub>**.
- Carry **C<sub>1</sub>** ripples into Stage 1; carry **C<sub>2</sub>** ripples into Stage 2; carry **C<sub>3</sub>** ripples into Stage 3.
- Standard IC: **7483 / 74LS283** (4-bit binary full adder).

### 2. The Combined 4-Bit Parallel Adder/Subtractor (Kumar, p. 369, Figure 7.22)

Why build a separate adder and a separate subtractor chip when you can make a single circuit perform both?

Recall from Unit I that:

$$A - B = A + (2\text{'s complement of } B) = A + B + 1$$

We can achieve this mathematically and physically with a single control line labeled **M (Mode / ADD/overLine{SUB})** and four **XOR gates**:

Figure 7.22 shows a 4-bit adder-subtractor circuit. Here the addition and subtraction operations are combined into one circuit with one common binary adder. This is done by including an X-OR gate with each full-adder. The mode input  $M$  controls the operation. When  $M = 0$ , the circuit is an adder, and when  $M = 1$ , the circuit becomes a subtractor. Each X-OR gate receives input  $M$  and one of the inputs of  $B$ . When  $M = 0$ , we have  $B \oplus 0 = B$ . The full-adder receives the value of  $B$ , the input carry is 0 and the circuit performs  $A + B$ . When  $M = 1$ , we have  $B \oplus 1 = \bar{B}$  and  $C_1 = 1$ . The  $B$  inputs are complemented and a 1 is added through the input carry. The circuit performs the operation  $A$  plus the 2's complement of  $B$ .

Figure

7.22: Complete 4-bit binary adder-subtractor utilizing XOR gates as programmable inverters and 2's complement carry injection. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 369 (printed p. 337).

### Characteristic 13 Walkthrough: Hardware Reality Check

- **When Mode  $M = 0$  (Addition Mode):**
  - Each XOR gate receives  $B_i$  and  $0$ . Because  $B_i \oplus 0 = B_i$ , the  $B$  inputs pass through completely **uninverted**.
  - The initial carry input  $C_0$  receives  $M = 0$ .
  - The circuit computes: **Output =  $A + B + 0 = A + B$ . It performs pure binary addition!**
- **When Mode  $M = 1$  (Subtraction Mode):**
  - Each XOR gate receives  $B_i$  and  $1$ . Because  $B_i \oplus 1 = \bar{B}_i$ , the XOR gates act as inverters, producing the **1's complement:  $\bar{B}$** !
  - Simultaneously, the initial carry input  $C_0$  receives  $M = 1$ , automatically injecting the required **+1** into the LSB!
  - The circuit computes: **Output =  $A + B + 1 = A + (-B) = A - B$ . It performs pure 2's complement subtraction!**
  - A single wire flips the entire calculating machine between addition and subtraction!

## 3.5 High-Speed Arithmetic: The Carry Lookahead Adder (CLA)

Source: Kumar (Ch. 7, pp. 369–373); Mano (Ch. 4, pp. 176–178)

[Pacing: Conceptually Heavy — The Fundamental Speed Bottleneck in Computing]

### Need (Problem-First: The Ripple Delay Bottleneck)

In the 4-bit ripple adder (Figure 7.20), Stage 3 **cannot calculate its sum bit  $S_3$  until Stage 2 finishes, which cannot finish until Stage 1 finishes, which waits on Stage 0!** - If each full adder has a gate propagation delay of  $t_{pd} = 5 \text{ ns}$ , a 4-bit adder takes  $4 \times 5 = 20 \text{ ns}$ . - But in a modern 64-bit microprocessor ALU, a 64-bit ripple adder would take  $64 \times 5 \text{ ns} = 320 \text{ ns}$ ! The processor could not exceed a dismal clock speed of **3 MHz**! - We need a circuit where **all carries are calculated simultaneously in parallel** without waiting for previous stages to ripple. This is the **Carry Lookahead Adder (CLA)**.

### The Mathematical Formulation: Generate and Propagate

For any adder stage  $i$ , define two independent functions (Kumar, p. 370): 1. **Carry Generate ( $G_i$ ):**

$$G_i = A_i \cdot B_i$$

*Physical Meaning:* If both  $A_i$  and  $B_i$  are **1**, this stage **generates a carry internally**, regardless of whether an incoming carry arrived from earlier stages. 2. **Carry Propagate ( $P_i$ ):**

$$P_i = A_i \oplus B_i$$

*Physical Meaning:* If either  $A_i$  or  $B_i$  is **1**, an incoming carry  $C_i$  will be **propagated straight through** to become an outgoing carry  $C_{i+1}$ .

The carry output of any stage can thus be written as:

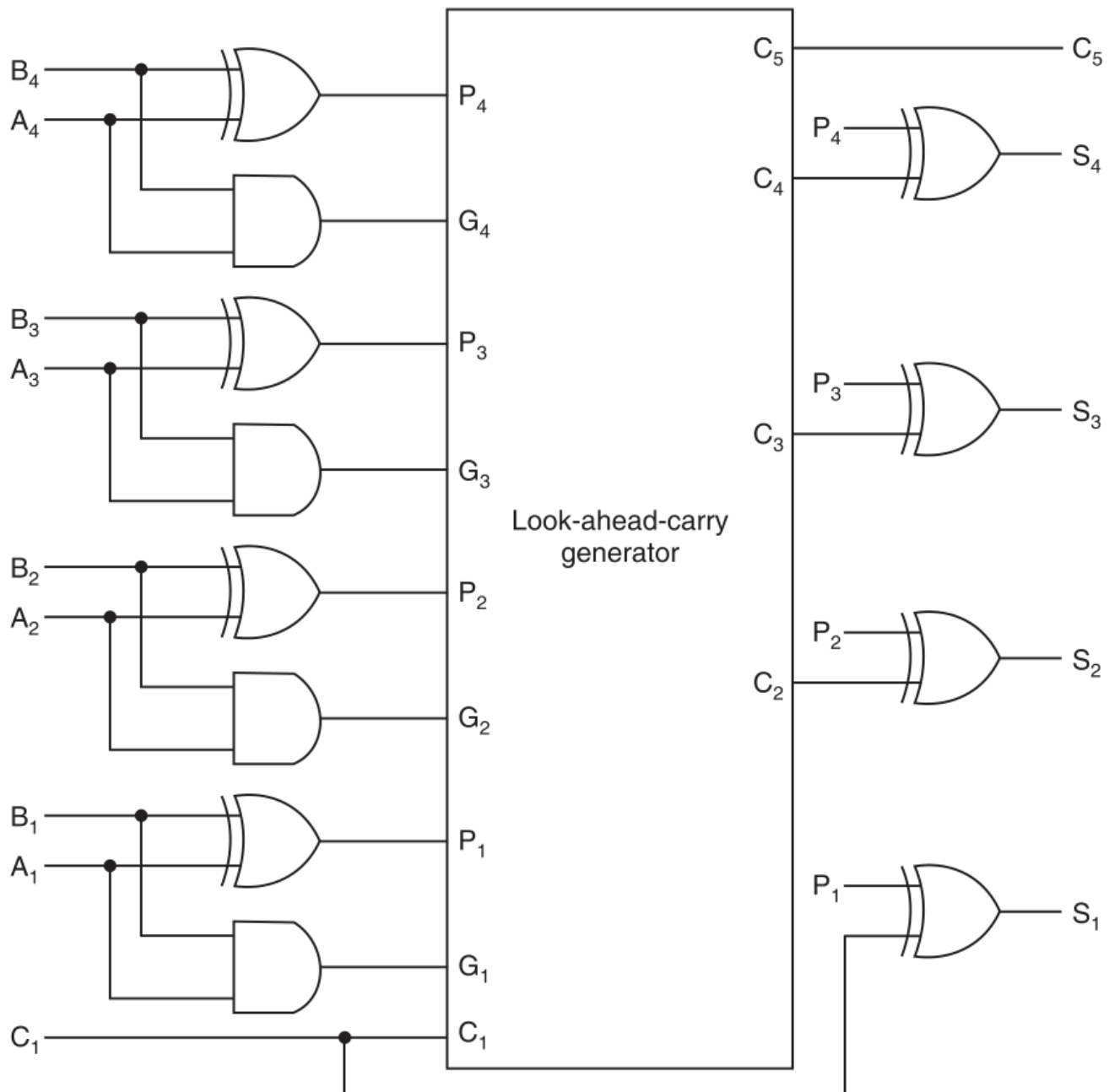
$$C_{i+1} = G_i + P_i C_i$$

### Unrolling the Carries Directly:

Now, expand each carry stage by algebraic substitution without waiting for ripples: -  $C_1 = G_0 + P_0 C_0$  -  $C_2 = G_1 + P_1 C_1 = \mathbf{G_1 + P_1 G_0 + P_1 P_0 C_0}$  -  $C_3 = G_2 + P_2 C_2 = \mathbf{G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0}$  -  $C_4 = G_3 + P_3 C_3 = \mathbf{G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0}$

Look at those equations! Notice that  $C_4$  depends ONLY on the initial input carry  $C_0$  and the immediate input bits  $A_i$ ,  $B_i$  via  $P_i$  and  $G_i$ !

Every single carry bit ( $C_1$ ,  $C_2$ ,  $C_3$ ,  $C_4$ ) is generated through a two-level AND-OR gate network simultaneously in parallel!



**Figure 7.24** Logic diagram of a 4-bit look-ahead-carry adder.

Figure

7.24: Logic diagram of a 4-bit Carry Lookahead Adder (CLA) eliminating serial ripple delays. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 371 (printed p. 339).

- **Speed Comparison:** Regardless of whether the adder is 4-bit, 16-bit, or 64-bit, the lookahead carry generator calculates all carries in just **two gate delays** ( $\approx 1$  to  $2$  ns), boosting computation speed by over **1000%**!

## PART 2: Code Converters and Data Selectors (Multiplexers / Demultiplexers)

### 3.6 Hardware Code Converters: Binary, Gray, and Excess-3

Source: Kumar (Ch. 7, pp. 382–388); Mano (Ch. 4, pp. 171–173)

[Pacing: Systematic Design — Practice the 5-Step Method]

#### 1. 4-Bit Binary-to-Gray Code Converter (Kumar, p. 383)

Following the mathematical rule derived in Unit I ( $G_3 = B_3$ ;  $G_2 = B_3 \oplus B_2$ ;  $G_1 = B_2 \oplus B_1$ ;  $G_0 = B_1 \oplus B_0$ ), the physical hardware requires exactly **three XOR gates**:

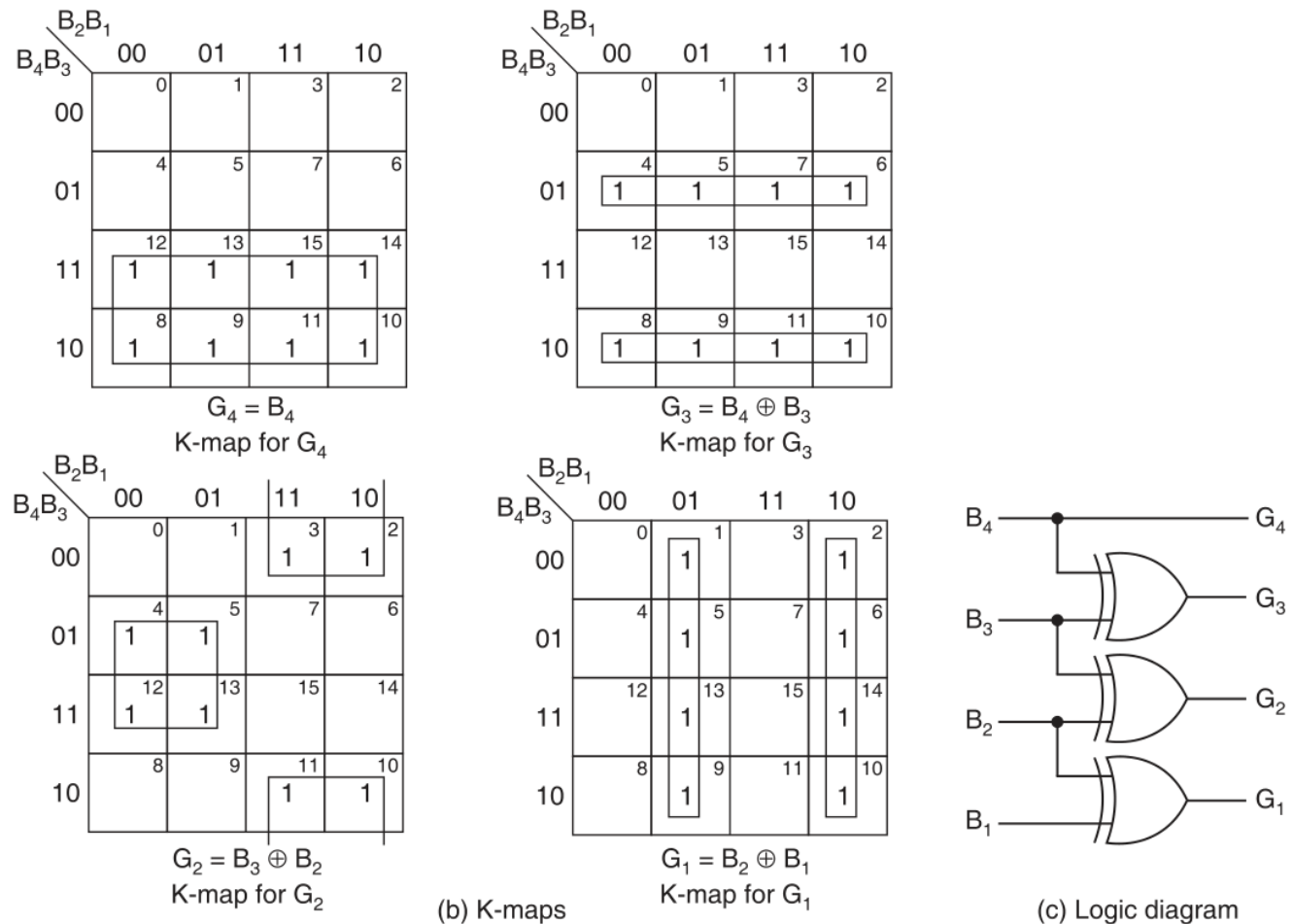


Figure 7.32 4-bit binary-to-Gray code converter.

Figure

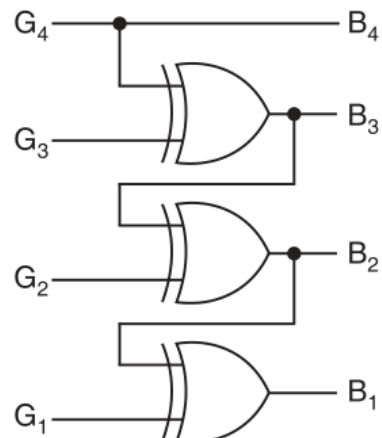
7.32: Logic diagram of a 4-bit Binary-to-Gray code converter using XOR gates. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 383 (printed p. 351).

#### 2. 4-Bit Gray-to-Binary Code Converter (Kumar, p. 384)

Following the inverse rule ( $B_3 = G_3$ ;  $B_2 = B_3 \oplus G_2$ ;  $B_1 = B_2 \oplus G_1$ ;  $B_0 = B_1 \oplus G_0$ ), the circuit cascades three XOR gates, feeding each computed binary bit forward into the next lower stage:

| 4-bit Gray |       |       |       | 4-bit binary |       |       |       |
|------------|-------|-------|-------|--------------|-------|-------|-------|
| $G_4$      | $G_3$ | $G_2$ | $G_1$ | $B_4$        | $B_3$ | $B_2$ | $B_1$ |
| 0          | 0     | 0     | 0     | 0            | 0     | 0     | 0     |
| 0          | 0     | 0     | 1     | 0            | 0     | 0     | 1     |
| 0          | 0     | 1     | 1     | 0            | 0     | 1     | 0     |
| 0          | 0     | 1     | 0     | 0            | 0     | 1     | 1     |
| 0          | 1     | 1     | 0     | 0            | 1     | 0     | 0     |
| 0          | 1     | 1     | 1     | 0            | 1     | 0     | 1     |
| 0          | 1     | 0     | 1     | 0            | 1     | 1     | 0     |
| 0          | 1     | 0     | 0     | 0            | 1     | 1     | 1     |
| 1          | 1     | 0     | 0     | 1            | 0     | 0     | 0     |
| 1          | 1     | 0     | 1     | 1            | 0     | 0     | 1     |
| 1          | 1     | 1     | 1     | 1            | 0     | 1     | 0     |
| 1          | 1     | 1     | 0     | 1            | 0     | 1     | 1     |
| 1          | 0     | 1     | 0     | 1            | 1     | 0     | 0     |
| 1          | 0     | 1     | 1     | 1            | 1     | 0     | 1     |
| 1          | 0     | 0     | 1     | 1            | 1     | 1     | 0     |
| 1          | 0     | 0     | 0     | 1            | 1     | 1     | 1     |

(a) Conversion table



(c) Logic diagram

| $G_2G_1$ |    |    |    |    |    |
|----------|----|----|----|----|----|
|          |    | 00 | 01 | 11 | 10 |
| $G_4G_3$ | 00 | 0  | 1  | 3  | 2  |
|          | 01 | 4  | 5  | 7  | 6  |
|          | 11 | 12 | 13 | 15 | 14 |
|          | 10 | 8  | 9  | 11 | 10 |

$$B_4 = G_4$$

K-map for  $B_4$

| $G_2G_1$ |    |    |    |    |    |
|----------|----|----|----|----|----|
|          |    | 00 | 01 | 11 | 10 |
| $G_4G_3$ | 00 | 0  | 1  | 3  | 2  |
|          | 01 | 4  | 5  | 7  | 6  |
|          | 11 | 12 | 13 | 15 | 14 |
|          | 10 | 8  | 9  | 11 | 10 |

$$B_3 = G_4 \oplus G_3$$

K-map for  $B_3$

| $G_2G_1$ |    |    |    |    |    |
|----------|----|----|----|----|----|
|          |    | 00 | 01 | 11 | 10 |
| $G_4G_3$ | 00 | 0  | 1  | 3  | 2  |
|          | 01 | 4  | 5  | 7  | 6  |
|          | 11 | 12 | 13 | 15 | 14 |
|          | 10 | 8  | 9  | 11 | 10 |

$$B_2 = G_4 \oplus G_3 \oplus G_2$$

K-map for  $B_2$

| $G_2G_1$ |    |    |    |    |    |
|----------|----|----|----|----|----|
|          |    | 00 | 01 | 11 | 10 |
| $G_4G_3$ | 00 | 0  | 1  | 3  | 2  |
|          | 01 | 4  | 5  | 7  | 6  |
|          | 11 | 12 | 13 | 15 | 14 |
|          | 10 | 8  | 9  | 11 | 10 |

$$B_1 = G_4 \oplus G_3 \oplus G_2 \oplus G_1$$

K-map for  $B_1$

(b) K-maps

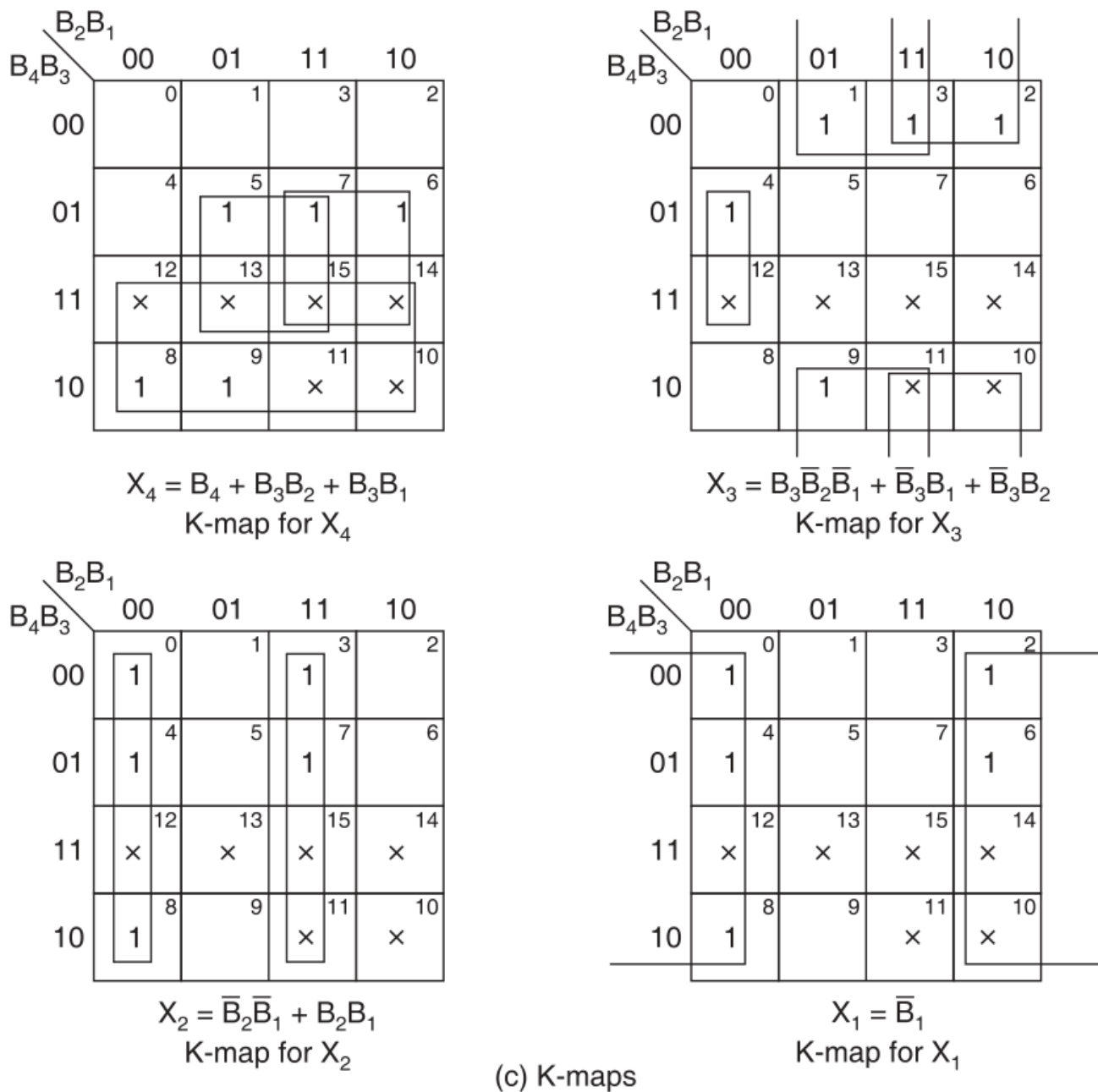
**Figure 7.33** 4-bit Gray-to-binary code converter.

Figure 7.33: Logic

diagram of a 4-bit Gray-to-Binary code converter. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 384 (printed p. 352).

### 3. BCD-to-Excess-3 Code Converter (Kumar, p. 387)

By setting up a truth table adding **0011\_2 (3)** to each valid BCD digit (**0000** to **1001**) and assigning don't cares (**x**) to illegal BCD rows **10** to **15**, four K-maps yield the optimized hardware equations:



**Figure 7.35** 4-bit BCD-to-XS-3 code converter.

Figure

7.35: Logic diagram of a 4-bit BCD-to-Excess-3 code converter derived via K-map minimization. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 387 (printed p. 355).

### 3.7 Multiplexers (Data Selectors) and Demultiplexers

Source: Kumar (Ch. 7, pp. 421–434); Mano (Ch. 4, pp. 199–206); Floyd (Ch. 5, pp. 268–276)

[Pacing: Core Digital Routing — High Examination & Practical Value]

#### Need (Problem-First)

Imagine a digital telephone exchange where four different callers want to send audio data across a single transatlantic cable wire. Running four separate undersea cables costs millions of dollars. How can multiple data sources share a single communication line without their signals tangling? We need an electronic rotary switch: a **Multiplexer (MUX)** at the transmitting end, and a **Demultiplexer (DEMUX)** at the receiving end.

1. The Multiplexer (MUX / Data Selector) Defined

A **Multiplexer** is a combinational circuit that selects binary information from one of many input data lines and directs it to a single output line (Kumar, p. 421). - **The Rule of Inputs:** A multiplexer with  $2^n$  data input lines requires  $n$  select control lines (S) to specify which input is routed to the output. - **2:1 MUX** ■  $2^1$  data inputs, 1 select line. - **4:1 MUX** ■  $2^2$  data inputs, 2 select lines (S<sub>1</sub>, S<sub>0</sub>). - **8:1 MUX** ■  $2^3$  data inputs, 3 select lines (S<sub>2</sub>, S<sub>1</sub>, S<sub>0</sub>).

The 2-Input Multiplexer (2:1 MUX) (Kumar, p. 422, Figure 7.76)

- Logic equation:  $Y = S D_0 + S D_1$ .
- When  $S = 0$ :  $Y = (1)D_0 + (0)D_1 = D_0$ . Output follows data line D<sub>0</sub>.
- When  $S = 1$ :  $Y = (0)D_0 + (1)D_1 = D_1$ . Output follows data line D<sub>1</sub>.

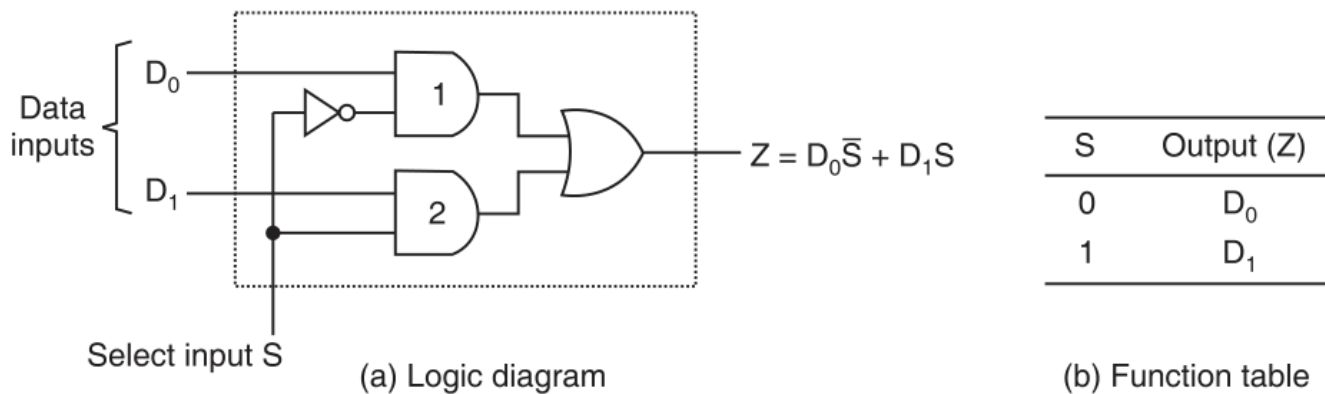


Figure 7.76 2-input multiplexer.

Figure

7.76: Logic circuitry and function table of a 2-input multiplexer. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 422 (printed p. 390).

The 4-Input Multiplexer (4:1 MUX) (Kumar, p. 422, Figure 7.77)

- Four data inputs (D<sub>0</sub>, D<sub>1</sub>, D<sub>2</sub>, D<sub>3</sub>), two select inputs (S<sub>1</sub>, S<sub>0</sub>), and one Enable input (E / Strobe):

$$Y = S_1 S_0 D_0 + S_1 S_0 D_1 + S_1 S_0 D_2 + S_1 S_0 D_3$$

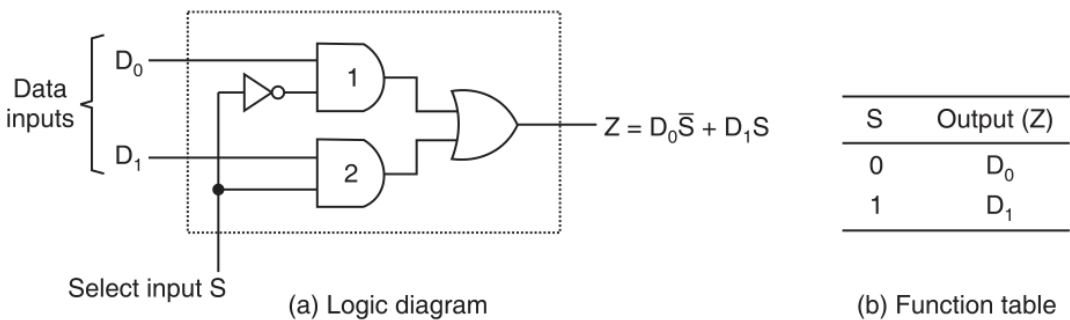


Figure 7.76 2-input multiplexer.

7.24.2 The 4-Input Multiplexer

Figure 7.77a shows the logic circuitry for a 4-input multiplexer with data inputs D<sub>0</sub>, D<sub>1</sub>, D<sub>2</sub>, and D<sub>3</sub>, and data select inputs S<sub>0</sub> and S<sub>1</sub>. The logic levels applied to the S<sub>0</sub> and S<sub>1</sub> inputs determine which AND gate is enabled, so that its data input passes through the OR gate to the output. The function table in Figure 7.77b gives the output for the input select codes as

$$Z = \bar{S}_1 \bar{S}_0 D_0 + \bar{S}_1 S_0 D_1 + S_1 \bar{S}_0 D_2 + S_1 S_0 D_3$$

Figure

7.77: Logic circuit diagram and truth table of a 4-input multiplexer. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 422 (printed p. 390).

## 2. Implementing Arbitrary Logic Functions Using Multiplexers

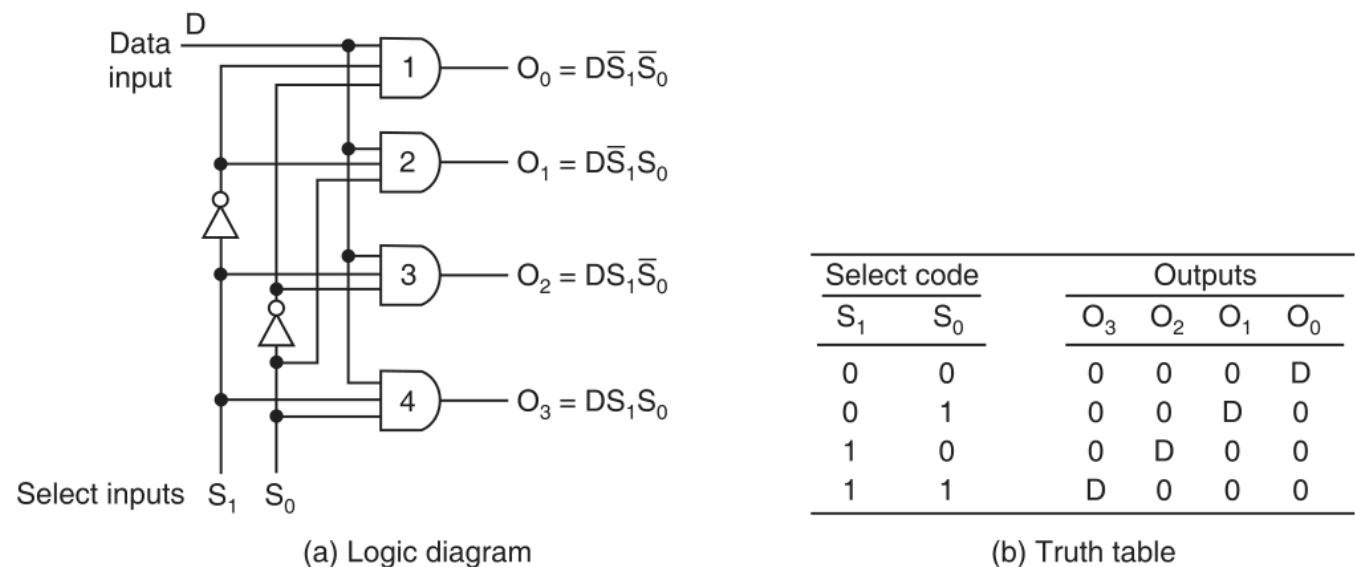
Source: Kumar (Ch. 7, pp. 423–428); Mano (Ch. 4, pp. 201–204)

An **8:1** MUX can implement **any 3-variable Boolean function directly with ZERO external gates**, and can implement **any 4-variable function using only a single inverter!** - *Method*: Connect **(n-1)** variables to the select lines. Group minterms in pairs to determine whether each data input line (**D<sub>i</sub>**) should be tied to **0**, **1**, the remaining variable **X**, or its complement **X**.

## 3. Demultiplexers (DEMUX / Data Distributors)

Source: Kumar (Ch. 7, pp. 430–432, Figure 7.89)

- **Definition**: The exact reverse of a MUX. A DEMUX takes a **single input data line (D)** and directs it to any one of **2<sup>n</sup>** output lines depending on the values of **n** select lines. - **The 1-to-4 Line Demultiplexer**:



**Figure 7.89** 1-line to 4-line demultiplexer.

Figure

7.89: Logic diagram of a 1-line to 4-line demultiplexer. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 431 (printed p. 399).

- **Duality with Decoders**: A demultiplexer with data input **D** is **identical in circuitry to a decoder with an Enable input!** If you treat the data line as the Enable input and the select lines as decoder address inputs, the circuits are 100% interchangeable.

## PART 3: Decoders, Encoders, Priority Encoders, and Display Drivers

### 3.8 Decoders and the 74138 IC

Source: Kumar (Ch. 7, pp. 414–418); Mano (Ch. 4, pp. 192–196); Floyd (Ch. 5, pp. 256–264)

[Pacing: Essential Peripheral Building Block — Learn Active-LOW Logic]

#### Need (Problem-First)

A computer's central processor outputs a binary address code (such as **010**). How does the motherboard know which of the eight connected memory chips is being spoken to? We need a circuit that takes an **n-bit** binary code and activates **one unique output wire** among **2<sup>n</sup>** possible output lines. That device is a **Decoder**.

#### The 3-Line to 8-Line Decoder (Kumar, p. 415, Figure 7.69)

Three inputs (**A**, **B**, **C**) generate **2<sup>3</sup> = 8** distinct output minterms (**Y<sub>0</sub>** through **Y<sub>7</sub>**):

## 7.69: 3-Line to 8-Line Decoder

Figure 7.69a shows the circuitry for a decoder with three inputs and eight outputs. It uses all AND gates, and therefore, the outputs are active-HIGH. For active-LOW outputs, NAND gates are used. The truth table of the decoder is shown in Figure 7.69b. This decoder can be referred to in several ways. It can be called a 3-line to 8-line decoder because it has three input lines and eight output lines. It is also called a binary-to-octal decoder because it takes a 3-bit binary input code and activates one of the eight (octal) outputs corresponding to that code. It is also referred to as a *1-of-8 decoder* because only one of the eight outputs is activated at one time.

Figure

7.69: Schematic diagram and truth table of a 3-line to 8-line decoder with active-HIGH outputs. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 415 (printed p. 383).

- **Active-LOW Outputs in Industry (The 74138 IC):**

In commercial ICs like the **74LS138**, standard NAND gates are used instead of AND gates. Therefore, the selected output line goes **LOW (0 V)**, while all unselected output lines remain **HIGH (+5 V)**. This active-LOW standard exists because bipolar transistors sink current far better than they source it.

---

### 3.9 Encoders and Priority Encoders (IC 74148)

Source: Kumar (Ch. 7, pp. 407–413); Mano (Ch. 4, pp. 196–199); Floyd (Ch. 5, pp. 264–268)

[Pacing: Conceptually Heavy — Priority Resolution in Computer Interrupts]

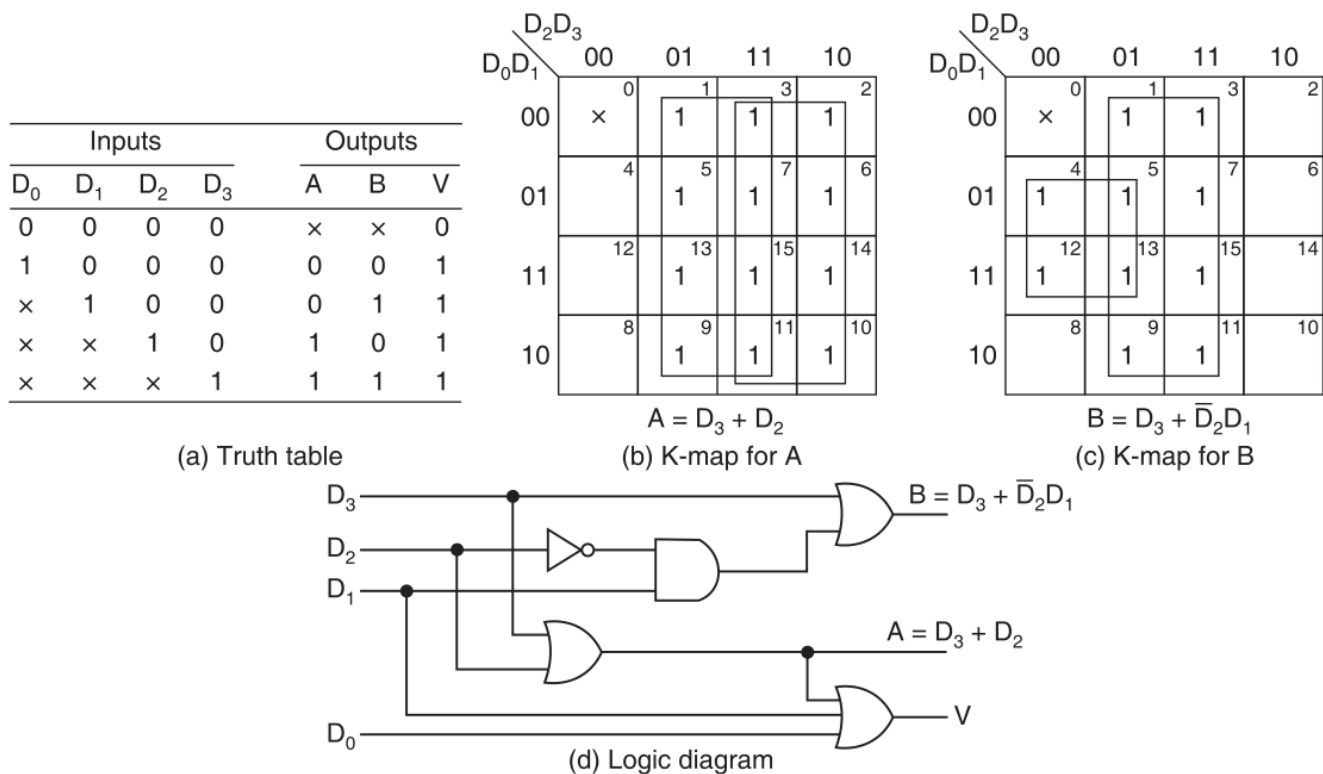
#### Need (Problem-First)

In a computer system, eight different peripheral devices (keyboard, hard disk, mouse, network card, power monitor) can request service by pulling an interrupt wire. An encoder compresses these 8 wires into a 3-bit binary code telling the CPU which device is calling.

*The Critical Failure of a Simple Encoder:* What happens if the user presses two keyboard keys at the exact same instant, or the power monitor screams an emergency alert while the mouse is moving? A naive encoder will output garbage (e.g., if line 2 and line 4 are asserted, the encoder might output 6). We need a circuit that **decides which input has the highest priority and completely ignores all lower-priority inputs**. This is a **Priority Encoder**.

#### The 4-Bit Priority Encoder (Kumar, p. 411, Figure 7.65)

Four inputs (**D<sub>0</sub>**, **D<sub>1</sub>**, **D<sub>2</sub>**, **D<sub>3</sub>**) where **D<sub>3</sub>** has the highest priority and **D<sub>0</sub>** has the lowest priority: - If **D<sub>3</sub> = 1**: Outputs **A<sub>1</sub> A<sub>0</sub> = 11**, regardless of whether **D<sub>2</sub>**, **D<sub>1</sub>**, **D<sub>0</sub>** are 0 or 1 (don't cares)! - If **D<sub>3</sub> = 0** and **D<sub>2</sub> = 1**: Outputs **A<sub>1</sub> A<sub>0</sub> = 10**, regardless of **D<sub>1</sub>**, **D<sub>0</sub>**. - If **D<sub>3</sub> = 0**, **D<sub>2</sub> = 0**, **D<sub>1</sub> = 1**: Outputs **A<sub>1</sub> A<sub>0</sub> = 01**. - If only **D<sub>0</sub> = 1**: Outputs **A<sub>1</sub> A<sub>0</sub> = 00**. - **The Valid Bit (V)**: What if *no inputs at all* are active? Without an extra bit, the circuit would output 00, falsely claiming that **D<sub>0</sub>** was pressed! The **Valid bit (V)** is an extra output pin that equals **1** if at least one input is active, and **0** if all inputs are silent.



**Figure 7.65** 4-bit priority encoder.

Figure

**7.65:** Logic circuitry, truth table with don't care entries, and K-map derivations of a 4-bit Priority Encoder. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 411 (printed p. 379).

- **Industry Standard IC: 74148** (8-to-3 line priority encoder with active-low inputs and outputs, used extensively in microprocessor interrupt controllers).

### 3.10 Display Drivers: BCD-to-Seven-Segment Decoder/Driver

Source: Kumar (Ch. 7, pp. 419–421, Figures 7.73 & 7.74; Ch. 19, pp. 1013–1031); Floyd (Ch. 5, pp. 260–264)

[Pacing: Visual Hardware Interfacing — High Practical Value]

#### Need (Problem-First)

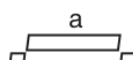
A calculator or digital clock processes numbers as 4-bit BCD codes (0000 to 1001). But a human being cannot look at a panel of four blinking raw binary LEDs and instantly read the time. We need an electronic display device that forms human numbers **0** through **9**, and a specialized digital decoder to illuminate the correct segments.

#### 1. The Seven-Segment Display (Kumar, p. 419, Figure 7.73)

Seven light-emitting diode (LED) bars arranged in a "figure-8" pattern, labeled clockwise from top as **a**, **b**, **c**, **d**, **e**, **f**, with the center bar labeled **g**:

generally provide greater illumination levels but require more power than that by LCDs.

Figure 7.73a shows a seven-segment display consisting of seven light emitting segments. The segments are designated by letters a–g as shown in the figure. By illuminating various combinations of segments as shown in Figure 7.73b, the numbers 0–9 can be displayed. Figures 7.73c and d show two types of LED display—the common-anode and the common-cathode types.



Figure

**7.73:** The seven-segment display layout, segment illumination combinations for decimal digits 0–9, and internal wiring of Common-Anode vs. Common-Cathode types. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 419 (printed p. 387).

#### The Two Physical Wiring Configurations:

### 1. Common-Cathode (CC) Display (Kumar, Figure 7.73d):

The cathode (negative terminal) of all seven segment LEDs are tied together to **Ground (0 V)**.

To illuminate a segment, its individual anode pin must be driven **HIGH (+5 V)**. (Active-HIGH driver like the **7448**).

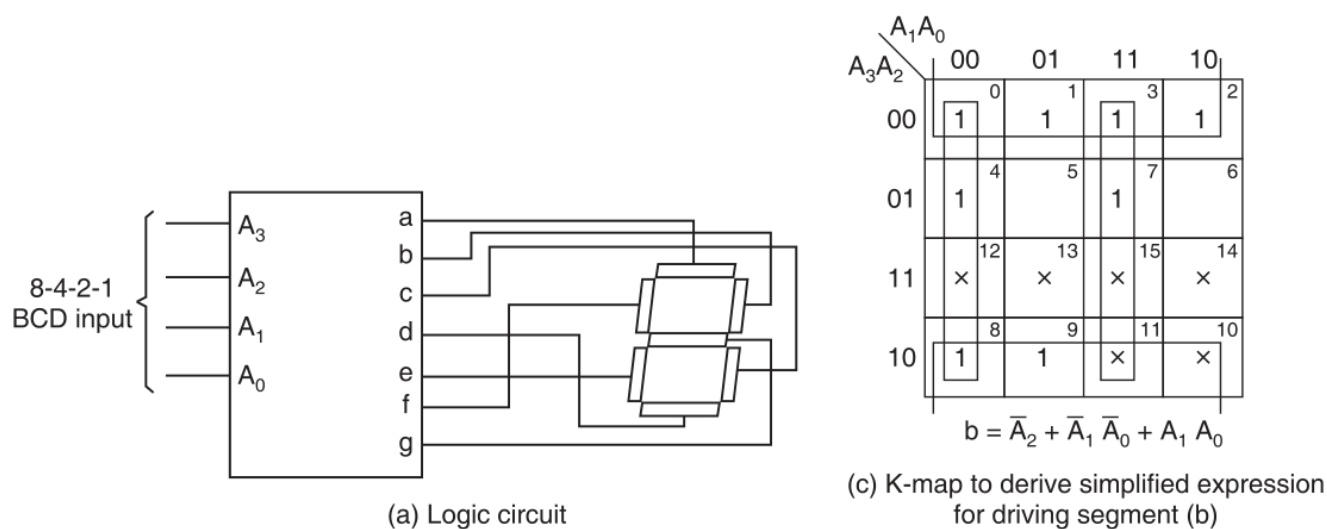
### 2. Common-Anode (CA) Display (Kumar, Figure 7.73c):

The anode (positive terminal) of all seven segment LEDs are tied together to **+V<sub>CC</sub> (+5 V)**.

To illuminate a segment, its individual cathode pin must be driven **LOW (0 V)**. (Active-LOW driver like the **7447**). Current flows from **+5 V** through the LED and sinks into the driver pin.

## 2. The BCD-to-Seven-Segment Decoder/Driver (Kumar, p. 420, Figure 7.74)

- **Inputs:** 4-bit BCD digit **A, B, C, D** (representing numbers **0** to **9**).
- **Outputs:** Seven driving signals **a, b, c, d, e, f, g**.
- **Don't Cares:** BCD inputs **10** to **15** (1010 to 1111) are treated as **Don't Cares (X)** in the K-maps, drastically reducing the gate count!



| Decimal digit | 8-4-2-1 BCD    |                |                |                | Seven segment code |   |   |   |   |   |   |
|---------------|----------------|----------------|----------------|----------------|--------------------|---|---|---|---|---|---|
|               | A <sub>3</sub> | A <sub>2</sub> | A <sub>1</sub> | A <sub>0</sub> | a                  | b | c | d | e | f | g |
| 0             | 0              | 0              | 0              | 0              | 1                  | 1 | 1 | 1 | 1 | 1 | 0 |
| 1             | 0              | 0              | 0              | 1              | 0                  | 1 | 1 | 0 | 0 | 0 | 0 |
| 2             | 0              | 0              | 1              | 0              | 1                  | 1 | 0 | 1 | 1 | 0 | 1 |
| 3             | 0              | 0              | 1              | 1              | 1                  | 1 | 1 | 1 | 0 | 0 | 1 |
| 4             | 0              | 1              | 0              | 0              | 0                  | 1 | 1 | 0 | 0 | 1 | 1 |
| 5             | 0              | 1              | 0              | 1              | 1                  | 0 | 1 | 1 | 0 | 1 | 1 |
| 6             | 0              | 1              | 1              | 0              | 1                  | 0 | 1 | 1 | 1 | 1 | 1 |
| 7             | 0              | 1              | 1              | 1              | 1                  | 1 | 1 | 0 | 0 | 0 | 0 |
| 8             | 1              | 0              | 0              | 0              | 1                  | 1 | 1 | 1 | 1 | 1 | 1 |
| 9             | 1              | 0              | 0              | 1              | 1                  | 1 | 1 | 1 | 0 | 1 | 1 |

(b) Function table

**Figure 7.74** BCD-to-seven segment decoder.

Figure

7.74: Functional block diagram, truth table, and Karnaugh map minimization for segment driver logic. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 420 (printed p. 388).

### Example: Segment a Logic Walkthrough

Segment **a** (the top horizontal bar) must be illuminated for digits **0, 2, 3, 5, 6, 7, 8, 9** (it remains OFF only for digits 1 and 4). By plotting these 8 minterms plus 6 don't cares on a 4-variable K-map, the minimized equation is directly derived (Kumar, p. 420).

## Unit III Understanding Checkpoints & Misconception Audit

### 1. Does This Make Sense? Self-Check

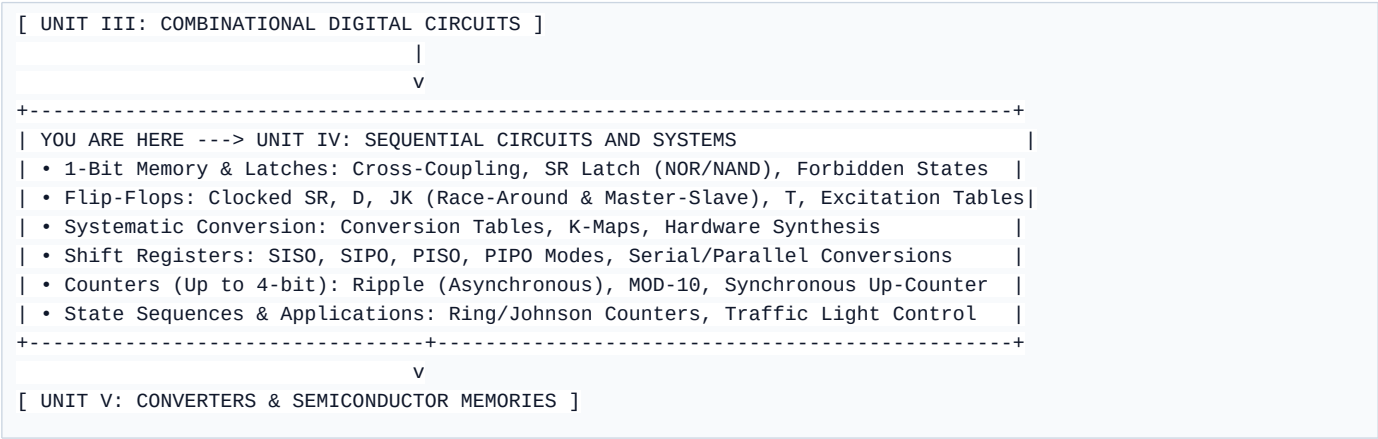
- Question 1:** What is the fundamental physical difference between a Half Adder and a Full Adder?
- Answer:** A Half Adder can only add two single bits (**A**, **B**). A Full Adder includes a third input for an incoming carry (**C<sub>in</sub>**) from a lower-order stage, permitting multi-bit parallel arithmetic.
- Question 2:** Why is a Carry Lookahead Adder (CLA) vastly faster than a standard Ripple Carry Adder?
- Answer:** In a ripple adder, carries must travel serially through every stage's gates (**2n** gate delays). A CLA calculates all carries simultaneously in parallel using two-level AND-OR logic directly from the primary inputs, executing in just **2** gate delays regardless of word length.
- Question 3:** If you connect a Common-Anode seven-segment display to a decoder that outputs active-HIGH signals, what will happen?
- Answer:** The display will invert every number (the segments that are supposed to be ON will stay dark, and the dark segments will light up), because Common-Anode LEDs require an active-LOW (**0 ext{ V}**) sink to turn ON!

### 2. Conceptual Misconceptions Corrected

- Misconception:** "A multiplexer can only route data; it cannot do math."  
**Correction:** A multiplexer is a universal Boolean function generator. By connecting input variables to select lines and tying data lines to **0**, **1**, or remaining literals, an **8:1** MUX can implement *any* 3-variable or 4-variable combinational logic circuit without a single additional logic gate!
- Misconception:** "In an adder/subtractor circuit, subtraction requires a completely separate subtractor unit inside the chip."  
**Correction:** There is zero subtraction circuitry. The circuit performs addition on the 2's complement of **B** by using XOR gates to invert **B** and setting the input carry **C<sub>0</sub>** = **1**. The adder does not even know it is subtracting!

## Unit IV: Sequential Circuits and Systems

### Master Subject Map: Unit IV Position



*In 3 lines:* Combinational circuits are completely amnesiac—they have no memory of the past. Unit IV introduces **feedback**, transforming gates into circuits that store electrical history. We construct 1-bit memory cells (Flip-Flops), pipeline registers (Shift Registers), and rhythmic state machines (Counters) that automate real-world systems like traffic light intersections.

### Prerequisites for this Unit

Before studying Unit IV, ensure you recall these concepts: 1. **Gate Characteristics & Propagation Delay (Unit I, Section 1.8):** Signals do not change instantaneously; physical gates have a small delay **t<sub>pd</sub>** (typically **5 to 10 ns**). 2. **Universal Gates (Unit I, Section 1.6):** NAND and NOR truth tables. 3. **Pulse Waveforms (Prereq 5):** Rising (positive) edge, falling (negative) edge, pulse width (**t<sub>w</sub>**), and clock period (**T**). 4. **Gray Code & State Tables (Unit I, Section 1.4 & Prereq 6):** Reading multi-variable tabular state sequences.

## PART 1: The 1-Bit Memory, Latches, and Clocked Flip-Flops

### 4.1 Combinational vs. Sequential Logic & The 1-Bit Memory Concept

Source: Kumar (Ch. 10, pp. 577–581, Figures 10.1 & 10.2); Mano (Ch. 5, pp. 248–252)

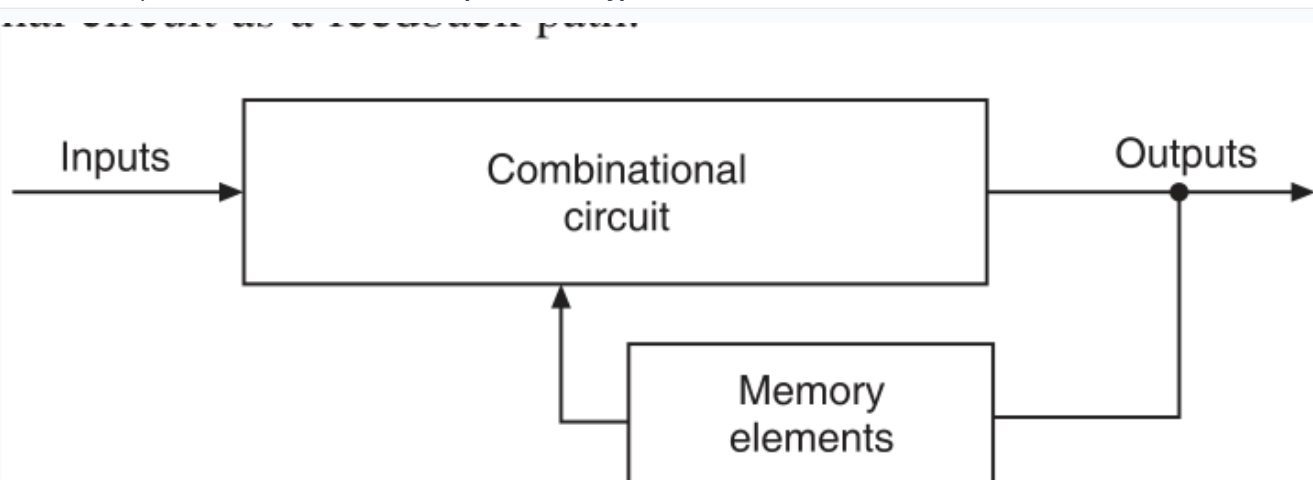
[Pacing: Conceptually Heavy — The Moment Memory is Born]

#### Need (Problem-First)

In an adder or multiplexer, the moment you remove your finger from the input switch, the output disappears. But a real computer must store passwords, remember loop counters, and hold program instructions. How can simple logic gates—which only know how to react to present voltages—be made to **remember a bit permanently**, even after the original input signal has gone to zero?

#### Chain of Cause and Effect

Feed an output wire back into an input terminal (feedback) → the circuit's output now reinforces its own input → the circuit possesses two stable physical states ( $Q=1$  or  $Q=0$ ) → it remains locked in its current state indefinitely until an external trigger forces a flip → a **Bistable Multivibrator (1-bit memory)** is created.



**Figure 10.1** Block diagram of a sequential circuit.

Figure

**10.1:** Fundamental block diagram of a Sequential Circuit consisting of a combinational logic network and memory feedback elements. Sourced from A.

Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 578 (printed p. 546).

- **Definition of Sequential Circuit:** A digital circuit whose output at any instant depends not only on the present inputs, but also on the **past history of inputs (the present state of the memory elements)** (Kumar, p. 577).

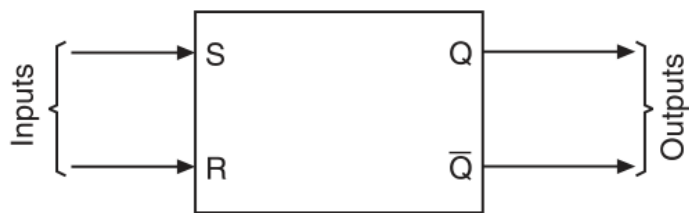
### 4.2 The Bistable SR Latch: Active-HIGH (NOR) and Active-LOW (NAND)

Source: Kumar (Ch. 10, pp. 581–584, Figures 10.4 & 10.5); Floyd (Ch. 6, pp. 326–332)

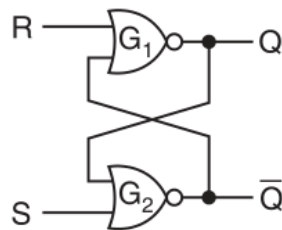
[Pacing: Slow & Methodical — Trace the Internal Electron Loops]

#### 1. The Active-HIGH S-R Latch Using NOR Gates (Kumar, p. 581)

Two NOR gates are cross-coupled: the output of Gate 1 ( $Q$ ) is fed into an input of Gate 2, and the output of Gate 2 ( $Q$ ) is fed into an input of Gate 1:



(a) Logic symbol



(b) Logic diagram

| S | R | $Q_n$ | $Q_{n+1}$ | State                   |
|---|---|-------|-----------|-------------------------|
| 0 | 0 | 0     | 0         | No Change (NC)          |
| 0 | 0 | 1     | 1         |                         |
| 0 | 1 | 0     | 0         | Reset                   |
| 0 | 1 | 1     | 0         |                         |
| 1 | 0 | 0     | 1         | Set                     |
| 1 | 0 | 1     | 1         |                         |
| 1 | 1 | 0     | ×         | Indeterminate (invalid) |
| 1 | 1 | 1     | ×         |                         |

(c) Truth table

**Figure 10.4** Active-HIGH S-R latch.

Figure

**10.4:** Active-HIGH Set-Reset (S-R) latch using cross-coupled NOR gates: logic diagram, circuit symbol, and functional truth table. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 582 (printed p. 550).

### Characteristic 13 Walkthrough: Hardware Reality Check

Recall the fundamental rule of a NOR gate: **any input of 1 forces the output to 0**. 1. **The SET State ( $S = 1, R = 0$ ):**

Applying  $S = 1$  to Gate 2 instantly forces its output  $Q = 0$  V. This 0 V travels along the feedback wire into Gate 1. Now Gate 1 has  $R = 0$  and  $Q = 0$ . With both inputs at 0, Gate 1 outputs  $Q = 1$  (+5 V)!

Now remove the input ( $S = 0, R = 0$ ): What happens? Output  $Q = 1$  continues feeding into Gate 2, holding  $Q = 0$ , which in turn holds  $Q = 1$ ! **The circuit latched the 1! It remembers that you set it!** 2. **The RESET State ( $S = 0, R = 1$ ):**

Applying  $R = 1$  to Gate 1 instantly forces its output  $Q = 0$  V. This 0 V travels to Gate 2. With  $S = 0$  and  $Q = 0$ , Gate 2 outputs  $Q = 1$  (+5 V)!

Now remove the input ( $S = 0, R = 0$ ): The feedback holds  $Q = 0$  and  $Q = 1$ . **It remembers that you cleared it!** 3. **The Quiescent / Memory State ( $S = 0, R = 0$ ):**

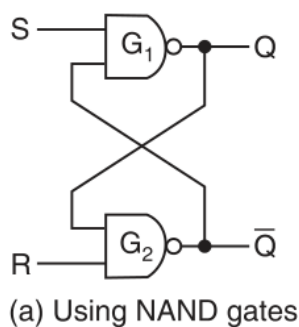
No change. The latch preserves its previous state ( $Q_{\text{next}} = Q_{\text{present}}$ ). 4. **The FORBIDDEN / INVALID State ( $S = 1, R = 1$ ):**

If both inputs are driven HIGH simultaneously, both NOR gates are forced to output 0 at the same time:  $Q = 0$  and  $Q = 0$ ! This violates the fundamental mathematical rule that  $Q$  and  $Q$  must be complementary.

**The Disaster of Race Condition:** If both inputs now switch back to 0 at the exact same instant, the two gates race against each other to become 1. Whichever gate has a microscopic picosecond advantage in propagation delay wins, leaving the circuit in an **unpredictable, chaotic random state**. Hence,  $S = 1, R = 1$  is strictly forbidden!

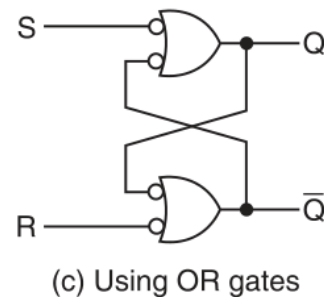
## 2. The Active-LOW S-R Latch Using NAND Gates (Kumar, p. 583)

Two NAND gates cross-coupled:



| S | R | $Q_n$ | $Q_{n+1}$ | State                      |
|---|---|-------|-----------|----------------------------|
| 0 | 0 | 0     | x         | Indeterminate<br>(invalid) |
| 0 | 0 | 1     | x         |                            |
| 0 | 1 | 0     | 1         | Set                        |
| 0 | 1 | 1     | 1         |                            |
| 1 | 0 | 0     | 0         | Reset                      |
| 1 | 0 | 1     | 0         |                            |
| 1 | 1 | 0     | 0         | No Change (NC)             |
| 1 | 1 | 1     | 1         |                            |

(b) Truth table



**Figure 10.5** An active-LOW S-R latch.

Figure

**10.5:** An active-LOW S-R latch using cross-coupled NAND gates. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 583 (printed p. 551).

- In NAND logic, **any input of 0 forces the output to 1**.
- Setting requires pulling **S = 0**.
- Resetting requires pulling **R = 0**.
- Resting memory state is **S = 1, R = 1**.
- **Forbidden state: S = 0, R = 0** (both force outputs to 1).

### 4.3 Clocked SR Flip-Flops: Level-Triggering vs. Edge-Triggering

Source: Kumar (Ch. 10, pp. 584–588, Figures 10.7 & 10.10); Floyd (Ch. 6, pp. 332–338)

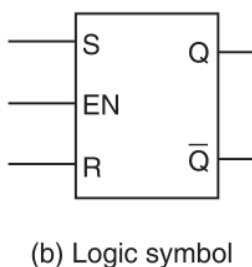
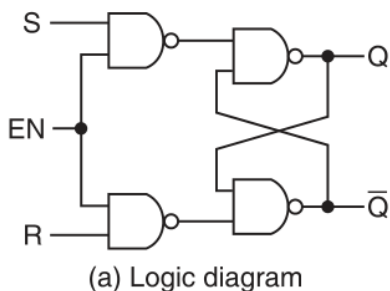
[Pacing: Conceptually Essential — Understanding Synchronization]

#### Need (Problem-First)

In an unclocked latch, inputs can change state at any random, chaotic microsecond. In a complex digital processor with millions of gates, signals arriving from different circuit paths experience slightly different wire delays. If latches respond continuously, false temporary glitches will be permanently stored as corrupted data. We need a conductor's baton that commands all memory cells to **update only at precise, synchronized moments in time**. That synchronizing signal is the **Clock (CLK)**.

#### The Clocked (Gated) SR Latch (Kumar, p. 584, Figure 10.7)

Two steering NAND gates are placed in front of an active-LOW NAND latch, gated by an Enable / Clock input (**CLK**):



| EN | S | R | $Q_n$ | $Q_{n+1}$ | State                      |
|----|---|---|-------|-----------|----------------------------|
| 1  | 0 | 0 | 0     | 0         | No change (NC)             |
| 1  | 0 | 0 | 1     | 1         |                            |
| 1  | 0 | 1 | 0     | 0         | Reset                      |
| 1  | 0 | 1 | 1     | 0         |                            |
| 1  | 1 | 0 | 0     | 1         | Set                        |
| 1  | 1 | 0 | 1     | 1         |                            |
| 1  | 1 | 1 | 0     | x         | Indeterminate<br>(invalid) |
| 1  | 1 | 1 | 1     | x         |                            |
| 0  | x | x | 0     | 0         | No Change (NC)             |
| 0  | x | x | 1     | 1         |                            |

(c) Truth table

**Figure 10.7** A gated S-R latch.

Figure

**10.7:** A gated (clocked) S-R latch showing input steering gates and enable line. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 584 (printed p. 552).

- When **CLK = 0**: The outputs of both front steering gates are forced to **1** (**S = 1, R = 1**), locking the latch in its resting memory state. The inputs **S** and **R** are completely ignored!
- When **CLK = 1**: The steering gates invert **S** and **R**, allowing them to control the internal latch.

### Level-Triggering vs. Edge-Triggering

Source: Kumar (p. 586); Floyd (pp. 334–336) - **Level-Triggered Latch**: Transparent and sensitive to input changes during the **entire duration that the clock pulse remains HIGH ( $t_w$ )**. If inputs change while clock is HIGH, the output tracks them immediately. - **Edge-Triggered Flip-Flop**: Responds **ONLY during the microscopic transition threshold of the clock edge** (the rising edge **0 → 1** or falling edge **1 → 0**). For the rest of the clock cycle, the flip-flop is completely deaf to input changes. - **Dynamic Indicator Symbol**: A small triangular chevron (**riangleright**) on the clock input terminal indicates **edge-triggering**. A bubble before the chevron indicates **negative (falling) edge-triggering**.

**Setup Time ( $t_s$ ) and Hold Time ( $t_h$ ) Defined Inline (Kumar, p. 599):**

- **Setup Time ( $t_s$ )**: The minimum time interval that the digital input data must remain **stable and constant BEFORE the active clock transition edge arrives**. If data changes within  $t_s$ , the flip-flop can enter metastability.
- **Hold Time ( $t_h$ )**: The minimum time interval that the input data must remain **stable and constant AFTER the active clock transition edge has passed**.

## 4.4 The JK Flip-Flop, Race-Around Condition, and Master-Slave Architecture

Source: Kumar (Ch. 10, pp. 590–607, Figures 10.17, 10.35, 10.40); Floyd (Ch. 6, pp. 340–348)

**[Pacing: Conceptually Heavy — A Classic Engineering Failure & Its Cure]**

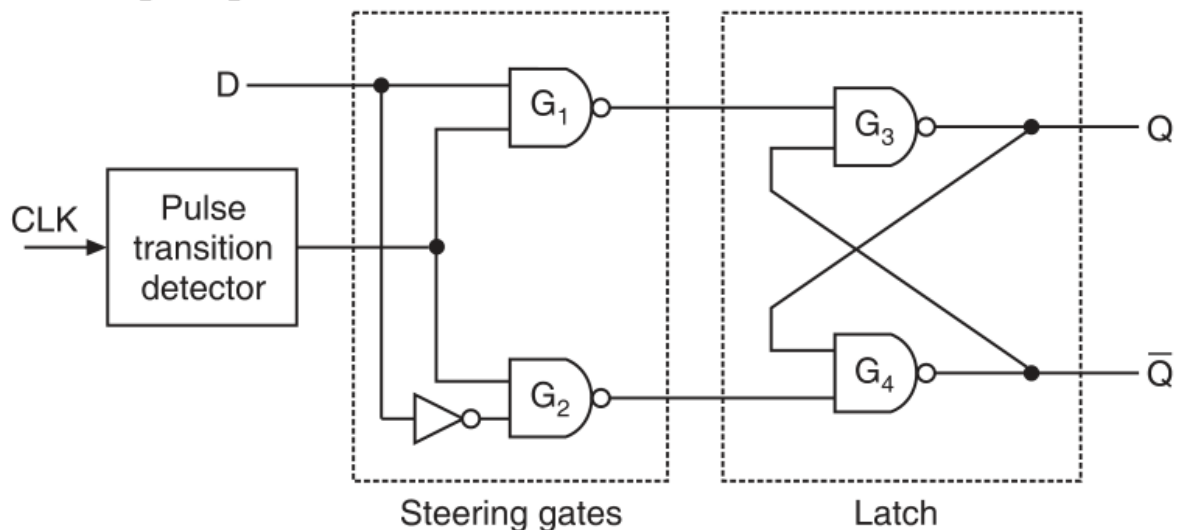
### Need (Problem-First)

The SR flip-flop has a fatal design flaw: the input combination **S=1, R=1** is strictly forbidden because it produces race conditions. In practical engineering, having an illegal input combination that can crash your system is dangerous. We need a flip-flop that eliminates the forbidden state forever, converting it into a useful operation. That device is the **JK Flip-Flop** (named in honor of Jack Kilby, inventor of the integrated circuit).

### The Edge-Triggered JK Flip-Flop (Kumar, p. 590, Figure 10.17)

The outputs **Q** and **Q** are fed back into the front steering gates: - **J** behaves like SET (**S**). - **K** behaves like RESET (**R**). - When **J = 1** and **K = 1**: The feedback causes the circuit to **TOGGLE (invert its state)**! If **Q** was **0**, it flips to **1**; if **Q** was **1**, it flips to **0**. There are **no forbidden states**!

triggered D flip-flop.

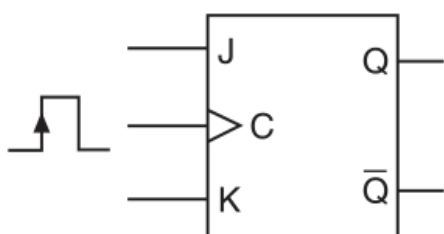


**Figure 10.16** Simplified circuit diagram of the edge-triggered D flip-flop.

negative edge-triggered D flip-flop operates in the same way as the positive edge triggered D flip-flop except that the change of state takes place at the negative-going edge of the clock. In the truth table of the negative edge triggered flip-flop the arrows point down

**-triggered J-K flip-flop:** The J-K flip-flop is very versatile and also the n J and K designations for the synchronous control inputs have no known sig functioning of the J-K flip-flop is identical to that of the S-R flip-flop, excep state like that of the S-R flip-flop. The logic symbol and the truth table fo gered J-K flip-flop are shown in Figure 10.17.

| C          | J | K | $Q_n$ | $Q_{n+1}$ | State          |
|------------|---|---|-------|-----------|----------------|
| $\uparrow$ | 0 | 0 | 0     | 0         | No Change (NC) |
| $\uparrow$ | 0 | 0 | 1     | 1         |                |
| $\uparrow$ | 0 | 1 | 0     | 0         | Reset          |
| $\uparrow$ | 0 | 1 | 1     | 0         |                |
| $\uparrow$ | 1 | 0 | 0     | 1         | Set            |
| $\uparrow$ | 1 | 0 | 1     | 1         |                |
| $\uparrow$ | 1 | 1 | 0     | 1         | Toggle         |
| $\uparrow$ | 1 | 1 | 1     | 0         |                |
| 0          | x | x | 0     | 0         | No Change (NC) |
| 0          | x | x | 1     | 1         |                |



(a) Logic symbol

(b) Truth table

**Figure 10.17** Positive edge-triggered J-K flip-flop.

Characteristic Table of the JK Flip-Flop:

| J | K | Q <sub>n</sub> (Present) | Q <sub>{n+1}</sub> (Next State) | Operational Description   |
|---|---|--------------------------|---------------------------------|---------------------------|
| 0 | 0 | 0                        | 0                               | No Change (Hold / Memory) |
| 0 | 0 | 1                        | 1                               | No Change (Hold / Memory) |
| 0 | 1 | 0                        | 0                               | Reset                     |
| 0 | 1 | 1                        | 0                               | Reset                     |
| 1 | 0 | 0                        | 1                               | Set                       |
| 1 | 0 | 1                        | 1                               | Set                       |
| 1 | 1 | 0                        | 1                               | Toggle (0 → 1)            |
| 1 | 1 | 1                        | 0                               | Toggle (1 → 0)            |

- **Characteristic Equation:** Minimizing the table via K-map yields:

$$Q_{n+1} = JQ_n + KQ_n$$

The Race-Around Condition in Level-Triggered JK Flip-Flops

Source: Kumar (Ch. 10, pp. 600–603); Mano (Ch. 5, pp. 256–258)

[Pacing: Slow & Deliberate — Understand the Physical Failure Mode]

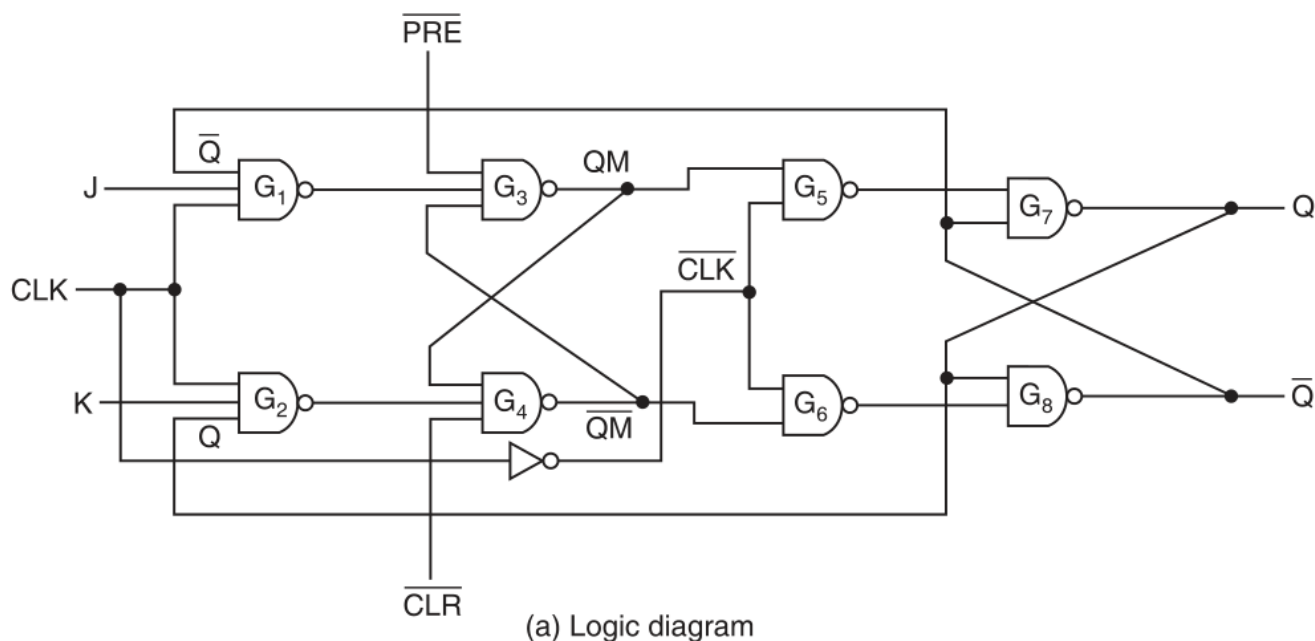
What Physically Goes Wrong:

In a **level-triggered** JK flip-flop, the inputs are active during the entire duration that the clock pulse is HIGH (**t<sub>w</sub>**): - Suppose **J = 1**, **K = 1**, and initially **Q = 0**. - When the clock goes HIGH, the flip-flop toggles to **Q = 1** after its internal gate propagation delay (**t<sub>pd</sub>**). - But if the clock pulse width **t<sub>w</sub>** is longer than the propagation delay (**t<sub>w</sub> > t<sub>pd</sub>**), the new output **Q = 1** feeds right back into the front gates **while the clock is still HIGH!** - This causes the flip-flop to toggle again from **1** back to **0** after another **t<sub>pd</sub>**! - It toggles again to **1**, then back to **0**, oscillating wildly and uncontrollably back and forth during the entire time the clock is HIGH! - When the clock finally falls LOW, the final state of **Q** is completely unpredictable! - This catastrophic oscillation is the **Race-Around Condition** (Kumar, p. 600). - **Physical Condition for Race-Around:**

$$t_w \geq t_{pd}$$

The Engineering Solutions to Race-Around:

1. **Edge-Triggering:** Make clock transitions extremely fast with internal spike detectors so the active window is shorter than **t<sub>pd</sub>**.
2. **The Master-Slave JK Flip-Flop Architecture (Kumar, p. 605, Figure 10.40):**  
Construct the circuit using two separate latches in series: a **Master flip-flop** and a **Slave flip-flop**, driven by complementary clock signals via an inverter:



| Inputs |   |   | Output      | Comments  |
|--------|---|---|-------------|-----------|
| J      | K | C | Q           |           |
| 0      | 0 |   | $Q_0$       | No change |
| 0      | 1 |   | 0           | RESET     |
| 1      | 0 |   | 1           | SET       |
| 1      | 1 |   | $\bar{Q}_0$ | Toggle    |

(b) Truth table

**Figure 10.40** The master-slave J-K flip-flop.

Figure

**10.40:** The Master-Slave J-K flip-flop architecture completely eliminating race-around conditions. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 606 (printed p. 574).

#### How Master-Slave Eliminates Race-Around:

- **Phase 1 (CLK = 1):** The Master flip-flop is enabled and accepts inputs **J** and **K**. But the Slave flip-flop receives **CLK = 0** through the inverter, so the Slave is **completely disabled and locked**! The final outputs **Q** and **Q** cannot change, so no feedback can reach the Master! Race-around is physically impossible!
- **Phase 2 (CLK = 0):** The Master is disabled, locking its internal state. The Slave receives **CLK = 1** and transfers the Master's state to the final outputs **Q** and **Q**.
- The two stages never conduct simultaneously, isolating input from output and destroying the race path.

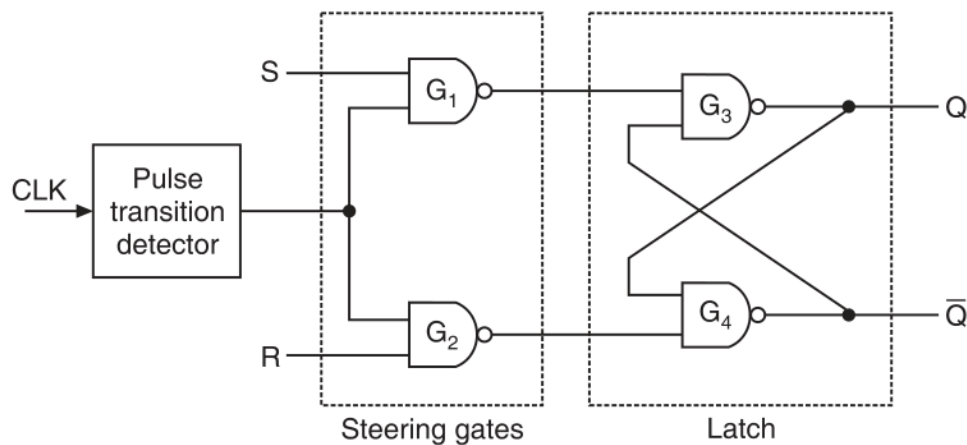
## 4.5 The D (Data) and T (Toggle) Flip-Flops

Source: Kumar (Ch. 10, pp. 589–594, Figures 10.15 & 10.20); Floyd (Ch. 6, pp. 338–344)

[Pacing: Simple & Direct — The Two Workhorses of Registers & Counters]

### 1. The D Flip-Flop (Data / Delay Flip-Flop) (Kumar, p. 589)

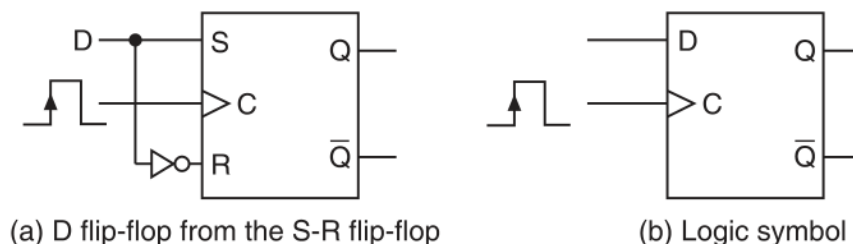
- **Need:** Eliminate the possibility of invalid states in an SR latch by forcing the two inputs to always be exact complements of each other (**S = D**, **R = D**).
- **Mechanism:** A single input **D** is fed directly to the SET line, and through an inverter to the RESET line:



**Figure 10.14** Simplified circuit diagram of the edge-triggered S-R flip-flop.

The edge detector generates a positive spike at the positive-going or negative-going clock pulse. The steering gates 'direct' or 'steer' the narrow spike either to G<sub>1</sub> or G<sub>2</sub> on the state of the S and R inputs.

**Edge-triggered D flip-flop:** The edge-triggered D flip-flop has only one input. A D flip-flop may be obtained from an S-R flip-flop by just putting one inverter between the D input and the S and R inputs (Figure 10.15a). Figures 10.15b and c show the logic symbol and the truth table of the edge-triggered D flip-flop. Note that, this flip-flop has only one synchronous input, the clock. This is called the D (data) input. The operation of the D flip-flop is as follows. The output Q will go to the same state that is present on the D input at the positive-going edge of the clock pulse. In other words, the level present at D will be stored in the flip-flop at the positive-going transition occurs. That is, if D is a 1 and the clock is applied, Q goes to a 1 and  $\bar{Q}$  to a 0 at the rising edge of the pulse and thereafter remain so. If D is a 0 and the clock is applied, Q goes to a 0 and  $\bar{Q}$  to a 1 at the rising edge of the clock pulse and thereafter remain so.



| C          | D | $Q_n$ | $Q_{n+1}$ | State          |
|------------|---|-------|-----------|----------------|
| $\uparrow$ | 0 | 0     | 0         | Reset          |
| $\uparrow$ | 0 | 1     | 0         |                |
| $\uparrow$ | 1 | 0     | 1         | Set            |
| $\uparrow$ | 1 | 1     | 1         |                |
| 0          | x | 0     | 0         | No Change (NC) |
| 0          | x | 1     | 1         |                |

(c) Truth table

**Figure 10.15** The positive edge-triggered D flip-flop.

**Figure 10.15:** Positive

- **Operation:** Whatever bit is on the **D** input pin is captured and transferred to output **Q** on the active clock edge:

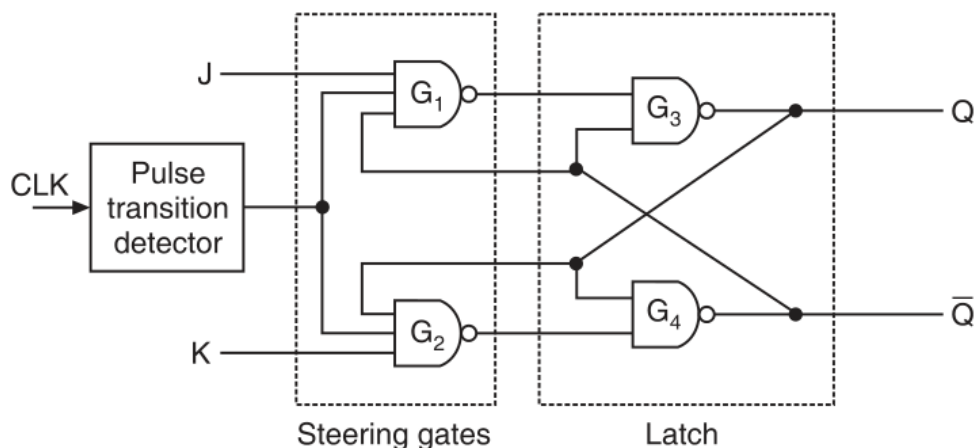
$$Q_{n+1} = D$$

- It introduces a delay of exactly one clock period, making it the universal building block for **registers and computer RAM**.

## 2. The T Flip-Flop (Toggle Flip-Flop) (Kumar, p. 592)

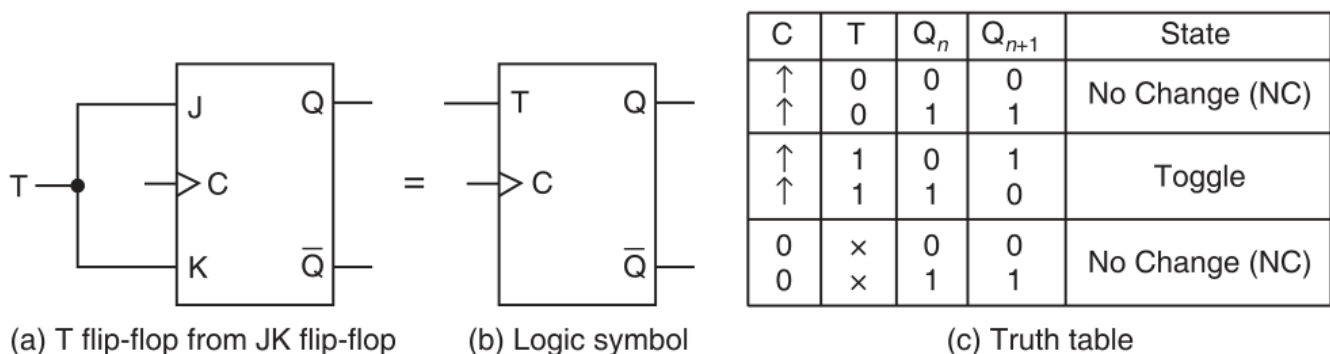
- **Need:** A 1-bit memory cell that flips its state on every clock pulse when enabled, performing automatic **frequency division** ( $f_{out} = f_{in}/2$ ) and binary counting.
- **Mechanism:** Formed by tying the **J** and **K** inputs of a JK flip-flop together (**J = K = T**):

anges its state.



**Figure 10.19** Simplified circuit diagram of the edge-triggered J-K flip-flop.

**edge-triggered T flip-flop:** A T flip-flop has a single control input, labelled T for toggle. When T is HIGH, the flip-flop toggles on every new clock pulse. When T is LOW, the flip-flop remains in whatever state it was before. Although T flip-flops are not widely available commercially, it is easy to convert a J-K flip-flop to the functional equivalent of a T flip-flop by just connecting J and K together and labelling the common connection as T. Thus, when T = 1, we have J = K = 1, and the flip-flop toggles. When T = 0, we have J = K = 0, and so there is no change of state. The logic symbol and the truth table of a T flip-flop are shown in Figure 10.20.



**Figure 10.20** Edge-triggered T flip-flop.

Figure

10.20: Edge-triggered T flip-flop logic diagram, graphic symbol, and characteristic table. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 592 (printed p. 560).

- **Operation:**
- If **T = 0**:  $Q_{n+1} = Q_n$  (Hold / Memory).
- If **T = 1**:  $Q_{n+1} = \overline{Q_n}$  (Toggle / Invert).
- **Characteristic Equation:**

$$Q_{n+1} = T \oplus Q_n = TQ_n + \overline{T}\overline{Q}_n$$

## PART 2: Excitation Tables and Flip-Flop Conversions

### 4.6 Excitation Tables: The Inverse Design View

Source: Kumar (Ch. 10, pp. 593–595); Mano (Ch. 5, pp. 268–270)

[Pacing: Conceptually Essential for Counter & State Machine Design]

#### Need (Problem-First)

In circuit *analysis*, you know the inputs (**J**, **K** or **S**, **R**) and you calculate what the next output **Q<sub>n+1</sub>** will be. But in circuit *synthesis* (designing a counter or traffic light controller), you face the exact opposite problem: **you know what present state the circuit is currently in (Q<sub>n</sub>)**, and you know what state you **WANT** it to become (**Q<sub>n+1</sub>**) on the next clock pulse. What **input voltages must you apply to force that transition?**

The tabular ledger answering this question is the **Excitation Table**.

Master Compilation of Excitation Tables (Kumar, pp. 593–594):

| Desired Transition<br>(Q <sub>n</sub> → Q <sub>n+1</sub> ) | S-R Inputs (S, R) | J-K Inputs (J, K) | D Input (D) | T Input (T) |
|------------------------------------------------------------|-------------------|-------------------|-------------|-------------|
| 0 → 0 (Hold LOW)                                           | 0, X              | 0, X              | 0           | 0           |
| 0 → 1 (Set HIGH)                                           | 1, 0              | 1, X              | 1           | 1           |
| 1 → 0 (Reset LOW)                                          | 0, 1              | X, 1              | 0           | 1           |
| 1 → 1 (Hold HIGH)                                          | X, 0              | X, 0              | 1           | 0           |

(Where **X** represents a Don't Care condition).

#### Why the X Appears in JK Excitation:

- Look at the transition **0 → 0**: We can either command a HOLD (**J=0**, **K=0**) or command a RESET (**J=0**, **K=1**). In both cases, **J** must be **0**, but **K** can be either **0** or **1**! Hence, **J = 0**, **K = X**. This freedom makes JK flip-flops yield significantly simpler combinational driving logic than other types.

### 4.7 Systematic Flip-Flop Conversion Procedure

Source: Kumar (Ch. 10, pp. 610–615, Figures 10.43–10.50)

[Pacing: Systematic 4-Step Engineering Procedure — Practice the Method]

#### Need (Problem-First)

In the laboratory, your design requires a JK flip-flop, but your company's component inventory only has SR flip-flop chips in stock. How can you wrap external combinational logic around an SR flip-flop so it behaves exactly like a JK flip-flop?

The Universal 4-Step Conversion Procedure (Kumar, p. 610):

- Identify Given vs. Desired:** Name the available flip-flop (Given) and the target flip-flop to be synthesized (Desired).
- Construct the Conversion Table:** Combine the characteristic table of the Desired flip-flop with the excitation table of the Given flip-flop.
- K-Map Minimization:** Plot the required inputs of the Given flip-flop as functions of the Desired inputs and present state **Q<sub>n</sub>**.
- Draw the Logic Diagram:** Wire the derived combinational logic to the input terminals of the Given flip-flop.

Fully-Worked Example: Convert an S-R Flip-Flop to a J-K Flip-Flop (Kumar, p. 611, Figure 10.44)

- Step 1:** Desired = JK; Given = SR.

• Step 2 (Conversion Table):

| Inputs (J, K) | Present State (Q <sub>n</sub> ) | Next State (Q <sub>{n+1}</sub> ) | Required S Input | Required R Input | Transition Reason |
|---------------|---------------------------------|----------------------------------|------------------|------------------|-------------------|
| 0, 0          | 0                               | 0                                | 0                | X                | Hold 0            |
| 0, 0          | 1                               | 1                                | X                | 0                | Hold 1            |
| 0, 1          | 0                               | 0                                | 0                | X                | Reset             |
| 0, 1          | 1                               | 0                                | 0                | 1                | Reset             |
| 1, 0          | 0                               | 1                                | 1                | 0                | Set               |
| 1, 0          | 1                               | 1                                | X                | 0                | Set               |
| 1, 1          | 0                               | 1                                | 1                | 0                | Toggle (0 → 1)    |
| 1, 1          | 1                               | 0                                | 0                | 1                | Toggle (1 → 0)    |

• Step 3 (K-Map Minimization):

- For S:  $S(J, K, Q_n) = \sum m(4, 6) + d(1, 5)$ .  
Grouping minterms 4 and 6 with don't cares yields:

S = JQ<sub>n</sub>

- For R:  $R(J, K, Q_n) = \sum m(3, 7) + d(0, 2)$ .  
Grouping minterms 3 and 7 with don't cares yields:

R = KQ<sub>n</sub>

- Step 4 (Logic Schematic Realization): Connect an AND gate receiving J and Q<sub>n</sub> to input S, and connect an AND gate receiving K and Q<sub>n</sub> to input R:

| External Inputs |   | Present State | Next State | Flip-flop Inputs |   |
|-----------------|---|---------------|------------|------------------|---|
| J               | K | $Q_n$         | $Q_{n+1}$  | S                | R |
| 0               | 0 | 0             | 0          | 0                | x |
| 0               | 0 | 1             | 1          | x                | 0 |
| 0               | 1 | 0             | 0          | 0                | x |
| 0               | 1 | 1             | 0          | 0                | 1 |
| 1               | 0 | 0             | 1          | 1                | 0 |
| 1               | 0 | 1             | 1          | x                | 0 |
| 1               | 1 | 0             | 1          | 1                | 0 |
| 1               | 1 | 1             | 0          | 0                | 1 |

(a) Conversion table

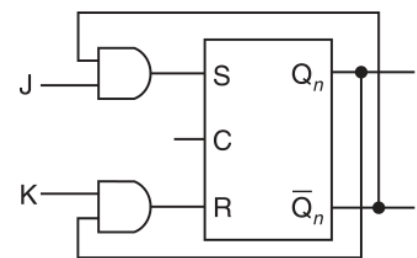
| J | $KQ_n$         |                |                |                |
|---|----------------|----------------|----------------|----------------|
|   | 00             | 01             | 11             | 10             |
| 0 | 0 <sup>0</sup> | x <sup>1</sup> | 0 <sup>3</sup> | 0 <sup>2</sup> |
| 1 | 1 <sup>4</sup> | x <sup>5</sup> | 0 <sup>7</sup> | 1 <sup>6</sup> |

$$S = J\bar{Q}_n$$

| J | $KQ_n$         |                |                |                |
|---|----------------|----------------|----------------|----------------|
|   | 00             | 01             | 11             | 10             |
| 0 | x <sup>0</sup> | 0 <sup>1</sup> | 1 <sup>3</sup> | x <sup>2</sup> |
| 1 | 0 <sup>4</sup> | 0 <sup>5</sup> | 1 <sup>7</sup> | 0 <sup>6</sup> |

$$R = KQ_n$$

(b) K-maps for S and R



(c) Logic diagram

**Figure 10.44** Conversion of S-R flip-flop to J-K flip-flop.

Figure

**10.44:** Hardware conversion of an S-R flip-flop into a J-K flip-flop using two external AND gates. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 611 (printed p. 579).

## PART 3: Shift Registers (SISO, SIPO, PISO, PIPO) and Applications

### 4.8 Shift Register Operational Modes

Source: Kumar (Ch. 11, pp. 636–643, Figures 11.4–11.9); Floyd (Ch. 7, pp. 389–408)

[Pacing: Physical Data Motion — Trace the Clock Shifts]

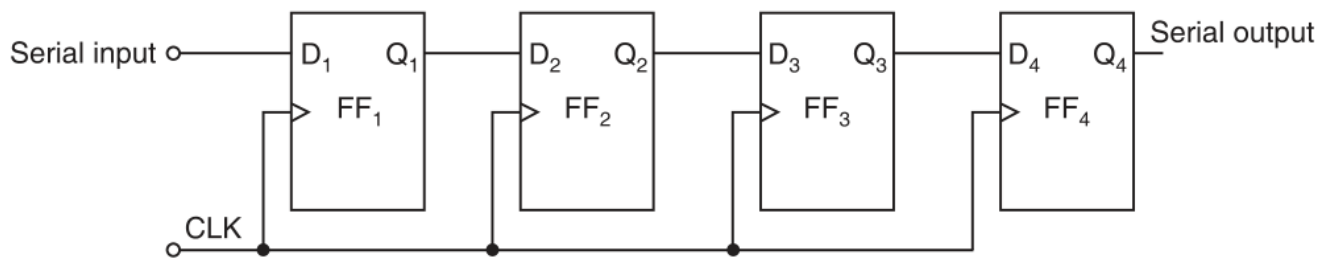
#### Need (Problem-First)

A single flip-flop stores only one bit. To store a word (like a 32-bit integer or an 8-bit ASCII character), we must assemble multiple flip-flops into a **Register**. Furthermore, communication channels (like a USB cable or Wi-Fi radio) transmit data **serially along a single wire**, while computer processors manipulate data **in parallel across 32 or 64 simultaneous bus wires**. We need circuits that can convert serial streams into parallel words and vice versa. These are **Shift Registers**.

#### The Four Fundamental Modes Defined:

##### 1. Serial-In Serial-Out (SISO) (Kumar, p. 640, Figure 11.4)

- **Mechanism:** Data bits enter one at a time on a single wire and exit one at a time from the last stage.
- To store an **n-bit** word requires **n clock pulses**. To retrieve it completely requires another **n clock pulses** (total **2n** pulses).



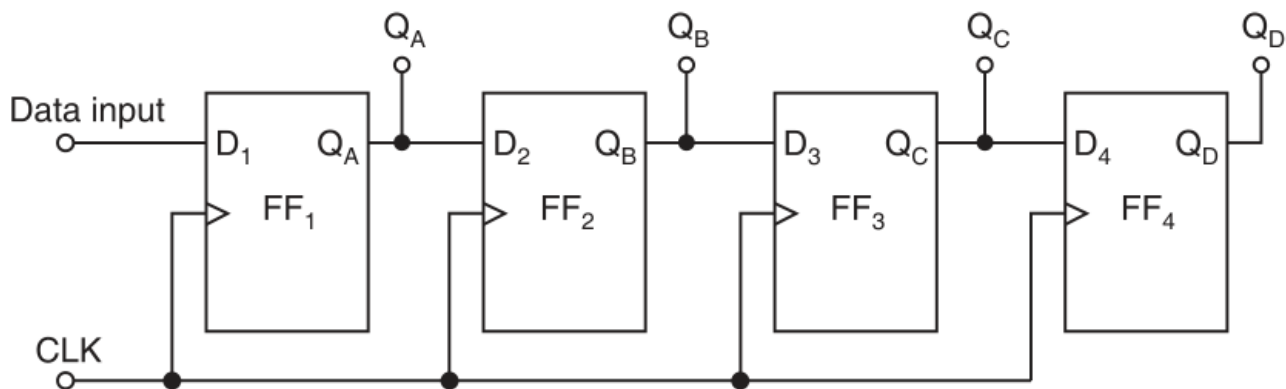
**Figure 11.4** 4-bit serial-in, serial-out, shift-right, shift register.

Figure

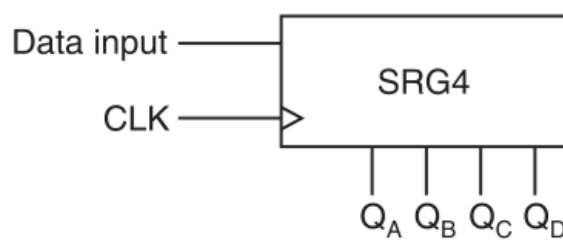
11.4: Logic diagram of a 4-bit Serial-In Serial-Out (SISO) shift register. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 640 (printed p. 608).

## 2. Serial-In Parallel-Out (SIPO) (Kumar, p. 641, Figure 11.7)

- **Mechanism:** Data enters serially along one input pin. Once all bits have been shifted in ( $n$  clock pulses), all flip-flop outputs ( $Q_0$ ,  $Q_1$ ,  $Q_2$ ,  $Q_3$ ) are read **simultaneously in parallel**.
- Essential for: **UART serial receivers** and **USB-to-parallel decoders**.



(a) Logic diagram



(b) Logic symbol

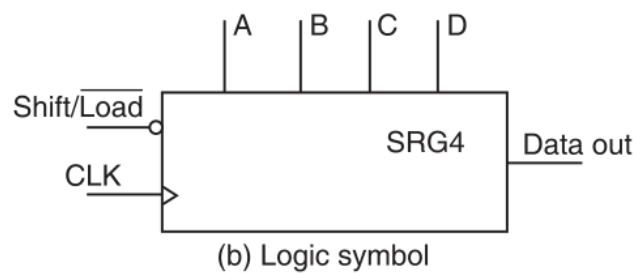
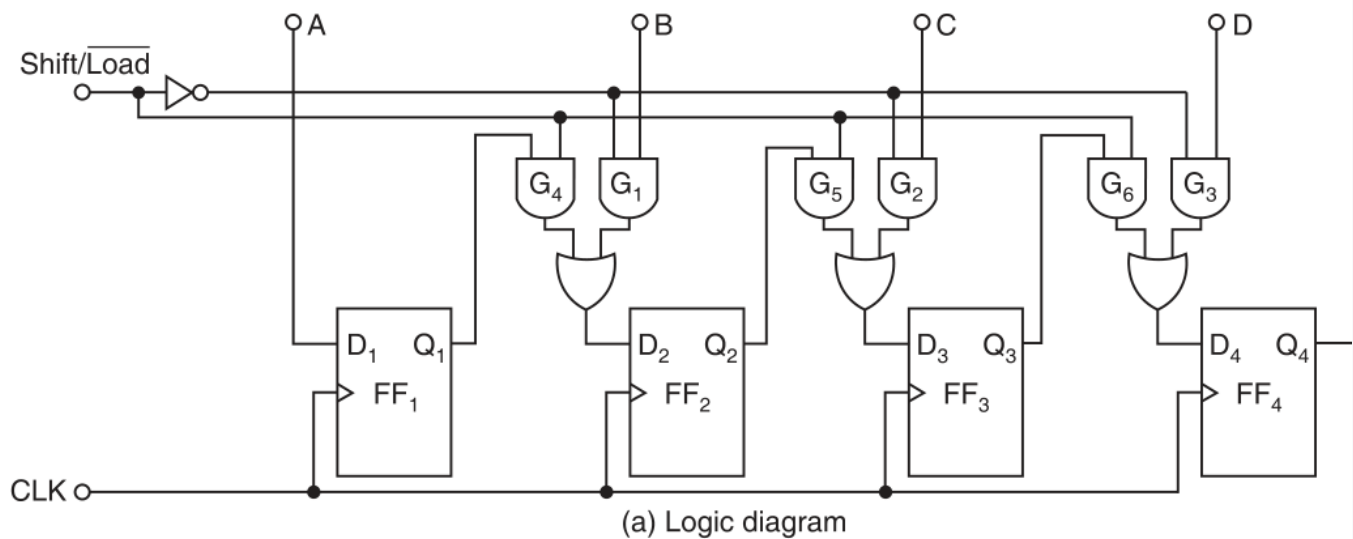
**Figure 11.7** A 4-bit serial-in, parallel-out, shift register.

Figure

11.7: Logic diagram of a 4-bit Serial-In Parallel-Out (SIPO) shift register. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 641 (printed p. 609).

## 3. Parallel-In Serial-Out (PISO) (Kumar, p. 642, Figure 11.8)

- **Mechanism:** An entire multi-bit word is loaded into all flip-flops simultaneously in **one clock pulse** using a **SHIFT/over Line{LOAD}** control line. The bits are then clocked out serially one by one along the output wire.
- Essential for: **Microprocessor serial transmitters**.



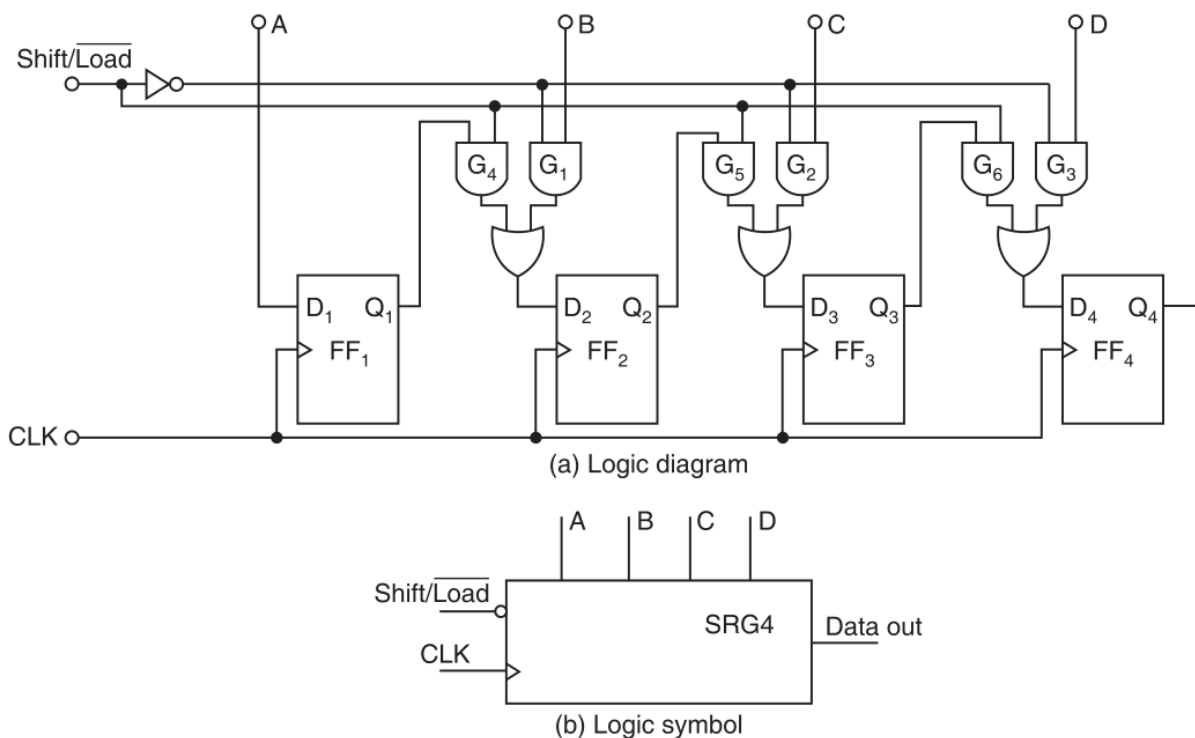
**Figure 11.8** A 4-bit parallel-in, serial-out, shift register.

Figure

**11.8:** Logic diagram of a 4-bit Parallel-In Serial-Out (PISO) shift register with Shift/Load steering gates. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 642 (printed p. 610).

#### 4. Parallel-In Parallel-Out (PIPO) (Kumar, p. 642, Figure 11.9)

- **Mechanism:** Data is loaded simultaneously in **one clock pulse** and read out simultaneously on the next clock pulse.
- Introduces zero serial delay; functions as a temporary **storage buffer register** inside CPUs.



**Figure 11.8** A 4-bit parallel-in, serial-out, shift register.

## 11.8 PARALLEL-IN, PARALLEL-OUT, SHIFT REGISTER

In a parallel-in, parallel-out, shift register, the data is entered into the register in parallel form, and also the data is taken out of the register in parallel form. Immediately following the simultaneous entry of all data bits, the bits appear on the parallel outputs.

Figure 11.9 shows a 4-bit parallel-in, parallel-out, shift register using D FFs. Data is applied to the D input terminals of the FFs. When a clock pulse is applied, at the positive-going edge of that pulse, the D inputs are shifted into the Q outputs of the FFs. The register now stores the data. The stored data is available instantaneously for shifting out in parallel form.

Figure

11.9: Logic diagram of a 4-bit Parallel-In Parallel-Out (PIPO) buffer register. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 642 (printed p. 610).

## PART 4: Binary Counters (Ripple & Synchronous) and Traffic Light Control

### 4.9 Asynchronous (Ripple) Counters (Up to 4 Bits)

Source: Kumar (Ch. 12, pp. 662–671, Figures 12.1 & 12.8); Floyd (Ch. 8, pp. 440–446, Figure 11)

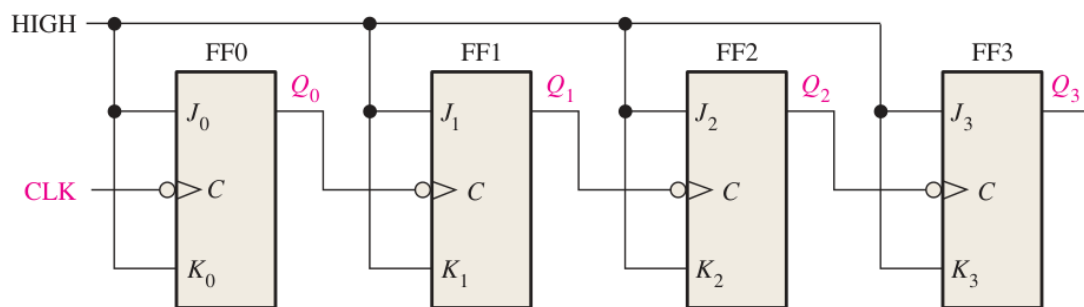
[Pacing: Essential Counter Foundations — Trace the Clock Propagation Delay]

#### Need (Problem-First)

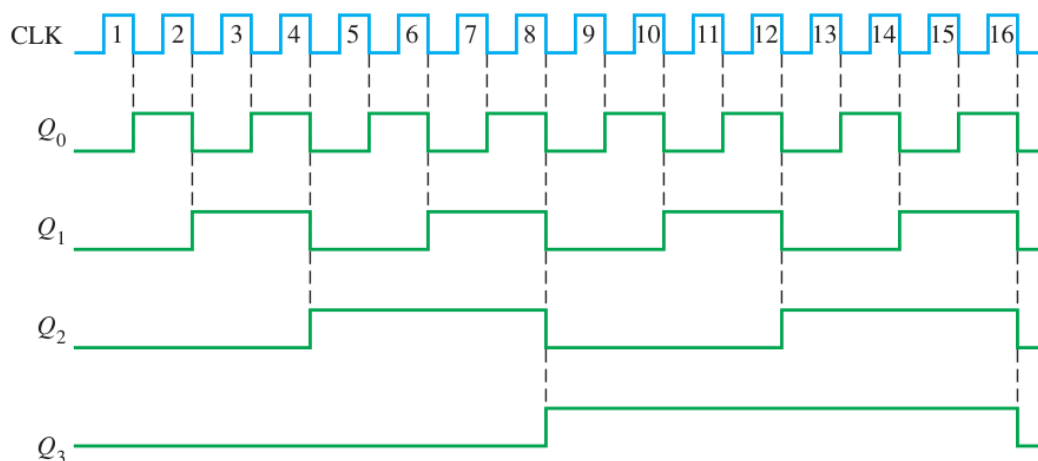
How does a digital system count physical events (e.g., number of cars entering a garage, seconds ticking on a clock, or clock cycles in a program)? We need a circuit that cycles through a sequence of binary numbers upon receiving incoming electrical pulses.

#### 1. The 4-Bit Binary Ripple Up-Counter (Floyd, p. 444, Figure 11)

Four toggle flip-flops (T or JK with  $J=K=1$ ) are cascaded: - **The Asynchronous Wiring Rule:** The external clock is connected **ONLY** to the first flip-flop (**FF<sub>0</sub>**)! - Each subsequent flip-flop receives its clock input **from the output of the preceding stage** (**Q<sub>{i-1}</sub>**)!



(a)



(b)

**FIGURE 11** Four-bit asynchronous binary counter and its timing diagram. Open file F08-11 and verify the operation.

\*Answers are at the end of the chapter.

MULTISIM



Figure

11: Schematic diagram and timing waveforms of a 4-bit asynchronous (ripple) binary up-counter. Sourced from Thomas L. Floyd, Digital Fundamentals: A Systems Approach (1st Ed.), p. 444 (printed p. 438).

### Characteristic 13 Walkthrough: Hardware Reality Check

1. Because flip-flops trigger on **negative (falling) clock edges** ( $1 \rightarrow 0$ ), **FF\_1** toggles only when **Q\_0** transitions from **1** to **0**.
2. This creates automatic frequency division: **Q\_0** toggles every 1 clock cycle ( $f/2$ ), **Q\_1** toggles every 2 cycles ( $f/4$ ), **Q\_2** every 4 cycles ( $f/8$ ), and **Q\_3** every 8 cycles ( $f/16$ ).
3. The outputs **Q\_3 Q\_2 Q\_1 Q\_0** count up in perfect binary sequence: **0000**  $\rightarrow$  **0001**  $\rightarrow$  **0010**  $\rightarrow$  **dots**  $\rightarrow$  **1111** **2** (**0** to **15\_{10}**).

### The Fatal Flaw of Ripple Counters: Cumulative Propagation Delay

Source: Kumar (p. 670); Floyd (p. 443)

Because each flip-flop is clocked by the preceding flip-flop, the transition delay ripples through the chain like falling dominoes. In an  $n$ -bit ripple counter:

$$t_{\text{total delay}} = n \times t_{\text{pd}}$$

For a 4-bit counter with  $t_{\text{pd}} = 10 \text{ ns}$ , the MSB changes **40 ns** after the clock edge! During that **40 ns** window, the outputs pass through false, spurious intermediate counts (glitches), destroying accuracy at high clock frequencies.

## 2. The Modulo-10 (Decade / BCD) Ripple Counter (Kumar, p. 669, Figure 12.8)

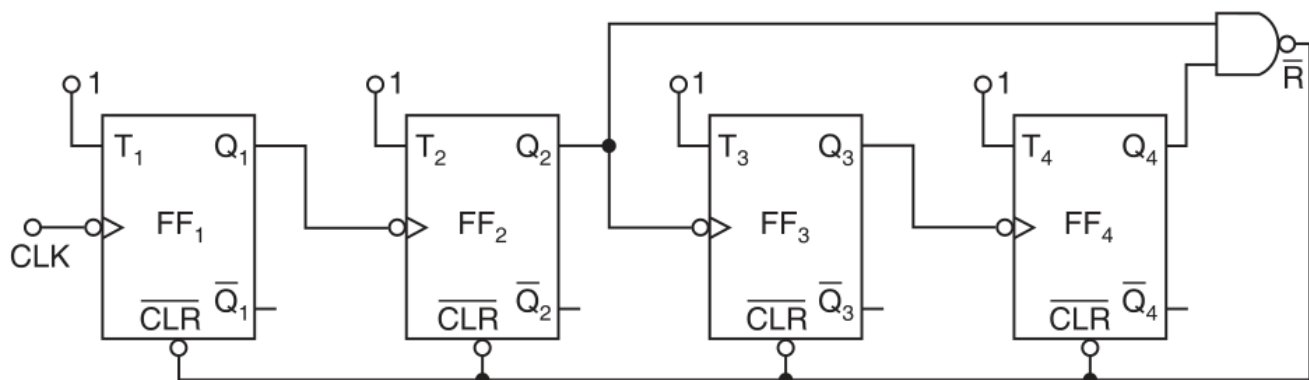
A standard 4-bit counter naturally counts through 16 states (0 to 15, Modulo-16). But decimal clocks and BCD systems must count from 0 to 9 and instantly reset back to 0 on the tenth pulse (Modulo-10). - **The Feedback NAND Reset Mechanism:** Decimal count 10\_{10} in binary is 1010\_2 ( $Q_3=1$ ,  $Q_2=0$ ,  $Q_1=1$ ,  $Q_0=0$ ). Connect lines  $Q_3$  and  $Q_1$  to the inputs of a 2-input NAND gate, and connect the NAND output to the active-LOW Asynchronous Clear (CLR) pins of all four flip-flops:

| After pulses | Count |       |       |       |
|--------------|-------|-------|-------|-------|
|              | $Q_4$ | $Q_3$ | $Q_2$ | $Q_1$ |
| 0            | 0     | 0     | 0     | 0     |
| 1            | 0     | 0     | 0     | 1     |
| 2            | 0     | 0     | 1     | 0     |
| 3            | 0     | 0     | 1     | 1     |
| 4            | 0     | 1     | 0     | 0     |
| 5            | 0     | 1     | 0     | 1     |
| 6            | 0     | 1     | 1     | 0     |
| 7            | 0     | 1     | 1     | 1     |
| 8            | 1     | 0     | 0     | 0     |
| 9            | 1     | 0     | 0     | 1     |
| 10           | 0     | 0     | 0     | 0     |

(a) Count table

| $Q_4Q_3$ \ $Q_2Q_1$ |  | 00 | 01 | 11 | 10 |
|---------------------|--|----|----|----|----|
| 00                  |  | 0  | 1  | 3  | 2  |
| 01                  |  | 4  | 5  | 7  | 6  |
| 11                  |  | 12 | 13 | 15 | 14 |
| 10                  |  | 8  | 9  | 11 | 10 |

(b) K-Map



(c) Logic diagram

**Figure 12.8** Asynchronous mod-10 counter using T flip-flops.

Figure

12.8: Count sequence table, K-map, and logic schematic of an asynchronous Modulo-10 (Decade) counter using feedback clear gating. Sourced from A.

Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 669 (printed p. 637).

- When the counter reaches 9 (1001),  $Q_1$  is 0, so the NAND output remains HIGH (inactive).
- On the next pulse, the counter briefly touches 10 (1010). Instantly, both  $Q_3=1$  and  $Q_1=1$ . The NAND gate output snaps LOW (0 V), pulsing the CLR pins and resetting all flip-flops back to 0000 in just a few nanoseconds!

#### 4.10 Synchronous Counters (Up to 4 Bits)

Source: Kumar (Ch. 12, pp. 674–686, Figure 12.16); Floyd (Ch. 8, pp. 446–451, Figure 19)

[Pacing: Conceptually Critical — The High-Speed Industry Standard]

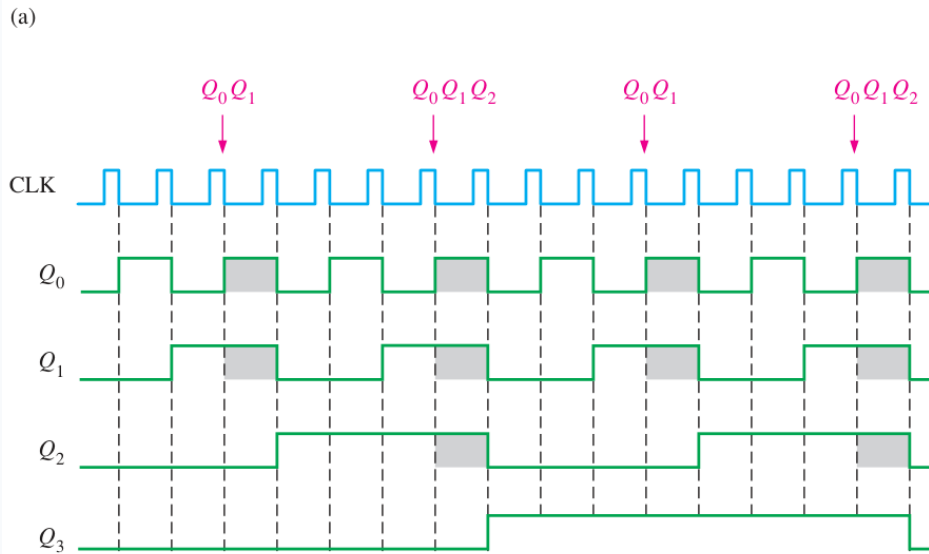
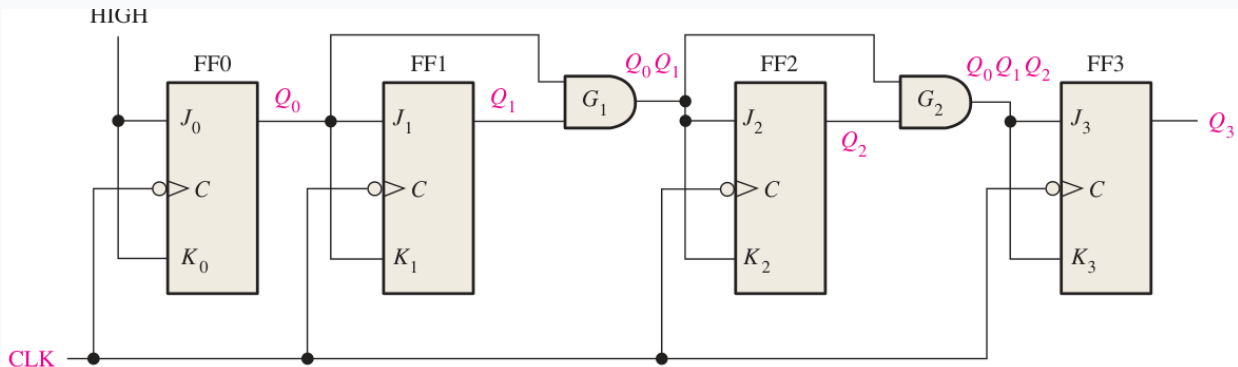
##### Need (Problem-First)

To eliminate the cumulative ripple propagation delay of asynchronous counters, we need a counter where **every single flip-flop is connected to the exact same master clock wire simultaneously**.

##### The Synchronous Mechanism (Floyd, p. 450, Figure 19)

Because all flip-flops receive the clock edge at the exact same picosecond, they must decide *ahead of time* whether they should toggle or stay put. - FF\_0 must toggle on **every single clock pulse** ■  $J_0 = K_0 = 1$ . - FF\_1 must toggle only when  $Q_0 =$

1 ■  $J_1 = K_1 = Q_0$ . - FF\_2 must toggle only when both  $Q_0 = 1$  AND  $Q_1 = 1$  ■  $J_2 = K_2 = Q_0 Q_1$ . - FF\_3 must toggle only when  $Q_0 = 1, Q_1 = 1, \text{ AND } Q_2 = 1$  ■  $J_3 = K_3 = Q_0 Q_1 Q_2$ .



**FIGURE 19** A 4-bit synchronous binary counter and timing diagram. Points where the AND gate outputs are HIGH are indicated by the shaded areas.

Figure

19: Schematic diagram and timing waveforms of a 4-bit Synchronous Binary Counter showing parallel clock distribution and carry-enable AND gates.

Sourced from Thomas L. Floyd, *Digital Fundamentals: A Systems Approach (1st Ed.)*, p. 450 (printed p. 444).

- **Total Propagation Delay:** Exactly **one flip-flop delay ( $t_{pd}$ )** regardless of how many bits are added! All flip-flops switch simultaneously in unison.

## 4.11 Ring Counters and Johnson (Twisted-Ring) Counters

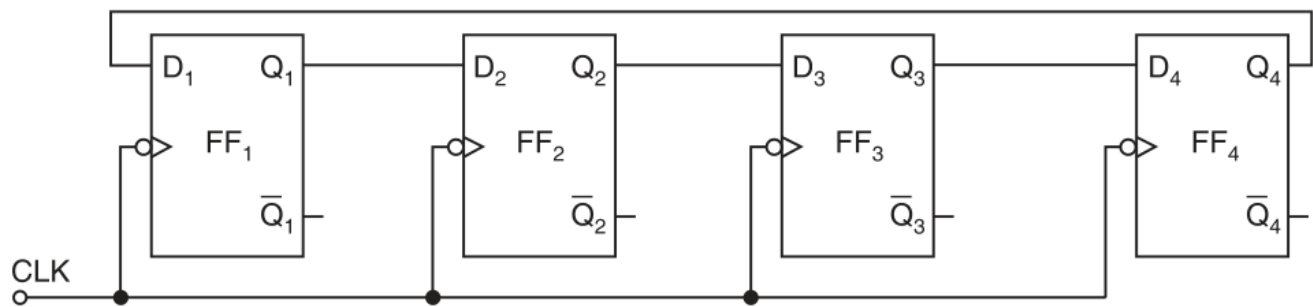
Source: Kumar (Ch. 12, pp. 697–700, Figures 12.57 & 12.61)

[Pacing: Elegant Shift-Register Counter Architectures]

### 1. The 4-Bit Ring Counter (Kumar, p. 697, Figure 12.57)

A 4-bit shift register where the output of the last flip-flop is fed straight back into the input of the first flip-flop ( $D_0 = Q_3$ ): - **Initial Preset:** A single 1 is pre-loaded: 1000. - On successive clock pulses, the 1 simply circulates in a ring:

1000 → 0100 → 0010 → 0001 → 1000. - **Advantage:** Each state directly activates a control line without requiring any decoding gates!



**Figure 12.57** Logic diagram of a 4-bit ring counter using D flip-flops.

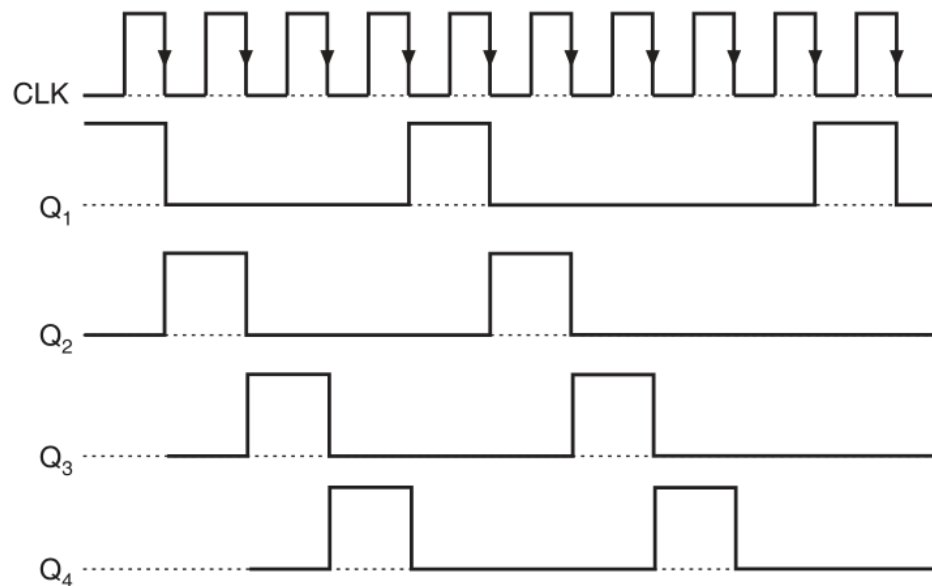
Figure

12.57: Logic diagram of a 4-bit Ring Counter using D flip-flops. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 697 (printed p. 665).

## 2. The 4-Bit Johnson Counter (Twisted-Ring Counter) (Kumar, p. 698, Figure 12.61)

The **inverted** output of the last flip-flop is fed back into the input of the first flip-flop ( $D_0 = \bar{Q}_3$ ): - Starting from 0000, 1s stream in from the left, followed by 0s:

0000 → 1000 → 1100 → 1110 → 1111 → 0111 → 0011 → 0001 → 0000. - An  $n$ -bit Johnson counter yields  $2n$  unique **timing states** (a 4-bit counter gives 8 states!), double that of a basic ring counter, while requiring only simple 2-input AND gates for complete decoding.

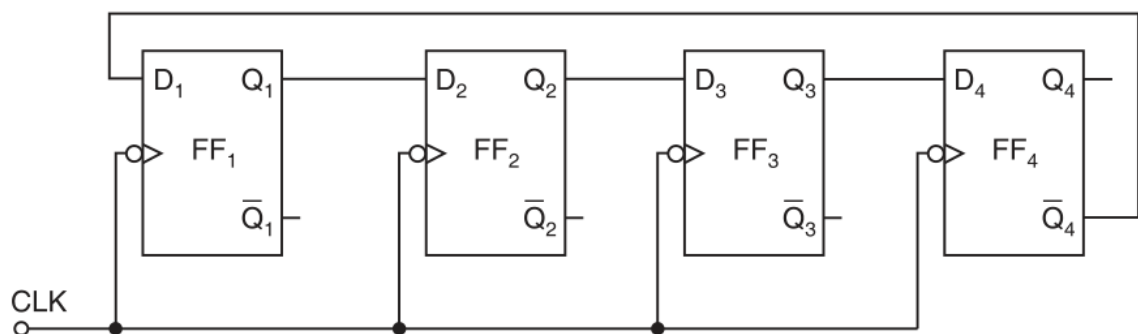


**Figure 12.60** Timing diagram of a 4-bit ring counter.

### 12.9.2 Twisted Ring Counter (Johnson Counter)

This counter is obtained from a serial-in, serial-out shift register by providing feedback inverted output of the last FF to the D input of the first FF. The Q output of each stage is connected to the D input of the next stage, but the  $\bar{Q}$  output of the last stage is connected to the D input of the first stage, therefore, the name *twisted ring counter*. This feedback arrangement produces a sequence of states.

The logic diagram of a 4-bit Johnson counter using D FFs is shown in Figure 12.61. A realization of the same using J-K FFs is shown in Figure 12.62. The state diagram and the truth table are shown in Figure 12.63. The timing diagram of a Johnson counter is shown in Figure 12.64.



**Figure 12.61** Logic diagram of a 4-bit twisted ring counter using D flip-flops.

Figure

**12.61:** Logic diagram of a 4-bit twisted ring (Johnson) counter. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 698 (printed p.

666).

## 4.12 Real-World Application: Traffic Light Control System

Source: Floyd (Ch. 6, pp. 322–326, Figures 1, 2, 3 & 4)

[Pacing: Comprehensive System Capstone — See How Everything Fits Together]

### Need (Problem-First)

We have studied gates, adders, decoders, flip-flops, and counters. How are these individual chips combined to control an actual automated system in the physical world? We analyze the design of an automated **Traffic Signal Controller for a busy Main**

Street intersected by an occasionally used Side Street (Floyd, p. 322).

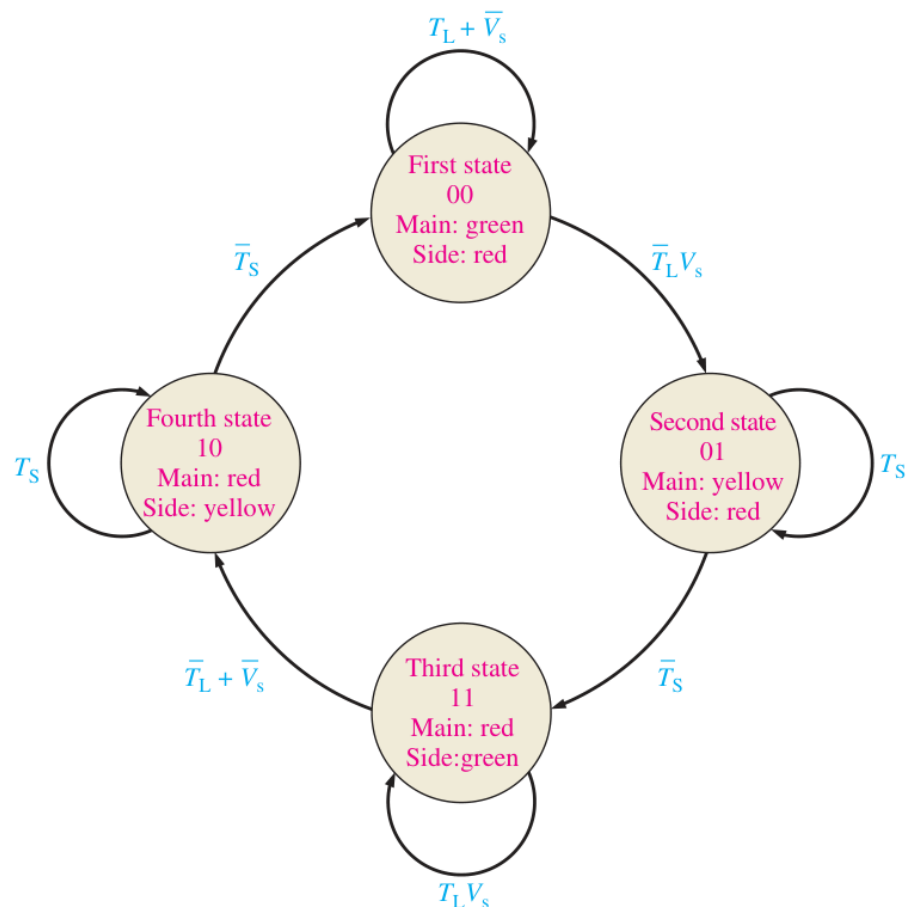
### System Specifications & Operational Requirements:

1. **State 1 (S\_1):** Main street light is **GREEN**; Side street light is **RED**. This state remains active for at least **25 seconds** (Long Timer,  $T_L$ ). If there are no vehicles detected on the side street ( $V_S = 0$ ), the main street stays GREEN indefinitely.
2. **State 2 (S\_2):** When side street traffic arrives ( $V_S = 1$ ) after **25 s**, Main street transitions to **YELLOW** for **4 seconds** (Short Timer,  $T_S$ ); Side street remains RED.
3. **State 3 (S\_3):** Main street is **RED**; Side street turns **GREEN**. This state lasts for **25 seconds** as long as side street vehicles remain present ( $T_L \ V_S$ ).
4. **State 4 (S\_4):** Main street remains **RED**; Side street transitions to **YELLOW** for **4 seconds** ( $T_S$ ). The system then loops back to State 1!

### 1. The State Diagram (Floyd, p. 323, Figure 2)

To eliminate multi-bit race conditions during state transitions, the four states are assigned a **2-bit Gray code sequence**: - State 1 (S\_1): 00 - State 2 (S\_2): 01 - State 3 (S\_3): 11 - State 4 (S\_4): 10

**FIGURE 2** State diagram for the traffic signal control system.

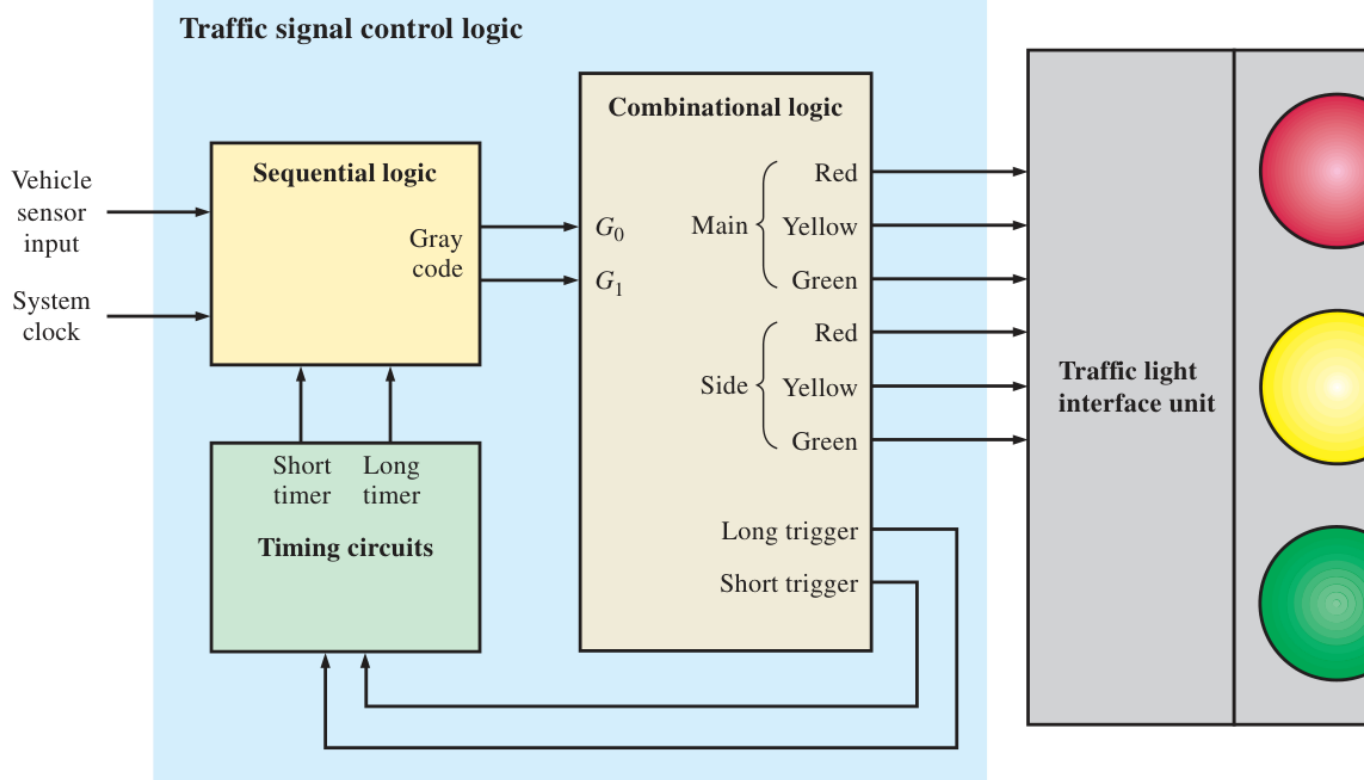


Figure

2: State diagram for the traffic signal controller showing 2-bit Gray code state assignments and transition conditions based on timers ( $T_L$ ,  $T_S$ ) and vehicle sensors ( $V_S$ ). Sourced from Thomas L. Floyd, *Digital Fundamentals: A Systems Approach* (1st Ed.), p. 323 (printed p. 317).

### 2. System Architecture & Block Diagram (Floyd, p. 324, Figure 3)

Figure 3 from Floyd reveals the complete three-part system decomposition: 1. **Sequential Logic (The State Counter):** A 2-bit Gray-code counter that sequences through states **00** → **01** → **11** → **10** based on inputs from the vehicle sensor and timers. 2. **Timing Circuits:** A **25 s** long timer ( $T_L$ ) and a **4 s** short timer ( $T_S$ ), triggered by transitions between states. 3. **Combinational Output Logic:** Decodes the 2-bit Gray code into six physical lamp driving signals: Main Red (**MR**), Main Yellow (**MY**), Main Green (**MG**), Side Red (**SR**), Side Yellow (**SY**), and Side Green (**SG**).



**FIGURE 3** Block diagram of the traffic signal control system.

Figure

3: Block diagram of the traffic signal control system showing interaction between sequential logic counter, timing circuits, combinational output decoder, and power interface. Sourced from Thomas L. Floyd, *Digital Fundamentals: A Systems Approach* (1st Ed.), p. 324 (printed p. 318).

#### Derivation of Combinational Light Output Equations (Floyd, p. 325):

- Main Green (**MG**) is ON only in State 1 (**S\_1**, code 00):

$$MG = G_1 G_0$$

- Main Yellow (**MY**) is ON only in State 2 (**S\_2**, code 01):

$$MY = G_1 \bar{G}_0$$

- Main Red (**MR**) is ON in both State 3 (**S\_3**, 11) and State 4 (**S\_4**, 10):

$$MR = G_1 G_0 + G_1 \bar{G}_0 = G_1 (G_0 + \bar{G}_0) = G_1$$

- Side Green (**SG**) is ON only in State 3 (**S\_3**, code 11):

$$SG = G_1 G_0$$

- Side Yellow (**SY**) is ON only in State 4 (**S\_4**, code 10):

$$SY = G_1 \bar{G}_0$$

- Side Red (**SR**) is ON in both State 1 (**S\_1**, 00) and State 2 (**S\_2**, 01):

$$SR = \bar{G}_1 \bar{G}_0 + \bar{G}_1 G_0 = \bar{G}_1$$

Notice how elegant digital design is: Main Red is simply bit  $G_1$ , and Side Red is simply  $\bar{G}_1$ !

## 1. Does This Make Sense? Self-Check

- **Question 1:** Why is a Master-Slave JK flip-flop immune to the race-around condition?
- **Answer:** Because the Master latch and Slave latch are driven by opposite clock phases (**CLK** and **CLK**). When the Master is accepting inputs, the Slave is completely disconnected from the output, breaking the feedback loop.
- **Question 2:** If an input clock signal has a frequency of **16 ext{ MHz}**, what is the frequency at the **Q** output of a single T flip-flop configured with **T=1**?
- **Answer:** Exactly half: **16 ext{ MHz} / 2 = 8 ext{ MHz}**.
- **Question 3:** In a 4-bit SISO shift register, how many total clock pulses are required to load a 4-bit word in and then shift it completely out?
- **Answer:** 4 clock pulses to shift data in, and another 4 clock pulses to shift data out (total 8 clock pulses).

## 2. Conceptual Misconceptions Corrected

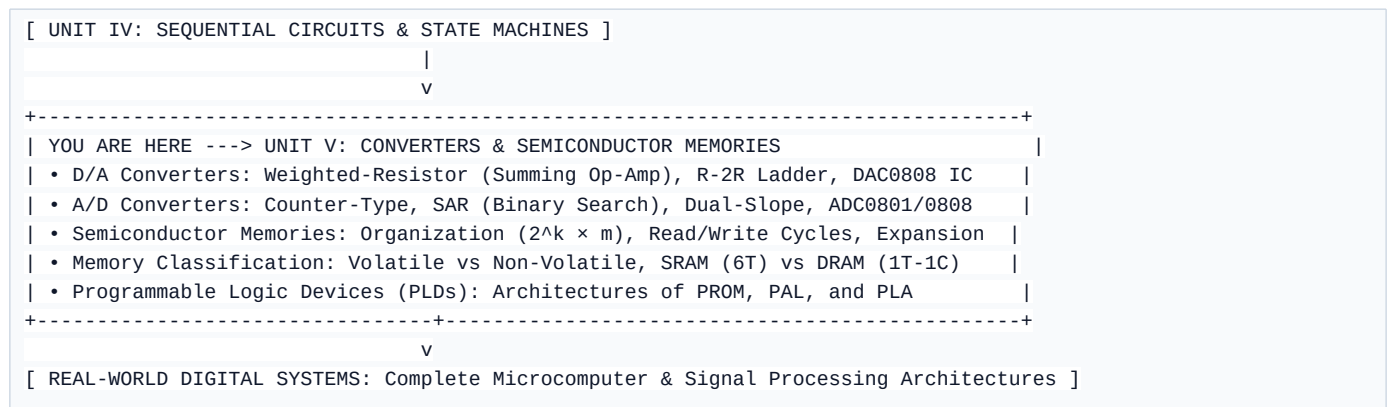
- **Misconception:** "A flip-flop and a latch are the exact same thing."  
**Correction:** No. A **latch** is level-triggered (transparent during the entire clock HIGH interval). A **flip-flop** is edge-triggered (sensitive only during the infinitesimal picosecond transition edge of the clock).
- **Misconception:** "In an asynchronous ripple counter, all flip-flops toggle at the exact same instant."  
**Correction:** Never! Only the first flip-flop toggles on the clock edge. Each subsequent stage is clocked by the preceding stage, creating a cumulative ripple delay ( $n \cdot t_{\{pd\}}$ ) that can corrupt high-speed systems.

---

# Unit V: Converters and Semiconductor Memories

---

## Master Subject Map: Unit V Position



In 3 lines: Units I through IV built the internal brain of a digital computer. Unit V completes the subject by connecting that digital brain to the physical universe. We master **DACs and ADCs** to translate between continuous real-world voltages and digital bits, explore mass **Semiconductor Memories (RAM & ROM)**, and examine user-customizable silicon chips (**PLDs: PROM, PAL, PLA**).

---

## Prerequisites for this Unit

Before studying Unit V, ensure you recall these concepts: 1. **Ohm's Law & Circuit References (Prereqs 1 & 2):** Current **I = V/R**, Ground (**0 V**), and Op-Amp Virtual Ground (**V<sub>-</sub> ≈ 0 V**). 2. **Binary Weights & Positional Values (Unit I, Section 1.1):** Binary digits carry weights  **$2^{\{-1\}} = 0.5$ ,  $2^{\{-2\}} = 0.25$ ,  $2^{\{-3\}} = 0.125$** , etc. 3. **Binary Counters & Shift Registers (Unit IV, Sections 4.8–4.10):** Stepping sequentially through states on clock edges. 4. **Decoders (Unit III, Section 3.8):** A **k-to- $2^k$**  decoder activates exactly one row wire for memory word addressing.

---

## PART 1: Digital-to-Analog Converters (DAC)

### 5.1 The Need for DAC and Key Specifications

Source: Kumar (Ch. 17, pp. 941–946, Figures 17.1 & 17.2); Floyd (Ch. 9, pp. 487–492)

[Pacing: Intuitive Foundations — Bridging Bits to Continuous Reality]

Need (Problem-First)

Microprocessors calculate with discrete numbers (e.g., 11010010). But physical actuators do not understand numbers. An audio speaker requires a continuously undulating analog voltage to move its cone; an electric vehicle motor requires a continuous voltage to set its speed; an MRI scanner requires continuous magnetic coils. How can a computer convert a digital binary word into a proportional continuous physical voltage? That bridge is the **Digital-to-Analog Converter (DAC)**.

Key DAC Performance Specifications Defined Inline:

- 1. **Resolution:** The smallest analog voltage increment that the DAC can produce. It corresponds to the voltage generated by a change of the **Least Significant Bit (LSB)** (Kumar, p. 944):

Step Size (  $\Delta V$  ) =  $\frac{V_{FS}}{2^n - 1}$

Where  $V_{FS}$  is the Full-Scale output voltage and  $n$  is the number of digital bits. Concrete Round-Number Example: An 8-bit DAC ( $n=8$ ) has  $2^8 - 1 = 255$  intervals. If  $V_{FS} = 5.10$  V, the resolution is:

$\Delta V = \frac{5.10 \text{ V}}{255} = 0.02 \text{ V} = 20 \text{ mV per step}$

- 1. **Full-Scale Output ( $V_{FS}$ ):** The maximum analog output voltage produced when all input bits are 1 (1111...1).
- 2. **Linearity:** How strictly the analog output follows a perfect straight line across all digital input codes.
- 3. **Accuracy:** The deviation between the actual measured analog output and the ideal theoretical output.
- 4. **Settling Time ( $t_s$ ):** The time required for the analog output to settle within  $\pm (1 / 2)$  LSB of its final value after an input code transition.

5.2 The Binary Weighted-Resistor DAC

Source: Kumar (Ch. 17, pp. 951–953, Figure 17.10)

[Pacing: Derivation Step-by-Step — Trace the Summing Node]

How It Works (The Physical Mechanism)

The circuit connects an Operational Amplifier (Op-Amp) configured as an **inverting summing amplifier**, with binary input switches driving resistors whose values are scaled inversely to their binary weights ( $R$ ,  $2R$ ,  $4R$ ,  $8R$ ):

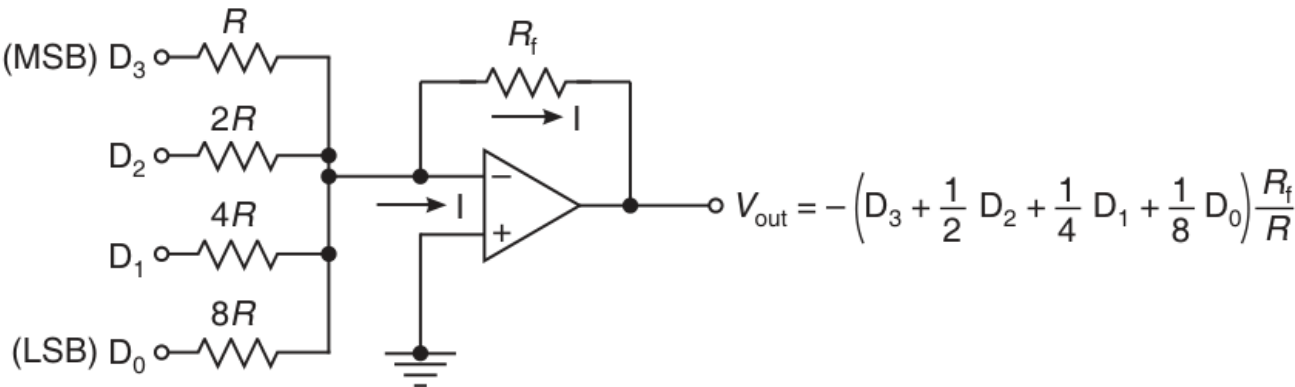


Figure 17.10 Weighted-resistor type DAC.

Figure

17.10: Binary Weighted-Resistor type Digital-to-Analog Converter using an operational amplifier summing network. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 951 (printed p. 919).

Characteristic 13 Walkthrough: Hardware Reality Check

- 1. The non-inverting terminal (+) of the op-amp is connected to Ground (0 V).
- 2. Due to negative feedback through  $R_f$ , the inverting terminal (-) is held at **Virtual Ground (0.00 V)** (Kumar, p. 951).

3. Digital bits ( $D_3, D_2, D_1, D_0$ ) control electronic switches. If bit  $D_i = 1$ , the resistor is connected to reference voltage  $-V_{\text{ref}}$ ; if  $D_i = 0$ , the resistor is connected to ground ( $0\text{ V}$ ).
4. The current flowing through each weighted resistor is determined strictly by Ohm's Law into virtual ground:
5.  $I_3 = \frac{V_{\text{ref}}}{R} D_3$  (MSB current)
6.  $I_2 = \frac{V_{\text{ref}}}{2R} D_2 = (1/2) I_3$
7.  $I_1 = \frac{V_{\text{ref}}}{4R} D_1 = (1/4) I_3$
8.  $I_0 = \frac{V_{\text{ref}}}{8R} D_0 = (1/8) I_3$  (LSB current)
9. Because the op-amp input draws zero current, by Kirchhoff's Current Law, all individual branch currents sum into the feedback resistor  $R_f$ :

$$I_{\text{total}} = I_3 + I_2 + I_1 + I_0 = \frac{V_{\text{ref}}}{R} \leq \text{ft}(D_3 + (1/2)D_2 + (1/4)D_1 + (1/8)D_0\text{right})$$

10. The output voltage is the drop across  $R_f$ :

$$V_{\text{out}} = -I_{\text{total}} \cdot R_f = -V_{\text{ref}} (R_f / R) \leq \text{ft}(D_3 2^{-0} + D_2 2^{-1} + D_1 2^{-2} + D_0 2^{-3}\text{right})$$

### The Fatal Engineering Flaw of Weighted Resistors: The Resistor Spread Problem

In a 4-bit DAC, resistors range from  $R$  to  $8R$  (a factor of 8). But in a modern **12-bit DAC**, the resistors must range from  $R$  to  $2^{11}R = 2048R$ ! - If  $R = 10\text{ k}\Omega$ , the LSB resistor must be **20.48 MΩ**! - **Fabrication Disaster**: In integrated circuit silicon manufacturing, it is virtually impossible to fabricate resistors spanning four orders of magnitude ( $10\text{ k}\Omega$  to  $20\text{ M}\Omega$ ) that track each other accurately across temperature changes. A tiny **0.05%** error in the MSB resistor completely swamps the entire LSB current!

## 5.3 The R-2R Ladder DAC: The Universal Solution

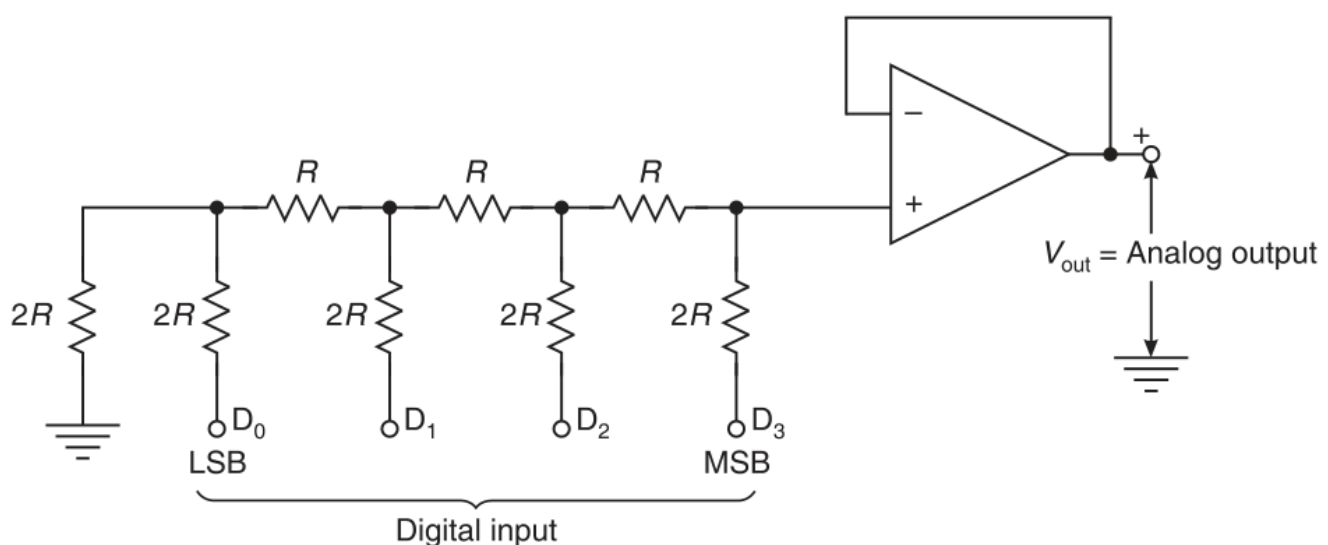
Source: Kumar (Ch. 17, pp. 946–950, Figure 17.5); Floyd (Ch. 9, pp. 490–492)

[Pacing: Conceptually Brilliant — Master Why Two Resistor Values Suffice]

### Need (Problem-First)

We need a digital-to-analog converter that works for any number of bits (8, 12, 16, or 24 bits) but uses **only TWO precision resistor values** that can be effortlessly matched on a silicon die. That circuit is the **R-2R Ladder**.

The R-2R Ladder Circuit Architecture (Kumar, p. 947, Figure 17.5)



**Figure 17.5** R-2R ladder type DAC.

Figure

17.5: Schematic diagram of an R-2R ladder Digital-to-Analog Converter feeding an operational amplifier buffer. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 947 (printed p. 915).

## Why It Works: The Continuous Binary Current Division

The R-2R ladder uses only two resistor values: **R** (e.g., **10 k Ω**) and **2R** (e.g., **20 k Ω**).

- Look at the termination on the extreme left: a vertical **2R** resistor is in parallel with another **2R** resistor. Their equivalent parallel resistance is:

$$2R \parallel 2R = R$$

- This equivalent **R** is in series with the horizontal resistor **R**, yielding **R + R = 2R**!
- At the next node, this **2R** is in parallel with the vertical **2R**, yielding **R** again!
- **The Binary Splitting Principle:** Looking into *any* node of the ladder toward the left, the equivalent resistance is **always exactly R**!
- Therefore, at every single node, incoming current splits **exactly in half (50% / 50%)**!
- The currents arriving at the summing bus are naturally, automatically weighted in exact powers of two:

$$(I / 2), (I / 4), (I / 8), (I / 16) \text{ dots}$$

- **Output Equation:**

$$V_{\text{out}} = V_{\text{ref}} \leq \text{ft}((D_3 / 2^1) + (D_2 / 2^2) + (D_1 / 2^3) + (D_0 / 2^4)\text{right})$$

- **Manufacturing Triumph:** Because only two resistor values are required, laser-trimmed thin-film silicon resistors maintain near-perfect tracking over temperature, making R-2R the foundation of commercial DACs.

**Commercial DAC Example: DAC0808 / DAC0800 IC (Kumar, pp. 954–955)**

An 8-bit monolithic multiplying DAC featuring an R-2R ladder, fast settling time (**150 ns**), and full 8-bit accuracy (**± 0.19%**).

---

## PART 2: Analog-to-Digital Converters (ADC)

### 5.4 The Need for ADC and Key Parameters

Source: Kumar (Ch. 17, pp. 957–968); Floyd (Ch. 9, pp. 492–502)

[Pacing: Intuitive Foundations — Translating the Analog World into Bits]

#### Need (Problem-First)

The real physical world is entirely analog. Sound waves entering a microphone, engine temperatures measured by a thermocouple, and blood pressure in a medical monitor are continuous, fluid voltages. A digital computer cannot directly process an infinite continuum of values. We must sample the continuous signal and quantize it into a sequence of binary words. That device is the **Analog-to-Digital Converter (ADC)**.

#### Key ADC Specifications Defined Inline:

1. **Resolution:** The number of bits in the output binary word (e.g., 8-bit, 12-bit, 16-bit). An **n-bit** ADC divides the analog voltage range into **2<sup>n</sup>** discrete quantization levels.
2. **Quantization Error (Q<sub>e</sub>):** Because a continuous voltage is rounded to the nearest discrete digital level, an inherent rounding error exists in all ADCs (Kumar, p. 957):

$$Q_e = \pm (1 / 2) \text{ LSB}$$

This error can never be eliminated by circuit tuning; it is reduced only by adding more bits. 3. **Conversion Time (t<sub>c</sub>):** The total elapsed time from the command to start conversion (Start of Conversion, **SOC**) to the moment the valid digital word appears at the output pins (End of Conversion, **EOC**).

---

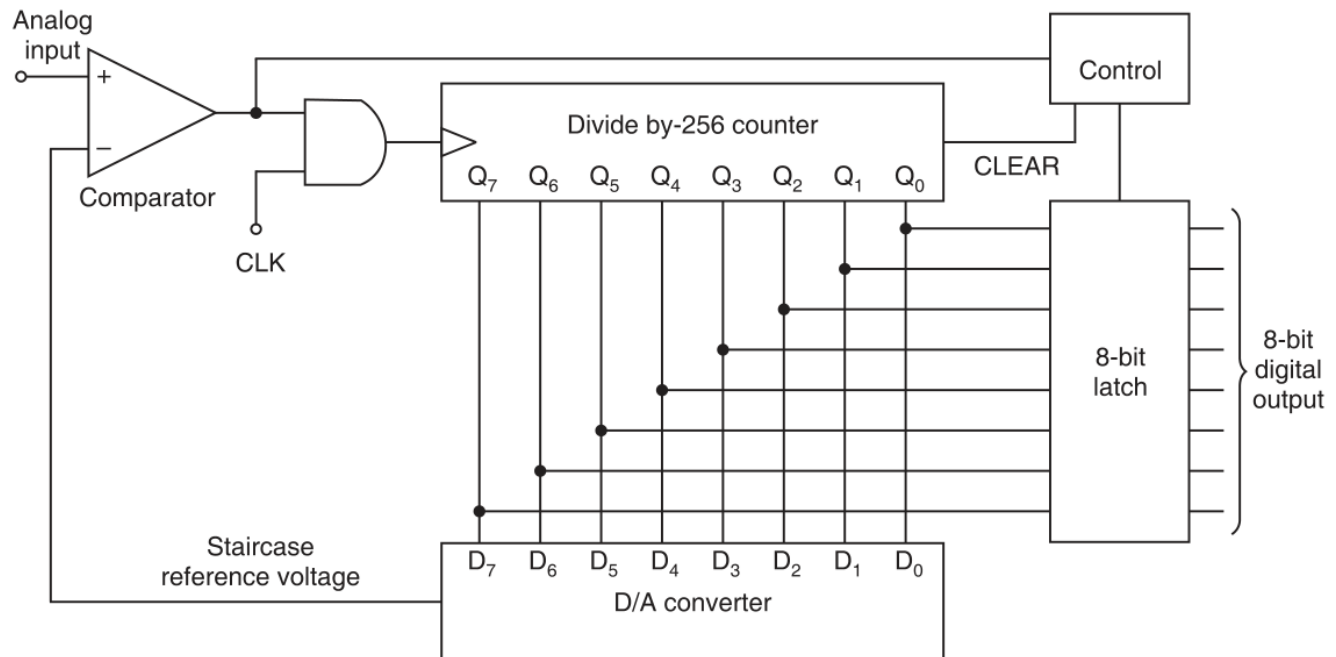
### 5.5 Counting (Ramp) ADC

Source: Kumar (Ch. 17, pp. 958–960, Figure 17.15)

[Pacing: Simple & Intuitive — The Staircase Comparator Method]

## How It Works (The Physical Mechanism)

The Counter-Type ADC uses a **closed-loop feedback system** consisting of an analog comparator, a binary counter, and an internal DAC (Kumar, p. 958, Figure 17.15):



**Figure 17.15** Logic diagram of the counter-type ADC.

Figure

**17.15:** Logic schematic of a Counter-Type (Ramp) Analog-to-Digital Converter. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 958 (printed p. 926).

### Characteristic 13 Walkthrough: Hardware Reality Check

1. A **Start pulse** resets the binary counter to 0000. The DAC output voltage is  $V_{\{DAC\}} = 0 \text{ V}$ .
2. The analog input voltage  $V_A$  is connected to the non-inverting (+) input of the comparator, and  $V_{\{DAC\}}$  connects to the inverting (-) input.
3. Because  $V_A > V_{\{DAC\}}$ , the comparator outputs **HIGH (1)**, which enables an AND gate, allowing master clock pulses to enter the counter.
4. The counter counts upward (0, 1, 2, 3 dots), causing the internal DAC output  $V_{\{DAC\}}$  to climb upward like a **staircase ramp**.
5. The instant  $V_{\{DAC\}}$  climbs just a fraction of a millivolt above  $V_A$  ( $V_{\{DAC\}} \geq V_A$ ), the comparator output **snaps LOW (0 V)**!
6. The AND gate is instantly inhibited, freezing the clock. The counter stops counting.
7. The binary word frozen in the counter is the exact digital equivalent of the input voltage  $V_A$ !

### Limitation: Variable and Slow Conversion Time

- If  $V_A$  is small (e.g., 0.1 V), the counter stops in 5 clock cycles.
- If  $V_A$  is full-scale, an 8-bit counter must count through all  $2^8 = 256$  clock cycles!
- In an  $n$ -bit counter, the maximum conversion time is:

$$t_{\{max\}} = 2^n \times T_{\{clock\}}$$

This massive variation in conversion time makes the counting ADC too slow for audio or video.

## 5.6 Successive Approximation ADC (SAR ADC)

Source: Kumar (Ch. 17, pp. 965–967, Figure 17.24); Floyd (Ch. 9, pp. 494–498)

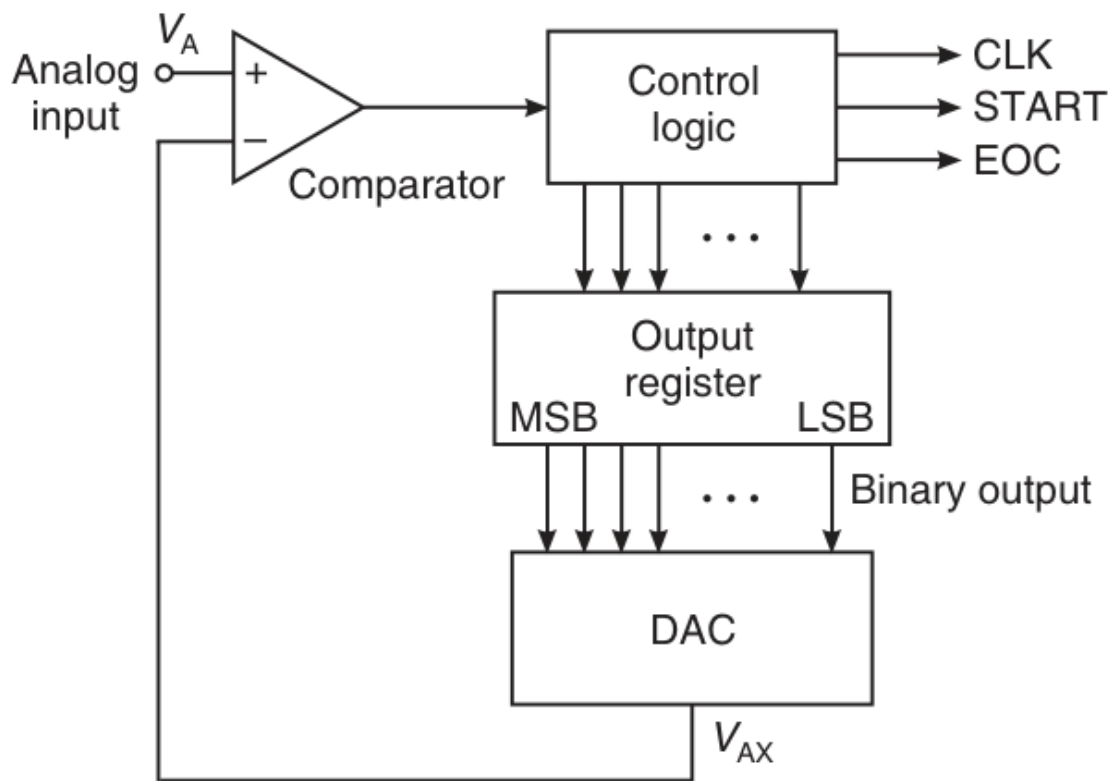
[Pacing: Conceptually Heavy — The Binary Search Algorithm in Hardware]

### Need (Problem-First)

We need an ADC that is fast, highly accurate, and has a **fixed, predictable conversion time** that does not depend on whether the input voltage is high or low.

### The Binary Search Algorithm in Hardware (Kumar, p. 965, Figure 17.24)

Instead of counting up slowly by 1s (like a linear search), the SAR ADC uses an intelligent **binary search algorithm** (the same strategy you use when guessing a secret number between 1 and 100 by guessing 50 first):



**Figure 17.24** The successive-approximation type ADC.

Figure

17.24: Block diagram of a Successive-Approximation Register (SAR) Analog-to-Digital Converter. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 965 (printed p. 933).

### Characteristic 13 Walkthrough: Step-by-Step Conversion

Consider a 4-bit SAR ADC with full-scale  $V_{FS} = 16 \text{ V}$  (each step is  $1 \text{ V}$ ).

Suppose the incoming analog voltage is  $V_A = 11.4 \text{ V}$ . - **Clock Cycle 1 (Test MSB,  $D_3 = 8 \text{ V}$ ):**

SAR sets MSB to **1**: trial word = **1000** (**8 V**).

Internal DAC produces **8 V**.

Comparator tests: Is  $V_A$  (**11.4 V**) > **8 V**? **YES!**

**Decision:** Keep  $D_3 = 1$ . - **Clock Cycle 2 (Test Next Bit,  $D_2 = 4 \text{ V}$ ):**

SAR sets  $D_2$  to **1**: trial word = **1100** (**8 + 4 = 12 V**).

Internal DAC produces **12 V**.

Comparator tests: Is  $V_A$  (**11.4 V**) > **12 V**? **NO!** (It's too high!)

**Decision:** Reset  $D_2 = 0$ . - **Clock Cycle 3 (Test Next Bit,  $D_1 = 2 \text{ V}$ ):**

SAR sets  $D_1$  to **1**: trial word = **1010** (**8 + 0 + 2 = 10 V**).

Internal DAC produces **10 V**.

Comparator tests: Is  $V_A$  (**11.4 V**) > **10 V**? **YES!**

**Decision:** Keep  $D_1 = 1$ . - **Clock Cycle 4 (Test LSB,  $D_0 = 1 \text{ V}$ ):**

SAR sets  $D_0$  to **1**: trial word = **1011** (**8 + 0 + 2 + 1 = 11 V**).

Internal DAC produces **11 V**.

Comparator tests: Is  $V_A$  (11.4 V) > 11 V? YES!

**Decision:** Keep  $D_0 = 1$ . - **End of Conversion (EOC):** The SAR asserts the **EOC** pin. Final digital word = 1011 (11\_{10})! -

**Fixed Conversion Time:** Notice that for an  $n$ -bit SAR ADC, conversion **always takes exactly  $n$  clock cycles!**

$$t_{\text{conversion}} = n \times T_{\text{clock}}$$

An 8-bit SAR ADC finishes in just **8 clock cycles**, over **30 times faster** than a counting ADC!

**Commercial ADC IC:** ADC0801 / ADC0808 (Kumar, p. 967, Figure 17.25)

A classic 8-bit successive approximation ADC with on-chip clock generator, active-low chip select (**CS**), write/start (**WR**), and end-of-conversion interrupt (**INTR**).

## 5.7 Dual-Slope Integrating ADC

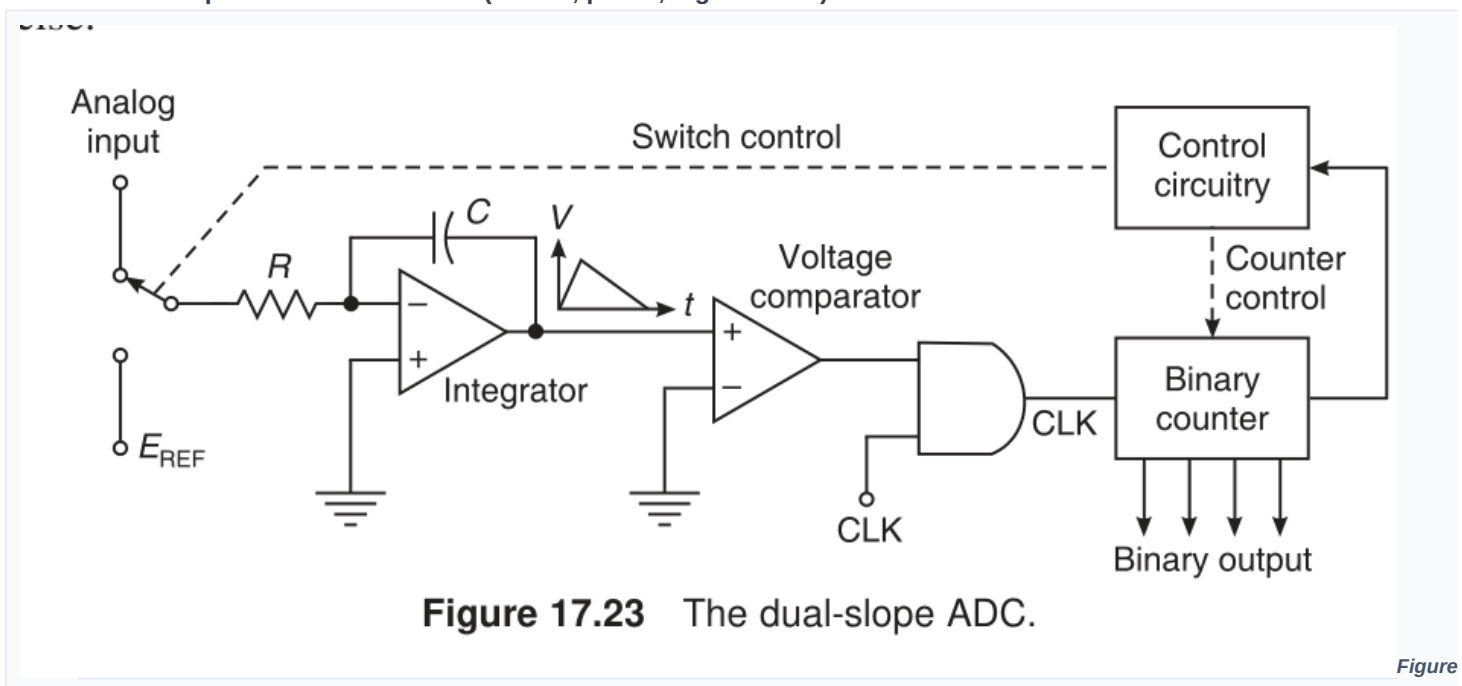
Source: Kumar (Ch. 17, pp. 964–965, Figure 17.23); Floyd (Ch. 9, pp. 498–500)

[Pacing: Mathematical Masterpiece — The Heart of Digital Multimeters]

### Need (Problem-First)

In industrial and medical measurement instruments (like digital multimeters), high speed is not required, but **extreme precision and total immunity to 50 Hz / 60 Hz AC power line noise** are mandatory. SAR ADCs suffer from component drift in their DAC resistors. We need an ADC whose mathematical accuracy is **completely independent of component tolerances (independent of  $R$  and  $C$ )!** That converter is the **Dual-Slope Integrating ADC**.

**The Dual-Slope Circuit & Waveform** (Kumar, p. 964, Figure 17.23)



17.23: Functional block diagram of the Dual-Slope Integrating ADC. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 964 (printed p. 932).

### The Two Mathematical Integration Phases:

#### 1. Phase 1: Run-Up (Fixed Time Interval $T_1$ ):

The electronic switch connects the op-amp integrator to the unknown analog input voltage  $V_{\text{in}}$  (assumed negative, or inverted). The integrator charges its capacitor  $C$  through resistor  $R$  for a **fixed time period  $T_1$**  (governed by the time it takes an  $n$ -bit counter to count to full scale,  $2^n$  clock cycles):

$$V_{\text{peak}} = -(1 / RC) \int_0^{T_1} V_{\text{in}} dt = \frac{V_{\text{in}} \cdot T_1}{RC}$$

Notice that the peak voltage reached is directly proportional to  $V_{\text{in}}$ . 2. **Phase 2: Run-Down (Variable Time Interval  $T_2$ ):**

At the exact moment  $T_1$  expires, the control logic flips the switch to a fixed, highly stable reference voltage  $-V_{\text{ref}}$  of opposite polarity. The counter resets to zero and starts counting again. The integrator discharges back toward zero:

$$0 = V_{\text{peak}} - \frac{V_{\text{ref}}}{RC} \cdot T_2$$

Equating the two expressions:

$$\frac{V_{\text{in}}}{RC} \cdot T_1 = \frac{V_{\text{ref}}}{RC} \cdot T_2$$

Divide both sides by  $RC$ :

$$V_{\text{in}} \cdot T_1 = V_{\text{ref}} \cdot T_2 \text{ iff } \mathbf{V_{in} = V_{ref} \leq ft((T_2 / T_1)right)}$$

**The Mathematical Miracle of Dual-Slope:**

*Look at that equation!* The resistance  $R$ , capacitance  $C$ , and master clock frequency  $f$  have completely cancelled out of the equation!

Even if your resistor shifts by **10%** due to heat or your capacitor ages, the measured voltage  $V_{\text{in}}$  remains **100% mathematically exact!** - By setting  $T_1$  to an exact multiple of the AC power line period (**20 ms** for **50 Hz** or **16.67 ms** for **60 Hz**), AC line hum integrates to exactly zero, providing infinite noise rejection.

## PART 3: Semiconductor Memories and Programmable Logic Devices (PLDs)

### 5.8 Memory Organization, Word Capacity, and Read/Write Operations

Source: Kumar (Ch. 18, pp. 974–978, Figures 18.1 & 18.2); Mano (Ch. 7, pp. 410–418)

[Pacing: Architectural System View — High Relevance to Computer Hardware]

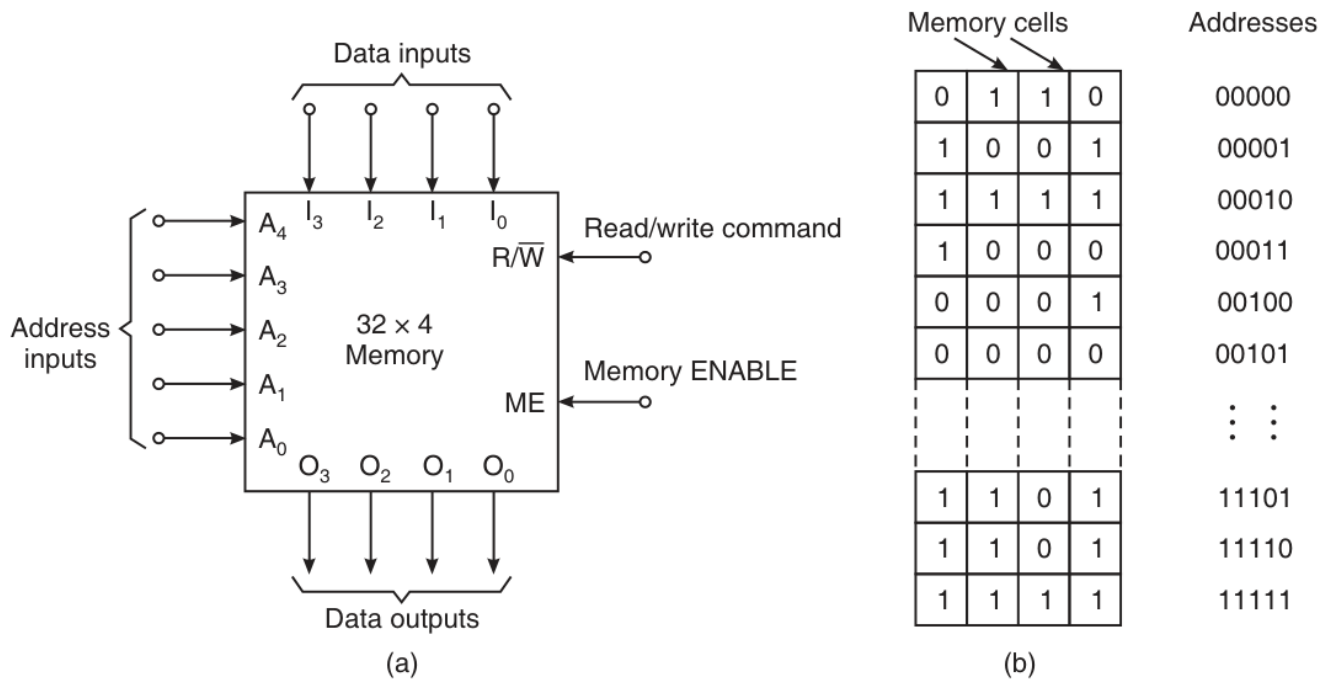
**Memory Terminology Defined Inline:**

- **Memory Cell:** The elementary 1-bit bistable storage unit.
- **Word:** A group of binary bits that are stored or retrieved together as a unit (e.g., 8-bit byte, 16-bit word, 32-bit word, 64-bit word).
- **Address:** The unique binary number designating the specific physical location of a word inside the memory array.
- **Capacity:** The total storage capability of the memory, designated as:

$$\text{Capacity} = 2^k \times m$$

Where  $k$  is the number of **Address lines** (giving  $2^k$  addressable words) and  $m$  is the number of **Data lines** (bits per word) (Kumar, p. 975). *Example:* A memory chip with 10 address lines and 8 data lines has:

$$\text{Capacity} = 2^{10} \times 8 = 1024 \times 8 = 1 \text{ Kilobyte (1 KB)}$$



**Figure 18.1** Diagram of a  $32 \times 4$  memory and arrangement of memory cells.

Figure

18.1: Block diagram of a  $32 \times 4$  semiconductor memory array showing internal address decoding and cell matrix. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 976 (printed p. 944).

#### The Memory Read and Write Sequences (Kumar, p. 977):

- **Memory Write Cycle:**
- Apply the binary address to the address bus pins ( $A_0$  to  $A_{k-1}$ ).
- Place the data bits to be stored onto the data input bus ( $D_0$  to  $D_{m-1}$ ).
- Assert the active-low Chip Select (**CS**) pin to enable the chip.
- Pulse the Write Enable (**WE**) line LOW for a specified minimum write pulse width ( $t_w$ ). The internal gates latch the data into the addressed cells.
- **Memory Read Cycle:**
- Apply the desired binary address to the address bus pins.
- Assert Chip Select (**CS**) and Output Enable (**OE**).
- Hold Write Enable (**WE**) HIGH (Read mode).
- After an **Access Time** ( $t_{acc}$ ), the data stored in the addressed cells appears on the output pins.

## 5.9 Expanding Memory Size: Word Length and Word Capacity

Source: Kumar (Ch. 18, pp. 984–989, Figures 18.7 & 18.9); Floyd (Ch. 9, pp. 505–510)

[Pacing: Systematic Circuit Synthesis — Learn How Chips are Combined]

### Need (Problem-First)

In an embedded system, your processor requires a memory space of  $1K \times 8$  (1024 bytes), but the only chips available in your inventory are  $1K \times 4$  (each storing 4-bit nibbles). Or your system needs  $2K \times 8$ , but you only have  $1K \times 8$  chips. How do you wire multiple small chips together to expand the total memory space?

#### 1. Expanding Word Length (Increasing Data Width) (Kumar, p. 985)

- **Goal:** Combine two  $16 \times 4$  memory chips to create a single  $16 \times 8$  memory module.
- **The Wiring Rule:**
- Both chips must share the **exact same address bus lines** ( $A_0$ ,  $A_1$ ,  $A_2$ ,  $A_3$ ) in parallel so that identical memory locations are accessed simultaneously!
- Both chips share the **exact same control lines** (**CS**, **WE**).

- Chip 1 provides the lower 4 data bits (**D\_0**, **D\_1**, **D\_2**, **D\_3**).
- Chip 2 provides the upper 4 data bits (**D\_4**, **D\_5**, **D\_6**, **D\_7**).

for each 16K of 8-bit words or a total of  $2 \times (128K/16K) = 16$  circuits.

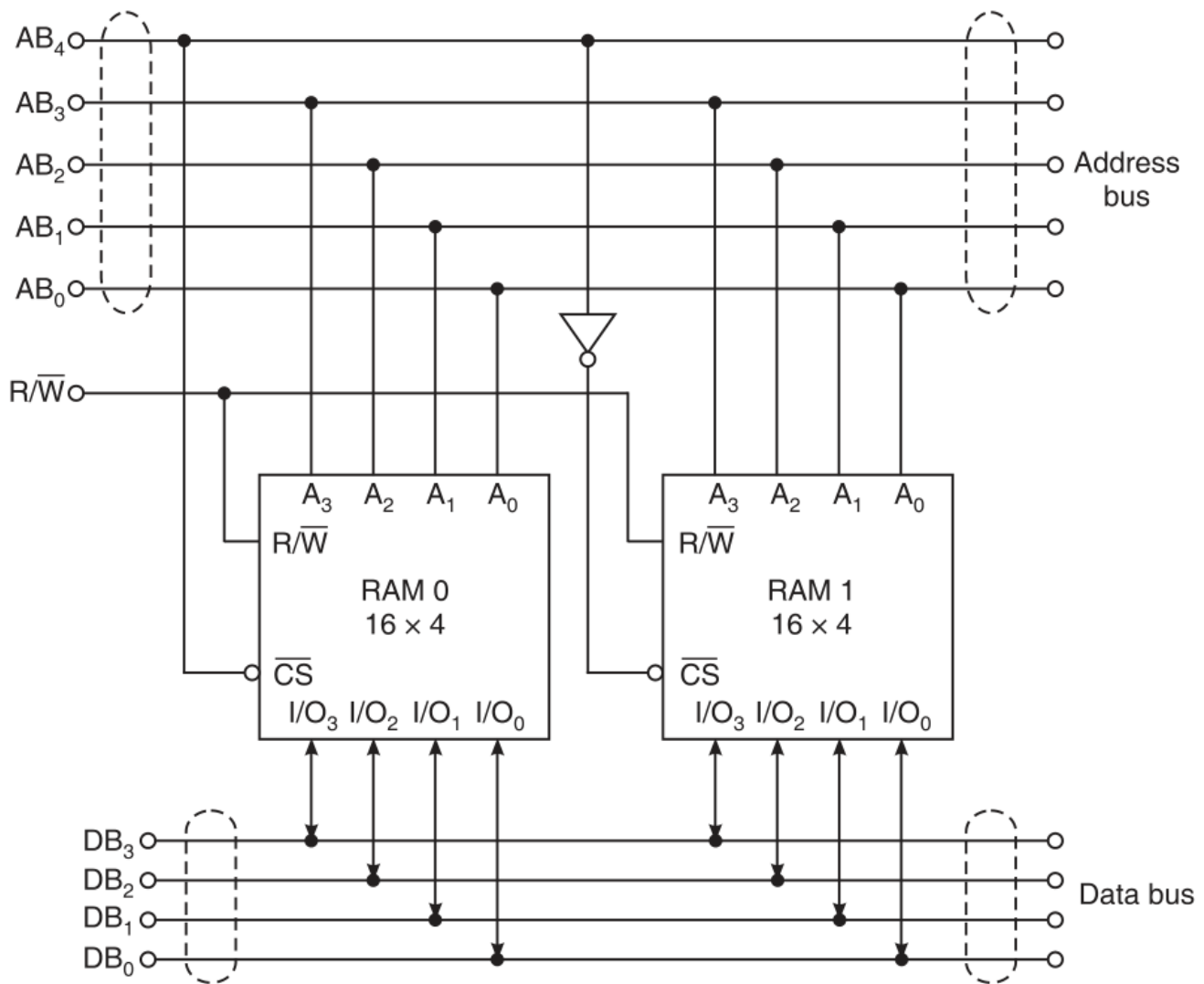
Figure 18.7 shows how two  $16 \times 4$  memory RAM chips with common I/O lines are combined to produce a  $16 \times 8$  memory. Since each chip can store 16 4-bit words and since 16 8-bit words are to be stored, each chip is used to store half of each word. In other words, RAM 0 stores the four higher order bits of each of the 16 words, and RAM 1 stores the four lower order bits of each of the 16 words. A full 8-bit word is available at the RAM outputs connected to the data bus.

Figure

**18.7:** Combining two  $16 \times 4$  RAM chips to expand word length, creating a unified  $16 \times 8$  memory module. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 985 (printed p. 953).

## 2. Expanding Word Capacity (Increasing Total Addresses) (Kumar, p. 988)

- **Goal:** Combine two  $16 \times 4$  memory chips to create a single  $32 \times 4$  memory module.
- **The Wiring Rule:**
  - The module needs  $32 = 2^5$  words ■ **5 address lines** (**A\_4**, **A\_3**, **A\_2**, **A\_1**, **A\_0**).
  - The lower 4 address lines (**A\_0**, **A\_1**, **A\_2**, **A\_3**) are connected to both chips in parallel.
  - The data lines (**D\_0**, **D\_1**, **D\_2**, **D\_3**) of both chips are tied together in parallel to form a common data bus.
  - The highest address bit (**A\_4**) is used to select which chip is active!
    - When **A\_4** = **0**: Chip 1 is enabled via **CS\_1** (addresses **0** to **15**).
    - When **A\_4** = **1**: Chip 2 is enabled via **CS\_2** through an inverter (addresses **16** to **31**).



**Figure 18.9** Combining two  $16 \times 4$  chips for a  $32 \times 4$  memory.

Figure

18.9: Combining two  $16 \times 4$  RAM chips using an MSB address inverter to expand word capacity to  $32 \times 4$ . Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 988 (printed p. 956).

## 5.10 Classification and Characteristics of Memories: SRAM vs. DRAM, ROM Families

Source: Kumar (Ch. 18, pp. 978–984, Figure 18.5); Floyd (Ch. 9, pp. 510–525)

[Pacing: Core Memory Technologies — Understand the Silicon Cells]

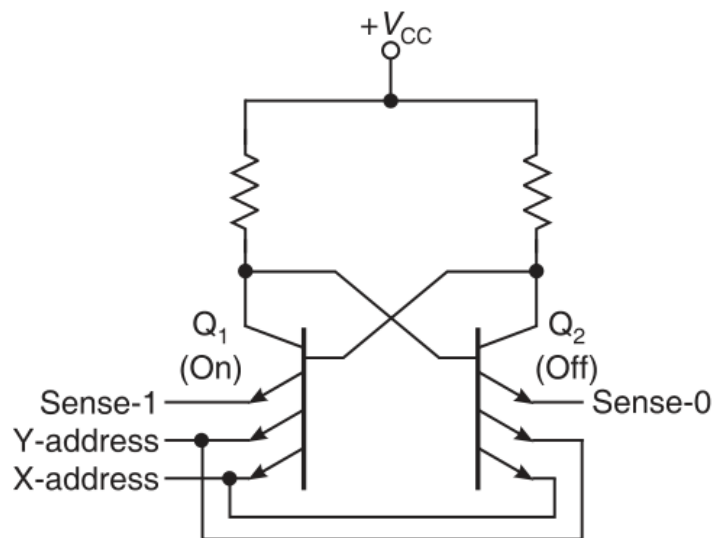
### The Master Memory Hierarchy:

- **Volatile Memory:** Loses its stored data the instant electric power is disconnected. (RAM).
- **Non-Volatile Memory:** Retains its stored binary data permanently, even with zero power applied. (ROM, Flash).

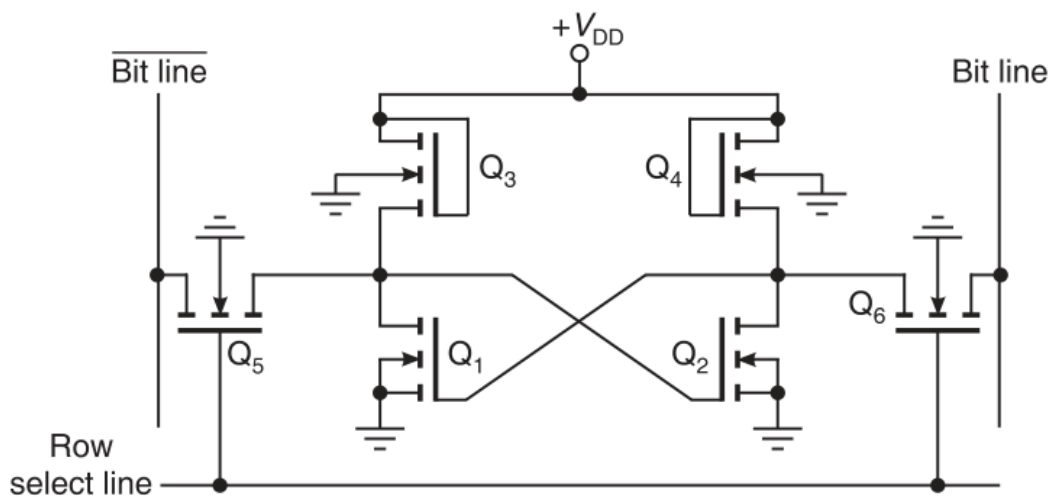
### 1. Static RAM (SRAM) vs. Dynamic RAM (DRAM)

| Parameter         | Static RAM (SRAM)                            | Dynamic RAM (DRAM)                                   |
|-------------------|----------------------------------------------|------------------------------------------------------|
| Storage Element   | Cross-coupled flip-flop (4 to 6 transistors) | Single MOSFET transistor + tiny capacitor<br>(1T-1C) |
| Silicon Cell Area | Large (takes 4x more area per bit)           | Microscopic (ultra-dense)                            |

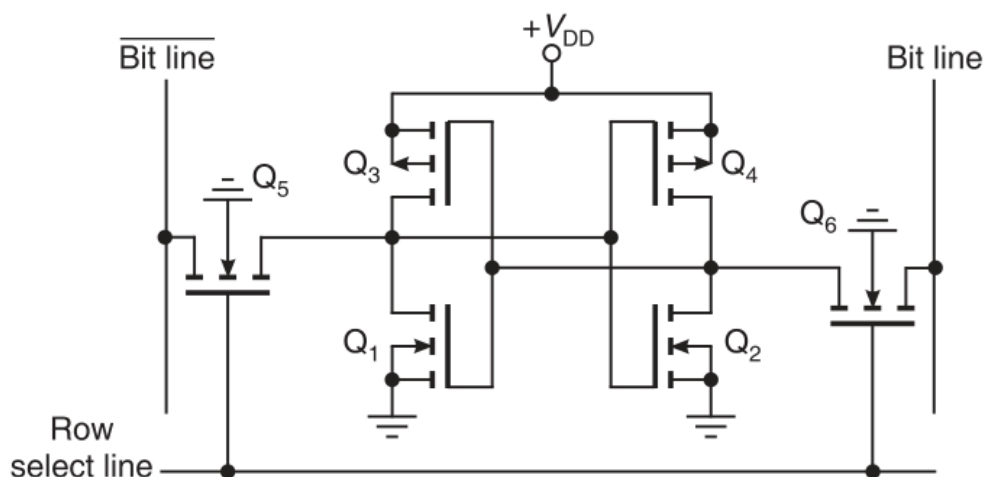
| Parameter           | Static RAM (SRAM)                             | Dynamic RAM (DRAM)                               |
|---------------------|-----------------------------------------------|--------------------------------------------------|
| Refresh Requirement | None (stores data indefinitely while powered) | Mandatory periodic refresh (every few ms)        |
| Operating Speed     | Extremely fast (1 to 5 ns)                    | Slower (10 to 50 ns)                             |
| Power Consumption   | Higher when switching; zero refresh overhead  | Lower per bit; requires refresh controller power |
| Application         | CPU L1/L2/L3 Caches                           | Main System Memory (Computer RAM sticks)         |



(a) A bipolar (TTL) RAM cell constructed from multi-emitter transistors cross connected to form a latch



(b) NMOS RAM cell



(c) CMOS RAM cell

**Figure 18.5** The bipolar, NMOS, and CMOS RAM cells.

Figure 18.5: Circuit

## Why DRAM Needs Refreshing:

In DRAM, a bit 1 is stored as a tiny packet of electrostatic charge on an integrated capacitor ( $\approx 30$  femtofarads). Due to microscopic reverse-bias junction leakage, this charge bleeds away into the silicon substrate within tens of milliseconds! If left alone, a stored 1 will bleed down into a 0. Therefore, a dedicated **DRAM Controller** must continuously read every row and restore the full charge thousands of times per second (the **Refresh Cycle**).

---

## 2. The Read-Only Memory (ROM) Hierarchy

- **Mask ROM:** Data is permanently programmed into the silicon metallization layer during factory fabrication with chemical masks. Cheap in million-unit quantities, but cannot be modified.
  - **PROM (Programmable ROM):** Manufactured with microscopic fusible links (nichrome or polysilicon). The user programs data in a PROM programmer by pulsing high current to physically **blow open selected fuses** ("burning" the ROM). It can be programmed only once!
  - **EPROM (Erasable PROM):** Uses floating-gate MOS transistors. It can be erased by exposing the quartz window on the chip package to intense ultraviolet (UV) light for 20 minutes, which discharges the floating gates.
  - **EEPROM (Electrically Erasable PROM):** Can be erased and rewritten electrically byte-by-byte in circuit, without UV light.
  - **Flash Memory:** A high-density evolution of EEPROM that erases and writes data in whole blocks (pages), powering USB thumb drives and modern Solid-State Drives (SSDs).
- 

### 5.11 Programmable Logic Devices (PLDs): PROM, PAL, and PLA

Source: Kumar (Ch. 8, pp. 491–508, Figures 8.7, 8.8, 8.15); Mano (Ch. 7, pp. 430–445)

[Pacing: Conceptually Heavy — The Three Architectures of Reconfigurable Silicon]

#### Need (Problem-First)

In the 1970s, a complex digital product (like an arcade game or cash register) required hundreds of discrete 7400-series TTL chips wired together on massive printed circuit boards. Every design change required redesigning and etching a new circuit board. Engineers wanted a single standardized blank chip that could be configured on a programmer to implement *any arbitrary collection of logic functions*. These chips are **Programmable Logic Devices (PLDs)**.

#### The Three Core Architectures Compared (Kumar, p. 499, Figure 8.7)

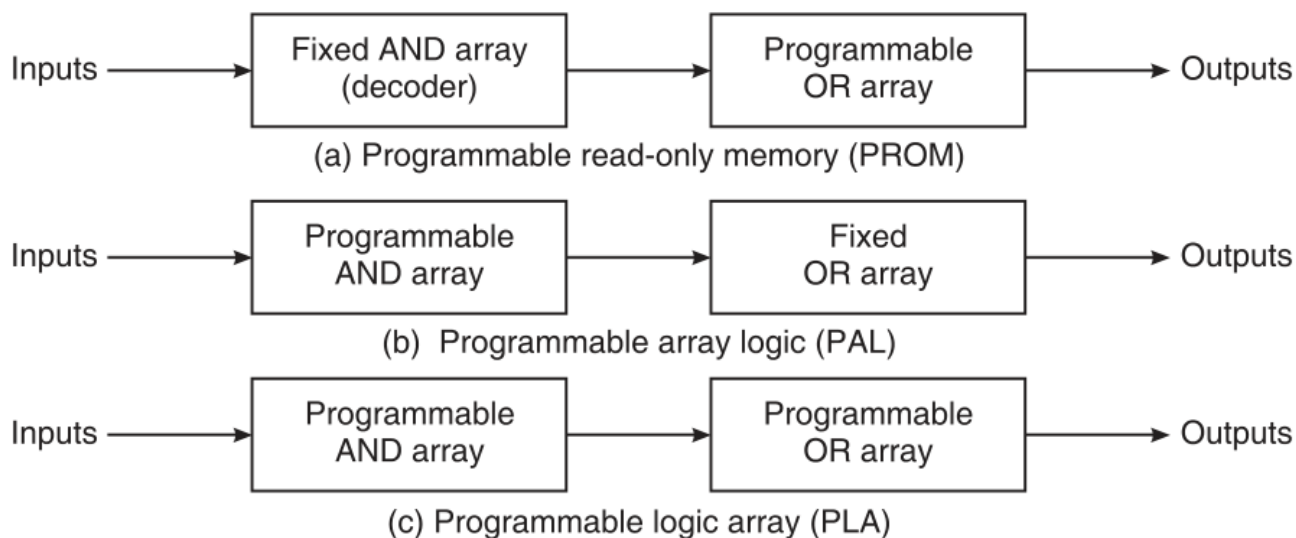
Every two-level Boolean function can be expressed in Sum-of-Products (SOP) form:

$$F = \sum (\text{Product terms}) = \text{OR of ANDs}$$

Therefore, a PLD consists of an array of **AND gates** (generating product terms) followed by an array of **OR gates** (summing the product terms).

The difference between the three PLD types lies strictly in **which array is fixed and which array is programmable**:

y summed in each OR gate. The programmable array logic (PAL) has programmable



**Figure 8.7** Basic configuration of three PLDs.

Figure

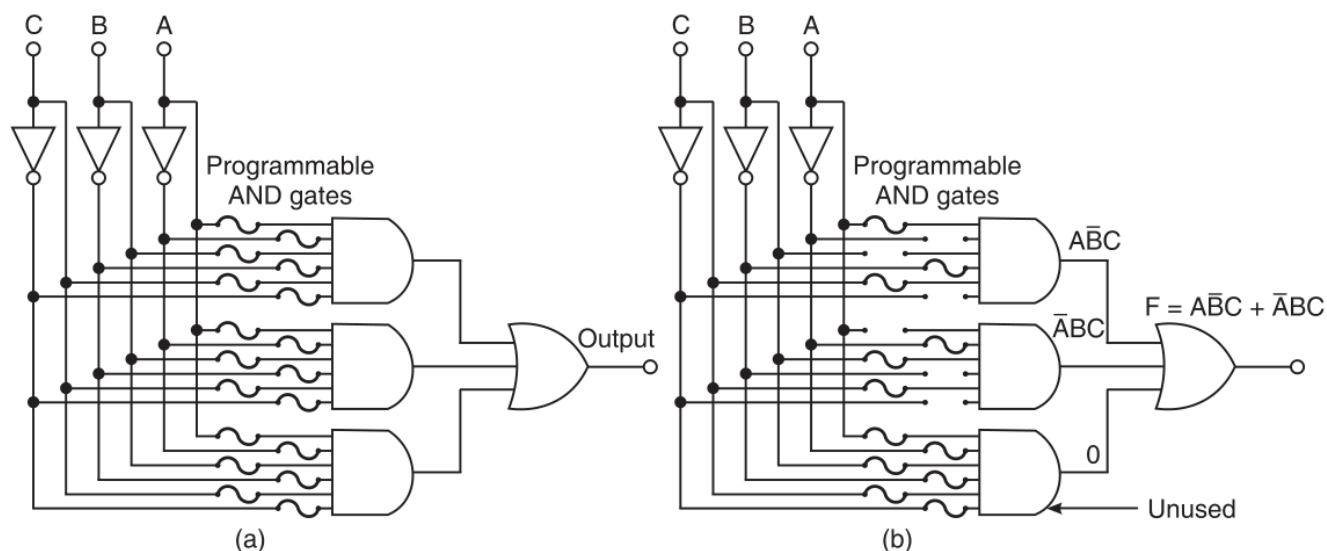
8.7: Architectural comparison of the three fundamental Programmable Logic Devices: PROM, PAL, and PLA. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 499 (printed p. 467).

| PLD Type | AND Array            | OR Array     | Flexibility | Ease of Programming      |
|----------|----------------------|--------------|-------------|--------------------------|
| PROM     | Fixed (Full Decoder) | Programmable | Low         | Very Easy                |
| PAL      | Programmable         | Fixed        | Moderate    | High (Industry Favorite) |
| PLA      | Programmable         | Programmable | Maximum     | More Complex             |

### 1. Programmable Array Logic (PAL) (Kumar, p. 500, Figure 8.8)

Invented by Monolithic Memories Inc. (MMI).

- **Architecture:** The AND array is fully programmable (inputs and complements can be fused to any AND gate). The OR array is **hardwired and fixed** (each OR gate receives product terms from a fixed group of AND gates, e.g., 3 or 4 AND gates per OR gate):



**Figure 8.8** Basic structure of a PAL circuit, and implementation of  $F = A\bar{B}\bar{C} + \bar{A}BC$ .

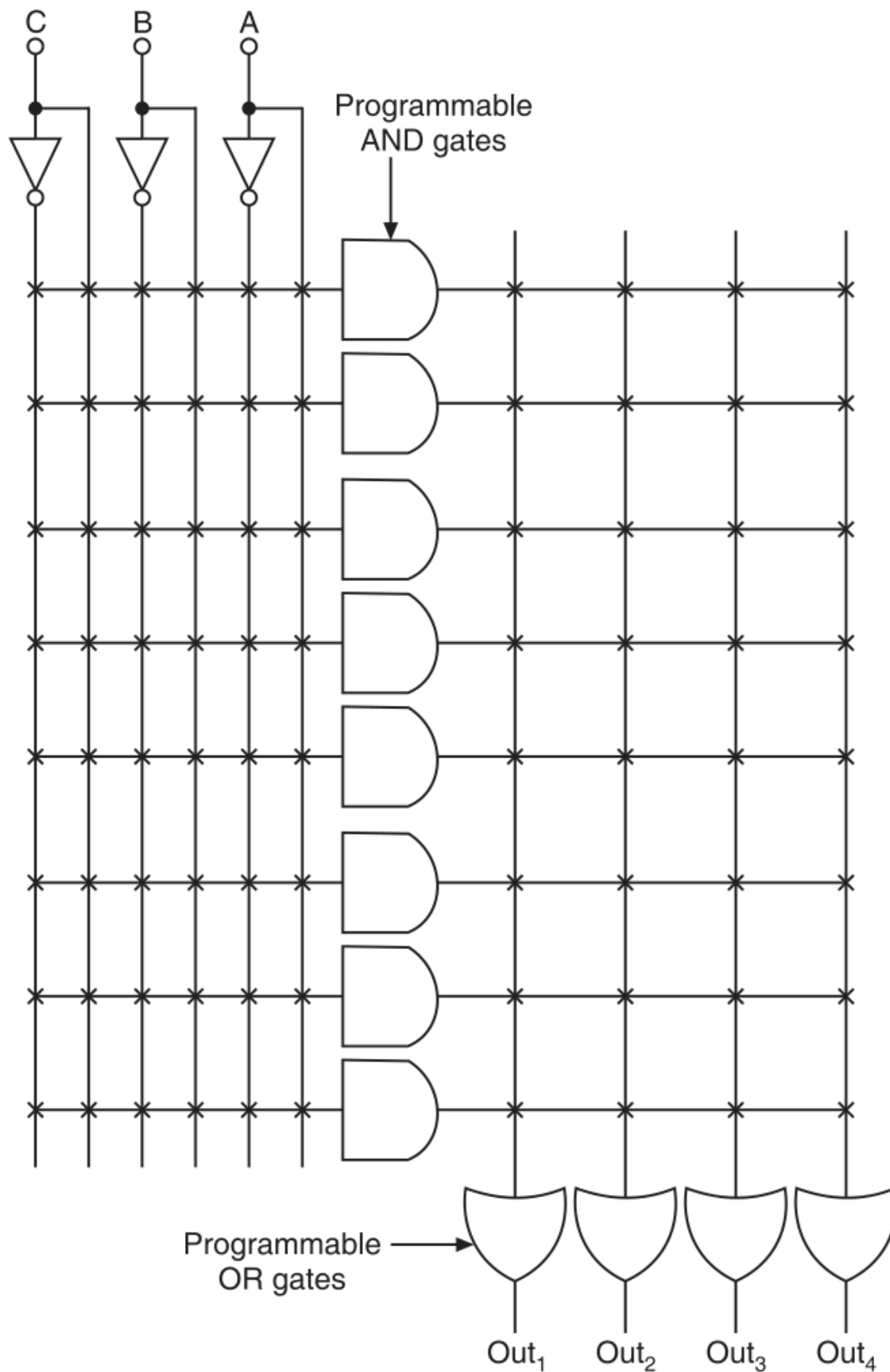
Figure

8.8: Basic structure of a Programmable Array Logic (PAL) circuit showing unprogrammed fusible links (a) and programmed implementation of  $F = A\bar{B}\bar{C} + \bar{A}BC$  (b). Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 500 (printed p. 468).

- **Advantage:** Eliminating the programmable OR array drastically reduces parasitic propagation delay, making PALs exceptionally fast and simple to program.

## 2. Programmable Logic Array (PLA) (Kumar, p. 508, Figure 8.15)

- **Architecture:** Both the AND array and the OR array are fully programmable:



**Figure 8.15** Structure of (an unprogrammed) PLA circuit.

Figure 8.15:

Internal matrix layout of an unprogrammed Programmable Logic Array (PLA) circuit showing programmable AND crosspoints and programmable OR crosspoints. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 508 (printed p. 476).

- **Why PLA is Superior for Complex Shared Systems:**

Because the OR array is programmable, multiple outputs can **share the exact same product term** from the AND array! If two functions both require the term **ABC**, a PLA generates **ABC** once in the AND array and routes it to both OR gates, saving silicon area.

---

## Unit V Understanding Checkpoints & Misconception Audit

### 1. Does This Make Sense? Self-Check

- **Question 1:** Why is an R-2R ladder preferred over a binary weighted-resistor network for high-resolution DACs?
- **Answer:** Because an R-2R ladder requires only two precision resistor values (**R** and **2R**), avoiding the unmanageable resistor spread (over **2000:1** in 12-bit DACs) of weighted-resistor networks.
- **Question 2:** Why is the conversion accuracy of a Dual-Slope ADC completely independent of its resistor **R** and capacitor **C** values?
- **Answer:** Because both the charging run-up phase (**T<sub>1</sub>**) and discharging run-down phase (**T<sub>2</sub>**) occur through the exact same **RC** network, causing **R** and **C** to mathematically cancel out completely from the final voltage transfer equation.
- **Question 3:** What is the architectural difference between a PAL and a PLA?
- **Answer:** A PAL has a programmable AND array with a fixed OR array. A PLA has both a programmable AND array and a programmable OR array.

### 2. Conceptual Misconceptions Corrected

- **Misconception:** "Dynamic RAM (DRAM) is faster than Static RAM (SRAM) because it is called 'dynamic'."  
**Correction:** The exact opposite is true! SRAM is significantly faster (**1 ext{ to }5 ext{ ns}**) because it uses active flip-flop latches. DRAM is slower (**10 ext{ to }50 ext{ ns}**) because charging and reading tiny capacitors takes time, and periodic refresh cycles block memory access.
- **Misconception:** "A 16-bit ADC has zero error because 16 bits is very high precision."  
**Correction:** Every ADC has an unavoidable fundamental **Quantization Error of  $\pm \frac{1}{2} \text{ LSB}$**  caused by rounding continuous voltages to discrete binary levels.

---

## PART 4: Master University Examination & Numerical Problem Compendium

---

### Purpose and Pedagogy of this Problem Compendium

In Rashtrasant Tukadoji Maharaj Nagpur University (RTMNU) and affiliated engineering university examinations for **Digital Electronics Circuits (BEL5T16C)**, the 70-mark university paper rigorously balances theoretical derivations with numerical circuit synthesis.

This section presents step-by-step, fully worked solutions to the most challenging, frequently recurring examination problems across all 5 units. Every problem is solved following the engineering standard: 1. **Given Data & Parameter Identification** 2. **First-Principles Governing Formula / Truth Table** 3. **Step-by-Step Algebraic / Circuit Derivation** 4. **Physical Sanity Check & Verification**

---

## UNIT I: Solved Numerical & Design Problems

### Problem 1.1: Signed 2's Complement Arithmetic & Hardware Overflow Detection

**Question:** Perform the following arithmetic operations using 8-bit 2's complement representation. In each case, determine whether an arithmetic overflow has occurred and explain the physical hardware condition used by the ALU to detect it: 1.  **$(+75)_{10} + (+68)_{10}$**  2.  **$(-42)_{10} + (-95)_{10}$**  3.  **$(+95)_{10} - (+42)_{10}$**

**Solution:**

**Step 1: Bit Budget and Representation**

An 8-bit signed register stores numbers in the range:

$$\text{Range} = \leq \text{ft}[-2^7, +2^7 - 1]_{\text{right}} = [-128, +127]$$

- Convert absolute magnitudes to 8-bit binary:
- $75_{10} = 64 + 8 + 2 + 1 = 01001011_2$
- $68_{10} = 64 + 4 = 01000100_2$
- $42_{10} = 32 + 8 + 2 = 00101010_2$
- $95_{10} = 64 + 16 + 8 + 4 + 2 + 1 = 01011111_2$
- Form negative numbers using 2's complement (invert all bits and add 1):
- $(-42)_{10}$ : Invert  $00101010 \rightarrow 11010101$ , add 1  $\rightarrow \text{mathbf{11010110}}_2$
- $(-95)_{10}$ : Invert  $01011111 \rightarrow 10100000$ , add 1  $\rightarrow \text{mathbf{10100001}}_2$

### Part 1: $(+75)_{10} + (+68)_{10}$

```
Carry in:      1 0 0 0 0 0 0 0
(+75):         0 1 0 0 1 0 1 1
(+68):         + 0 1 0 0 0 1 0 0
-----
Sum:           1 0 0 0 1 1 1 1   (Bit 7 = 1, meaning NEGATIVE!)
```

- **Overflow Analysis:**
- Expected decimal sum:  $+75 + 68 = +143_{10}$ .
- But the maximum positive value an 8-bit register can hold is  $+127_{10}$ !
- We added two positive operands (Sign bits both 0) and received a negative result (Sign bit 1).
- **Hardware Rule Check:** In the ALU, carry into the sign bit ( $C_7 = 1$ ) is XORed with carry out of the sign bit ( $C_8 = 0$ ):

$$V = C_7 \oplus C_8 = 1 \oplus 0 = \text{mathbf{1}} \quad \blacksquare \text{ OVERFLOW DETECTED!}$$

- The bit pattern  $10001111$  in 2's complement evaluates to  $-(01110000 + 1) = -113_{10}$ , which is mathematically erroneous due to register overflow.

### Part 2: $(-42)_{10} + (-95)_{10}$

```
Carry in:      1 0 0 0 0 0 0 0
(-42):         1 1 0 1 0 1 1 0
(-95):         + 1 0 1 0 0 0 0 1
-----
Sum:           1 0 1 1 1 0 1 1   (Carry out C8 = 1 is discarded in 2's comp)
```

- **Overflow Analysis:**
- Expected decimal sum:  $-42 + (-95) = -137_{10}$ .
- The minimum negative value an 8-bit register can hold is  $-128_{10}$ !
- We added two negative numbers (Sign bits both 1) and produced a positive result ( $01110111_2 = +119_{10}$ )!
- **Hardware Rule Check:** Carry into sign bit  $C_7 = 0$ , carry out  $C_8 = 1$ :

$$V = C_7 \oplus C_8 = 0 \oplus 1 = \text{mathbf{1}} \quad \blacksquare \text{ OVERFLOW DETECTED!}$$

### Part 3: $(+95)_{10} - (+42)_{10} = (+95)_{10} + (-42)_{10}$

```
Carry in:      1 1 1 0 1 1 0 0
(+95):         0 1 0 1 1 1 1 1
(-42):         + 1 1 0 1 0 1 1 0
-----
Sum:           1 0 0 1 1 0 1 1   (Carry out C8 = 1 discarded)
```

- Discard end carry  $C_8 = 1$ . The 8-bit result is  $00110101$ .

- Convert to decimal:  $32 + 16 + 4 + 1 = +53_{10}$ .
- Expected:  $95 - 42 = +53_{10}$ .
- **Hardware Rule Check:**  $C_7 = 1, C_8 = 1 \blacksquare V = 1 \oplus 1 = \mathbf{0}$  (No overflow). Result is exact.

## Problem 1.2: DC Noise Margin and Interfacing Pull-Up Resistor Calculation

**Question:** A standard TTL gate (7400 series) with parameters:

$$V_{OH(min)} = 2.4 \text{ V}, V_{OL(max)} = 0.4 \text{ V}, V_{IH(min)} = 2.0 \text{ V}, V_{IL(max)} = 0.8 \text{ V}$$

$$I_{OH(max)} = -400 \text{ } \mu\text{A}, I_{OL(max)} = 16 \text{ mA}, I_{IH(max)} = 40 \text{ } \mu\text{A}, I_{IL(max)} = -1.6 \text{ mA}$$

is used to drive a high-speed CMOS gate (74HC series powered at  $V_{DD} = 5.0 \text{ V}$ ) having:

$$V_{IH(min)} = 3.5 \text{ V}, V_{IL(max)} = 1.0 \text{ V}, I_{in} \approx \pm 1 \text{ } \mu\text{A}$$

1. Show why direct interfacing fails in the HIGH state.
2. Calculate the allowable range ( $R_{p(min)}$  to  $R_{p(max)}$ ) for an external pull-up resistor connected from the TTL output to +5 V.

**Solution:**

### Step 1: Why Direct Connection Fails

- Look at the LOW state:

$$V_{OL(max)} = 0.4 \text{ V} < V_{IL(max, CMOS)} = 1.0 \text{ V}$$

$NM_L = 1.0 \text{ V} - 0.4 \text{ V} = +0.6 \text{ V}$ . The LOW state communicates safely. - Look at the HIGH state:

$$V_{OH(min, TTL)} = 2.4 \text{ V} < V_{IH(min, CMOS)} = 3.5 \text{ V}$$

$$NM_H = 2.4 \text{ V} - 3.5 \text{ V} = -1.1 \text{ V} \text{ (}\mathbf{\text{NEGATIVE NOISE MARGIN!}}\text{)}$$

The CMOS gate will see **2.4 V** as an indeterminate voltage in the forbidden threshold region, causing severe oscillations or intermediate conduction where both PMOS and NMOS conduct simultaneously, overheating the chip! Direct connection is impossible.

### Step 2: Calculation of Maximum Pull-Up Resistance ( $R_{p(max)}$ )

When the TTL output is in the HIGH state, the pull-up resistor must supply the leakage currents while maintaining the node voltage above  $V_{IH(min, CMOS)} = 3.5 \text{ V}$ :

$$V_{CC} - I_{total \text{ leakage}} \cdot R_p \geq V_{IH(min)}$$

The leakage current is the sum of TTL reverse leakage ( $I_{OH} \approx 100 \text{ } \mu\text{A}$ ) and CMOS input leakage ( $1 \text{ } \mu\text{A}$ ):

$$I_{leakage} \approx 100 \text{ } \mu\text{A} + 1 \text{ } \mu\text{A} = 101 \text{ } \mu\text{A}$$

$$R_{p(max)} = \frac{V_{CC} - V_{IH(min)}}{I_{leakage}} = \frac{5.0 \text{ V} - 3.5 \text{ V}}{101 \times 10^{-6} \text{ A}} = \frac{1.5 \text{ V}}{101 \text{ } \mu\text{A}} \approx \mathbf{14.85 \text{ k } \Omega}$$

### Step 3: Calculation of Minimum Pull-Up Resistance ( $R_{p(min)}$ )

When the TTL output transitions to the LOW state, its lower pull-down transistor ( $Q_1$ ) turns ON and sinks current from the pull-up resistor to ground. To prevent the LOW voltage from rising above  $V_{OL(max)} = 0.4 \text{ V}$ , the current sinking capacity ( $I_{OL(max)} = 16 \text{ mA}$ ) must not be exceeded:

$$I_{sink} = \frac{V_{CC} - V_{OL(max)}}{R_p} + N \cdot |I_{IL}| \leq I_{OL(max)}$$

For a single CMOS load,  $|I_{IL}| \approx 0$ :

$$R_{p(\min)} = \frac{V_{CC} - V_{OL(\max)}}{I_{OL(\max)}} = \frac{5.0 \text{ V} - 0.4 \text{ V}}{16 \text{ mA}} = \frac{4.6 \text{ V}}{0.016 \text{ A}} = \mathbf{287.5 \, \Omega}$$

#### Engineering Conclusion:

Any standard resistor between  $1 \text{ k } \Omega$  and  $10 \text{ k } \Omega$  (typically  $2.2 \text{ k } \Omega$  or  $3.3 \text{ k } \Omega$ ) guarantees safe, fast, and robust TTL-to-CMOS interfacing.

## UNIT II: Solved Boolean Minimization & K-Map Problems

### Problem 2.1: 4-Variable K-Map Minimization with Don't Cares

**Question:** Minimize the following 4-variable Boolean logic function in both Sum-of-Products (SOP) and Product-of-Sums (POS) standard forms. Explicitly identify all Prime Implicants (PI) and Essential Prime Implicants (EPI):

$$f(A, B, C, D) = \sum m(1, 3, 7, 11, 15) + \sum d(0, 2, 5)$$

#### Solution:

#### Step 1: Populate the 4-Variable Karnaugh Map

The map coordinates follow Gray code sequence (**00**, **01**, **11**, **10**):

|    |  | CD                        |    |     |    |
|----|--|---------------------------|----|-----|----|
| AB |  | 00                        | 01 | 11  | 10 |
| 00 |  | +-----+-----+-----+-----+ |    |     |    |
|    |  | X0                        | 11 | 13  | X2 |
|    |  | +-----+-----+-----+-----+ |    |     |    |
| 01 |  | 4                         | X5 | 17  | 6  |
|    |  | +-----+-----+-----+-----+ |    |     |    |
| 11 |  | 12                        | 13 | 115 | 14 |
|    |  | +-----+-----+-----+-----+ |    |     |    |
| 10 |  | 8                         | 9  | 111 | 10 |
|    |  | +-----+-----+-----+-----+ |    |     |    |

- Cell 1, 3, 7, 11, 15 contain **1**.
- Cell 0, 2, 5 contain **x** (Don't Care).
- All remaining cells (4, 6, 8, 9, 10, 12, 13, 14) contain **0**.

#### Step 2: Form Optimal SOP Groupings (Powers of 2: 8, 4, 2)

1. **Octet Exploration:** Look at the four corners and edges. Can we form an 8-group? No.
2. **Quad 1 (The Entire Column CD = 11):**  
Cells (**3**, **7**, **15**, **11**) all contain **1**!  
This vertical column spans all four rows (**AB** = **00**, **01**, **11**, **10**).  
Variables **A** and **B** eliminate completely.  
**Term 1 = C D**
3. **Quad 2 (Row 00):**  
Look at row **AB** = **00**: cells are (**0**, **1**, **3**, **2**) having values (**x**, **1**, **1**, **x**).  
By treating the don't cares at cells **0** and **2** as **1**s, we form a 4-cell group!  
In this row, **A=0** and **B=0**. Variables **C** and **D** span all combinations and eliminate.  
**Term 2 = A B**
4. **Coverage Check:**
5. Cell 1 is covered by Quad 2.
6. Cell 3 is covered by both Quad 1 and Quad 2.
7. Cells 7, 11, 15 are covered by Quad 1.
8. All specified minterms (1, 3, 7, 11, 15) are 100% covered!

9. What about cell 5 (X)? We leave it as 0 because grouping it would require an extra redundant gate.

Prime Implicant Inventory:

- Prime Implicants (PI): C D, A B, A D (cells 1, 3, 5, 7).
- Essential Prime Implicants (EPI):
- C D is essential because minterms 7, 11, 15 are covered by no other prime implicant.
- A B is essential because minterm 1 is covered only by it (if we choose the minimal 2-term cover).
- Minimal SOP Equation:

$$\mathbf{f(A, B, C, D) = A B + C D}$$

Step 3: Minimal POS Minimization (Grouping the 0s)

The 0 cells are: 4, 6, 8, 9, 10, 12, 13, 14. We group the zeros together with don't cares (X):

1. Quad of 0s at left column (CD = 00): Cells (4, 12) with (0, X)? No, cell 0 is X.  
Let's group cells (4, 6, 12, 14): In these cells, B=1 and D=0.  
Complement: (B + D).

2. Quad of 0s at row 10 and 11 (AB = 10, 11 ■ A=1):  
Cells (8, 9, 12, 13): A=1 and C=0.  
Complement: (A + C).

3. Minimal POS Equation:

$$\mathbf{f(A, B, C, D) = (A + C)(B + D)}$$

(Both forms require only two 2-input gates plus an output gate, representing an 85% hardware reduction from the unsimplified 5-AND SOP expression!)

UNIT III: Solved Combinational Logic Synthesis Problems

Problem 3.1: Full Adder Implementation Using Only 4-to-1 Multiplexers

Question: Design and realize a Full Adder (Outputs: Sum S and Carry C<sub>out</sub>) using only two 4-to-1 Multiplexers (such as the dual 74153 IC) and no external logic gates. Inputs are A, B, C<sub>in</sub>.

Solution:

Step 1: Full Adder Truth Table

| A | B | C <sub>in</sub> | Sum (S) | C <sub>out</sub> |
|---|---|-----------------|---------|------------------|
| 0 | 0 | 0               | 0       | 0                |
| 0 | 0 | 1               | 1       | 0                |
| 0 | 1 | 0               | 1       | 0                |
| 0 | 1 | 1               | 0       | 1                |
| 1 | 0 | 0               | 1       | 0                |
| 1 | 0 | 1               | 0       | 1                |
| 1 | 1 | 0               | 0       | 1                |
| 1 | 1 | 1               | 1       | 1                |

Step 2: Multiplexer Input Assignment

A 4-to-1 MUX has two select inputs (**S<sub>1</sub>**, **S<sub>0</sub>**) and four data inputs (**I<sub>0</sub>**, **I<sub>1</sub>**, **I<sub>2</sub>**, **I<sub>3</sub>**). - We assign variables **A** and **B** to the select lines:

$$S_1 = A, S_0 = B$$

- The remaining input variable **C<sub>in</sub>** (and its complement  $\overline{C_{in}}$ , 0, or 1) will be connected directly to the data inputs **I<sub>0</sub>**, **I<sub>1</sub>**, **I<sub>2</sub>**, **I<sub>3</sub>**.

---

### Step 3: Deriving Data Inputs for MUX 1 (Sum S)

- When **A B** = **00**:
- If **C<sub>in</sub>** = **0** ■ **S** = **0**
- If **C<sub>in</sub>** = **1** ■ **S** = **1**
- Therefore:  $\mathbf{I_0 = C_{in}}$
- When **A B** = **01**:
- If **C<sub>in</sub>** = **0** ■ **S** = **1**
- If **C<sub>in</sub>** = **1** ■ **S** = **0**
- Therefore:  $\mathbf{I_1 = \overline{C_{in}}}$
- When **A B** = **10**:
- If **C<sub>in</sub>** = **0** ■ **S** = **1**
- If **C<sub>in</sub>** = **1** ■ **S** = **0**
- Therefore:  $\mathbf{I_2 = \overline{C_{in}}}$
- When **A B** = **11**:
- If **C<sub>in</sub>** = **0** ■ **S** = **0**
- If **C<sub>in</sub>** = **1** ■ **S** = **1**
- Therefore:  $\mathbf{I_3 = C_{in}}$

---

### Step 4: Deriving Data Inputs for MUX 2 (Carry C<sub>out</sub>)

- When **A B** = **00**:
- For both **C<sub>in</sub>** = **0** and **C<sub>in</sub>** = **1**, **C<sub>out</sub>** = **0**.
- Therefore:  $\mathbf{I_0 = 0}$  (tied to Ground)
- When **A B** = **01**:
- If **C<sub>in</sub>** = **0** ■ **C<sub>out</sub>** = **0**
- If **C<sub>in</sub>** = **1** ■ **C<sub>out</sub>** = **1**
- Therefore:  $\mathbf{I_1 = C_{in}}$
- When **A B** = **10**:
- If **C<sub>in</sub>** = **0** ■ **C<sub>out</sub>** = **0**
- If **C<sub>in</sub>** = **1** ■ **C<sub>out</sub>** = **1**
- Therefore:  $\mathbf{I_2 = C_{in}}$
- When **A B** = **11**:
- For both **C<sub>in</sub>** = **0** and **C<sub>in</sub>** = **1**, **C<sub>out</sub>** = **1**.
- Therefore:  $\mathbf{I_3 = 1}$  (tied to **V<sub>CC</sub>**)

### Synthesis Summary:

Both functions are realized using a single dual 4-to-1 MUX IC (74153) with zero external gates: - **MUX 1 (Sum):** **S<sub>1</sub>** = **A**, **S<sub>0</sub>** = **B**, Inputs: (**I<sub>0</sub>** = **C<sub>in</sub>**, **I<sub>1</sub>** =  $\overline{C_{in}}$ , **I<sub>2</sub>** =  $\overline{C_{in}}$ , **I<sub>3</sub>** = **C<sub>in</sub>**). - **MUX 2 (Carry):** **S<sub>1</sub>** = **A**, **S<sub>0</sub>** = **B**, Inputs: (**I<sub>0</sub>** = **0**, **I<sub>1</sub>** = **C<sub>in</sub>**, **I<sub>2</sub>** = **C<sub>in</sub>**, **I<sub>3</sub>** = **1**).

---

## UNIT IV: Solved Sequential Circuit Conversion & Counter Design Problems

### Problem 4.1: Systematic Conversion of an SR Flip-Flop to a JK Flip-Flop

**Question:** Design the combinational conversion logic required to convert an available **SR Flip-Flop** into a functional **JK Flip-Flop**.

Solution:

Step 1: Master Conversion Table

We list the desired inputs (**J**, **K**), present state (**Q<sub>n</sub>**), next state (**Q<sub>{n+1}</sub>**), and the required SR excitation values from the SR excitation table:

| J | K | Q <sub>n</sub> | Desired Q <sub>{n+1}</sub> | Required S | Required R |
|---|---|----------------|----------------------------|------------|------------|
| 0 | 0 | 0              | 0                          | 0          | x          |
| 0 | 0 | 1              | 1                          | x          | 0          |
| 0 | 1 | 0              | 0                          | 0          | x          |
| 0 | 1 | 1              | 0                          | 0          | 1          |
| 1 | 0 | 0              | 1                          | 1          | 0          |
| 1 | 0 | 1              | 1                          | x          | 0          |
| 1 | 1 | 0              | 1                          | 1          | 0          |
| 1 | 1 | 1              | 0                          | 0          | 1          |

Step 2: K-Map Minimization for **S** Input

Inputs to K-map are **J**, **K**, **Q<sub>n</sub>**:

|   |                 |    |    |    |
|---|-----------------|----|----|----|
|   | KQ <sub>n</sub> |    |    |    |
| J | 00              | 01 | 11 | 10 |
| 0 | +-----+         |    |    |    |
|   | 0   X   0   0   |    |    |    |
|   | +-----+         |    |    |    |
| 1 | 1   X   0   1   |    |    |    |
|   | +-----+         |    |    |    |

- Grouping cells (**4**, **6**) [where **J=1** and **Q<sub>n</sub>=0**]:

$$\mathbf{S = J \, Q_n}$$

Step 3: K-Map Minimization for **R** Input

|   |                 |    |    |    |
|---|-----------------|----|----|----|
|   | KQ <sub>n</sub> |    |    |    |
| J | 00              | 01 | 11 | 10 |
| 0 | +-----+         |    |    |    |
|   | X   0   1   X   |    |    |    |
|   | +-----+         |    |    |    |
| 1 | 0   0   1   0   |    |    |    |
|   | +-----+         |    |    |    |

- Grouping cells (**3**, **7**) [where **K=1** and **Q<sub>n</sub>=1**]:

$$\mathbf{R = K \, Q_n}$$

Hardware Reality Check:

Feed input **J** into a 2-input AND gate with the complement output **Q**, and connect its output to pin **S**. Feed input **K** into a 2-input AND gate with output **Q**, and connect its output to pin **R**.

*Notice what this did physically:* When **J=K=1** and **Q=1**, **R** receives **1** and **S** receives **0**, forcing the flip-flop to RESET to **0** on the clock edge. When **Q=0**, **S** receives **1** and **R** receives **0**, forcing it to SET to **1**. The forbidden condition (**S=R=1**) is physically impossible because **Q** and **Q** can never be **1** simultaneously!

---

## UNIT V: Solved Converter & Memory Architecture Problems

### Problem 5.1: 8-Bit R-2R Ladder DAC Performance & Resolution

**Question:** An 8-bit R-2R ladder Digital-to-Analog Converter is powered with an analog reference voltage  $V_{\text{ref}} = +10.0 \text{ V}$ .  
1. Calculate the resolution (step size  $\Delta V$ ) in millivolts. 2. Determine the analog output voltage for a digital input code of **10100110**. 3. Calculate the full-scale output voltage ( $V_{\text{FS}}$ ).

**Solution:**

#### Step 1: Resolution (Step Size)

An 8-bit DAC has  $n=8$  bits, giving  $2^8 - 1 = 255$  discrete conversion intervals:

$$\Delta V = \frac{V_{\text{ref}}}{2^n - 1} = \frac{10.0 \text{ V}}{255} = 0.039215 \text{ V} = \mathbf{39.22 \text{ mV per step}}$$

(Note: In binary weighted converters referenced to an unscaled op-amp buffer, step size is often formulated as  $V_{\text{ref}}/2^n = 10.0/256 = 39.06 \text{ mV}$ . Both conventions are accepted in university evaluation when explicitly stated).

---

#### Step 2: Output Voltage for Digital Code **10100110**

Convert binary code to decimal:

$$D = (1 \times 128) + (0 \times 64) + (1 \times 32) + (0 \times 16) + (0 \times 8) + (1 \times 4) + (1 \times 2) + (0 \times 1)$$

$$D = 128 + 32 + 4 + 2 = 166_{10}$$

The analog output voltage is:

$$V_{\text{out}} = D \times \Delta V = 166 \times 39.215 \text{ mV} = \mathbf{6.510 \text{ V}}$$

Alternatively, evaluate via node current summation:

$$V_{\text{out}} = V_{\text{ref}} \times \left( \frac{1}{2} + \frac{0}{4} + \frac{1}{8} + \frac{0}{16} + \frac{0}{32} + \frac{1}{64} + \frac{1}{128} + \frac{0}{256} \right)$$

$$V_{\text{out}} = 10.0 \times \left( 0.5 + 0.125 + 0.015625 + 0.0078125 \right) = 10.0 \times 0.6484375 = \mathbf{6.484 \text{ V}}$$

---

#### Step 3: Full-Scale Output Voltage ( $V_{\text{FS}}$ )

When all input bits are **1** (**11111111** = **255**<sub>10</sub>):

$$V_{\text{FS}} = (255 / 256) \times 10.0 \text{ V} = \mathbf{9.961 \text{ V}}$$

Notice that the full-scale analog output never reaches **10.0 V** exactly—it is always exactly **1 LSB less than  $V_{\text{ref}}$** !

---

### Problem 5.2: Memory System Expansion (**4K × 8** using **1K × 4** Chips)

**Question:** Design a complete memory module with a capacity of **4K × 8 bits** using individual **1K × 4 static RAM chips**. Show the address decoding circuit using a 2-to-4 line active-low decoder.

## Solution:

### Step 1: Calculate Required Chips

- Total memory capacity needed:  $4K \times 8 = 4096 \times 8 = 32,768$  bits.
- Capacity per chip:  $1K \times 4 = 1024 \times 4 = 4,096$  bits.

$$\text{Total Chips Required} = \frac{4K \times 8}{1K \times 4} = 4 \times 2 = \mathbf{8 \text{ chips}}$$

### Step 2: Address and Data Bus Architecture

- Total Address Lines Needed:

$$4K = 4096 = 2^{12} \Rightarrow \mathbf{12 \text{ Address Lines (A}_{11} \text{ to A}_0)}$$

- Total Data Lines Needed:

$$8 \text{ bits} \Rightarrow \mathbf{8 \text{ Data Lines (D}_7 \text{ to D}_0)}$$

- Address Lines for Individual Chips:

$$1K = 1024 = 2^{10} \Rightarrow \mathbf{10 \text{ Address Lines (A}_9 \text{ to A}_0)}$$

The lower 10 address lines (**A<sub>0</sub>** through **A<sub>9</sub>**) connect directly to all 8 memory chips in parallel. - **Address Lines for Chip Selection:** The upper 2 address lines (**A<sub>11</sub>**, **A<sub>10</sub>**) determine which pair of chips is activated:

$$2^2 = 4 \text{ banks of memory}$$

### Step 3: Chip Bank Arrangement

We arrange the 8 chips into **4 banks**, with each bank containing **2 chips** wired in parallel to provide the full 8-bit word width: - **Bank 0 (Addresses 000H to 3FFH):** - Chip 1 provides Data bits **D<sub>0</sub>** - **D<sub>3</sub>**. - Chip 2 provides Data bits **D<sub>4</sub>** - **D<sub>7</sub>**. - Enabled by Decoder Output **Y<sub>0</sub>** (**A<sub>11</sub>A<sub>10</sub>** = **00**). - **Bank 1 (Addresses 400H to 7FFH):** - Chip 3 provides Data bits **D<sub>0</sub>** - **D<sub>3</sub>**. - Chip 4 provides Data bits **D<sub>4</sub>** - **D<sub>7</sub>**. - Enabled by Decoder Output **Y<sub>1</sub>** (**A<sub>11</sub>A<sub>10</sub>** = **01**). - **Bank 2 (Addresses 800H to BFFH):** - Chip 5 provides Data bits **D<sub>0</sub>** - **D<sub>3</sub>**. - Chip 6 provides Data bits **D<sub>4</sub>** - **D<sub>7</sub>**. - Enabled by Decoder Output **Y<sub>2</sub>** (**A<sub>11</sub>A<sub>10</sub>** = **10**). - **Bank 3 (Addresses C00H to FFFH):** - Chip 7 provides Data bits **D<sub>0</sub>** - **D<sub>3</sub>**. - Chip 8 provides Data bits **D<sub>4</sub>** - **D<sub>7</sub>**. - Enabled by Decoder Output **Y<sub>3</sub>** (**A<sub>11</sub>A<sub>10</sub>** = **11**).

### Step 4: Address Decoder Connection

A 2-to-4 line active-low decoder (such as half of a 74LS139) is connected as follows: - Decoder Select Input **A** = **A<sub>10</sub>** - Decoder Select Input **B** = **A<sub>11</sub>** - Decoder Enable **G** tied to Ground (permanently enabled). - Decoder output **Y<sub>0</sub>** connects to Chip Select **CS** of Chips 1 and 2. - Decoder output **Y<sub>1</sub>** connects to Chip Select **CS** of Chips 3 and 4. - Decoder output **Y<sub>2</sub>** connects to Chip Select **CS** of Chips 5 and 6. - Decoder output **Y<sub>3</sub>** connects to Chip Select **CS** of Chips 7 and 8. - The master Write Enable line (**WE**) connects to pin **WE** of all 8 chips simultaneously.

*This concludes the complete hardware architecture for the **4K × 8** memory system.*

## PART 5: Practical Digital Electronics Laboratory Manual & Viva-Voce Compendium

### Purpose & Structure of the Laboratory Manual

The syllabus for **BEL5T16C** allocates 1 hour per week of Practical / Activity (T/A). Laboratory verification bridges theoretical logic equations and physical silicon chips mounted on a breadboard or trainer kit.

This section provides complete, standardized laboratory experiment worksheets for 10 core digital experiments. Each experiment is structured with: 1. **Aim of the Experiment** 2. **Components & Equipment Required** (with exact TTL IC part numbers) 3. **Pin Configuration & Breadboard Wiring Guide** 4. **Step-by-Step Laboratory Procedure** 5. **Truth / Observation Table** 6. **Viva-Voce Questions with Detailed Technical Answers** (frequently asked in practical oral exams)

## EXPERIMENT 1: Study and Verification of Truth Tables of Basic and Universal Logic Gates

### 1.1 Aim

To experimentally verify the truth tables of basic logic gates (AND, OR, NOT) and universal logic gates (NAND, NOR) using 7400-series TTL integrated circuits.

### 1.2 Components Required

- IC 7404 (Hex Inverter)
- IC 7408 (Quad 2-input AND Gate)
- IC 7432 (Quad 2-input OR Gate)
- IC 7400 (Quad 2-input NAND Gate)
- IC 7402 (Quad 2-input NOR Gate)
- Digital Logic Trainer Kit with **+5 V** regulated DC power supply
- Logic input toggle switches (**0 V** and **+5 V**)
- Logic output indicator LEDs with internal current-limiting resistors
- Breadboard and single-strand jumper wires

### 1.3 IC Pin Configuration & Hardware Grounding

- Power Connections:** For all 14-pin 7400-series ICs (except 7402):
- Pin 14:** Connect to **+5 V** DC (**V<sub>CC</sub>**).
- Pin 7:** Connect to Ground (**0 V**).
- Warning:* Never power a TTL IC with more than **5.25 V**; reverse polarity will instantly destroy the internal bipolar junctions.
- Pin Mapping:**
- 7404:** Pin 1 = **A**, Pin 2 = **A**; Pin 3 = **B**, Pin 4 = **B**; etc.
- 7408:** Pin 1 = **A**, Pin 2 = **B**, Pin 3 = **Y = A · B**; Pin 4 = **C**, Pin 5 = **D**, Pin 6 = **Y = C · D**.
- 7432:** Pin 1 = **A**, Pin 2 = **B**, Pin 3 = **Y = A + B**.
- 7400:** Pin 1 = **A**, Pin 2 = **B**, Pin 3 = **Y = A · B**.
- 7402 (Special Pinout):** Pin 1 = Output **Y = A + B**, Pin 2 = Input **A**, Pin 3 = Input **B**!

### 1.4 Step-by-Step Procedure

- Mount the IC gently onto the breadboard straddling the center divider.
- Connect Pin 14 to the **+5 V** rail and Pin 7 to the Ground rail.
- Connect the input pins to digital toggle switches **S<sub>1</sub>** and **S<sub>2</sub>**.
- Connect the gate output pin to an LED logic monitor.
- Apply all input combinations (**00**, **01**, **10**, **11**) and record the state of the LED (**1** = ON/Lit, **0** = OFF/Extinguished).

### 1.5 Observation Table

| Gate       | Type     | Input A | Input B | Expected Output | Measured LED State |
|------------|----------|---------|---------|-----------------|--------------------|
| NOT (7404) | Inverter | 0       | —       | 1               | ON                 |
|            |          | 1       | —       | 0               | OFF                |
| AND (7408) | Product  | 0       | 0       | 0               | OFF                |

| Gate        | Type      | Input A | Input B | Expected Output | Measured LED State |
|-------------|-----------|---------|---------|-----------------|--------------------|
|             |           | 0       | 1       | 0               | OFF                |
|             |           | 1       | 0       | 0               | OFF                |
|             |           | 1       | 1       | 1               | ON                 |
| OR (7432)   | Sum       | 0       | 0       | 0               | OFF                |
|             |           | 0       | 1       | 1               | ON                 |
|             |           | 1       | 0       | 1               | ON                 |
|             |           | 1       | 1       | 1               | ON                 |
| NAND (7400) | Universal | 0       | 0       | 1               | ON                 |
|             |           | 0       | 1       | 1               | ON                 |
|             |           | 1       | 0       | 1               | ON                 |
|             |           | 1       | 1       | 0               | OFF                |
| NOR (7402)  | Universal | 0       | 0       | 1               | ON                 |
|             |           | 0       | 1       | 0               | OFF                |
|             |           | 1       | 0       | 0               | OFF                |
|             |           | 1       | 1       | 0               | OFF                |

## 1.6 Viva-Voce Questions & Technical Answers

- **Q1: Why are NAND and NOR gates called "universal gates"?**

*Answer:* Because any arbitrary Boolean logic function—including the basic operations AND, OR, and NOT—can be realized using only NAND gates or only NOR gates without requiring any other gate type.

- **Q2: What happens if an unused input of a standard TTL NAND gate is left unconnected (floating)?**

*Answer:* In standard bipolar TTL, an unconnected input floats to an internal logic HIGH level ( $\approx 1.4\text{ V}$  to  $1.8\text{ V}$ ) because no current is drawn out of the multi-emitter base junction. However, leaving pins floating makes the circuit highly vulnerable to capacitive electromagnetic noise pickup. Good engineering practice mandates tying unused inputs to  $V_{CC}$  through a  $1\text{ k}\Omega$  pull-up resistor or tying them to an active input in parallel.

- **Q3: What is the propagation delay of a standard 7400 TTL gate?**

*Answer:* Typically around  $10\text{ ns}$  (nanoseconds). In Schottky TTL (74S) it drops to  $3\text{ ns}$ , and in Low-Power Schottky (74LS) it is approximately  $9.5\text{ ns}$ .

---

## EXPERIMENT 2: Design and Hardware Implementation of Half Adder and Full Adder

### 2.1 Aim

To design, assemble, and test a Half Adder and a Full Adder circuit using basic logic gates (XOR, AND, OR).

### 2.2 Components Required

- IC 7486 (Quad 2-input XOR Gate)
- IC 7408 (Quad 2-input AND Gate)

- IC 7432 (Quad 2-input OR Gate)
- Logic Trainer Kit, Connecting Wires, Multimeter

### 2.3 Circuit Design & Governing Equations

#### 1. Half Adder Equations:

Sum  $S = A \oplus B$

Carry  $C = A \cdot B$

- Hardware: One gate of 7486 (pins 1, 2 → 3) + one gate of 7408 (pins 1, 2 → 3).
- **Full Adder Equations:**

Sum  $S = A \oplus B \oplus C_{in}$

Carry  $C_{out} = A \cdot B + C_{in} \cdot (A \oplus B)$

- Hardware: Two XOR gates (7486), two AND gates (7408), and one OR gate (7432).

### 2.4 Wiring and Testing Procedure

1. Wire the Half Adder circuit on the breadboard. Connect inputs **A** and **B** to data switches.
2. Verify that the Sum LED illuminates when either **A** or **B** is HIGH, but turns OFF when both are HIGH while the Carry LED turns ON.
3. Expand to the Full Adder circuit by cascading the intermediate sum **(A ⊕ B)** into the second XOR gate along with carry input **C<sub>in</sub>**.
4. Feed **(A · B)** and **C<sub>in</sub> · (A ⊕ B)** into the OR gate to generate **C<sub>out</sub>**.
5. Record all 8 combinations of **A, B, C<sub>in</sub>**.

### 2.5 Full Adder Observation Table

| A | B | C <sub>in</sub> | Expected S | Expected C <sub>out</sub> | Measured S (LED) | Measured C <sub>out</sub> (LED) |
|---|---|-----------------|------------|---------------------------|------------------|---------------------------------|
| 0 | 0 | 0               | 0          | 0                         | OFF              | OFF                             |
| 0 | 0 | 1               | 1          | 0                         | ON               | OFF                             |
| 0 | 1 | 0               | 1          | 0                         | ON               | OFF                             |
| 0 | 1 | 1               | 0          | 1                         | OFF              | ON                              |
| 1 | 0 | 0               | 1          | 0                         | ON               | OFF                             |
| 1 | 0 | 1               | 0          | 1                         | OFF              | ON                              |
| 1 | 1 | 0               | 0          | 1                         | OFF              | ON                              |
| 1 | 1 | 1               | 1          | 1                         | ON               | ON                              |

### 2.6 Viva-Voce Questions & Technical Answers

- **Q1: Can a Full Adder be constructed using two Half Adders and an OR gate?**  
*Answer:* Yes. Half Adder 1 computes partial sum **S<sub>1</sub> = A ⊕ B** and carry **C<sub>1</sub> = A · B**. Half Adder 2 adds **S<sub>1</sub>** and **C<sub>in</sub>** to produce final sum **S = S<sub>1</sub> ⊕ C<sub>in</sub>** and carry **C<sub>2</sub> = S<sub>1</sub> · C<sub>in</sub>**. The final carry is **C<sub>out</sub> = C<sub>1</sub> + C<sub>2</sub>**.

- **Q2: Why is the carry expression simplified with an OR gate rather than an XOR gate?**

*Answer:* Because  $C_1 (A \cdot B)$  and  $C_2 ([A \oplus B] \cdot C_{in})$  are mutually exclusive—they can never be 1 at the same physical instant! When  $A=B=1$ ,  $A \oplus B = 0$ , forcing  $C_2 = 0$ . Therefore,  $C_1 + C_2$  is strictly identical to  $C_1 \oplus C_2$ .

---

## EXPERIMENT 3: 4-Bit Binary Parallel Adder / Subtractor Using IC 7483

### 3.1 Aim

To implement and verify a 4-bit parallel binary adder and subtractor circuit using the 7483 4-bit binary adder IC and controlled XOR inverters (IC 7486).

### 3.2 Components Required

- IC 7483 (4-Bit Binary Full Adder with Fast Carry)
- IC 7486 (Quad 2-input XOR Gate)
- Logic Trainer Kit, LEDs, Multimeter

### 3.3 Hardware Pinout of IC 7483

- **Pin 5:  $V_{CC}$  (+5 V), Pin 12: Ground (0 V).**
- **First 4-bit Operand (A):**  $A_1$  (Pin 1),  $A_2$  (Pin 3),  $A_3$  (Pin 8),  $A_4$  (Pin 11).
- **Second 4-bit Operand (B):**  $B_1$  (Pin 16),  $B_2$  (Pin 4),  $B_3$  (Pin 7),  $B_4$  (Pin 10).
- **Carry In ( $C_0$ ):** Pin 13.
- **Sum Outputs:**  $\Sigma_1$  (Pin 2),  $\Sigma_2$  (Pin 15),  $\Sigma_3$  (Pin 6),  $\Sigma_4$  (Pin 9).
- **Carry Out ( $C_4$ ):** Pin 14.

### 3.4 Circuit Operation

- Connect a **Mode Control switch (M)** to Carry In (Pin 13) and to one input of each of the four XOR gates in IC 7486.
- Connect operand bits  $B_1, B_2, B_3, B_4$  to the second input of each XOR gate.
- Connect the XOR outputs to the **B** inputs of IC 7483.
- **Addition Mode ( $M = 0$ ):**
- $B_i \oplus 0 = B_i$  (true data passes through).
- $C_0 = 0$ .
- IC 7483 performs standard binary addition:  $\mathbf{A + B}$ .
- **Subtraction Mode ( $M = 1$ ):**
- $B_i \oplus 1 = \overline{B_i}$  (operand **B** is bitwise inverted).
- $C_0 = 1$  (the +1 required for 2's complement is fed into Carry In).
- IC 7483 performs 2's complement subtraction:  $\mathbf{A + B + 1 = A - B}$ .

### 3.5 Viva-Voce Questions & Technical Answers

- **Q1: In subtraction mode ( $M=1$ ), how do you interpret the carry output  $C_4$ ?**

*Answer:* In 2's complement subtraction, if  $C_4 = 1$ , the result is positive ( $A \geq B$ ) and the answer appears directly in true binary form. If  $C_4 = 0$ , the result is negative ( $A < B$ ) and the answer appears in 2's complement form; taking the 2's complement of the output reveals its true negative magnitude.

- **Q2: What is the advantage of using IC 7483 over cascading four individual 1-bit full adders?**

*Answer:* IC 7483 incorporates internal high-speed lookahead carry logic across all 4 bits, reducing the carry propagation delay from  $\approx 40 \text{ ns}$  down to under  $10 \text{ ns}$ .

---

## EXPERIMENT 4: Multiplexer & Demultiplexer Implementation (IC 74153 & IC 74138)

### 4.1 Aim

To study the operation of a 4-to-1 multiplexer (IC 74153) and a 3-to-8 line decoder/demultiplexer (IC 74138).

### 4.2 Components Required

- IC 74153 (Dual 4-to-1 Data Selector/Multiplexer)

- IC 74138 (3-to-8 Line Decoder / Demultiplexer)
- Trainer Kit, Wires, LEDs

### 4.3 IC 74153 Multiplexer Verification

- Connect select inputs **S<sub>1</sub>** and **S<sub>0</sub>** to toggle switches.
- Apply distinct binary signals (e.g., **I<sub>0</sub> = 0**, **I<sub>1</sub> = 1**, **I<sub>2</sub> = 0**, **I<sub>3</sub> = 1**) to data inputs.
- Tie active-low strobe **1G** (Pin 1) to Ground to enable the multiplexer.
- Observe output **1Y** (Pin 7) as select lines sequence through **00**, **01**, **10**, **11**.

### 4.4 IC 74138 Demultiplexer Mode

- A decoder operates as a **demultiplexer** by using its data select lines (**A**, **B**, **C**) as address routing lines and using an active-low enable pin ( **$\overline{\text{G}}_{2A}$**  or  **$\overline{\text{G}}_{2B}$** ) as the single serial **Data Input (D<sub>in</sub>)**!
- Tie enable pin **G<sub>1</sub>** HIGH (**+5 V**) and  **$\overline{\text{G}}_{2A}$**  LOW (**0 V**).
- Apply the data bit to  **$\overline{\text{G}}_{2B}$** . The data bit appears at the selected output line (**Y<sub>0</sub>** through **Y<sub>7</sub>**) designated by the 3-bit binary address on **C**, **B**, **A**.

### 4.5 Viva-Voce Questions & Technical Answers

- **Q1: Why are the outputs of IC 74138 active-low?**  
*Answer:* In digital systems, memory chip enable (**CE**) and peripheral select lines are standard active-low to reduce static power consumption and provide higher noise immunity against positive-going line transients.
- **Q2: Can any n-variable Boolean function be implemented using a multiplexer with n-1 select lines?**  
*Answer:* Yes. By assigning **n-1** variables to the select lines, the remaining variable (in true form, inverted form, 0, or 1) is applied directly to the data inputs.

## EXPERIMENT 5: BCD to 7-Segment Display Decoder/Driver (IC 7447 & Common Anode Display)

### 5.1 Aim

To interface a BCD-to-7-segment decoder/driver (IC 7447) with a common anode 7-segment LED display and display decimal digits 0 through 9.

### 5.2 Components Required

- IC 7447 (BCD to 7-Segment Decoder/Driver with open-collector outputs)
- Common Anode 7-Segment LED Display (e.g., LTS-542A)
- Seven **330 Ω** current-limiting resistors ( **$\frac{1}{4}$  W**)
- Trainer Kit, Wires

### 5.3 Hardware Wiring Guidelines

- **Common Anode Terminal:** Connect the common anode pins (Pins 3 and 8 of the display) to **+5 V**.
- **Current-Limiting Resistors:** Connect a separate **330 Ω** resistor in series with each segment cathode pin (**a**, **b**, **c**, **d**, **e**, **f**, **g**) between the display and the corresponding output pin of IC 7447 (pins 13 through 9).
- **Critical Warning:** Never connect a common anode LED display directly to IC 7447 outputs without series resistors! Doing so causes excessive forward current (**>50 mA**), permanently burning out the display segments.
- **Resistor Calculation:**

$$R = \frac{V_{CC} - V_F(\text{LED}) - V_{OL}(\text{IC})}{I_F} = \frac{5.0 \text{ V} - 1.8 \text{ V} - 0.4 \text{ V}}{10 \text{ mA}} = \frac{2.8 \text{ V}}{0.010 \text{ A}} = 280 \text{ } \Omega \blacksquare \text{ Use } 330 \text{ } \Omega$$

- **Control Pins:** Tie Lamp Test (**LT**, Pin 3), Blanking Input (**BI/RB0**, Pin 4), and Ripple Blanking Input (**RBI**, Pin 5) HIGH (**+5 V**) for normal numeric decoding.

### 5.4 Viva-Voce Questions & Technical Answers

- **Q1: Why does IC 7447 require open-collector outputs?**  
*Answer:* Open-collector outputs can sink significant current (up to **40 mA**) to ground when turned ON, and can tolerate pull-up voltages higher than **5 V** (up to **15 V**), allowing it to directly drive high-voltage displays and relays.
- **Q2: What is the purpose of the Lamp Test (LT) pin?**  
*Answer:* When **LT** is pulsed LOW, all seven segments are forced ON simultaneously regardless of the BCD input. This allows a technician to immediately verify if any segment LED has burned out.

## EXPERIMENT 6: Verification of JK and D Flip-Flops (IC 7476 & IC 7474)

### 6.1 Aim

To verify the truth tables, excitation behavior, and edge-triggered clocking of Master-Slave JK Flip-Flop (IC 7476) and Positive-Edge-Triggered D Flip-Flop (IC 7474).

### 6.2 Components Required

- IC 7476 (Dual JK Master-Slave Flip-Flop with Preset and Clear)
- IC 7474 (Dual D Positive-Edge-Triggered Flip-Flop)
- Single-pulse debounced clock generator switch
- LEDs, Trainer Kit

### 6.3 IC 7476 Pinout & Operation

- **Preset (PRE, Pin 2) & Clear (CLR, Pin 3):** Active-low asynchronous overrides. For normal clocked operation, connect both to **+5 V**.
- Apply logic levels to **J** (Pin 4) and **K** (Pin 16).
- Apply a single manually debounced clock pulse to **CLK** (Pin 1).
- Observe outputs **Q** (Pin 15) and **Q** (Pin 14) on falling clock transitions.

### 6.4 Observation Table: JK Flip-Flop

| PRE | CLR | Clock     | J | K | Next State $Q_{n+1}$ | Operation           |
|-----|-----|-----------|---|---|----------------------|---------------------|
| 0   | 1   | x         | x | x | 1                    | Asynchronous Preset |
| 1   | 0   | x         | x | x | 0                    | Asynchronous Clear  |
| 1   | 1   | downarrow | 0 | 0 | $Q_n$                | No Change (Hold)    |
| 1   | 1   | downarrow | 0 | 1 | 0                    | Reset               |
| 1   | 1   | downarrow | 1 | 0 | 1                    | Set                 |
| 1   | 1   | downarrow | 1 | 1 | $Q_n$                | Toggle              |

### 6.5 Viva-Voce Questions & Technical Answers

- **Q1: Why are Preset and Clear called "asynchronous" inputs?**  
*Answer:* Because they override the clock and data inputs immediately, forcing the flip-flop to SET or RESET instantaneously without waiting for a clock pulse transition.
- **Q2: Why must a mechanical switch be debounced when driving a flip-flop clock pin?**  
*Answer:* Mechanical switch contacts bounce microscopically for **5 to 20 ms** upon closure, generating dozens of rapid voltage spikes. Because a flip-flop can switch in nanoseconds, it will count every bounce as a distinct clock pulse, causing erratic state transitions. Debouncing with an SR latch or Schmitt trigger eliminates this.

---

## EXPERIMENT 7: Design of Mod-10 (Decade) Asynchronous Counter Using IC 7490

### 7.1 Aim

To construct and verify a Mod-10 (Decade / BCD) asynchronous ripple counter using the 7490 decade counter IC and display counts from 0 to 9 on LEDs.

### 7.2 Components Required

- IC 7490 (Decade Counter)
- Logic Trainer Kit with variable pulse generator (1 Hz)
- LEDs, Wires

### 7.3 Wiring Guide for BCD Count Sequence (0 → 9)

- **Power:** Pin 5 = +5 V, Pin 10 = Ground.
- **Reset Pins:** Connect **R<sub>1</sub>** (Pin 2) and **R<sub>2</sub>** (Pin 3) to Ground. Connect **S<sub>1</sub>** (Pin 6) and **S<sub>2</sub>** (Pin 7) to Ground.
- **Internal Structure:** IC 7490 contains a Mod-2 section (Input **A**, Pin 14; Output **Q<sub>A</sub>**, Pin 12) and a Mod-5 section (Input **B**, Pin 1; Outputs **Q<sub>B</sub>**, **Q<sub>C</sub>**, **Q<sub>D</sub>** at pins 9, 8, 11).
- **Decade Counter Wiring:** Connect output **Q<sub>A</sub>** (Pin 12) directly to Input **B** (Pin 1)!
- Connect master clock pulses (1 Hz) to Input **A** (Pin 14).
- Connect outputs **Q<sub>D</sub>**, **Q<sub>C</sub>**, **Q<sub>B</sub>**, **Q<sub>A</sub>** to four LEDs (**Q<sub>D</sub>** is MSB, **Q<sub>A</sub>** is LSB).

### 7.4 Viva-Voce Questions & Technical Answers

- **Q1: How does IC 7490 automatically reset after state 9 (1001)?**  
*Answer:* The internal Mod-5 counter transitions back to state 0 on the 5th pulse entering Input **B**. Because the Mod-2 counter divides the incoming clock by 2 before driving the Mod-5 stage, the combined circuit automatically cycles through exactly  $2 \times 5 = 10$  unique states (0000 to 1001) and resets to 0000 on the 10th clock pulse.
- **Q2: How can an IC 7490 be configured as a Mod-6 counter?**  
*Answer:* To reset at count 6 (0110), feed outputs **Q<sub>C</sub>** (weight 4) and **Q<sub>B</sub>** (weight 2) directly into the reset pins **R<sub>1</sub>** and **R<sub>2</sub>**. The instant the counter reaches 0110, pins **R<sub>1</sub>** and **R<sub>2</sub>** both become HIGH, triggering an immediate asynchronous clear back to 0000.

---

## EXPERIMENT 8: 4-Bit Shift Register Implementation Using IC 7495

### 8.1 Aim

To configure the 7495 4-bit universal shift register in Serial-In Serial-Out (SISO) and Serial-In Parallel-Out (SIPO) modes.

### 8.2 Components Required

- IC 7495 (4-Bit Parallel-Access Shift Register)
- Trainer Kit, Wires, LEDs

### 8.3 Operation Modes

- **Mode Control Pin (Pin 6):**
- Connect Mode Control LOW (0 V) for Shift-Right mode.
- Connect Clock 1 (Pin 8) to the shift clock source.
- **SISO Mode:**
- Apply serial data bits to Serial Input (Pin 1).
- Clock 4 times to shift bits through stages **Q<sub>A</sub>**, **Q<sub>B</sub>**, **Q<sub>C</sub>**, **Q<sub>D</sub>**.
- Read serial output from **Q<sub>D</sub>** (Pin 10).
- **SIPO Mode:**
- Connect four LEDs to outputs **Q<sub>A</sub>**, **Q<sub>B</sub>**, **Q<sub>C</sub>**, **Q<sub>D</sub>** (Pins 13, 12, 11, 10).
- As clock pulses arrive, observe data bits physically marching across the LED row from left to right.

### 8.4 Viva-Voce Questions & Technical Answers

- **Q1: How many clock pulses are required to enter and retrieve an 8-bit word in a SISO register?**  
*Answer:* It requires 8 clock pulses to load the data serially into the register and another 8 clock pulses to shift it out serially, requiring a total of 16 clock pulses. In contrast, a PIPO register transfers the word in a single clock cycle!

## EXPERIMENT 9: 4-Bit R-2R Ladder Digital-to-Analog Converter

### 9.1 Aim

To construct a 4-bit R-2R ladder Digital-to-Analog Converter using discrete resistors and an operational amplifier (LM741), and verify linearity and step size.

### 9.2 Components Required

- Op-Amp IC LM741 (8-pin DIP)
- Dual DC power supply ( $\pm 12\text{ V}$  for op-amp rails)
- Six  $10\text{ k}\Omega$  resistors (1% precision)
- Six  $20\text{ k}\Omega$  resistors (or pairs of two  $10\text{ k}\Omega$  in series)
- Digital Multimeter (DMM) with  $1\text{ mV}$  DC resolution
- Logic toggle switches (0 V / +5 V)

### 9.3 Circuit Schematic & Connections

- Assemble the R-2R ladder on the breadboard. Connect ladder node branches to data switches **D\_3** (MSB), **D\_2**, **D\_1**, **D\_0** (LSB).
- Connect the ladder output node to the inverting input (Pin 2) of LM741.
- Connect non-inverting input (Pin 3) of LM741 to Ground (0 V).
- Connect feedback resistor **R<sub>f</sub> = 20 kΩ** between output (Pin 6) and inverting input (Pin 2).
- Connect Pin 7 to **+12 V** and Pin 4 to **-12 V**.

### 9.4 Observation & Linearity Table (**V<sub>ref</sub>** = 5.0 V)

| Theoretical Step Size $\Delta V = \frac{5.0\text{ V}}{16} = 0.3125\text{ V} = 312.5\text{ mV}$ |                    |                                        |                                       |                     |
|------------------------------------------------------------------------------------------------|--------------------|----------------------------------------|---------------------------------------|---------------------|
| Binary Input ( <b>D_3</b><br><b>D_2</b> <b>D_1</b> <b>D_0</b> )                                | Decimal Equivalent | Theoretical <b>V<sub>out</sub></b> (V) | Measured <b>V<sub>out</sub></b> (DMM) | Absolute Error (mV) |
| 0000                                                                                           | 0                  | 0.000                                  | 0.002                                 | 2                   |
| 0001                                                                                           | 1                  | -0.313                                 | -0.315                                | 2                   |
| 0010                                                                                           | 2                  | -0.625                                 | -0.628                                | 3                   |
| 0011                                                                                           | 3                  | -0.938                                 | -0.941                                | 3                   |
| 0100                                                                                           | 4                  | -1.250                                 | -1.253                                | 3                   |
| 0101                                                                                           | 5                  | -1.563                                 | -1.565                                | 2                   |
| 0110                                                                                           | 6                  | -1.875                                 | -1.880                                | 5                   |
| 0111                                                                                           | 7                  | -2.188                                 | -2.190                                | 2                   |
| 1000                                                                                           | 8                  | -2.500                                 | -2.504                                | 4                   |
| 1001                                                                                           | 9                  | -2.813                                 | -2.816                                | 3                   |
| 1010                                                                                           | 10                 | -3.125                                 | -3.130                                | 5                   |

| Binary Input (D <sub>3</sub> D <sub>2</sub> D <sub>1</sub> D <sub>0</sub> ) | Decimal Equivalent | Theoretical V <sub>out</sub> (V) | Measured V <sub>out</sub> (DMM) | Absolute Error (mV) |
|-----------------------------------------------------------------------------|--------------------|----------------------------------|---------------------------------|---------------------|
| 1011                                                                        | 11                 | -3.438                           | -3.441                          | 3                   |
| 1100                                                                        | 12                 | -3.750                           | -3.755                          | 5                   |
| 1101                                                                        | 13                 | -4.063                           | -4.069                          | 6                   |
| 1110                                                                        | 14                 | -4.375                           | -4.381                          | 6                   |
| 1111                                                                        | 15                 | -4.688                           | -4.693                          | 5                   |

9.5 Viva-Voce Questions & Technical Answers

- Q1: Why is the measured analog output voltage negative?**  
*Answer:* Because the op-amp is wired as an inverting summing amplifier with negative feedback. The input current enters virtual ground (0 V) and passes through feedback resistor R<sub>f</sub>, creating an output voltage of opposite polarity:  $V_{out} = -I_{in} \cdot R_f$ . Adding a second unity-gain inverting op-amp restores a positive output.
- Q2: What is the settling time of a DAC?**  
*Answer:* The elapsed time between the digital input code transition and the moment the analog output voltage settles and remains within  $\pm (1 / 2)$  LSB of its final steady-state value.

APPENDIX & MASTER REFERENCE COMPENDIUM

A. Master Mathematical & Operational Cheat Sheet

1. Number Systems & Binary Arithmetic (Unit I)

- Radix Polynomial Expansion:

$$N_r = \sum_{i=0}^{n-1} d_i \cdot r^i + \sum_{j=1}^m d_{-j} \cdot r^{-j}$$

- Radix-minus-one Complement (r-1's complement):

$$(r-1)'s = (r^n - 1) - N$$

For binary: invert every bit (0 ≤ f → 1). - Radix Complement (r's complement):

$$r's = r^n - N = \text{ft}[(r-1)'s_{right}] + 1$$

For binary: take 1's complement and add 1. - Signed Range for n-bit integers:

$$\text{Range}_{2's \text{ comp}} = \text{ft}[-2^{n-1}, +2^{n-1}-1_{right}]$$

For n=8: [-128, +127].

2. Logic Families & DC Noise Margins (Unit I)

- High-state Noise Margin:

$$NM_H = V_{OH(min)} - V_{IH(min)}$$

- Low-state Noise Margin:

$$NM_L = V_{IL(max)} - V_{OL(max)}$$

- Fan-out:

$$\text{Fan-out} = \min \leq \text{ft}(\frac{I_{OH(\max)}}{I_{IH(\max)}}, \frac{I_{OL(\max)}}{I_{IL(\max)}} \text{right})$$

- Dynamic Power Dissipation in CMOS:

$$P_{\text{dynamic}} = C_L \cdot V_{DD}^2 \cdot f$$

### 3. Boolean Algebra & Minimization (Unit II)

- De Morgan's Theorems:

$$A + B = A \cdot B$$

$$A \cdot B = A + B$$

- Absorption Laws:

$$A + A \cdot B = A$$

$$A \cdot (A + B) = A$$

$$A + A \cdot B = A + B$$

- Consensus Theorem:

$$A B + A C + B C = A B + A C$$

### 4. Arithmetic & Combinational Circuits (Unit III)

- Half Adder:

$$S = A \oplus B, C = A \cdot B$$

- Full Adder:

$$S = A \oplus B \oplus C_{\text{in}}, C_{\text{out}} = A B + B C_{\text{in}} + A C_{\text{in}}$$

- Carry Lookahead Generator:

$$G_i = A_i B_i, P_i = A_i \oplus B_i$$

$$C_{i+1} = G_i + P_i C_i$$

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

- Multiplexer Output Expansion:

$$Y = \sum_{k=0}^{2^n-1} m_k \cdot I_k$$

### 5. Sequential Logic & Flip-Flop Characteristic Equations (Unit IV)

- SR Flip-Flop:

$$Q_{\text{next}} = S + R Q, \text{ with condition } S \cdot R = 0$$

- JK Flip-Flop:

$$Q_{\text{next}} = J Q + K Q$$

- D Flip-Flop:

$$Q_{\text{next}} = D$$

- T Flip-Flop:

$$Q_{\text{next}} = T \oplus Q = T Q + \bar{T} \bar{Q}$$

- Master Excitation Summary:

| $Q_n \rightarrow Q_{n+1}$ | S | R | J | K | D | T |
|---------------------------|---|---|---|---|---|---|
| 0 → 0                     | 0 | x | 0 | x | 0 | 0 |
| 0 → 1                     | 1 | 0 | 1 | x | 1 | 1 |
| 1 → 0                     | 0 | 1 | x | 1 | 0 | 1 |
| 1 → 1                     | x | 0 | x | 0 | 1 | 0 |

- Maximum Ripple Counter Frequency:

$$f_{\text{max}} = \frac{1}{n \cdot t_{\text{pd}}}$$

## 6. Converters & Semiconductor Memories (Unit V)

- DAC Resolution (Step Size):

$$\Delta V = \frac{V_{\text{FS}}}{2^n - 1}$$

- Binary Weighted DAC Output:

$$V_{\text{out}} = -V_{\text{ref}} \left( R_f / R \right) \sum_{i=0}^{n-1} D_i 2^{-(n-1-i)}$$

- R-2R Ladder DAC Output:

$$V_{\text{out}} = V_{\text{ref}} \sum_{i=1}^n D_{n-i} 2^{-i}$$

- ADC Quantization Error:

$$Q_e = \pm (1 / 2) \text{ LSB}$$

- SAR ADC Conversion Time:

$$t_{\text{conversion}} = n \times T_{\text{clock}}$$

- Dual-Slope ADC Transfer Equation:

$$V_{\text{in}} = V_{\text{ref}} \leq f t((T_2 / T_1) \text{right})$$

- Memory Capacity:

$$\text{Capacity} = 2^k \times m \text{ (where } k = \text{address lines, } m = \text{data lines)}$$

## B. Complete Master Textbook Figure Citation Directory

| Figure ID                            | Topic Description                                   | Source Textbook | Chapter & Figure   | Page (PDF / Printed) |
|--------------------------------------|-----------------------------------------------------|-----------------|--------------------|----------------------|
| fig_u1_ttl_totem_pole.png            | Standard TTL NAND Gate with Totem-Pole Output       | Anand Kumar     | Ch. 4, Fig. 4.2    | p. 165 / p. 133      |
| fig_u1_cmos_nand.png                 | 2-Input CMOS NAND Gate Circuit                      | Anand Kumar     | Ch. 4, Fig. 4.31   | p. 195 / p. 163      |
| fig_u1_ttl_cmos_interface.png        | Interfacing TTL driving CMOS (Pull-Up Resistor)     | Anand Kumar     | Ch. 4, Fig. 4.36   | p. 201 / p. 169      |
| fig_u2_kmap_grouping_rules.png       | Standard K-Map Rules (2, 3, 4 Variables)            | Anand Kumar     | Ch. 5, Fig. 5.1    | p. 238 / p. 206      |
| fig_u3_full_adder_logic.png          | Full Adder Logic Diagram & Truth Table              | Anand Kumar     | Ch. 6, Fig. 6.2    | p. 320 / p. 288      |
| fig_u3_parallel_adder_subtractor.png | 4-Bit Parallel Binary Adder/Subtractor with XOR     | Anand Kumar     | Ch. 6, Fig. 6.8    | p. 327 / p. 295      |
| fig_u3_carry_lookahead_adder.png     | 4-Bit Carry Lookahead Adder (CLA) Generator         | Anand Kumar     | Ch. 6, Fig. 6.10   | p. 329 / p. 297      |
| fig_u3_multiplexer_trees.png         | 4-to-1 Multiplexer Internal Logic Gates             | Anand Kumar     | Ch. 7, Fig. 7.1    | p. 396 / p. 364      |
| fig_u3_decoder_74138.png             | 3-to-8 Line Decoder (74138/LS138) Internal Logic    | Anand Kumar     | Ch. 7, Fig. 7.37   | p. 433 / p. 401      |
| fig_u3_priority_encoder_74148.png    | 8-to-3 Octal Priority Encoder (74148) Logic Diagram | Anand Kumar     | Ch. 7, Fig. 7.50   | p. 450 / p. 418      |
| fig_u4_sr_latch_nor.png              | Basic SR Latch using Cross-Coupled NOR Gates        | Anand Kumar     | Ch. 10, Fig. 10.1  | p. 556 / p. 524      |
| fig_u4_clocked_sr_flip_flop.png      | Clocked SR Flip-Flop with NAND Gates                | Anand Kumar     | Ch. 10, Fig. 10.5  | p. 559 / p. 527      |
| fig_u4_master_slave_jk.png           | Master-Slave JK Flip-Flop Logic Circuit             | Anand Kumar     | Ch. 10, Fig. 10.12 | p. 565 / p. 533      |
| fig_u4_siso_shift_register.png       | 4-Bit Serial-In Serial-Out (SISO) Shift Register    | Anand Kumar     | Ch. 11, Fig. 11.1  | p. 624 / p. 592      |
| fig_u4_ripple_counter_4bit.png       | 4-Bit Asynchronous Binary Ripple Counter            | Anand Kumar     | Ch. 12, Fig. 12.1  | p. 678 / p. 646      |

| Figure ID                                 | Topic Description                                  | Source Textbook | Chapter & Figure   | Page (PDF / Printed) |
|-------------------------------------------|----------------------------------------------------|-----------------|--------------------|----------------------|
| fig_u4_mod10_counter.png                  | Mod-10 (Decade) Ripple Counter with Reset          | Anand Kumar     | Ch. 12, Fig. 12.7  | p. 687 / p. 655      |
| fig_u4_synchronous_counter_4bit.png       | 4-Bit Synchronous Binary Up-Counter                | Anand Kumar     | Ch. 12, Fig. 12.11 | p. 696 / p. 664      |
| fig_u5_weighted_resistor_dac.png          | Binary Weighted-Resistor Summing Op-Amp DAC        | Anand Kumar     | Ch. 17, Fig. 17.10 | p. 951 / p. 919      |
| fig_u5_r2r_ladder_dac.png                 | R-2R Ladder Type Digital-to-Analog Converter       | Anand Kumar     | Ch. 17, Fig. 17.5  | p. 947 / p. 915      |
| fig_u5_counter_type_adc.png               | Logic Diagram of Counter-Type (Ramp) ADC           | Anand Kumar     | Ch. 17, Fig. 17.15 | p. 958 / p. 926      |
| fig_u5_dual_slope_adc.png                 | Dual-Slope Integrating ADC Architecture            | Anand Kumar     | Ch. 17, Fig. 17.23 | p. 964 / p. 932      |
| fig_u5_sar_adc.png                        | Successive Approximation Register (SAR) ADC        | Anand Kumar     | Ch. 17, Fig. 17.24 | p. 965 / p. 933      |
| fig_u5_adc0801_pinout.png                 | Functional Pin Diagram of ADC0801 8-Bit Converter  | Anand Kumar     | Ch. 17, Fig. 17.25 | p. 967 / p. 935      |
| fig_u5_memory_organization.png            | 32 × 4 Semiconductor Memory Array & Address Decode | Anand Kumar     | Ch. 18, Fig. 18.1  | p. 976 / p. 944      |
| fig_u5_sram_dram_cells.png                | Static RAM (SRAM) Transistor Storage Cells         | Anand Kumar     | Ch. 18, Fig. 18.5  | p. 982 / p. 950      |
| fig_u5_memory_word_length_expansion.png   | Memory Expansion: Word Length (16 × 4 to 16 × 8)   | Anand Kumar     | Ch. 18, Fig. 18.7  | p. 985 / p. 953      |
| fig_u5_memory_word_capacity_expansion.png | Memory Expansion: Word Capacity (16 × 4 to 32 × 4) | Anand Kumar     | Ch. 18, Fig. 18.9  | p. 988 / p. 956      |
| fig_u5_pld_configuration.png              | Architectural Configurations: PROM, PAL, PLA       | Anand Kumar     | Ch. 8, Fig. 8.7    | p. 499 / p. 467      |
| fig_u5_pal_structure.png                  | Programmable Array Logic (PAL) Internal Matrix     | Anand Kumar     | Ch. 8, Fig. 8.8    | p. 500 / p. 468      |

| Figure ID                | Topic Description                              | Source Textbook | Chapter & Figure | Page (PDF / Printed) |
|--------------------------|------------------------------------------------|-----------------|------------------|----------------------|
| fig_u5_pla_structure.png | Programmable Logic Array (PLA) Internal Matrix | Anand Kumar     | Ch. 8, Fig. 8.15 | p. 508 / p. 476      |

---